Kodewala Academy

Transforming code into career

Core Java & Advance Java

Kodewala

Overview of Java

What is Java?

A high-level, platform-independent, object-oriented programming language developed by Sun Microsystems (now owned by Oracle).

History and Evolution of Java:

Java was developed in 1991 and released in 1995

Versions from Java 1.0 to the latest LTS (e.g., Java 21).

LTS (Long Term Support) releases, meaning they receive extended support from Oracle and other vendors.

Why Do we use Java?

Use cases: Web development, mobile apps (Android), enterprise applications (large-scale software), game development, etc.

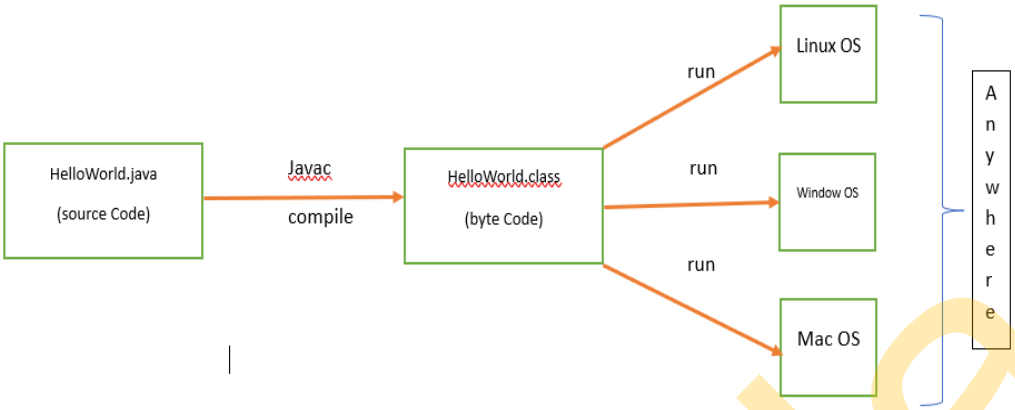
Note: Oracle Corporation acquired Sun Microsystems in 2010. Since then, Java has been licensed and maintained by Oracle Corporation.

Features of Java:

1. **Platform Independent:** Run code anywhere with JVM on any operating system (Window, Linux, Mac etc.).

   When we write and compile a Java program, the result is a .class file, which contains the **bytecode**. This bytecode is platform-independent, meaning it can be compiled on one operating system and executed on any other operating system. This concept is known as **WORA** (Write Once, Run Anywhere)

Kodewala



2. **Object-Oriented:** Java is an object-oriented programming (OOP) language, where everything is treated as an object. OOP means organizing software into different objects that combine both data and behaviour.

   OOP simplifies software development and maintenance by following a set of principles, which are:

   Object: An instance of a class, containing both data and methods.

   Class: A blueprint for creating objects, defining their properties and behaviours.

   Inheritance: A way for one class to inherit properties and behaviours from another.

   Polymorphism: The ability for different objects to respond to the same method in their own way.

   Abstraction: Hiding complex implementation details, showing only essential features.

   Encapsulation: Bundling data and methods together, restricting access to the internal state of objects.

3. **Robust and Secure:** Java offers built-in **garbage collection**, **strong memory management**, and **runtime error checking** to enhance program efficiency and reliability.

4. **High Performance:** With Just-In-Time (JIT) compilation.

Kodewala

Industry Experience (Real-World Example)

- ✓ "**Write Once, Run Anywhere**" (**WORA**) principle has been a game-changer. Applications built on Windows servers can seamlessly run on Linux-based servers without modification.

- ✓ Java's built-in **garbage collection** and runtime error checks helped ensure smooth operation in applications that handle millions of transactions daily (e.g., payment gateways or banking systems).

Interview Experience

- ✓ If you have compiled the java code in window machine, would you be able to run on Linux?
- ✓ What is WORA, and how does Java achieve it?
- ✓ Explain the significance of Java being platform-independent.

Kodewala

Your First Java Code: Hello World

Prerequisite: Installation of Java, Notepad++

Steps to Implement Java Program

- Write the java program (Creating the java class)
- Compile the program (Using javac command)
- Run / Execute the program (using java command)

Write the java program (Creating the java class)

Note: We will be creating the java program using Notepad++.

```
// This is you first Java program.
// FileName : "HelloWorld.java".

public class HelloWorld

{

public static void main(String args[])
    {

        // Your program begins with a call to main(String args[]) method
        // Prints "Welcome to the Kodewala!" to the command window.

        System.out.println("Welcome to the Kodewala!");

    }

}
```

Kodewala

1. Class Definition

This line uses the keyword **class** to declare that a new class is being defined.

```
public class HelloWorld

 {
 // Statements, Methods, Logics etc.
 }
```

2. HelloWorld

"HelloWorld" is the name of the class and in java everything will be between the opening curly brace "**{**" and the closing curly brace "**}**".

3. main() Method

In the Java, every application must have a **main** method. The main method is the entry point of your Java application, and it is mandatory in a Java program.

```
public static void main(String[] args)
```

4. Explanation of the main method Syntax

- ✓ ***public:*** JVM can call the method from anywhere with in Java universe.
- ✓ ***static:*** The main method is to be called without an object creation. The modifiers are public and static can be written in either order.
- ✓ ***void:*** The main method does not return anything.
- ✓ ***main()****:* Name configured in the JVM. The main method must be inside the class definition. The JVM executes the codes starting always from the main function.

1. ***String[]*** : The main method accepts a single argument, an array of elements of type String. This would be used to receive a command line argument for the application.

5. Code inside the main() method

```
System.out.println("Welcome to the Kodewala!");
```

The statement can be broken into 3 parts which can be understood separately:

1. **System**: It is a predefined java class.
2. **out**: This is an instance of PrintStream type, which is a public and static member field of the System class.
3. **println():** This is a method in PrintStream. It prints any argument passed to it and adds a new line to the output.

Kodewala

Now Let's compile and run the program.

1. **Compiling the Program**

   - ✓ After successfully setting up the environment, we can open a terminal in Mac/Windows/Unix and go to the directory where the file, "HelloWorld.java" is present.
   - ✓ Now, to compile the "HelloWorld" program, execute the compiler – **javac**, to specify the name of the source file on the command line, as shown below.

     <You program directory> javac HelloWorld.java

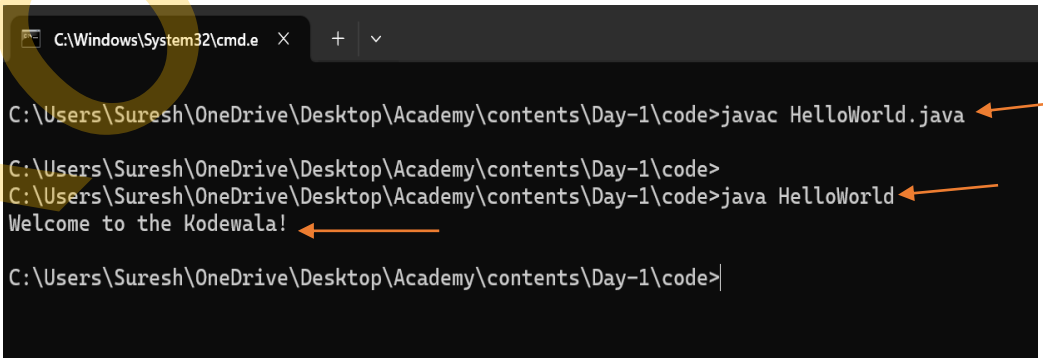
   - ✓ Once you run the above command, the compiler compile the code and create "HelloWorld.class" in the same directory. The .class contains the bytecode version of the program.

2. **Running / Executing the Program**

   - ✓ Now, to execute our program, **JVM** (Java Virtual Machine) needs to be called using Java, specifying the name of the **class** file on the command line, as shown below.

     <You program directory> java HelloWorld

   - ✓ This will print "Welcome to the Kodewala!" to the command screen.
   - ✓ Compile and run in window



   - ✓ With this you are able to code and run the Java HelloWorld program. Fantastic New!

Point To Note: "In java file name and class name should be the same". When the above law is violated a compiler error message will appear as below. We will discuss about the file name rules in next sessions. As of now just remember the rule: "**Class Name should be same as file Name**"

Kodewala

Industry Experience

- ✓ Debugging the code and understanding the error and exception.
- ✓ Improve debugging skills.
- ✓ Run without main method.
- ✓ Verify the generate bytecode / class files.

Interview Experience

- ✓ Why main method is static?
- ✓ Can we run the program without main method?
- ✓ What is the command used to compile and run the java program?
- ✓ What is the purpose the String args[] ?
- ✓ Explain the System.out.println()
- ✓ File Name vs Class name
- ✓ How many class files will be created for each java file?

Kodewala

JDK, JRE, and JVM

JDK

Definition: JDK stands for Java Development Kit, which is a complete toolkit for Java development. It includes everything needed to develop Java applications.

The main components of the JDK (Java Development Kit) are:

- ✓ **JRE**: JDK includes the Java Runtime Environment, which provides the runtime libraries and JVM required to run Java programs.
- ✓ **Compilers**: JDK contains a compiler (javac) that translates Java source code into bytecode.
- ✓ **Development Tools**: JDK provides additional tools such as javap (disassembler), javadoc (document generator), and jdb (debugger), which are essential for building, debugging, and documenting Java programs.
- ✓ **Libraries**: JDK comes with a set of standard libraries (Java API) that provide functionality like I/O operations, networking, database connections, etc.
- ✓ **JVM**: JDK includes the Java Virtual Machine, which is responsible for running the bytecode.

Purpose: JDK is primarily used by developers to write, compile, and debug Java applications. It is required for anyone developing Java programs from scratch.

JRE (Java Runtime Environment):

Definition: JRE stands for Java Runtime Environment. It provides the necessary environment for running Java applications but does not include development tools.

The main components of the JRE (Java Runtime Environment) are:

- ✓ **JVM**: JRE includes the Java Virtual Machine, which interprets and runs the compiled Java bytecode.
- ✓ **Libraries**: JRE provides the core set of libraries required to run Java programs, such as the Java standard library (java.lang, java.util, etc.).
- ✓ **Native Libraries**: JRE also includes platform-specific native libraries needed for running Java applications.

Purpose: JRE is used by users or systems that only need to run Java applications, not develop them. It does not include compilers, debuggers, or other development tools that are part of the JDK.

Kodewala

JVM

- ✓ JVM stands for Java Virtual Machine, which is not visible to the user.
- ✓ It is a part of the JRE (Java Runtime Environment).
- ✓ JVM is responsible for executing Java programs.
- ✓ It allocates memory required for program execution.
- ✓ JVM de-allocates memory when it's no longer needed.
- ✓ It converts bytecode into a machine-understandable format.
- ✓ JVM will be created when user execute the “java” command and it will be destroyed once program is executed.

Summary of Differences:

- ✓ **JDK** is a **development toolkit** that includes tools for compiling and debugging Java applications, and it contains **JRE**.
- ✓ **JRE** is the **runtime environment** for running Java applications; it includes the **JVM** but does not have development tools like compilers.
- ✓ **JVM** is the **virtual machine** that executes Java bytecode and manages memory during execution, and it is a key component of **JRE**.

Industry Experience

- ✓ The JRE is sufficient whenever the need is limited to running Java applications without development or debugging requirements.
- ✓ JDK is required for development

Interview Experience

- ✓ Can we install JRE without JDK?
- ✓ An application runs fine on one system but fails on another. Both have the JRE installed. What could be the problem?

Kodewala

Command Line Arguments in Java

About it

Java command-line argument is an argument, passed at the time of running the Java program (java HelloWorld “arg1” “arg2”).

In Java, the command line arguments passed from the console can be received in the Java program and they can be used as input to perform various activities. The users can pass the arguments during the execution bypassing the command-line arguments inside the main() method. This is used to make the program flexible and dynamic.

How to pass command line Args?

We need to pass the arguments as space-separated values. We can pass both strings and primitive data types (int, double, float, char, etc) as command-line arguments. These arguments convert into a string array and are provided to the main (String args[]) function as a string array argument. User need to convert back to desired format as per need.

When user supply the command line args, JVM will wraps up these command-line arguments into the args[ ] array that we pass into the main() method.

Write a program to read the command line arguments

```
public class CmdArgs
{

public static void main(String args[])
{
    String name = args[0];
    String location = args[1];
    System.out.println("Welcome to the "+name);
    System.out.println(name + " is located in "+location);
}

}
```

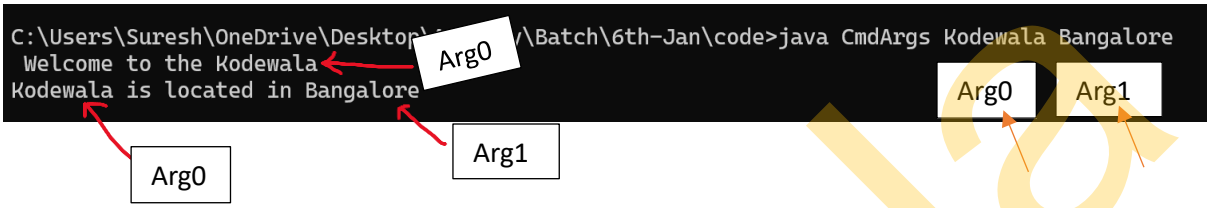
Kodewala

Kodewala Academy

How to pass command line arguments?

While running the java program, user need to pass the arguments as below.

<location>java <class name> <arg0> <arg1> <arg2> <arg…n>



Important Points to Remember:

- ✓ **String Input:** It takes input in the space separated format.
- ✓ **Dynamic-input:** Allows user to pass dynamic inputs.
- ✓ **Length**: We can check the length using .length property of the array.
- ✓ **Indexes**: We can take the values just using the indexes args[0] args[1] args[2] args[n]

Industry Experience

- ✓ If you are developing any application like reading file, user can pass file name as a command line argument.
- ✓ If you are developing the simple application like calculator, user can pass input using command line arguments.
- ✓ If you feel that any input may change in future, like data, filename etc. Use the command line arguments.

Interview Experience

- ✓ If you are developing the utility to read the file. What would be your approach to read the path, considering the path may change in future
- ✓ What will happed if you are readding the command line arguments but not passing them?
- ✓ How will you make sure that command line arguments are passed and in case user missed to pass the args, throw the warning. i.e “Please enter the input”

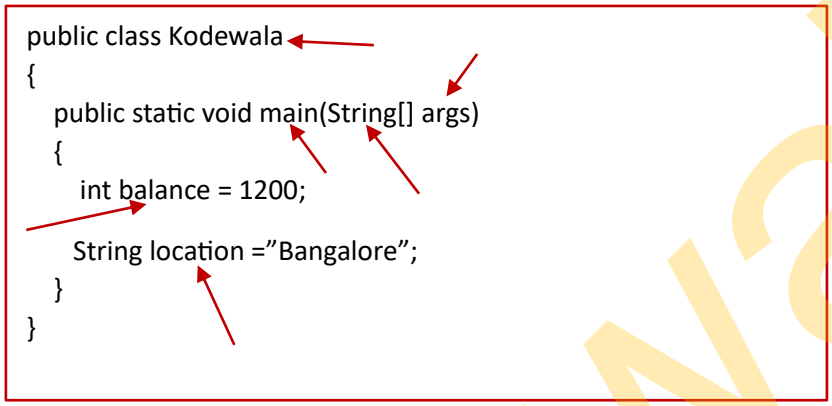
Kodewala

Java Identifiers

About it

An identifier in Java is the name given to **Variables, Classes, Methods, Packages, Interfaces,** etc. These are the unique names and every Java Variables must be identified with unique names.

Let's understand with simple example



In the above program, we have **total 6** identifiers as below.

- ✓ **Kodewala**: Class name
- ✓ **main**: Method name
- ✓ **String**: Predefined Class name
- ✓ **args**: Variable name
- ✓ **balance**: Variable name
- ✓ **location**: Variable name

In Java, there are rules for the identifiers

Java identifiers must adhere to the following rules, otherwise you will get compile time error/failure.

1. **Allowed Characters:** Identifiers can only include alphanumeric characters ([A-Z], [a-z], [0-9]), the dollar sign ($), and the underscore (_). For example, “Kodewala#” is not valid because it contains the special character #.
2. **Starting Character:** Identifiers cannot begin with a digit ([0-9]). For instance, “12Kodewala” is invalid because it starts with a number.
3. **Case Sensitivity:** Java identifiers are case-sensitive, meaning Example and example would be considered different.

Kodewala

4. **Length:** There is no restriction on the maximum length of an identifier. However, it is recommended to keep the length between 4 and 15 characters for better readability and maintainability.
5. **Reserved Words:** Reserved words in Java cannot be used as identifiers. For example, the statement int **for** = 1230; is invalid because while is a reserved word. Java has 50+ reserved words that cannot be used as identifiers.

Here are some examples of identifiers in Java.

Valid	Invalid	
FirstName	**Test Name**	// contains a space
FIRSTNAME	**34kodewala**	// Begins with a digit
firstname	**ca+t**	// plus sign is not an alphanumeric character
y	**variable-5**	// hyphen is not an alphanumeric character
j	**last_&_first**	// ampersand is not an alphanumeric character
y1		
firstname		
$firstname		
total_sum		
code123		

Kodewala

Reserved Words in Java

In programming languages, certain words are reserved to represent specific functionalities defined by the language. These are known as reserved words.

Category	Keywords
Primitive Data Types	`boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, `short`
Control Flow Statements	`if`, `else`, `switch`, `case`, `default`, `while`, `do`, `for`, `break`, `continue`, `return`
	`try`, `catch`, `finally`, `throw`, `throws`
Modifiers	`abstract`, `final`, `private`, `protected`, `public`, `static`, `synchronized`, `transient`
	`volatile`, `strictfp`
Class and Object	`class`, `interface`, `enum`, `extends`, `implements`, `this`, `super`, `instanceof`, `new`
Package Related	`import`, `package`
Exception Handling	`assert`
Multi-threading	`synchronized`, `volatile`
Special Purpose	`const` (reserved but not used), `goto` (reserved but not used), `null`, `true`, `false`
Miscellaneous	`native`

Kodewala

Important Points to Remember:

Always write code that is easy to read and self-explanatory. Let's understand with below example.

```java
public class FundTransfer {

    // Method to transfer funds between accounts
    public void transfer(String fromAccount, String toAccount, double amount) {
        if (amount > 0) {
            System.out.println("Transferred INR " + amount + " from " + fromAccount + " to " + toAccount);
        } else {
            System.out.println("Invalid transfer amount.");
        }
    }

    public static void main(String[] args) {
        FundTransfer fundTransfer = new FundTransfer();

        // Transfer funds from account 1234567 to account 4567893
        fundTransfer.transfer("1234567", "4567893", 500.00);
    }
}
```

Class Name: *FundTransfer*

- ✓ The class name is concise and descriptive. It clearly indicates that the class is designed to handle the process of transferring funds between accounts.
- ✓ **Why it matters:** The name provides clarity and helps other developers understand the class's purpose immediately, improving the readability of the code.

Method Name: *transfer*

- ✓ The method name transfer directly describes the action performed by the method - transferring money from one account to another.
- ✓ **Why it matters**: The method name is intuitive and easy to understand, making the code more readable and maintainable. It follows the principle of "self-describing" method names, where the method name communicates what it does.

Parameters: *fromAccount, toAccount, amount*

- ✓ fromAccount and toAccount: These parameters represent the account numbers involved in the transfer. Their names are straightforward and clearly indicate their purpose.
- ✓ amount: This parameter represents the amount to be transferred. It is also named clearly to avoid ambiguity.
- ✓ **Why it matters:** Descriptive parameter names enhance the readability of the code and help other developers understand what values are expected by the method.

Kodewala

Industry Experience

- ✓ Always give meaningful name for variable and classes, methods.
- ✓ If you are providing the meaningful name, it will be more readable and easier to debug the issues.

Interview Experience

- ✓ There would be question about writing the clean code and maintainability.

Class work

- ✓ Write a program to display employee details provided through command-line arguments, ensuring adherence to Java naming conventions.

Assignment

- ✓ Write a program demonstrating the use of various conventions and rules for naming identifiers.

Kodewala

Java Data Types

About it

Java is strongly typed language. In java each type of the data is predefined and it should be defined with one of the java data types.

Java has two categories in which data types are segregated.

1. **Primitive Data Type**: Including data types such as boolean, char, int, short, byte, long, float, and double.

2. **Non-Primitive or Object Data type:** such as String, Array, Person, Account etc.

A table displaying the primitive data types along with their size, range, and default value.

Data Type	Size	Range	Default Value
`byte`	8 bits	-128 to 127	0
`short`	16 bits	-32,768 to 32,767	0
`int`	32 bits	-2,147,483,648 to 2,147,483,647	0
`long`	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
`float`	32 bits	Approximately ±3.40282347E+38F	0.0f
`double`	64 bits	Approximately ±1.79769313486231570E+308	0.0d
`char`	16 bits	'\u0000' (0) to '\uffff' (65,535)	'\u0000'
`boolean`	1 bit	`true` or `false`	`false`

Note: Boolean size is specific to JVM. Many JVM store Boolean using 8 bits.

Let's write a program and explore the data types.

boolean:

```
public class PrimitiveDataType {

    public static void main(String[] args)
    {
        // declaring boolean variables
        boolean isValid=true;
        System.out.println(" Values : "+isValid);
    }

}
```

Kodewala

byte

```
public class PrimitiveDataType {

    public static void main(String[] args)
    {
        // declaring byte variables
        byte amount=124;
        System.out.println(" Values : "+amount);
    }

}
```

short

```
public class PrimitiveDataType {

    public static void main(String[] args)
    {
        // declaring short variables
        short amount=10;
        System.out.println(" Values : "+amount);
    }

}
```

Int:

```
public class PrimitiveDataType {

    public static void main(String[] args)
    {
        // declaring int variables
        int pen =100;
        int pencil =150;
        int total = pen+pencil;
        System.out.println(" Total Items : "+total);


    }

}
```

Kodewala

long

```
public class PrimitiveDataType {

    public static void main(String[] args)
    {
        // declaring long variables
        long amount=124523453453452L;
        System.out.println(" Values : "+amount);
    }

}
```

Explore the “byte” data type.

```
public class PrimitiveDataType {

    public static void main(String[] args) {

        // Exploring byte
        byte byteValue = 127;
        System.out.println("Byte value: " + byteValue);
        System.out.println("Byte size: " + Byte.SIZE + " bits");
        System.out.println("Byte range: " + Byte.MIN_VALUE + " to " + Byte.MAX_VALUE);
    }

}
```

Industry Experience

- ✓ Always choose the data type based on memory requirements and avoid using it without consideration.

Interview Experience

- ✓ In a large-scale application where memory usage is a concern, how would you decide between using int and long for storing numerical values?
- ✓ Suppose you are developing a system that tracks millions of user data, including large numerical identifiers. Which data type would you choose for user IDs, and why?

Class work

- ✓ Write a program to explore the primitive data types and print the employee’s salary details if permitted (use boolean).

Assignment

- ✓ Write a program to explores all the primitive data types and prints their default values, sizes, and ranges.

Kodewala

Java Data Types (Non-Primitive)

About it

Known as **reference types**, are data types that store references (memory addresses) to the actual data, rather than the data itself. These types are either built-in (e.g - String, Array) or all user-defined classes.

Key Features

- ✓ **Reference-Based:** Store memory addresses pointing to the data.
- ✓ **Defined by Java or the User:** Includes classes, arrays, interfaces, and enums.
- ✓ **Default Value:** Can be **null** if not initialized.
- ✓ **Dynamic in Nature:** Unlike primitive types, non-primitive types have no fixed size and can grow dynamically.

Creating the data type of **String** and **Account**

```
public class NonPrimitiveDataTypes {

    public static void main(String[] args) {

        String name = "Kodewala"; // This is non primitive data type
    }

}

class Account
{
    public void doFundTransfer()
    {
        Account account = new Account(); // This is non primitive data type
    }
}
```

Kodewala

Difference Between Primitive and Non-Primitive Data Types

Feature	Primitive Types	Non-Primitive Types
Definition	Store actual values.	Store references to objects.
Size	Fixed (depends on type).	Variable (depends on object).
Default Value	Depends on the type (e.g., `0`).	`null`.
Operations	Limited (arithmetic, etc.).	Extensive methods and features.
Examples	`int`, `float`, `char`, `boolean`	`String`, `Array`, `Class`, etc.

Industry Experience

- ✓ Handle non-primitive types carefully to avoid “NullPointerException”. Always validate before accessing reference variables.

Interview Experience

- ✓ What is the default value of a non-primitive variable?

Class work

- ✓ Write a program to explore the non-primitive data types default value.

Assignment

- ✓ Write a program to check if the non-primitive data type has null value or not.

Kodewala

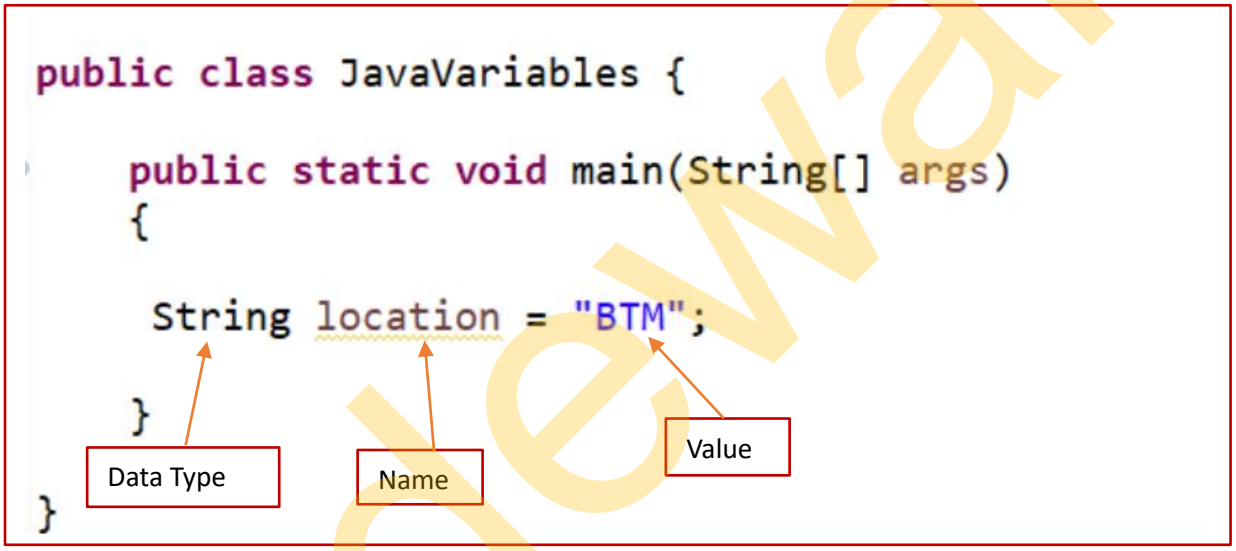
Java Variables

About it

Variables are like containers used to store data values. You can also think of them as names for specific memory locations where data is kept. Every variable has:

- ✓ **Data Type** – The type of data that it can hold. e.g - int, Boolean, string, float, char, etc.
- ✓ **Variable Name** – To identify the variable uniquely. (within the scope)
- ✓ **Value** – The data assigned to the variable

How to Declare Variables?



Another way, Declare the variable and assign the value later.

```
public class JavaVariables {

    int days = 100;              // Declaring and initializing variable

    int accountBalance ;         // declaring the variable

    public static void main(String[] args)
    {

     System.out.println("Welcome to the Kodewala!!");

    }

    public void doSomething()
    {
        accountBalance=100 ; // assigning the value
    }

}
```

kodewala

Kodewala Academy

Types of Java Variables

- ✓ Local Variables
- ✓ Instance Variables
- ✓ Static Variables

Local Variables:

- ✓ Local variables are defined with in methods or blocks (if, else etc) or constructors**.**
- ✓ The scope of local variables is with-in methods, blocks or constructors.
- ✓ Initialization of local variable is must. If you do not initialize it, you will get **compile** error.

```
public class LocalVariable {

    public static void main(String[] args)
    {

        // Declared a Local Variable
        int balance = 10;

        // This variable is local to this main method only
        System.out.println("Balance is : "+balance);

    }

}
```

Instance Variables:

- ✓ Instance variables are declared in a class but outside of method, block, constructor.
- ✓ Instance var are created when obj of class is created & destroyed when obj is destroyed.
- ✓ Initialization of an instance variable is not mandatory. Default value will be assigned as per data type. If it is type of int then default value would be 0.
- ✓ Instance variables should be accessed only by creating objects.
- ✓ We initialize instance variables using constructors while creating an object.
- ✓ We can initialize the instance variable inside initialization block as well.

```
public class InstanceVariable {
    int balance=10;
    public static void main(String[] args)
    {
        InstanceVariable instanceVariable = new InstanceVariable();

        System.out.println("Balance : "+instanceVariable.balance);
    }
}
```

Declaring and assigning the instance

Creating Object

Using the instance variable

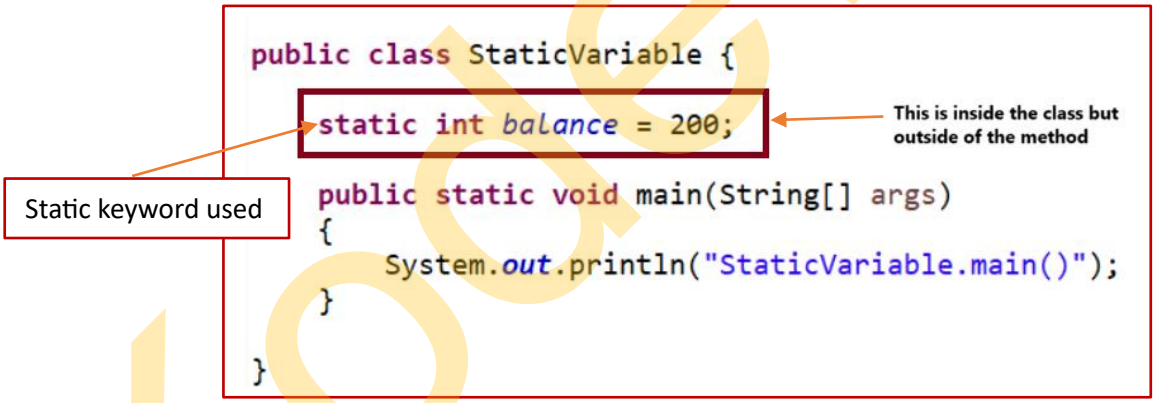
Kodewala

```java
public class InstanceVariable {

    int balance;

    public InstanceVariable() {
    this.balance=100;
    }
    public static void main(String[] args)
    {
        InstanceVariable instanceVariable = new InstanceVariable();


        System.out.println("Balance : "+instanceVariable.balance);
    }

}
```

Static Variables:

Static variables are also called class variables.

- ✓ Static variables are declared like instance variables but use the **static keyword.** These variables are written inside a class but outside any method, constructor.



- ✓ There is only one copy of a static variable for the whole class, no matter how many objects of the class you create.
- ✓ Static variables are created when the program starts and are destroyed when the program ends.
- ✓ You cannot declare static variables inside instance methods.
- ✓ You can use static blocks to initialize static variables

Kodewala

- ✓ If you don't mention the class name when accessing a static variable, the compiler will add it for you. However, to access a static variable from another class, you must use the class name because different classes may have static variables with the same name.

```java
public class StaticVariable {

    static int balance = 200;

    public static void main(String[] args)
    {
        // accessing the static variable directly, without class name.
        System.out.println(" Balance :"+balance);

        // access the static variable using class name.
        System.out.println(" Balance :"+StaticVariable.balance);
    }

}
```

Instance Variables vs Static Variables

Aspect	Instance Variables	Static Variables
Number of Copies	Each object has its own copy of the instance variable.	Only one copy exists per class, shared by all objects.
Memory Management	Not memory-efficient as each object has its own copy.	More memory-efficient since only one copy is shared.
Effect of Changes	Changes made in one object do not affect other objects.	Changes made to the static variable are reflected across all objects.
Access Method	Accessed using object references.	Accessed using the class name directly.
Lifecycle	Created when an object is created (using the `new` keyword) and destroyed when the object is destroyed.	Created when the program starts and destroyed when the program ends.

Industry Experience

- ✓ **Instance variables** are used in scenarios where each object of a class needs its own unique set of values. For example, in an e-commerce(amazon) app, each user's shopping cart might use instance variables to store item details. (quantify and total prices etc.)

- ✓ **Static** variables are commonly used for shared resources or constants, like maintaining a **global counter** for the number of active users in an application.

- ✓ Using instance variables where static variables are needed can lead to higher memory consumption.

Kodewala

Kodewala Academy

Interview Experience

- ✓ What is the difference between instance variables and static variables?
- ✓ How does the JVM manage memory for instance and static variables?
- ✓ Can a static method access instance variable directly? Why or why not?
- ✓ Design a class to represent a bank account where each account has a balance (instance variable), and all accounts share a single interest rate (static variable).
- ✓ How would you use static variables to count the total number of objects created for a class?
- ✓ What happens if an instance variable and static variable have the same name in a class?

Class work

- ✓ Write a program to use the instance and static variable and display the details. Instance variable can be employee detail like name, age, and department. Use company code as a static variable. Display the details in the console.

Assignment

- ✓ Write a program to count the total number of objects created for a class using a static variable.
- ✓ Create a class Student with the following details:

  Instance variables: name, rollNumber, marks.

  Static variable: schoolName.

  Write a method to display student details and assign a common school name for all students.

Kodewala

Scope of Variables

About it

In java there are mainly two category of variables, Member Variable (Instance and Static) and Local Variables.

Member Variable (Class Variable):

- ✓ These variables should be declared within the class but outside of any method. They can be accessed directly from anywhere within the class. Let's explore this with an example.

```java
public class OrderItem {

    // Member variable (declared inside the class, but outside any method)
    String itemName="iPhone";
    int price=1230000;

    public static void main(String[] args)
    {

        // Creating an object of the OrderItem class
        OrderItem orderItem = new OrderItem();

        // Accessing the member variables using the object
        orderItem.displayDetails();
    }

    // Method to display orderItem details
    public void displayDetails()
    {
        System.out.println("Item name: " + itemName);
        System.out.println("Price: " + price);
    }

}
```

- ✓ We can declare member/class variables anywhere in class, but outside methods.
- ✓ Access specified of member variables doesn't affect scope of them within a class.
- ✓ Member variables can be accessed outside a class with following rules.

Access Modifier	Same Package	Subclass (Different Package)	Anywhere (Java World)
Public	Yes	Yes	Yes
Protected	Yes	Yes	No
Default	Yes	No	No
Private	No	No	No

Kodewala

Local Variables (Method Level Scope)

- ✓ Variables declared inside a method have method level scope and can't be accessed outside the method.

```java
public class PlaceOrder {

    public void orderItem()
    {
        // Local variable
        String itemName = "Laptop"; // Declared and initialized inside the method
        int quantity = 2;           // Another local variable

        // Local variables must be initialized before use
        System.out.println("Order placed for " + quantity + " " + itemName + "(s).");

    }

    public static void main(String[] args)
    {
        // Creating an object of the PlaceOrder class
        PlaceOrder placeOrder = new PlaceOrder();
        placeOrder.orderItem();
    }
}
```

Key Points Regarding Variable Scope:

- ✓ Local variables don't exist after method's execution is over.
- ✓ In general, a set of curly brackets { } defines a scope.
- ✓ Any variable defined in a class outside of any method can be used by all member methods.

Industry Experience

Member Variables:

- ✓ Frequently used in real-world projects to define state or properties of objects, such as configuration settings, user details, or application state.
- ✓ Example: In a banking application, a BankAccount class may have member variables like accountNumber and balance that are accessed across multiple methods for transactions.

Local Variables:

- ✓ Commonly used in industry to perform temporary calculations, store method-specific data, or process intermediate results.
- ✓ Example: In a payment gateway, local variables are used within methods to hold temporary values like the transaction ID during a specific operation.

Interview Experience

- ✓ What are member variables, and how do they differ from local variables?
- ✓ Explain the scope and lifecycle of local variables.
- ✓ Why can't local variables have access modifiers in Java?

Kodewala

Kodewala Academy

Class work

- ✓ Create a class Employee with member variables like name, id, and salary. Write methods to update and display the values of these variables.

- ✓ Practice writing methods that calculate and return values using only local variables, such as a method to compute the sum of two numbers.

Assignment

- ✓ Create a Car class, write a method that calculates the final price after applying a discount. Use local variables to:
  - ➢ Store the discount rate (e.g., 10%).
  - ➢ Calculate the discounted price and return it.

Kodewala

Java Operators (Commonly Used in Industry)

About

Operators are special symbols that perform operations on variables or values.

Assignment Operator:

Assignment (=) operator is used to assign a value to any variable. Value given on the right-hand side of the operator is assigned to the variable on the left.

The format of the assignment operator is:

```
variable = value;
```

```
public class AssignmentOperator
{
    String name = "Kodewala";
    public static void main(String[] args)
    {
        int age = 15;
        System.out.println(age);
    }
}
```

Using = Operator to assign the value

Using = Operator to assign the value

Relational Operators

Relational operators are used to compare values, such as checking if they are **equal**, **greater** than, or **less** than. They return a **true** or **false** result after the comparison and are commonly used in loops and if-else conditions.

The format of the relational operator is:

```
variable relation_operator value
```

Relational operators compare values and return “boolean” results.

Kodewala

There are 6 type of the relational operators which we use generally in the industry.

- ✓ **==** , Equal to.
- ✓ **!=** , Not equal to.
- ✓ **<** , Less than.
- ✓ **<=** , Less than or equal to.
- ✓ **>** , Greater than.
- ✓ **>=** , Greater than or equal to.

Let's understand these with the below example.

```
public class RelationalOperators
{

    public static void main(String[] args)
    {

        int audiPrice = 8200000;
        int hondaPrice = 1500000;
        int marutiPrice = 500000;

        System.out.println("audiPrice > hondaPrice: " + (audiPrice > hondaPrice));
        System.out.println("audiPrice < hondaPrice: " + (audiPrice < hondaPrice));
        System.out.println("audiPrice >= b: " + (audiPrice >= hondaPrice));
        System.out.println("audiPrice <= b: " + (audiPrice <= hondaPrice));
        System.out.println("audiPrice == c: " + (audiPrice == marutiPrice));
        System.out.println("audiPrice != c: " + (audiPrice != marutiPrice));
    }

}
```

Logical Operators

Logical operators are used to perform "**AND**" and "**OR**" operations, similar to how they work in digital circuits. They also have a shortcut feature where the second condition is skipped if the result can already be determined from the first.

Logical operators are:

- ✓ **&&, Logical AND**: returns true when both conditions are true.
- ✓ **||, Logical OR**: returns true if at least one condition is true.
- ✓ **!, Logical NOT**: returns true when a condition is false and vice-versa

Kodewala

Let's understand the logical operators with an example:

```
public class LogicalOperators {

    public static void main(String[] args)
    {

        boolean a = true;
        boolean b = false;

        System.out.println("a && b: " + (a && b));
        System.out.println("a || b: " + (a || b));
        System.out.println("!a: " + (!b));

    }

}
```

Industry Experience

- ✓ Logical and relational operators are extensively used to validate user input or process data. For example, ensuring a user's age is within a certain range: if (age > 18 && age < 60).
- ✓ Access Control: Checking multiple conditions for user authentication, such as username and password validation or role-based access (if (isUserAdmin || isUserManager)).

Interview Experience

- ✓ What happens if the first condition in a logical AND (&&) operation is false?
- ✓ Write a program to determine if a number is divisible by both 3 and 5 using logical and relational operators.
- ✓ Given a range of temperatures (pass from command line), identify days where the temperature was both above 30°C and humidity below 50%.

Class work

- ✓ Basic Example: Write a program to check if a number is positive and even.

Assignment

- ✓ Write a program to check if a number is within a specific range (e.g., 10 to 50).
- ✓ Write a program to validate a user's age for driving eligibility (e.g., age > 18 && age < 70).

Kodewala

User Input – Interactive way

About

In our previous session, we captured user input using **command-line arguments** with String[] args. While this method works, it isn't interactive because all inputs need to be provided upfront when the program is executed.

To make the program more interactive, we can use the **Scanner** class. The Scanner class allows us to read input dynamically during program execution, making it more engaging for users.

Advantages of Using the Scanner Class

The **Scanner** class can:

- ✓ Read input from various sources like the console, files etc.
- ✓ Support different data types such as int, float, String, etc.
- ✓ Provide methods for handling multiple types of input effectively (e.g., nextInt(), nextLine()).

Follow below steps to take user input using Scanner class:

- ✓ Import the Scanner class using ***import java.util.Scanner;***
- ✓ Create the "**Scanner"** object and connect Scanner with "**System.in"** by passing it as an argument i.e. ***Scanner scanner = new Scanner(System.in);***
- ✓ Display a message prompting the user for input, and use appropriate methods from the Scanner class, such as **nextInt(), nextLine(), next(),** or **nextDouble(),** depending on the type of input required.

Let's understand with an example:

```java
import java.util.Scanner;

public class ScannerMethodsExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // Create a Scanner object for input

        // Using nextInt() to read an integer
        System.out.print("Enter your age: "); // Prompt user to enter their age
        int age = scanner.nextInt(); // Read an integer input

        // Clear the buffer
        scanner.nextLine(); // Consume the leftover newline character after nextInt()

        // Using nextLine() to read a full line of text
        System.out.print("Enter your full name: "); // Prompt user to enter their full name
        String fullName = scanner.nextLine(); // Read the entire line of text as input

        // Using next() to read a single word
        System.out.print("Enter your favorite color: "); // Prompt user to enter their favorite color
        String color = scanner.next(); // Read a single word as input

        // Using nextDouble() to read a decimal number
        System.out.print("Enter your weight in kilograms: "); // Prompt user to enter their weight
        double weight = scanner.nextDouble(); // Read a double value as input

        System.out.println("\n--- Collected Input ---");
        System.out.println("Age: " + age); // Display the age
        System.out.println("Full Name: " + fullName); // Display the full name
        System.out.println("Favorite Color: " + color); // Display the favorite color
        System.out.println("Weight: " + weight + " kg"); // Display the weight in kilograms

        scanner.close(); // Close the Scanner object to release resources
    }
}
```

Kodewala

How can I validate user input? For example, what if the user enters a junk string instead of a valid integer (like for age)?

```
// Using nextInt() to read an integer
System.out.print("Enter your age: "); // Prompt user to enter their age
while (!scanner.hasNextInt()) { // Check if the next input is not an integer
    System.out.println("Invalid input! Please enter a valid integer."); // Error message for invalid input
    scanner.next(); // Clear the invalid input from the scanner
}
int age = scanner.nextInt(); // Read the valid integer input
```

To handle cases where a user enters invalid input, such as a junk string instead of a valid integer (like for age), you can use the hasNextInt() method of the Scanner class. This method checks whether the next input token is a valid integer. If it's not, you can prompt the user to enter the input again.

Industry Experience

- ✓ In a real-world application, validating user input is crucial to ensure data integrity and prevent errors. For example, in an e-commerce platform, when users enter their age or date of birth during registration, the system must validate that the input is a valid number and not some random text. Incorrect or invalid data can lead to issues like incorrect user records or system crashes.

Interview Experience

- ✓ During an interview, you may be asked to demonstrate how you handle user input validation. For example, if the interviewer asks you to write a program that reads the user's age and ensures it is an integer, they will assess how you handle input errors.

Class work

- ✓ Write a program that reads multiple types of inputs, such as age (integer), name (string), and salary (double) and display it on console.

Assignment

- ✓ Write a program that reads multiple types of inputs, such as age (integer), name (string), and salary (double), and then validates them.

Kodewala

Kodewala Academy

Transforming code into career

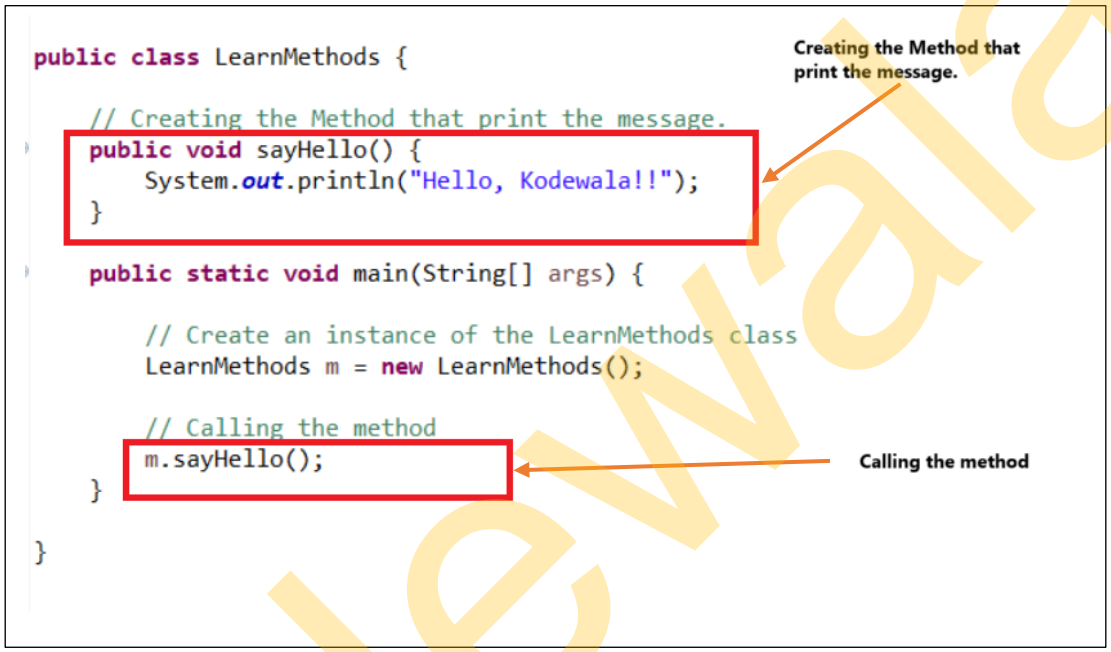
Core Java - Week 2 & 3

Kodewala

Java Methods

About

Java Methods are blocks of code that perform a specific task. A method allows user to reuse the code. All methods in Java must belong to a **class**. Methods expose the behavior of objects.




```
public class LearnMethods {

    // Creating the Method that print the message.
    public void sayHello() {
        System.out.println("Hello, Kodewala!!");
    }

    public static void main(String[] args) {

        // Create an instance of the LearnMethods class
        LearnMethods m = new LearnMethods();

        // Calling the method
        m.sayHello();
    }

}
```

Let's understand the method created in above code.

sayHello() is a simple method that prints a message "**Hello, Kodewala!!**". The method "**sayHello()**" has no parameters and does not return anything.

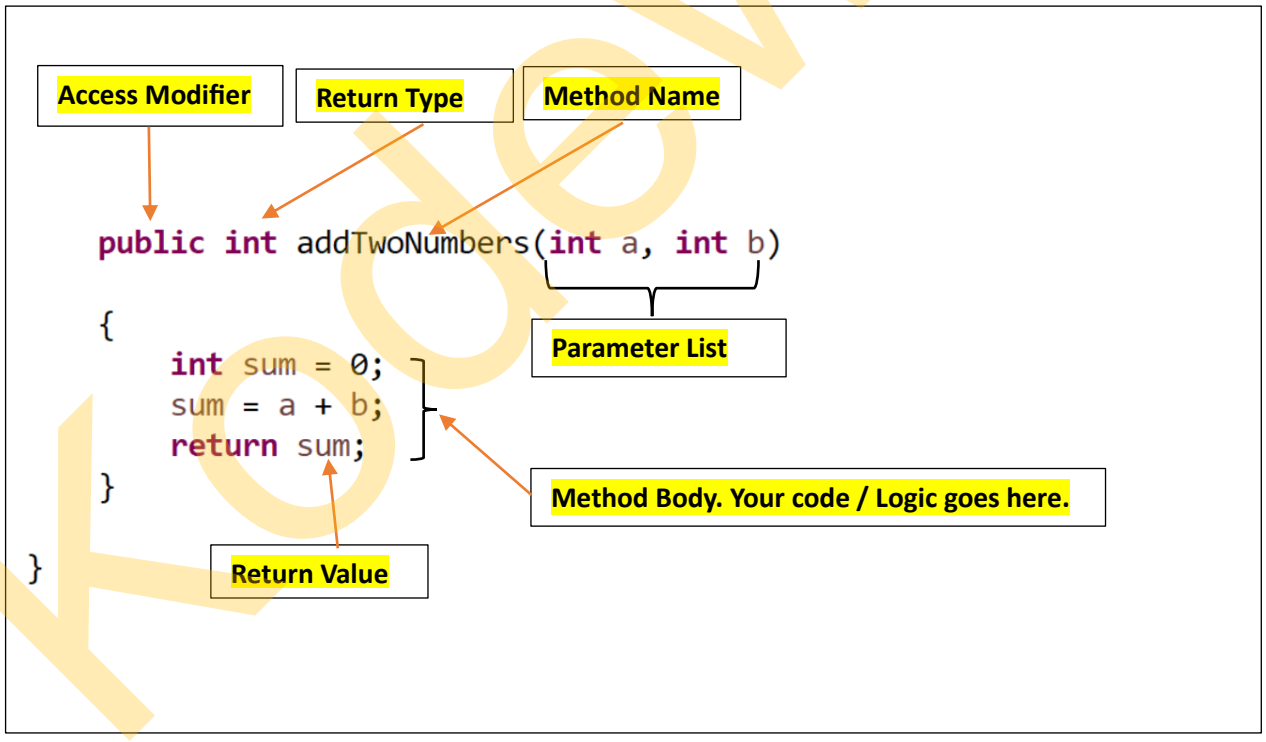
Syntax of a Java Method

```
<access_modifier> <return_type> <method_name>( list_of_parameters)
{
//  Method body: This is where your code or logic resides. In the industry, this is often referred to as business logic.
}
```

Kodewala

Elements of a Method Declaration:

1. **Access Modifier**: Defines the visibility of the method (e.g., public, private, protected, or default). This means who can call the method.
2. **Return Type**: Specifies the type of value the method will return (e.g., int, void, String). Use void if the method does not return anything.
3. **Method Name**: A unique identifier for the method, following Java naming conventions (e.g., camelCase). e.g – **sayHello()**
4. **Parameter List(Optional)**: Enclosed in parentheses, it specifies the input parameters the method accepts, including their data types. If no parameters are needed, the parentheses remain empty.
5. **Exception List (Optional)**: Declares exceptions the method might throw, using the throws keyword.
6. **Method Body**: The block of code enclosed within curly braces {} that contains the logic or operations to be performed.



Kodewala

Types of Methods in Java

1. Built-In Methods

✓ Built-In methods are the method that is already defined in the Java class libraries. It is also known as the standard library method or predefined method.

Example: random() and parseInt() are the built in methods.

```
Math.random();
Integer.parseInt("123");
```

2. User Defined Methods

✓ The method written by java developer is known as **"user defined method".** Developer can modify these methods as per the requirement.

Example: **sayHello()** and **addTwoNumbers()** are the built in methods which we written / developed in our previous example.

```
m.sayHello();
m.addTwoNumbers(8, 3);
```

Various Ways to Define Java Methods

In Java, methods can be created in two ways:

1. **Instance Method:** Defined within a class, these methods access instance data through an object reference.

Example:

```
// Instance Method
void methodName() {
   // Instance method body
}
```

Kodewala

2. **Static Method:** Defined within a class using the **"static"** keyword, these methods access static data through the class name.

   Example:

```
// Static Method
static void methodName() {
   // Static method body
}
```

Method Signature

The method signature includes the **method name** and the **parameter list**, which specifies:

- ✓ **Number of Parameters:** The total count of parameters.
- ✓ **Type of Parameters:** The data types of the parameters.
- ✓ **Order of Parameters:** The sequence in which the parameters are defined.

Note: The return type and exceptions are not part of the method signature.

```
int addTwoNumbers(int a, int b)
```

Method Signature of the above function:

Method Name	Number of Parameters	Type of Parameters
addTwoNumbers	**2**	**int**

Naming a Method

In Java, method names are typically either a single lowercase verb or a multi-word phrase that begins with a verb in lowercase, followed by an adjective or noun. In multi-word method names, the first letter of each subsequent word should be capitalized.(camelCase)

Rules for Naming a Method:

- ✓ The method name must be a verb and start with a lowercase letter.

Kodewala

- ✓ If the method name consists of multiple words, the first word should be a verb, followed by an adjective or noun.
- ✓ For multi-word method names, capitalize the first letter of each word except the first one. Examples include **addTwoNumbers**, **sayHello**, **doSomething**, and **printMessage**.
- ✓ Typically, a method name is unique within its class, but Java allows **method overloading**, so multiple methods in the same class can have the same name, as long as their parameter lists differ. Method Overloading will be covered in next sessions.

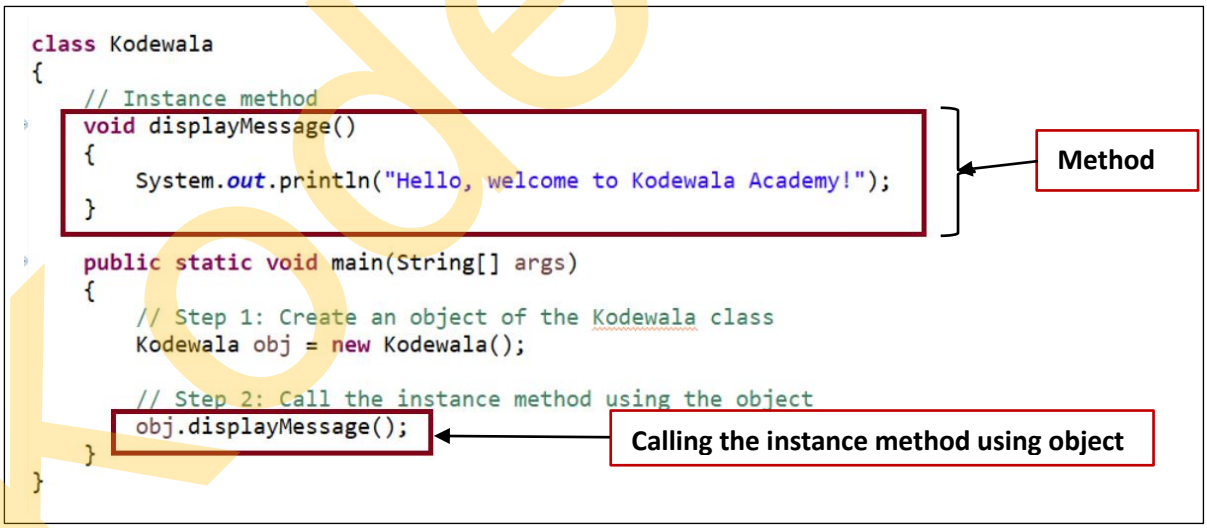
Method Calling

Method calling enables code reuse. To use a method's functionality, it must be explicitly called. There are three scenarios where a method returns control to the calling code:

1. When it completes all the statements in the method body.
2. When it encounters a return statement.
3. When it throws an exception.

Example: Method Calling Using an Object

In Java, instance methods are called using objects of the class in which the method is defined. The object acts as a reference to access the method and its functionality.



Explanation:

1. **Method Declaration**: The displayMessage() method is defined as an instance method inside the Kodewala class.
2. **Object Creation**: An object **obj** of the **Kodewala** class is instantiated using the new keyword.
3. **Method Call**: The displayMessage() method is invoked using the object reference (**obj.displayMessage()**).

Kodewala

Advantages of Methods

- ✓ **Reusability**: Methods enable you to write code once and reuse it multiple times, reducing redundancy and enhancing efficiency.
- ✓ **Abstraction**: Methods help abstract away complex logic, providing a simplified interface for other parts of the program or for external users to interact with.
- ✓ **Encapsulation**: Methods allow the encapsulation of complex logic and data, ensuring that the implementation details are hidden while exposing only the necessary functionality.
- ✓ **Modularity**: By dividing code into smaller, more manageable units, methods enhance modularity, making the code easier to understand, maintain, and debug.
- ✓ **Customization**: Methods can be easily tailored to perform specific tasks, making the program more flexible and adaptable.
- ✓ **Improved Performance**: Organizing code into well-structured methods improves readability and reduces repetition, which can lead to more efficient execution and easier optimization.

Industry Experience

- ✓ **Reusable Business Logic**: In real-world applications, methods simplify repetitive tasks. For instance, in a food delivery app, a method like calculateDeliveryFee(double distance) can be used to compute delivery charges for multiple orders based on the distance.

Interview Experience

- ✓ Why do we need methods in Java?
- ✓ What is the significance of a method signature?
- ✓ Explain the difference between static and instance methods with examples.
- ✓ Write a method to calculate the total price of items in a shopping cart.
- ✓ Create a method **sendEmail(**String recipient, String message) to simulate sending emails.

Class work

- ✓ Implement utility methods like **validateInput**(String input) to check user input for errors.

Assignment

- ✓ **Simple Calculator**:
  - Write methods to perform basic operations like addition, subtraction, multiplication, and division.
  - Use an object to call these methods from the main program.
- ✓ **Bank Application**:
  - Implement methods like *deposit*(double amount) and *checkBalance*() for basic banking operations.

Kodewala

- ✓ **Order Tracking System**:
  - Write a method trackOrder(int orderId) that simulates fetching the current status of an order.

Kodewala

Kodewala Academy

Java Access Modifier

About

In Java, access modifiers are used to control the visibility and accessibility of classes, constructors, variables, methods, and attributes. They help protect data, control who can access certain parts of a program, and provide security.

In this session, we will explore the different types of access modifiers in Java and how they are used.

Types of Access Modifiers

1. Default – No keyword required → **String sayHello()**
2. Private → **private String sayHello()**
3. Protected → **protected String sayHello()**
4. Public → **public String sayHello()**

Let's understand with code example:

1. **Default:** When no access modifier is specified for a class, method, or data member, it is given the default access modifier. The default access modifier allows access only within the same package.

   Example: Demonstrating Default Access Modifier Within the Same Package

   Let's use a real-world example involving a **'Customer'** class and an **'Order'** class in the same package to demonstrate the default access modifier.

   File 1: Customer.java

```
package com.kodewala.example;          Package

//Default access modifier example
class Customer      No access modifier. This means class has default access modifier.
{

    // Default access modifier
    String customerName = "Ajay";      This variable's access modifier is 'default'

    // Default access modifier method
    void sayHello(String name) {
        System.out.println("Welcome to the " + name);
    }
}
```

Kodewala

File 2: Order.java

```java
package com.kodewala.example;

//Same package
public class Order
{
    public static void main(String[] args)
    {
        // Creating an object of Customer class
        Customer customer = new Customer();

        // Accessing default access members within the same package
        customer.sayHello("Kodewala");
        System.out.println("Customer Name: " + customer.customerName);
    }
}
```

Explanation:

- ✓ The Customer and Order classes are in the same package.
- ✓ The **'customerName'** variable and **'sayHello'** method in the Customer class have the default access modifier, meaning they can be accessed from the **'Order'** class since both classes are in the same package.

  Note: Program will result in a compile-time error when attempting to access a class with a default access modifier from a different package.

Kodewala

2. Private Access Modifier

The private access modifier is used with the **'private'** keyword. Methods or variable declared as private are only accessible within the class where they are defined.

- ✓ Any other class of the same package will not be able to access these members.

Example: Private Access Modifier

In this example, I demonstrate how the private access modifier restricts access to a method and a data member, allowing them to be accessed only within the class where they are declared.

File: BankAccount.java

```java
package com.kodewala.example;

public class BankAccount
{
    // Private data member
    private double balance;

    // Private method
    private void updateBalance(double amount) {
        this.balance += amount;
    }

    // Public method to access the private method
    public void deposit(double amount) {
        if (amount > 0) {
            updateBalance(amount); // Accessing private method within the class
            System.out.println("Deposit successful. Current balance: " + balance);
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

These are the private members and will not be able to access outside this class.

File: Main.java

```java
package com.kodewala.example;

public class Main
{
    public static void main(String[] args) {
        // Creating an object of BankAccount class
        BankAccount account = new BankAccount();

        // Accessing the public method to deposit money
        account.deposit(1000);

        // Accessing the public method to get the balance
        System.out.println("Balance: " + account.getBalance());

        // The following lines will cause a compile-time error
        account.balance = 5000; // Cannot access private data member
        account.updateBalance(5000); // Cannot access private method
    }
}
```

As updateBalance method was declared as private, hence compile time error if same is accessed outside the class where it is declared.

Kodewala

Explanation:

- ✓ The balance variable and the updateBalance method are marked as private, meaning they can only be accessed within the BankAccount class.
- ✓ The deposit method is public, and it provides access to the private updateBalance method, allowing for controlled access to the private members.
- ✓ In the Main class, we cannot access the balance directly or call the updateBalance method because they are private. Attempting to do so will result in a compile-time error.

3. **Protected Access Modifier:** The protected access modifier is specified using the keyword protected. Methods or variables declared as protected are accessible within the same package and also in subclasses located in different packages.

   Example: Protected Access Modifier : Let's use the package 'com.kodewala.example' for the base class and create another package 'com.kodewala.client' for the subclass and main class to demonstrate the protected access modifier.

   File: Employee.java (in com.kodewala.example package)

```java
package com.kodewala.example;

public class Employee
{
    // Protected variable
    protected String employeeName;

    // Constructor to initialize employeeName
    public Employee(String name)
    {
        this.employeeName = name;
    }

    // Protected method
    protected void sayHello() {
        System.out.println("Hello, " + employeeName + "!");
    }
}
```

This is a protected method and will have visibility with-in same package and subclass

Now let's access the method of super class in the subclass which is declared in the different package.

```java
package com.kodewala.client;

import com.kodewala.example.Employee;

public class Manager extends Employee
{
    // Constructor to pass name to Employee
    public Manager(String name)
    {
        super(name);
    }

    public void displayGreeting() {
        // Accessing protected method in the subclass
        sayHello();
    }
}
```

Accessing the protected method 'sayHello' in the subclass. This method is declared in the super class i.e Employee

Kodewala

4. Public Access Modifier: The public access modifier is specified using the keyword public.

The public access modifier has the broadest scope of all access modifiers. Classes, methods, or variables declared as public can be accessed from anywhere in the program, without any restrictions on their accessibility.

Industry Experience

1. **Private:** In a banking application, the account balance is kept private to prevent unauthorized modifications. Only specific methods like deposit or withdraw can update the balance after proper validation.

2. **Protected:** In a payroll system, the bonus calculation logic is marked protected, allowing subclasses for different employee types (e.g., Manager, Developer) to implement custom bonus rules.

3. **Default:** In an e-commerce platform, helper methods for inventory updates are default, ensuring they are accessible only to classes in the same package, like the **InventoryManager** and **OrderProcessor**.

4. **Public:** In a hotel application, APIs like **searchHotel()** or **bookHotel()** are public to allow access to all users.

Interview Experience

- ✓ Difference between protected and default.
- ✓ Can a private method be overridden?
- ✓ How do you allow access within the same package and to subclasses in other packages?

Class work

- ✓ Create a Person class with all access modifiers and test them across classes in the same and different packages.
- ✓ Highlight how protected and default behave differently in inheritance and package access.

Assignment

- ✓ Design a **BankAccount** class where the balance is **private**, a **protected** method calculates the interest, and a **public** method displays the account details. Explain why each access modifier is used.

Kodewala

Kodewala Academy

How to Call a Method?

About

Calling a method / function helps us reuse code and keep our programs organized. In Java, methods are a group of statements that perform a specific task(i.e **addTwoNumbers()** , **println()** and **calculateInterest()** etc.) and can return a result. In this session, we will learn how to call different types of methods in Java with simple real-time examples.

1. Instance Method (method that belongs to an object)

Let's define a method. Since we are creating it ourselves, it is called a "**user-defined method**." After defining the method, we will call it (using object).

```java
package com.kodewala.methods;

public class Calculator
{
    // Define a method to add two numbers
    public int addNumbers(int num1, int num2)
    {
        int sum = num1 + num2; // Returns the sum of num1 and num2
        return sum;
    }

    public static void main(String[] args)
    {
        // Create an object of the Calculator class
        Calculator calculator = new Calculator();

        // Call the non-static method using the object
        int result = calculator.addNumbers(15, 25);

        // Print the result
        System.out.println("The sum is: " + result);
    }
}
```

The method adds two numbers and returns their sum. It takes two arguments, num1 and num2, and calculates the sum to return as the result.

Since the method is an instance type (non-static), we call it using an object and pass the required parameters.

2. Static Method (method that belongs to a class)

Now, let's define a static method. Since it belongs to the class itself, it is called a "static method." After defining the method, we will call it using the class name.

```java
package com.kodewala.methods;

public class MathOperations
{
    // Define a static method to multiply two numbers
    public static int multiplyNumbers(int num1, int num2)
    {
        return num1 * num2; // Returns the product of num1 and num2
    }

    public static void main(String[] args)
    {
        // Call the static method using the class name
        int result = MathOperations.multiplyNumbers(10, 5);

        // Print the result
        System.out.println("The product is: " + result);
    }
}
```

Defined the static method

Calling the static method using class name

Kodewala

Explanation:

- ✓ The class MathOperations contains a static method **multiplyNumbers**(), which multiplies two numbers.
- ✓ The static method is called in the main method using the class name **MathOperations.multiplyNumbers(10, 5)** without needing to create an object.
- ✓ The result is printed after calling the method.

Industry Experience: Calling Methods in Real-World Scenarios

- ✓ In the software development industry, methods are used extensively to break down complex tasks into manageable pieces.
- ✓ Calling methods is crucial for maintaining clean, reusable, and modular code. Methods such as mathematical calculations, data processing, and utility functions are frequently used in real-world applications.

Interview Experience

- ✓ What is the difference between calling a static method and an instance method?
- ✓ Can you give an example where you would use a static method vs an instance method? [**Hint]:** Use **static methods** for tasks that don't depend on specific object data, and use **instance methods** when the task needs data from a specific object.

Assignment

- ✓ Write a program that calls a method to compute the area of a rectangle. The method should take **length** and **width** as parameters and return the area.

Kodewala

Kodewala Academy

Static vs Instance Method?

About

In Java, methods are mainly of two types based on their relationship with the class:

1. **Static method**: This method belongs to the class itself, and you can call it without creating an object of the class.
2. **Instance method**: This method belongs to a specific object, and you need to create an object of the class to call it.

   In simple terms, static methods are for general actions related to the class, while instance methods are for actions tied to specific objects.

Static Method Example:

In a banking system, a static method in the **Bank** class could be used to calculate interest for all accounts, as the calculation is common to all and does not depend on specific account data.

```
package com.kodewala.methods;

public class Bank
{

    public static double calculateInterest(double balance)
    {
        return balance * 0.05; // Interest rate is the same for all accounts
    }
}
```

A static method is created using the **static** keyword and can be called without creating an object of the class. It is accessed by using the **class name.**

Static methods are stored in the class's memory (Permanent Generation space), not in an object's memory. They can be called directly using **ClassName.methodName(args).** Local variables and arguments used in static methods are stored in the stack.

Let's explore this with the following example.

```
package com.kodewala.methods;

public class TaxCalculator
{
    // Static method to calculate tax
    public static double calculateTax(double price)
    {
        return price * 0.18; // 18% tax
    }

    public static void main(String[] args)
    {
        // Call static method without creating an object
        double tax = TaxCalculator.calculateTax(1000);
        System.out.println("Tax amount: " + tax);
    }
}
```

The static method calculateTax belongs to the class TaxCalculator.

It is called directly using the class name TaxCalculator.calculateTax() without needing an instance.

Kodewala

Explanation:

- ✓ The static method **calculateTax** belongs to the class **TaxCalculator**.
- ✓ It is called directly using the class name **TaxCalculator.calculateTax()** without needing an instance/object.

Point to Remember:

- ✓ Static methods are common to all instances of a class.
- ✓ Since they are resolved at compile time using static binding, they cannot be overridden.
- ✓ If both a superclass and a subclass define static methods with the same name, this is known as **method hiding**, where the subclass method hides the superclass method, but it doesn't override it.

Instance Method Example:

In the same system (Banking), an instance method(non-static) in the Account class could be used to update or get the balance of a specific account.

```
package com.kodewala.methods;

public class Bank
{
    double balance;

    public void setBalance(double balance)
    {
        this.balance = balance; // Each account has its own balance
    }

}
```

Instance methods require an object of the class to be created before they can be called. To use an instance method, you first need to create an object of that class.

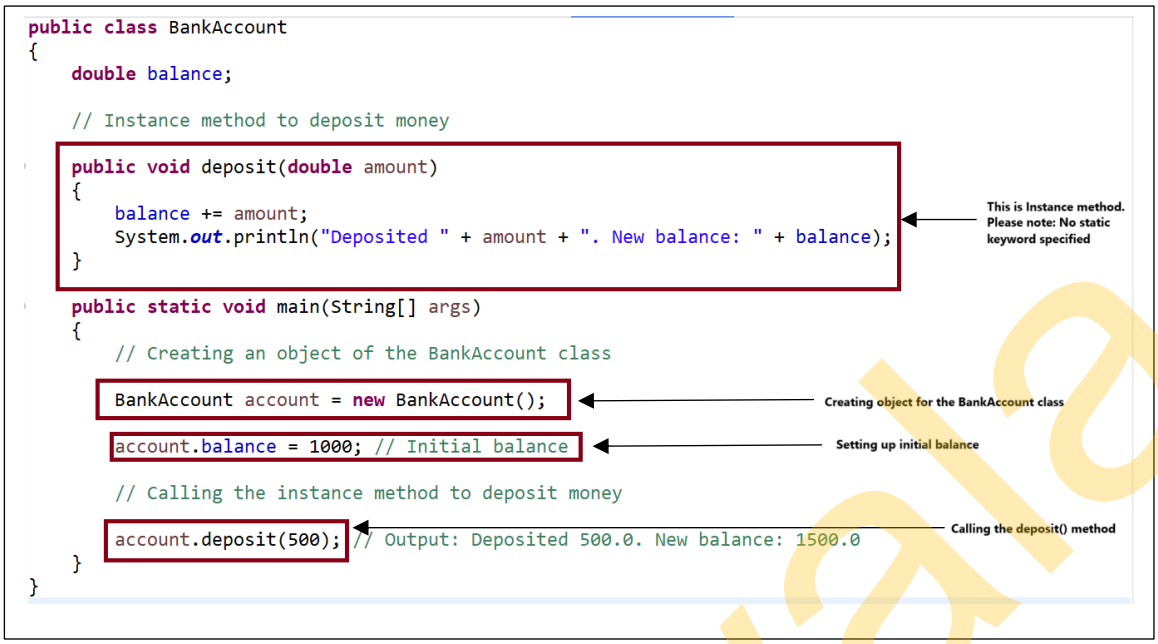
Instance methods are stored in the class's memory area (until Java 7, it was the Permanent Generation space, now replaced by Metaspace from Java 8). Their parameters, local variables, and return values are stored in the stack.

They can be accessed within the class or from other classes, depending on the method's access level.

Let’s explore this with the following example.

In a banking system, an instance method could be used to deposit money into a customer's account.

kodewala




```java
public class BankAccount
{
    double balance;

    // Instance method to deposit money
    public void deposit(double amount)
    {
        balance += amount;
        System.out.println("Deposited " + amount + ". New balance: " + balance);
    }

    public static void main(String[] args)
    {
        // Creating an object of the BankAccount class
        BankAccount account = new BankAccount();

        account.balance = 1000; // Initial balance

        // Calling the instance method to deposit money
        account.deposit(500); // Output: Deposited 500.0. New balance: 1500.0
    }
}
```

Explanation:

- ✓ The deposit method is an instance method because it operates on the balance of a specific BankAccount object.
- ✓ The method updates the balance for that particular account.

Point to Remember:

- ✓ Instance methods are tied to objects and require an object to be called.
- ✓ These methods use this pointer to refer to their specific object.
- ✓ They are stored in one memory location and can be overridden because they use dynamic binding at runtime, meaning the method to be called is decided during program execution.

Industry Experience:

- ✓ In industry, **static methods** are commonly used for utility functions, like calculating tax or logging messages, where the logic doesn't depend on specific objects. **Instance methods** are used for operations that modify or interact with specific data, like processing an order or updating a bank account balance.

Interview Experience

- ✓ What is the difference between an instance method and a static method?
- ✓ Interviewers may ask for examples of when to use static methods and whether they can be overridden or not.
- ✓ How would you manage global settings in a class using static methods?
- ✓ Can you explain how instance methods are used to manage specific object data?
- ✓ Can you override a static method in Java? Why or why not?
- ✓ Can an instance method be called without creating an object?

Kodewala

- ✓ How would you implement a utility function (like a calculation) in a system that doesn't require object-specific data?
- ✓ Share some of the examples where you have used static methods in your projects.

Kodewala

Kodewala Academy

Control Flow

If Statement

The Java **if statement** is the simplest decision-making statement. It is used to decide whether a specific block of code will execute based on a given condition. If the condition is true, the block of code runs, otherwise, it is skipped.

Syntax:

```
if(condition)
{
  // Statements to execute if condition is true, otherwise it is skipped
}
```

Working of the if statement:

1. The program checks the condition.
2. If the condition is true, the code inside the if block runs.
3. If the condition is false, the code inside the if block is skipped.
4. The program continues with the next statement after the if block.

Let's understand this with a code example

```
public class PositiveNumberCheck
{
    public static void main(String[] args)
    {
        int number = 10;

        // Call the method to check if the number is positive
        checkIfPositive(number);
    }

    // Function to check if a number is positive
    private static void checkIfPositive(int number)
    {
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
    }
}
```

If condition

kodewala

Explanation:

1. **Main Method**: The main method initializes the variable number and calls the method **checkIfPositive**.
2. **Method**: The **checkIfPositive** method contains the logic to check whether the number is positive or not.
3. **Industry Standard**: The function is defined with private static as it's utility-like and doesn't depend on instance variables.

If-else Statement

The **if-else statement** in Java is a simple way to make decisions in your program. It checks a condition, and if the condition is **true**, it runs one set of code. If the condition is false, it runs a different set of code. This helps control what the program does based on specific situations.

Syntax:

```
if(condition)
{
  // Statements to execute if condition is true.
}

else

{

// Code to execute if the condition is false

}
```

Let's understand this with a code example.

```
public class DecisionMakingExample
{
    public static void main(String[] args)
    {
        int number = -5; // You can change this value to test different scenarios.

        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        else
        {
            System.out.println("The number is not positive.");
        }
    }
}
```

Else condition, This will be executed in case if condition is false

kodewala

Explanation:

1. **Class Name**: DecisionMakingExample (follows class naming convention, as used in the industry).
2. **Main Method**: The entry point of the program.
3. **Logic**: The if condition checks whether the number is greater than 0. Based on the condition, it decides which block of code to execute.

if-else-if Statement

The **if-else-if** in Java is used when you need to test multiple conditions sequentially. It allows the program to check multiple conditions one by one and execute the corresponding block of code for the first true condition.

Syntax:

```
if (condition1) {
   // Code to execute if condition1 is true
} else if (condition2) {
   // Code to execute if condition2 is true
} else if (condition3) {
   // Code to execute if condition3 is true
} else {
   // Code to execute if none of the conditions are true
}
```

Let's understand this with a code example.

```
public class AmazonDiscount
{
    public static void main(String[] args)
    {
        double orderAmount = 8500.0; // Example: The total amount of the order

        if (orderAmount >= 10000)
        {
            System.out.println("Discount Applied: 20%");
        }
        else if (orderAmount >= 5000)
        {
            System.out.println("Discount Applied: 15%");
        }
        else if (orderAmount >= 2000)
        {
            System.out.println("Discount Applied: 10%");
        }
        else
        {
            System.out.println("No discount available.");
        }
    }
}
```

Kodewala

Explanation

- ✓ The program uses the **if-else-if** ladder to check conditions in sequence.
- ✓ It executes the first **true** condition and **ignores** the rest.
- ✓ If none of the conditions are true, the final else block runs, displaying "No discount available."

Industry Experience:

- ✓ In online platforms like Amazon, if-else-if is used to determine discounts. For example, orders above ₹10,000 get a 20% discount, while orders below ₹1,000 get no discount.

Interview Experience:

- ✓ In real-world software development, conditional statements are used everywhere, from determining access control (permissions i.e isAdmin , isManager etc.) to handling user input, performing calculations, and making business decisions based on input data.

Class Work:

- ✓ Write a program to check whether a number is even or odd using if-else.
  Hint: Use number % 2 == 0 for even numbers.

Home Work:

- ✓ Write a program to calculate the shipping cost based on the order amount:
  - Orders above ₹5,000: Free shipping.
  - Orders between ₹2,000 and ₹5,000: ₹100 shipping fee.
  - Orders below ₹2,000: ₹200 shipping fee.

Kodewala

Kodewala Academy

For Loop in Java

Foor Loop

A **for loop** in Java is a way to repeat a block of code multiple times. It runs the code as long as a specific condition is true. For loops are useful for going through a range of values, repeating tasks, or working with arrays and collections efficiently.

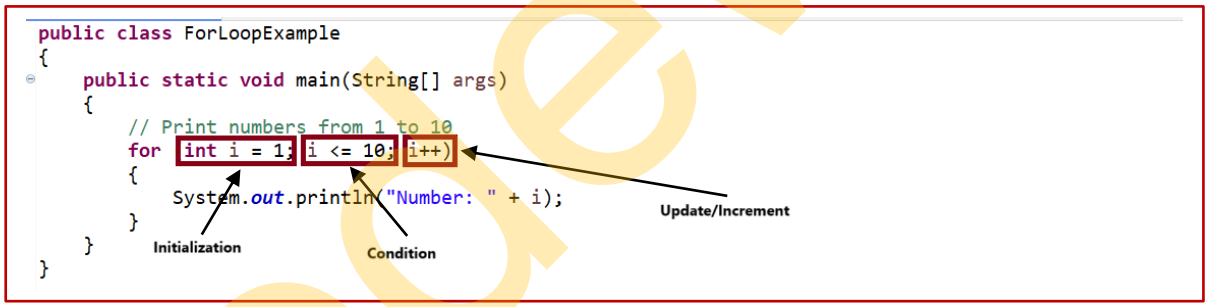
Syntax:

```
for (initialization; condition; update) {
   // Code to execute repeatedly
}
```

Explanation:

1. *Initialization*: This is executed only once, at the start of the loop. It's used to initialize the loop variable.
2. *Condition*: This is checked before each iteration. If the condition is true, the loop continues; otherwise, it stops.
3. *Update*: This is executed after each iteration. It updates the loop variable.

Let's print the 10 numbers using for loop.



Let's understand each step:

1. The program begins execution.
2. The for loop initializes **i** to 1.
3. The condition **i** <= 10 is checked:

- If true:
  - The statement inside the loop runs, printing the current value of **i**.
  - The value of **i** is incremented by 1.
- If false:
  - The loop ends, and the program continues.

4. These steps repeat until **i** reaches 11, at which point the condition **i** <= 10 fails.
5. Program completed.

Kodewala

Kodewala Academy

Let's write a program to print **"Welcome to Kodewala Academy"** 20 times using a for loop.

```
public class WelcomeMessage
{
    public static void main(String[] args)
    {
        // Loop to print the message 20 times
        for (int i = 1; i <= 20; i++)
        {
            System.out.println("Welcome to Kodewala Academy");
        }
    }
}
```

Nested Loops (Loop Inside Loop): A **nested loop** is a loop that occurs inside another loop. In other words, it is a loop within a loop. The outer loop runs once, and for each iteration of the outer loop, the inner loop will run completely.

Let's understand the nested loop with star pattern problem. Refer below.

```
*
**
***
****
*****
```

```
public class StarPattern
{
    public static void main(String[] args)
    {
        // Outer loop for rows
        for (int i = 1; i <= 5; i++)
        {
            // Inner loop for columns
            for (int j = 1; j <= i; j++)
            {
                System.out.print("*"); // Print star without a new line
            }
            System.out.println(); // Move to the next line after each row
        }
    }
}
```

Responsible for printing rows.

Responsible for printing columns

Explanation:

- ✓ The outer loop controls the number of rows.
- ✓ The inner loop prints the stars in each row. For each iteration of the outer loop, the number of stars printed increases by one.
- ✓ After printing the stars in each row, System.out.println() is used to move to the next line.

Kodewala

Industry Experience:

- ✓ Loops help automate repetitive tasks and process data efficiently. For example, a developer in banking might use loops to calculate the total transaction amount for many customers or process account details.

Interview Experience:

- ✓ In Java interviews, questions involving loops are common. Interviewers often test a candidate's understanding of using loops for simple tasks like summing an array, printing patterns, or working with data structures.
- ✓ Loops are also commonly used in questions related to string manipulation, array operations, and algorithm implementation.
- ✓ Example - Write a program to find the sum of all even numbers in an array using a loop.

Class Work:

- ✓ Print numbers from 1 to 10 using a loop.

Home Work:

- ✓ Sum of Odd Numbers in an Array
- ✓ Count Even Numbers in an Array

Kodewala

Kodewala Academy

While Loop in Java

While Loop

A **while loop** is used to execute a block of code repeatedly as long as a given condition is true. The condition is checked before the code inside the loop is executed, which means the loop might not run at all if the condition is false at the start.

Syntax:

```
while (condition) {
  // Code to be executed
}
```

Explanation:

1. **Condition**: A boolean expression that is evaluated before each iteration. If it is true, the loop continues; if false, the loop stops.

.

Let's print the 5 numbers using while loop.

```
public class WhileLoopExample
{
    public static void main(String[] args)
    {
        int i = 1;
        while (i <= 5)
        { // Condition: i should be less than or equal to 5
            System.out.println(i);
            i++; // Increment i after each iteration
        }
    }
}
```

Condition Checked Before Execution

Let's understand each step:

1. **Initialization**: The variable **i** is set to 1.
2. **Condition Check**: The loop checks if **i <= 5**. If **true**, it executes the loop body.
3. **Loop Body**:
   - ✓ Prints the current value of **i**
   - ✓ Increments **i** by **1**
4. **Repeat**: This process repeats until **i** exceeds **5**.
5. **Exit**: When i > 5, the loop stops.

Kodewala

Kodewala Academy

Let's Understand How a While Loop Can Be Used to Create a Number Guessing Game. User need to guess any single digit number (0 to 9)

```java
import java.util.Scanner;

public class GuessingGame
{

    public static void main(String[] args)
    {
        int userGuess = 0;
        int expectedNumber = 5;

        Scanner sc = new Scanner(System.in);

        while (userGuess != expectedNumber)
        {
            userGuess = sc.nextInt();
            if (userGuess == expectedNumber)
            {
                System.out.println("Congratulations! You guessed the correct number.");
            } else
            {
                System.out.println("Bad luck! Try again.");
            }
        }

        sc.close();
    }

}
```

This keep on executing until condition is false. In first iteration the condition will be true and enters. From 2nd iteration onward condition will be checke based on the user input(userGuess) and expected numbers.

Explanation

Initialize Variables:

- userGuess = 0: Start with an initial guess of 0.
- expectedNumber = 5: Set the correct number to 5.

Setup Scanner:

- Scanner sc = new Scanner(System.in); → Create a Scanner to read user input.

While Loop:

- The loop runs while userGuess is not equal to expectedNumber.

Get User Input:

- userGuess = sc.nextInt();: Get the user's guess.

Check Guess:

- If the guess is correct (userGuess == expectedNumber), print success message.
- If incorrect, print a try-again message.

Exit:

- The loop stops when the user guesses correctly.

Kodewala

When to use for Loop and When to use While Loop?

for loop: Use when you know how many times the loop should run.

- Example: Printing a set of numbers from 1 to 5.

while loop: Use when the number of iterations is unknown, and you want the loop to continue until a certain condition is met.

- Example: Repeating actions until a user guesses the correct number.

Do While Loop

✓ In Java, the do-while loop is an exit-controlled loop. This means it checks the condition after running the code inside the loop, so the loop will always run at least once.

Syntax

```
do {
   // Code to be executed
} while (condition);
```

Explanation:

1. The code inside the do block runs **at least once**, even if the condition is false initially.
2. After executing the code, the condition is checked.
3. If the condition is true, the loop repeats; otherwise, it stops.

Why Use a do-while Loop?

✓ Guarantees that the code inside the loop will run at least once.

✓ Useful in scenarios where you want to perform an action first and check the condition afterward.

Kodewala

Kodewala Academy

Industry Experience:

- ✓ In real-world applications, a **while loop** is useful when you need to repeat a task until a condition changes. For example, in a **banking system**, you might use a while loop to continuously check a user's account balance until it drops below a certain threshold.

Interview Experience:

- ✓ What is the difference between a for loop and a while loop, and in which scenario should each be used?

Class Work:

- ✓ Print numbers from 1 to 5 using a while loop:

Home Work:

- ✓ Ask the user to input a number greater than 10 using a while loop

Kodewala

Switch Statements

Switch Statements

The **switch statement** is used to handle multiple conditions in a simpler and more readable way compared to an **if-else-if** ladder. It allows you to execute different blocks of code based on the value of an expression.

The switch statement works with data types like byte, short, char, int, long, String, enums, or their wrapper classes (Integer, Short, Byte, Long). It makes code easier to manage when you need to choose between multiple options.

Syntax:

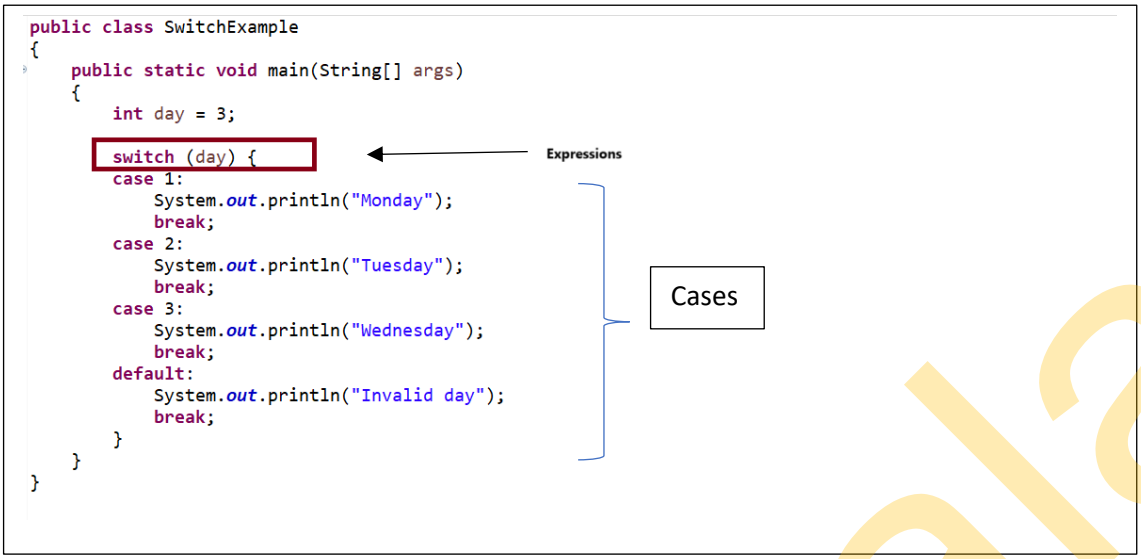
```
switch (expression) {
   case value1:
      // Code to execute if expression matches value1
      break; // Optional, exits the switch
   case value2:
      // Code to execute if expression matches value2
      break;
   // You can add more cases as needed
   default:
      // Code to execute if no cases match
      break;
}
```

Explanation:

1. **switch (expression):** The expression is evaluated, and its value is compared with each case.
2. **case value1**: If the expression matches value1, the code in this block is executed.
3. **break:** Exits the switch block. Without it, the execution continues to the next case.
4. **default:** Optional, executed if no case matches. Think of it like the "else" block in an if-else statement.

kodewala

Let's understand it with example



Steps:

1. The variable day is assigned a value of 3.
2. The switch statement evaluates the expression (day).
3. It matches day = 3 with case 3.
4. The code inside case 3 is executed: System.out.println("**Wednesday**");
5. The **break** prevents further execution of other cases.
6. Output: **Wednesday**.

If day was 5 (not matching any case), the default block would execute: System.out.println("Invalid day");.

Industry Experience:

- ✓ The switch statement is often used in applications where you need to handle multiple predefined options efficiently. For example, in a **billing system**, the switch statement can determine the discount percentage based on a customer's membership type (e.g., Silver, Gold, Platinum).

Interview Experience:

- ✓ Explain what happens if you omit the break statement in a switch. Write a program to demonstrate the concept of "fall-through"
  [Hint]: If break is missing, execution continues to the next case, regardless of whether the condition matches or not. This is called "fall-through."

Kodewala

Class Work:

- ✓ Write a Java program to print the name of the day based on the number input (1 for Monday, 7 for Sunday). If the input is not between 1 and 7, print "Invalid day".

Home Work:

- ✓ Write a program to display the month name based on the number input:

Kodewala

Break Statements

Break Statements

The **break statement** is a control flow statement used to stop the execution of a loop or a switch case. When the break statement is encountered inside a loop, it immediately exits the loop and moves control to the first statement after the loop.

Syntax:

```
for (initialization; condition; update)
{
        if (conditionToBreak)
        {
         break;  // Exits the loop
        }
}
```

How break Works in Loops:

1. The break statement is encountered inside the loop body.
2. When executed, it immediately terminates the loop, regardless of the remaining iterations.
3. The control is transferred to the first statement after the loop.

The following code shows how the break statement is used to terminate a for loop when the value of i equals 5.

```
public class BreakExample
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 10; i++)
        {
            if (i == 5) // Check if i is equal to 5
            {
                break; // Exit the loop
            }
            System.out.println("i: " + i); // Print values before the break
        }
        System.out.println("Loop has ended");
    }
}
```

Kodewala

Let's understand the use case:

Write a Java program to search for the product "apple16" in an array. Stop once the first occurrence of "apple16" is found using the break statement. Display a message when the product is found.

```
package com.kodewala.breakexample;

public class Break
{

    public static void main(String[] args)
    {

        String products[] = { "apple16", "samsung", "lg", "nokia", "tata", "apple16", "samsung34", "apple16" };

        for (int index = 0; index < products.length; index++)
        {

            if (products[index] == "apple16")
            {
                System.out.println(" I found my expectecd product " + products[index]);
                break;
            }
        }

    }

}
```

stop(break) once the first occurrence of "apple16" is found. it immediately exits the loop.

Explanation: The program searches for the product "**apple16**" in an array. It prints each product during the search and stops when "apple16" is found, using the break statement to terminate the loop early.

Industry Experience:

- ✓ In real-world applications, the break statement is often used to stop searching through large datasets or collections once the required result is found. For example, in an e-commerce system, a loop can stop searching for a product once it's located in the inventory.

Interview Experience:

- ✓ You may be asked to explain how the break statement works in loops or switch cases. Common coding questions include tasks like stopping a loop when a specific condition is met.

Class Work:

- ✓ Write a program to print numbers from 1 to 10, but stop the loop when the number 5 is reached using the break statement.

Kodewala

Home Work:

- ✓ Write a program to search for a specific number in an array. Stop the search and print "Number found" as soon as the number is located using break. If not found, print "Number not found."

Kodewala

Continue Statements

Continue Statements

The continue statement is used within loops like for, while, and do-while to skip the current iteration and jump directly to the next one.

Syntax:

```
for (initialization; condition; update) {
   if (conditionToSkip) {
      continue; // Skips the current iteration and moves to the next
   }
   // Code to execute if the conditionToSkip is false
}
```

How continue Works in Loops:

1. The continue statement is encountered inside the loop body, often within a conditional block.
2. When executed, it skips the current iteration and moves to the next iteration of the loop.
3. Control is transferred to the next iteration, checking the loop condition again (or updating in a for loop).

The following code shows When i equals 3, the loop skips printing 3 and continues with the next iteration.

```
public class ContinueExample
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            if (i == 3)
            {
                continue; // Skip the iteration when i equals 3
            }
            System.out.println(i); // Prints numbers except 3
        }
    }
}
```

Kodewala

Let's understand the use case:

Write a program to count the total number of products in an array, excluding those that start with "apple". Use the continue statement to skip the iteration for products starting with "apple" and print the total count of other products.

```
package com.kodewala.breakexample;

public class ContinueExample
{

    public static void main(String args[])
    {
        String products[] = { "apple16", "samsung", "lg", "nokia", "tata", "apple16", "samsung34", "apple16",
                "redmi" };


        int productCount = 0;
        for (int i = 0; i < products.length; i++)
        {
            String currentProduct = products[i];

            if (currentProduct.startsWith("apple"))
            {
                continue;
            }
            productCount = productCount + 1;
        }

        System.out.println(" Total product (Except Apple) " + productCount);
    }
}
```

Continue if product is apple and do nothing.

Explanation: The continue statement skips products starting with "apple" and doesn't add them to the count. This makes the code efficient by avoiding unnecessary processing.

Industry Experience:

- ✓ In an e-commerce application, the continue statement might be used to skip over invalid or empty records when processing customer orders. This ensures that only valid data is processed and stored.

Interview Experience:

- ✓ How would you use the continue statement in a loop to skip certain iterations based on a condition.

Class Work:

- ✓ printing numbers from 1 to 10, but skipping multiples of 3 using the continue statement.

Home Work:

- ✓ Implement a loop that processes a list of student grades and skips those below a passing threshold (e.g., 50%) using the continue statement.

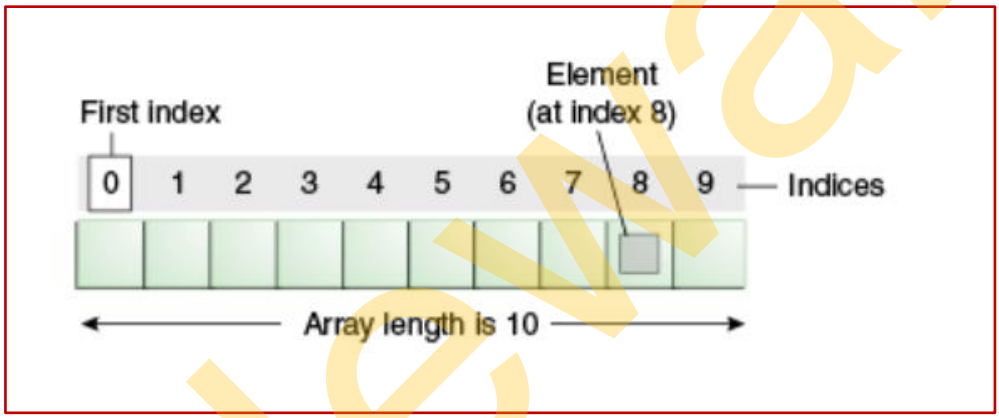
Kodewala

Kodewala Academy

Java Arrays

Array

- ✓ A Java array is an object that stores elements of the same type (i. e. int, String).
- ✓ These elements are kept in a continuous block of memory, making it easy to access them using their index.
- ✓ Arrays in Java are used to store a fixed number of similar items, meaning the size of the array is set when it is created and cannot be changed. This means, you need to decide the size of array while compile time and it cannot be changed at runtime.
- ✓ In Java, arrays are index-based, with the first element at index 0, the second element at index 1, and so on.
- ✓ This indexing system allows you to efficiently access elements by their position in the array.



Benefits

- ✓ **Code Optimization:** It makes the code optimized, we can store, retrieve or sort the data efficiently.
- ✓ **Random access:** We can get any data located at an index position.

Drawbacks

- ✓ **Size Limit:** Arrays have a fixed size and do not grow dynamically at runtime. If we are creating an array of size 10, this would not be change later.

Fundamental of Arrays

1. Array Declaration

To declare an array in Java, you use the following structure:

datatype[] arrayName; → **int[] numbers;**

Alternatively, it can be written as:

datatype arrayName[]; → ***int*** **numbers [];**

Kodewala

datatype: The data type of the array elements (e.g., int, String).

arrayName: The name of the array. This would be used to access an array.

2. Create an Array

To create an array, we need to allocate memory using 'new' keywords. Here we are creating an array of 10 integers.

```
// Creating an array of 10 integers

int[] numbers = new int[10];
```

This statement initializes an array called numbers with space for 10 integers. By default, each element in the array will be assigned the value 0. Please note: int default value is zero (0)

3. Assign the values to created array

```
numbers[0] = 5;
numbers[1] = 10;
numbers[2] = 15;
// Continue assigning values for other index.
```

4. Access an Element of an Array

To access the values in an array, you use the index of the array.

```
System.out.println(numbers[0]); // Accesses the first element (5)
System.out.println(numbers[2]); // Accesses the third element (15)
```

5. Reassign / Change the value of the array

To reassign values in an array, use the index of the array and assign a new value to it. For example:

```
numbers[0] = 50; // Reassign the first element to 50
numbers[2] = 100; // Reassign the third element to 100
```

6. Get the length of array

```
int[] numbers = {5, 10, 15, 20, 25};
int length = numbers.length;
System.out.println(length); // Outputs: 5
```

Kodewala

Kodewala Academy

Ways to initialize an array

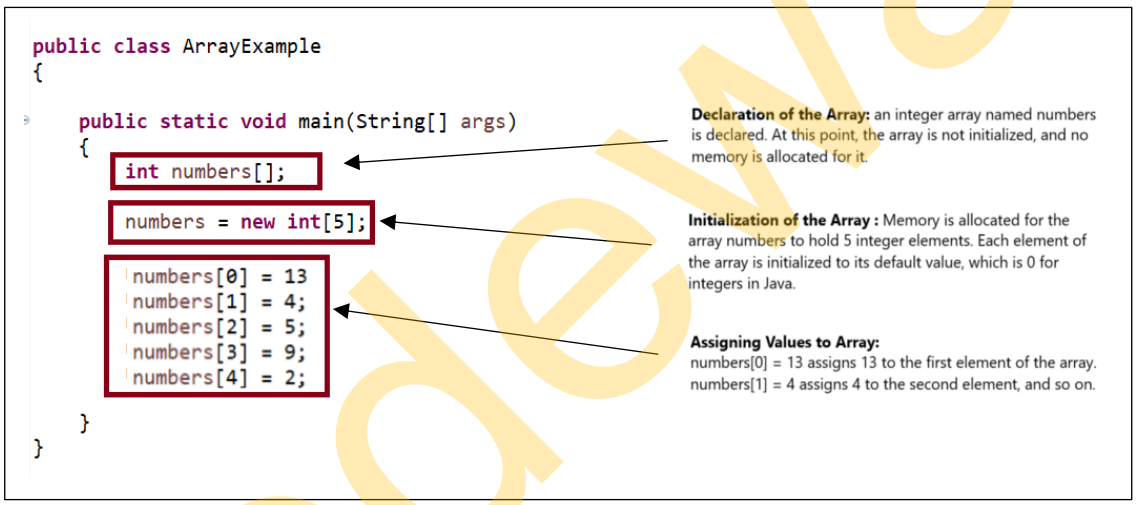
1. Directly Assign the Values

```
int[] oddNumbers = { 0, 2, 4, 6, 8 };  → // Directly assign values to the array during initialization:
```

2. Dynamic Initialization (using new keyword)

```
int[] evenNumbers = new int[5];→ // Array of size 5, all elements are 0 by default
```

Let's look at a code example that shows how to **declare an array**, **initialize it**, and **assign values** to its elements.



Things to Remember:

1. Indexing in Java Arrays:

- ✓ Array indexing starts from 0. For an array of size 5, the valid indices are 0 to 4.
- ✓ Trying to access an index outside this range (e.g., numbers[5]) will result in an **ArrayIndexOutOfBoundsException**.

2. Default Values:

- ✓ If you don't explicitly assign values to the array, all elements remain initialized to 0 (default for int type).

3. Accessing Array Elements:

- ✓ Use numbers[index] to retrieve or modify the value at a specific position in the array.

Kodewala

Now, let's retrieve / read access the values from the array 'numbers'.

1. **Accessing the values using index.**

   Directly use the array name and the desired index to read specific elements from the array. In this example, we are retrieving and reading the values at indexes 0 and 4.

```java
public class ArrayExample
{

    public static void main(String[] args)
    {
        int numbers[];

        numbers = new int[5];

        numbers[0] = 13;
        numbers[1] = 4;
        numbers[2] = 5;
        numbers[3] = 9;
        numbers[4] = 2;

        // Access and print the array values (method 1: accessing specific indices)

        System.out.println("Value at index 0 : " + numbers[0]); // Outputs 13
        System.out.println("Value at index 3 : " + numbers[3]); // Outputs 9

    }
}
```

Access and print the array values

2. Accessing the values using loop (Iteration).

```java
public class ArrayExample
{

    public static void main(String[] args)
    {
        int numbers[];

        numbers = new int[5];

        numbers[0] = 13;
        numbers[1] = 4;
        numbers[2] = 5;
        numbers[3] = 9;
        numbers[4] = 2;

        // Access and print the array values using a loop

        for (int i = 0; i < numbers.length; i++)
        {
            System.out.println("numbers[" + i + "] = " + numbers[i]);
        }

    }
}
```

The for loop iterates through the numbers array from index 0 to numbers.length - 1. In each iteration, it accesses the value at the current index (numbers[i]) and prints it along with its index.

Kodewala

```java
public class EmployeeSearch
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        // Create an array of employee names
        String[] employees = new String[4];

        // Adding employee names to the array
        employees[0] = "Mohit";
        employees[1] = "Raj";
        employees[2] = "Sanjay";
        employees[3] = "Neha";

        // Prompt user to enter employee name to search
        System.out.print("Enter Employee Name to search: ");
        String searchName = sc.nextLine();
        boolean found = false;

        for (int i = 0; i < employees.length; i++)
        {
            if (employees[i].equalsIgnoreCase(searchName))
            {
                System.out.println("Employee Found: " + employees[i]);
                found = true;
                break;
            }
        }
        if (!found)
        {
            System.out.println("Employee with name " + searchName + " not found.");
        }
        sc.close();
    }
}
```

Creating an Array

Adding the employee to the array.

For every iteration, we are checking if employee found or not. If found, we are breaking the loop and assigning the value to 'found' variable as true.

Explanation

1. Creating an array with 4 elements.
2. Storing employee names in the array.
3. Looping through the array to check if the name entered by the user exists.

Let's work on some coding challenges!

Write a program to store the prices of electronic items in an array along with their names in another array. Your task is to:

1. *Find the product with the highest price.*
2. *Print the name of the product and its price.*

```java
public class ElectronicItems
{
    public static void main(String[] args)
    {
        // Create an array of electronic items with their prices
        String[] products = { "Laptop", "Smartphone", "Tablet", "Smartwatch", "Headphones" };
        int[]    prices   = { 50000, 30000, 20000, 15000, 10000 };

        // Variables to store the index of the highest price
        int maxPriceIndex = 0;

        // Iterate through the array to find the highest price
        for (int i = 1; i < prices.length; i++)
        {
            if (prices[i] > prices[maxPriceIndex])
            {
                maxPriceIndex = i;
            }
        }

        // Print the product with the highest price
        System.out.println("The product with the highest price is: " + products[maxPriceIndex] + " with a price of Rs.
                + prices[maxPriceIndex]);
    }
}
```

Finding the highest price

Kodewala

Industry Experience:

- ✓ Arrays are used to store the inventory of products, enabling quick lookups when customers add items to their cart.

Interview Experience:

- ✓ How do you declare an array in Java?
- ✓ What is the default value of an array element in Java?
- ✓ What is the difference between an array and an ArrayList?
- ✓ How do you find the length of an array in Java?
- ✓ What happens if you try to access an array index out of bounds?
- ✓ Can an array in Java hold different data types?
- ✓ How do you initialize an array in Java?
- ✓ What is the purpose of the Arrays.sort() method?
- ✓ How would you reverse an array in Java?

Class Work:

- ✓ Creating, initializing, and printing electronic product items using arrays.

Home Work:

- ✓ Declare two arrays: one for storing the book titles and the other for storing their prices.
- ✓ Initialize the arrays with some sample book titles and prices.
- ✓ Print the list of books with their titles and prices in a formatted manner.

Kodewala

Kodewala Academy

Transforming code into career

Core Java Week - 4

Kodewala

Kodewala Academy

Java String

String

- ✓ A String in Java is an object that represents a sequence of characters.
- ✓ Strings are immutable, meaning once created, their content cannot be changed.
- ✓ If you try to change the string, in result you will get new object.

How to Create String

There are two ways to create string object.

1. Using **String literals**:

```
String s1 = "Kodewala";
```

2. Using the **new** keyword:

```
String s2 = new String("Kodewala");
```

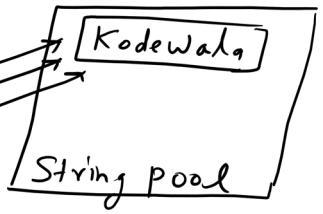
String Memory Management

1. **String Constant Pool (**If Objects are Created using String **Literals)**

- ✓ In Java, strings are stored in a special memory area called the **String Constant Pool**.
- ✓ When we create a string object using a **literal** (*String s1 = "*Kodewala*"*), Java checks if the same string("Kodewala") already exists in the pool. If it does, it reuses the same reference to save memory.
- ✓ No matter how many times you create a string with the same content, such as "Kodewala", only one object will be created in the String Pool.

Let's understand with an example:

```
public class Strings
{

    public static void main(String[] args)
    {
        String s1 = "Kodewala";
        String s2 = "Kodewala";
        String s3 = "Kodewala";

        System.out.println(s1 == s2);
        System.out.println(s1 == s3);

    }
}
```



Both returns true, because it refer to the same string in the pool

Kodewala

Explanation

1. When the code String s1 = "Kodewala"; is executed, JVM checks if the string literal "Kodewala" is already present in the **string constant pool (SCP).**

2. Since "Kodewala" is not present in the pool initially, it creates a new String object and stores it in the **string constant pool (SCP).**

3. For the **String s2 = "Kodewala";** and **String s3 = "Kodewala";** , the JVM finds that the string literal "**Kodewala**" is already present in the string pool, so it does not create new objects. Instead, s2 and s3 are simply references to the same String object stored in the pool.

4. **Number of String Objects Created**: Only **one** String object ("Kodewala") is created and stored in the string pool.

5. **Number of References**: Three references (s1, s2, and s3) point to the same String object in the string pool.

6. Since s1, s2, and s3 all reference the same String object in the string pool, both comparisons will return true.

Note: The == operator in Java is used to compare references of objects to check if they point to the same memory location. When applied to objects, it does not compare the actual content or values of the objects, but rather the memory addresses they refer to.

2. String Objects in the Heap (If Objects are Created using "new" keyword)

String s1 = new String("Kodewala");

- ✓ The new keyword explicitly creates a new String object on the heap.
- ✓ Unlike string literals, using the new keyword does not reuse objects from the String Constant Pool.
- ✓ Each call to **new** creates a distinct object on heap, even if the content is same.
- ✓ The object created using **new** keyword is allocated in the heap memory. This ensures that each object has its own memory space on the heap.
- ✓ If the string literal (e.g., "Kodewala") used in the new constructor is not already in the String Constant Pool, it gets added to the pool. This means one object created in String Constant Pool.
- ✓ Comparing strings created with **new** using == checks the memory addresses.

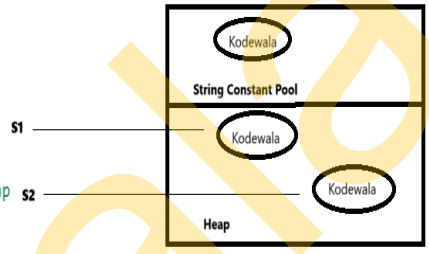
Kodewala

- ✓ Using **new** increases, the number of objects in memory, potentially impacting performance and memory usage.

- ✓ It is generally recommended to use **string literals** for common strings to take advantage of the **String Constant Pool**.

  Let's understand this with an example:

```java
public class StringExample
{
    public static void main(String[] args)
    {
        // Using new keyword to create String objects on the heap

        String s1 = new String("Kodewala"); // Creates one object in the heap

        String s2 = new String("Kodewala"); // Creates another object in the heap

        // Comparing objects using == (checks memory references)

        System.out.println("s1 == s2: " + (s1 == s2)); // false, because each new String creates a distinct object on
                                                       // the heap

        // Checking content equality using .equals
        System.out.println("s1.equals(s2): " + s1.equals(s2)); // true, because .equals compares content, not references
    }
}
```



Let's go through each step:

First call: new String("Kodewala")

- ✓ The **string literal** "Kodewala" is checked against the **String Constant Pool**.
- ✓ Since we are assuming "Kodewala" is **not already in the pool**, it gets **added** to the String Constant Pool.
- ✓ **Heap Object**: A new object is created in the **heap** because of the **new** keyword. This is a separate object from the String Constant Pool object.
- ✓ At this point, two objects have been created: one in the String Constant Pool and one in the heap. **This will be asked in each and every interview.**

Second call: new String("Kodewala")

- ✓ The **string literal** "Kodewala" is **already in the String Constant Pool** (from the first call), so no new object is added to the pool.
- ✓ **Another Heap Object**: The new String("Kodewala") creates another new **heap object** even though the content is the same as the first one.

Kodewala

Total How many objects will be created?

3 objects will be created:

1. **One object in the String Constant Pool**: The string "Kodewala" is added to the pool (if it's not already there).
2. **Two distinct objects in the heap**: One for s1 and another for s2. Both are created by the new keyword.

Points to Note:

String Constant Pool: The string literal "Kodewala" is added to the **String Constant Pool** the first time it is encountered (if it's not already there). This happens when you use the new keyword with the string literal, but the pool only holds one copy of the string "Kodewala" in memory.

Heap Memory: The new String("Kodewala") explicitly creates a **new object in the heap**. Each call to new creates a **distinct object** on the heap, even if the string content is the same.

Memory(address) Comparison:

- ✓ **s1 == s2** will be false because both s1 and s2 point to different objects in heap memory.
- ✓ **s1.equals(s2)** will be true because .equals() compares the **content** (the value of the string), not the memory address.

Summary of Object Creation:

3 objects:

- ✓ One in the **String Constant Pool** (if "Kodewala" is not already there).
- ✓ Two objects in the **heap**, one for each new String("Kodewala").

Why This Happens:

- The **String Constant Pool** helps save memory by ensuring that duplicate string literals are not stored multiple times in memory.
- The new keyword **always creates a new object** on the heap, even if the content of the string is the same. This is why two new String("Kodewala") calls create two distinct heap objects, even though the content is identical.

Kodewala

```java
package com.kodewala.string;

public class StringCreationExample
{
    public static void main(String[] args)
    {
        // Using string literal
        String s1 = "Kodewala";

        String s2 = "Kodewala";

        // Using the new keyword
        String s3 = new String("Kodewala");

        String s4 = new String("Kodewala");

        // Comparing memory address
        System.out.println("s1 == s2: " + (s1 == s2)); // true
        System.out.println("s3 == s4: " + (s3 == s4)); // false
        System.out.println("s1 == s3: " + (s1 == s3)); // false

        // Comparing content. Meaningfully equal or not.
        System.out.println("s1.equals(s2): " + s1.equals(s2)); // true
        System.out.println("s3.equals(s4): " + s3.equals(s4)); // true
        System.out.println("s1.equals(s3): " + s1.equals(s3)); // true
    }
}
```

Created one object in the String Constant Pool.

Refers to the same object in the String Constant Pool because object's content is same (Kodewala)

Creates a new object in the heap

Creates another new object in the heap

Let us break down each step of the above program.

Step 1: String s1 = "Kodewala";

- ✓ The literal "Kodewala" is checked in the **String Constant Pool**.
- ✓ Since "Kodewala" is not already in the pool, Object is created **in the pool**.
- ✓ Objects created so far: 1 object in the String Constant Pool.

Step 2: String s2 = "Kodewala";

- ✓ The literal "Kodewala" is already present in the **String Constant Pool** (from Step 1).
- ✓ **No new object is created**. Instead, s2 refers to the **same object** as s1 in the pool.
- ✓ Objects created so far: Still 1 object in the String Constant Pool.

Step 3: String s3 = new String("Kodewala");

- ✓ The new keyword creates a **new object** in the **heap** with the content "Kodewala".
- ✓ The literal "Kodewala" is already present in the **String Constant Pool**, so **no new object** is added to the pool.
- ✓ Objects created so far (**2 Objects**): **1 object** in the **String Constant Pool** and **1 new object** in the heap for **s3**.

Step 4: String s4 = new String("Kodewala");

- ✓ The new keyword creates **another new object** in the **heap** with the same content, "Kodewala".
- ✓ The literal "Kodewala" is already present in the **String Constant Pool**, so **no new object** is added to the pool.
- ✓ Objects created so far **1 object** in the String Constant Pool and **2 distinct objects** in the heap (one for s3 and another for s4).

Kodewala

Total Number of Objects Created:

1. **1 Object in the String Constant Pool**:
   - ✓ "Kodewala" is stored in the pool and shared by all literals with the same value.
2. **2 Objects in the Heap**:
   - ✓ One for s3 and another for s4, both created with the new keyword.

Total: 3 objects.

Why is String Immutable in Java?

In Java, strings are immutable, meaning that once a String object is created, it cannot be changed. If any modification is attempted, a new String object is created instead of altering the existing one. This immutability is achieved by design for efficiency, security, and thread safety.

Let us break it down further for better understanding:

How String Immutability Works:

1. **String object creation in the String Constant Pool (SCP):**
   - ✓ When we create a string using a string literal, it is stored in a special memory area called the String Constant Pool.
   - ✓ If a string with the same value already exists in the pool, the new reference will point to the existing object instead of creating a new one. This helps in memory optimization.

```
String s1 = "Kodewala"; // Stored in SCP
String s2 = "Kodewala"; // Points to the same object in SCP
```

   - ✓ If strings were mutable, modifying s1 would unintentionally modify s2 as well, leading to unexpected behavior.
2. **Heap and SCP Behavior:**
   - ✓ When we create a string using the **new** keyword, it creates a **new object** in the heap with the same value. As we know that, this is separate from the SCP.
   - ✓ Even in this case, immutability ensures that the content of the string in the heap cannot be modified, and any change results in a new object.

```
String s1 = new String("Kodewala"); // Creates a new object in the heap
String s2 = s1.concat(" Academy");  // Creates another new object in the heap
```

Kodewala

- ✓ **s1** still points to "Kodewala", and **s2** points to a new object with "Kodewala Academy". The original string remains unchanged. This means String is immutable. Once created cannot be changed.

Let's understand the immutability with a code example. Please go through the comments written along with code carefully.

```
package com.kodewala.string;

public class StringImmutability
{
    public static void main(String[] args)
    {
        String s1 = "Kodewala";                // Stored in SCP
        String s2 = s1.concat(" Academy");      // Creates a new object in the heap

        // Original string remains unchanged

        System.out.println("s1: " + s1);       // Outputs: Kodewala
        System.out.println("s2: " + s2);       // Outputs: Kodewala Academy

        // Memory references(Address)
        System.out.println("Are s1 and s2 the same? " + (s1 == s2)); // Outputs: false
    }
}
```

Read the comments carefully

Here is the detailed explanation of the above code.

Immutability:

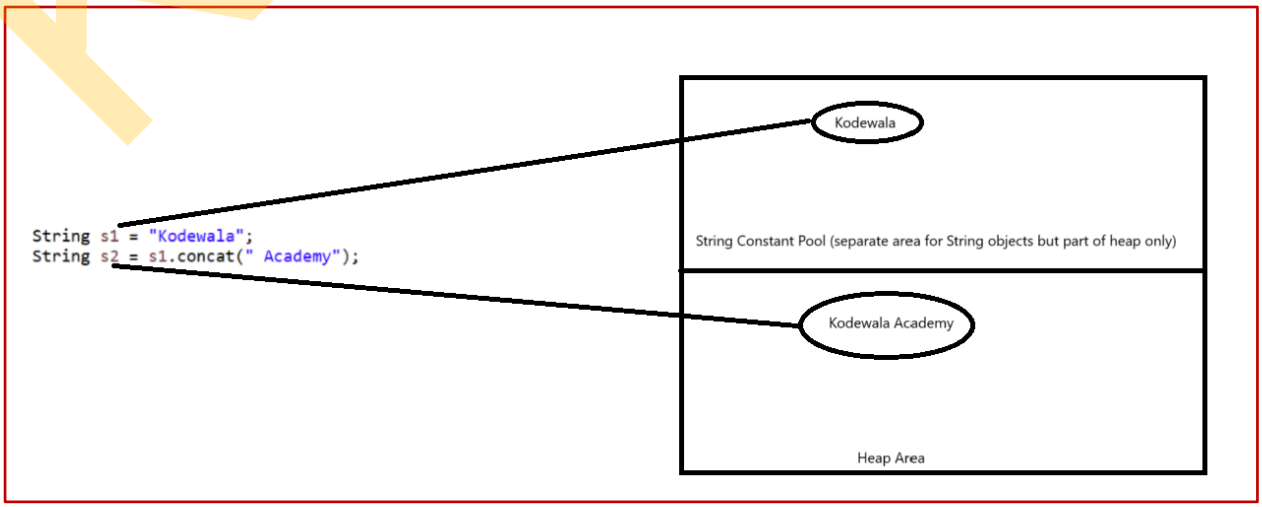
- ✓ "Kodewala" is stored in the String Constant Pool (SCP).
- ✓ s1.concat(" Academy") creates a new string ("Kodewala Academy") in the heap, not modifying the original "Kodewala".

Memory Allocation:

- ✓ s1 points to "Kodewala" in the SCP.
- ✓ s2 points to a new string ("Kodewala Academy") in the heap.

Result:

- ✓ s1 remains unchanged, and s2 is a new string. Hence, s1 == s2 returns false because they reference different memory locations.



Kodewala

Kodewala Academy

Why Did Java (Sun MicroSystems / Oracle) Designed Strings Immutable? Reason listed below.

- ✓ **Security**: Immutable strings protect sensitive data from being altered.
- ✓ **Performance**: They allow shared references, reducing memory usage.
- ✓ **Thread Safety**: Safe to use across multiple threads without data corruption.
- ✓ **Hashing**: Ensures consistent hash codes, especially in collections like HashMap.
- ✓ **String Pool**: Prevents unexpected changes in the string literal pool.
- ✓ **Optimization**: JVM can optimize memory by reusing immutable strings.

String Interning

What is String Interning?

String interning is a memory optimization technique where Java ensures that only **one copy** of each string literal exists in memory. This copy is stored in the **String Constant Pool (SCP)**, allowing the JVM to reuse it.

How Does Interning Work?

1. When a string literal like "Kodewala" is used, Java checks if it's already in the **SCP**:
    a. If it's not, it's added to the pool.
    b. If it exists, Java reuses the reference from the pool.
2. The **intern()** method is used to add a string to the pool or return the reference if it already exists.

Let's understand with an example:

```
public class StringInternExample
{
    public static void main(String[] args)
    {

        String s1 = "Kodewala Academy";   // String literal, automatically stored in the SCP


        String s2 = new String("Kodewala Academy"); // Creates a new String object on the heap


        // This will output 'false' because s2 points to the new object on the heap, not
        // the SCP.
        System.out.println(s1 == s2); // Output: false

        // Interning s2: This checks the SCP and returns the reference to the string in
        // the pool
        String s3 = s2.intern();


        // Now, s3 refers to the same reference in the SCP as s1

        System.out.println(s1 == s3); // Output: true
    }
}
```

Explanation:

1. **String s1 = "Kodewala Academy";**:
    ✓ "Kodewala Academy" is added to the **SCP**.
2. **String s2 = new String("Kodewala Academy");**:

Kodewala

- ✓ This creates a new **object on the heap**. The string "Kodewala Academy" is **already in the SCP**, but this new object is not automatically added to the SCP.

3. **s2.intern()**:
   - ✓ intern() checks if "Kodewala Academy" is in the **SCP**.
   - ✓ Since it's already there, intern() returns the reference to the string in the SCP, and s3 points to the **SCP object**.
   - ✓ **The heap object (s2) is not moved to the SCP**. Only s3 is now pointing to the SCP reference.

Summary:

- ✓ **new String("Kodewala Academy")** creates **two objects**:
  - ➢ **One in the String Constant Pool (SCP)** for the literal "Kodewala Academy".
  - ➢ **One on the heap** for the object created by new String("Kodewala Academy").
- ✓ The **intern() method** ensures that any string you create can **reuse a reference** from the SCP, avoiding duplicate string objects. It does not affect the original heap object but ensures the string is in the SCP and references are shared.

Industry Experience:

In the industry, **strings** are widely used for tasks like:

- ✓ **Handling User Input:** Strings are used to capture data like names, addresses, or other details from users.
- ✓ **Data Processing:** Strings are frequently manipulated when processing files (e.g., reading and writing data to CSV or JSON files) or cleaning data.

Strings are everywhere, and they're not going anywhere. You'll be working with them in almost every part of your program!

Interview Experience:

- ✓ What is the difference between String, StringBuilder, and StringBuffer in Java?
- ✓ What is String immutability, and why is String immutable in Java?
- ✓ How does the intern() method work in Java?
- ✓ What is the difference between == and .equals() when comparing strings?
- ✓ What is the String Constant Pool (SCP) in Java?
- ✓ How do you concatenate strings in Java? What is the most efficient way?
- ✓ What happens when you use the new String("Hello") constructor in Java?
- ✓ What is the significance of the intern() method?
- ✓ Explain the concept of String immutability and how it affects performance.
- ✓ What are the performance implications of using String vs StringBuilder?
- ✓ How do you reverse a string in Java?
- ✓ Explain how to check if a string is a palindrome in Java.
- ✓ What will happen if you compare two different string objects with ==?

Kodewala

Class Work:

- ✓ Create a string using both string literal and **new String().** Compare them using both **==** and **.equals()**. What do you notice about how the comparisons?
- ✓ Write a program that demonstrates the use of the **String Pool** and **interning**. Create two strings with the same value and check if they refer to the same memory location.

Home Work:

- ✓ Count the **total number of String objects** created in the below code.
- ✓ Identify which objects are stored in the **String Constant Pool (SCP)** and which are on the **heap**.
- ✓ Explain the role of intern() in this example.

```
package com.kodewala.string;

public class StringObjectCount
{
    public static void main(String[] args)
    {
        String str1 = "Java";
        String str2 = new String("Java");

        String str3 = str2.intern();
        String str4 = "Java";

        String str5 = new String("JavaScript").intern();

        System.out.println(str1 == str2);
        System.out.println(str3 == str4);
        System.out.println(str5 == "JavaScript");
    }
}
```

Kodewala

Kodewala Academy

Commonly Used String Method

1. Length and Character Access

✓ **length()**: Returns the length of the string.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";

        System.out.println(str.length()); // Output: 8
    }
}
```

✓ **charAt(int index)**: Returns the character at the specified index.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";

        System.out.println( str.charAt(2) );  // Output: d
    }
}
```

2. String Comparison

✓ **equals(Object obj):** Compares the content of two strings for equality.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String s1 = "Kodewala";
        String s2 = "Kodewala";

        System.out.println( s1.equals(s2) ); // Output: true
    }
}
```

✓ **equalsIgnoreCase(String anotherStr)**: Compares two strings, ignoring case.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String s1 = "Kodewala";
        String s2 = "KODEWALA";

        System.out.println( s1.equalsIgnoreCase(s2) ); // Output: true
    }
}
```

Kodewala

3. Searching and Substrings

- ✓ **indexOf(String str):** Returns the first occurrence index of a substring.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";
        System.out.println( str.indexOf("w") ); // Output: 4

        System.out.println( str.indexOf("M") ); // Return -1 if index is not found.
    }
}
```

- ✓ **lastIndexOf(String str):** Returns the last occurrence index of a substring.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala Academy";

        System.out.println( str.lastIndexOf("a") ); // Output: 11
    }
}
```

- ✓ **substring(int beginIndex):** Returns the substring from the specified index to the end.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";

        System.out.println( str.substring(4) ); // Output: wala
    }
}
```

- ✓ **substring(int beginIndex, int endIndex):** Returns the substring from beginIndex (inclusive) to endIndex (exclusive).

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";

        System.out.println(str.substring(0, 4)); // Output: Kode
    }
}
```

Kodewala

4. String Modification

- ✓ **toUpperCase():** Converts all characters to uppercase.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "kodewala";
        System.out.println( str.toUpperCase() ); // Output: KODEWALA
    }
}
```

- ✓ **toLowerCase():** Converts all characters to lowercase.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "KODEWALA";
        System.out.println(str.toLowerCase()); // Output: kodewala
    }
}
```

- ✓ **trim():** Removes leading and trailing whitespace.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "   Kodewala   ";

        System.out.println(str.trim()); // Output: Kodewala
    }
}
```

- ✓ **replace(char oldChar, char newChar):** Replaces all occurrences of a character with another character.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";

        System.out.println(str.replace('w', 'v')); // Output: Kodevala
    }
}
```

Kodewala

5. Splitting

- ✓ **split(String regex):** Splits the string into an array of substrings based on a delimiter(i.e , | $ # etc).

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala,Academy,BTM,Bangalore";

        String[] languages = str.split(","); // Splitting based on comma

        for (String lang : languages)
        {
            System.out.println(lang);
        }
        // Output:
        // Kodewala
        // Academy
        // BTM
        // Bangalore
    }
}
```

6. Checking String Content

- ✓ **contains(CharSequence s):** Checks if the string contains a specific sequence.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala Academy";
        System.out.println(str.contains("Academy")); // Output: true
    }
}
```

- ✓ **startsWith(String prefix):** Checks if the string starts with a specified prefix.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala Academy";
        System.out.println(str.startsWith("Kode")); // Output: true
    }
}
```

- ✓ **endsWith(String suffix):** Checks if the string ends with a specified suffix.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala Academy";
        System.out.println(str.endsWith("Academy")); // Output: true
    }
}
```

Kodewala

7. Conversions

- ✓ **toCharArray():** Converts the string to a character array.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str = "Kodewala";
        char[] chars = str.toCharArray();
        for (char c : chars)
        {
            System.out.print(c + " "); // Output: K o d e w a l a
        }
    }
}
```

8. String Concatenation: String concatenation is the process of joining two or more strings together.

- ✓ **Using the + Operator :** The simplest and most commonly used way to concatenate strings.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str1 = "Kodewala";
        String str2 = " Academy";
        String result = str1 + str2;
        System.out.println(result); // Output: Kodewala Academy
    }
}
```

- ✓ **Using concat() Method :** This method concatenates the specified string to the end of the current string.

```
package com.kodewala.string;

public class StringMethods
{
    public static void main(String[] args)
    {
        String str1 = "Kodewala";
        String str2 = " Academy";
        String result = str1.concat(str2);
        System.out.println(result); // Output: Kodewala Academy
    }
}
```

Kodewala

Industry Experience:

- ✓ **length()**: Commonly used to validate user input, like checking if a password meets minimum length requirements.
- ✓ **equals() and equalsIgnoreCase()**: Used to compare strings in APIs, like verifying usernames or search keywords.
- ✓ **concat()**: Useful when appending fixed strings to form file paths or dynamic messages.
- ✓ **substring()**: Used in data extraction tasks, like parsing account numbers or substrings from logs.
- ✓ **toUpperCase() and toLowerCase()**: Frequently used for standardizing user input (e.g., email normalization).

Interview Experience:

- ✓ **indexOf():** Asked in questions like finding the first occurrence of a character or substring.
- ✓ **lastIndexOf():** Interviewers often ask how to find the last occurrence of a character in a string.
- ✓ **replace():** Used in string manipulation problems, like replacing all spaces with underscores.
- ✓ **split():** Frequently tested in tasks like splitting CSV lines into fields or tokenizing a sentence.
- ✓ **trim():** A simple but common question to remove leading and trailing whitespaces.

Class Work:

- ✓ **charAt():** Write a program to print each character of a string using a loop.
- ✓ **substring():** Extract a portion of a string, such as extracting a domain name from an email.
- ✓ **replace():** Replace all spaces in a sentence with hyphens (-).
- ✓ **split():** Split a comma-separated string into an array and iterate through it.
- ✓ **concat():** Concatenate multiple strings and observe the result.

Home Work:

- ✓ **toCharArray():** Write a program to convert a string to a character array and count the vowels.
- ✓ **contains():** Check if a given string contains a specific word or character.
- ✓ **startsWith()** and **endsWith():** Write a program to validate if a file name starts with a prefix like "IMG_" or ends with ".jpg".

kodewala

Kodewala Academy

StringBuffer

StringBuffer is a class used to create and work with strings that can be changed. Unlike the String class, which creates a new object every time you modify it, StringBuffer lets you change the content of a string without creating a new object. StringBuffer is mutable (This mean it can be changed)

Key Features of the StringBuffer Class

- ✓ StringBuffer objects are mutable, that means, you can change the contents of the buffer without creating a new object.
- ✓ StringBuffer is thread-safe, meaning all its methods are **synchronized** to ensure safe access in multithreaded environments. We will read more about synchronization in upcoming sessions. As of now just remember that StringBuffer is **synchronized.**

StringBuffer.class

```
296     @Override
297     public synchronized void setCharAt(int index, char ch) {
298         toStringCache = null;
299         super.setCharAt(index, ch);
300     }
301
302     @Override
303     public synchronized StringBuffer append(Object obj) {
304         toStringCache = null;
305         super.append(String.valueOf(obj));
306         return this;
307     }
308
309     @Override
310     @IntrinsicCandidate
311     public synchronized StringBuffer append(String str) {
312         toStringCache = null;
313         super.append(str);
314         return this;
315     }
316
```

Synchronized means only one thread can access a method or block of code at a time to ensure thread safety.

- ✓ Since StringBuffer is thread-safe(**synchronized**), its operations may be slower compared to non-thread-safe alternatives due to the overhead of synchronization.
- ✓ The append() method in StringBuilder is used to add characters, strings, or other data types to the end of the current sequence, allowing efficient string concatenation.

Let's create a simple example using StringBuffer.

```
public class StringBufferExample
{
    public static void main(String[] args)
    {
        // Create a StringBuffer with initial content
        StringBuffer sb = new StringBuffer("Kodewala Academy");

        // Append location details
        sb.append(", BTM");
        sb.append(", Bangalore");

        // Print the final content
        System.out.println("Details: " + sb);

    }
}
```

StringBuffer object sb is created with the initial string "Kodewala Academy"

append() method is used to add ", BTM" and ", Bangalore" to the end of the current content in sb.

Details: Kodewala Academy, BTM, Bangalore

Kodewala

- ✓ The append() method is used to add new data to the end of the existing string in the buffer.
- ✓ StringBuffer modifies the string in-place without creating new objects, making it more efficient than using String for frequent changes.
- ✓ Unlike String, StringBuffer is mutable, meaning its content can be changed without creating a new object

Advantages of using StringBuffer in Java:

- **Mutable**: StringBuffer is mutable, meaning you can change its content after creation, unlike String which is immutable.
- **Efficient**: StringBuffer is more efficient for frequent modifications since it doesn't create new objects each time, unlike String which creates a new object with every change.

Note: Both **String** and **StringBuffer** are thread-safe.

- **StringBuffer** is synchronized, making it thread-safe, but concurrent access may affect performance.
- **String** is immutable, making it automatically thread-safe since its state cannot be changed.

Conclusion: If you need to make multiple modifications to a string, **StringBuffer** is more efficient than using regular **String** objects.

Commonly Used Methods of Java StringBuffer Class

Method	Action Performed
append()	Adds text at the end of the current content in the StringBuffer.
length()	Returns the number of characters currently stored in the StringBuffer.
capacity()	Returns the total allocated capacity (the amount of space reserved for the StringBuffer).
charAt()	Retrieves the character at the specified index in the StringBuffer.
delete()	Removes a sequence of characters from the StringBuffer between specified start and end indices.
deleteCharAt()	Removes the character at the specified index in the StringBuffer.
ensureCapacity()	Ensures the StringBuffer has at least the specified minimum capacity for better performance in large modifications.
insert()	Inserts text at a specified index position in the StringBuffer, shifting existing content to make room.
reverse()	Reverses the sequence of characters within the StringBuffer.
replace()	Replaces a specified range of characters in the StringBuffer with new text.

Kodewala

append() method : The append() method concatenates the given argument with this string.

```
public class StringBufferExample
{
    public static void main(String[] args)
    {
        // Create a StringBuffer with initial content
        StringBuffer sb = new StringBuffer("Hello");

        // Use append() method to add more text to the existing string
        sb.append(" World");
        sb.append(" from Kodewala Academy!");

        // Print the final content after appending
        System.out.println(sb); // Output: Hello World from Kodewala Academy!
    }
}
```

insert() method : The insert() method inserts the given string with this string at the given position.

```
public class StringBufferExample
{
    public static void main(String[] args)
    {
        // Create a StringBuffer with initial content
        StringBuffer sb = new StringBuffer("Welcome to");

        // Use insert() method to insert text at a specific position
        sb.insert(10, " Kodewala Academy!");

        // Print the final content after insertion
        System.out.println(sb); // Output: Welcome to Kodewala Academy!
    }
}
```

Capacity: If the capacity of a **StringBuffer** exceeds its current limit, the **StringBuffer** automatically increases its capacity to accommodate the new content. This process ensures that the StringBuffer can continue to store and modify strings without any issues.

- ✓ The default initial capacity of a StringBuffer is **16 characters**, unless a custom capacity or initial string is provided.
- ✓ When the content exceeds the current capacity, StringBuffer allocates a new internal storage array with **increased capacity**.
- ✓ Formula used for capacity calculation: **New Capacity = (Current Capacity * 2) + 2**

```
public class Capacity
{

    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer();

        int capacity = sb.capacity();

        System.out.println(" Initial capacity of string buffer is " + capacity); // Initial capacity of string buffer is 16

        // Append content that exceeds the current capacity

        sb.append("Welcome to Kodewala!!!!!");

        System.out.println(sb.capacity()); // capacity --> 34
    }
}
```

Kodewala

Kodewala Academy

Please make sure to try all the methods with examples at home and practice them.

Industry Experience:

- ✓ In a real-world application, StringBuffer is used to efficiently manage dynamic text updates in a user interface. For example, when a user is typing in a chat application, the application needs to frequently update the displayed text without creating new string objects each time.

Interview Experience:

- ✓ Interviewers often ask about StringBuffer when discussing performance optimizations in string handling. Be prepared to explain why StringBuffer is preferred over String in scenarios that involve frequent modifications, due to its mutable nature.
- ✓ When would you use StringBuffer instead of String or StringBuilder?
- ✓ What are the performance considerations when using StringBuffer?
- ✓ Why is the StringBuffer class declared as final?
- ✓ What happens if the capacity of a StringBuffer exceeds its limit?
- ✓ What is the difference between String and StringBuffer?

Class Work:

- ✓ Write a program to manipulate strings, focusing on methods like append(), insert(), and delete().

Home Work:

Create a Simple To-Do List:

- ✓ Objective: Use StringBuffer to manage a list of tasks.

Requirements:

- ✓ Add tasks to the list.
- ✓ Update a task description.
- ✓ Remove a task from the list.

Key focus: Use the append(), insert(), and delete() methods of StringBuffer.

Example Output :

To-Do List:

1. Buy groceries

2. Finish homework

3. Call the plumber

Updated List:

1. Buy groceries

2. Finish homework - Updated

3. Call the plumber

Kodewala

StringBuilder

StringBuilder is a class used to create and modify strings. **StringBuilder** is mutable, meaning the content of the string can be changed without creating a new object. However, StringBuilder is not thread-safe which making it faster. All the methods inside StringBuilder are no **synchronized**.

Key Features of StringBuilder

1. **Mutable Strings:**
   StringBuilder allows us to modify strings directly without creating new objects. This improves performance when you need to update strings frequently.
2. **Not Thread-Safe:**
   Unlike StringBuffer, StringBuilder is not synchronized, which means it is not safe to use in multi-threaded programs.
3. **Performance Advantage:**
   Because it does not have the overhead of synchronization, StringBuilder is faster than StringBuffer in single-threaded environments.

As we discussed in last session, **StringBuilder** and **StringBuffer** have almost the same methods because both are used to manipulate mutable strings. However, there is a key difference between the two:

StringBuffer:

- ✓ All its methods are **synchronized**, making it **thread-safe**.
- ✓ Suitable for use in multithreaded environments where multiple threads may access the same object.
- ✓ Synchronization makes it slightly slower than StringBuilder in single-threaded scenarios.

StringBuilder:

- ✓ Its methods are **not synchronized**, so it is **not thread-safe**.
- ✓ Suitable for single-threaded environments.
- ✓ Faster than StringBuffer in non-concurrent scenarios due to the lack of synchronization overhead.

Note: Interviewers often ask about the **differences between StringBuffer and StringBuilder**. So be prepared.

kodewala

Industry Experience:

Let's go through some the real-time use case of the StringBuilder.

Building Dynamic SQL Queries: Creating dynamic SQL queries based on user input or conditions:

```
public class StringBuilderExample
{
    public static void main(String[] args)
    {
        StringBuilder query = new StringBuilder("SELECT * FROM employees WHERE 1=1");
        boolean filterByDepartment = true;

        if (filterByDepartment)
        {
            query.append(" AND department = 'IT'");
        }

        boolean filterBySalary = true;

        if (filterBySalary)
        {
            query.append(" AND salary > 50000");
        }

        System.out.println(query.toString()); // SELECT * FROM employees WHERE 1=1 AND department = 'IT' AND salary > 50000

    }
}
```

initial SQL query "SELECT * FROM employees WHERE 1=1" is provided as the starting content for the StringBuilder object. The StringBuilder object can then be used to append more conditions to the SQL query dynamically.

Based on the condition, appending to the query

Based on the condition, appending to the query

Real-Time Web or App Responses: When dynamically constructing responses for APIs or web pages:

```
public class StringBuilderExample
{
    public static void main(String[] args)
    {
        StringBuilder response = new StringBuilder();

        response.append("{ \"status\": \"success\", ");
        response.append("\"data\": { \"id\": 123, \"name\": \"John\" } }");

        System.out.println(response.toString()); // { "status": "success", "data": { "id": 123, "name": "John" } }
    }
}
```

Interview Experience:

- ✓ Explain the difference between StringBuilder and StringBuffer?
- ✓ Why is StringBuilder preferred in scenarios where multiple string manipulations are required?
- ✓ In a multi-threaded environment, would you use StringBuilder? Why or why not?
- ✓ When would you use StringBuilder in a real-time project?
- ✓ What is the default capacity of StringBuilder when it is initialized?
- ✓ What happens when the length of the string exceeds the current capacity in StringBuilder?
- ✓ How can you change the capacity of a StringBuilder?

Kodewala

Kodewala Academy

Class Work:

- ✓ Create a Java program using StringBuilder to construct a dynamic SQL query based on various conditions. Depending on the conditions (e.g., user filters), append appropriate parts to the query string.

Home Work:

Write a program that generates an SQL query dynamically based on user input. The user can input filters such as:

- Department
- Salary range
- Joining date

Construct the query as follows:

- The base query should be SELECT * FROM employees WHERE 1=1.
- Append filters based on user input (if filters are provided).
- Print the final query.

Example:

- User inputs department = "IT", salary = 50000, and joining date = "2020-01-01".
- The final query should look like this:

SELECT * FROM employees WHERE 1=1 AND department = 'IT' AND salary > 50000 AND joining_date >= '2020-01-01'

Kodewala

Core Java Week - 5

Kodewala

Java OOPs Encapsulation

Java OOPs (**Object-Oriented Programming**) is a way of writing programs using objects. It helps organize code better and makes it easier to reuse and maintain.

Java follows Object-Oriented Programming (OOP) principles, which help in structuring code in a modular, reusable, and maintainable way. It allows developers to model real-world entities in the form of objects.

Let's discuss the four pillars of OOPs, with real world example.

1. Encapsulation – Data Hiding & Security

Encapsulation means wrapping the data (variables) and code (methods) together into a single unit, called a class. The data inside the class is private, and only specific methods (getters/setters) can access it.

- ✓ Encapsulation means keeping the data (variables) inside a class hidden from other classes. This data can only be accessed using methods within the same class.
- ✓ Hidden means? - Make the data(variable) private so it would not be visible outside class.
- ✓ To achieve encapsulation, we declare class variables as private and provide public methods (getters and setters) to read and modify the values safely.
- ✓ We often use "encapsulation" and "data hiding" interchangeably because both involve restricting direct access to data.

Let's understand "**Encapsulation**" with real world example.

Think of a **bank account**:

- ✓ You **can deposit or withdraw money** through an ATM or banking app or Gpay etc.
- ✓ But you **cannot directly change the balance** (you can't just write INR 1,00,000 in your account!).
- ✓ The bank provides **controlled access** using secure methods (e.g., withdrawal limit, PIN authentication etc).

This is exactly what **Encapsulation** does in Java:

data (balance) is hidden, and only specific operations (deposit, withdraw) are allowed.

Funny Story: Imagine a boyfriend who is super protective about his phone. He never lets his girlfriend see what is inside, but he allows her to use it for calls, music, and maps.

- ✓ His phone (private data) is locked – no one can access it directly.
- ✓ She can only use public apps like the camera or music (controlled access).
- ✓ He has a PIN code for extra security, and only he knows it (private method).

This is Encapsulation – hiding the sensitive details (phone contents) while allowing controlled access (public apps).

Kodewala

```java
class BankAccount
{
    private double balance; // Private data member

    public BankAccount(double balance) // Constructor to set initial balance
    {
        this.balance = balance;
    }

    public double getBalance() // Getter method to view balance (read-only access)
    {
        return balance;
    }

    public void deposit(double amount)  // Method to deposit money
    {
        if (amount > 0)
        {
            balance += amount;
            System.out.println("Deposited: " + amount);
        } else
        {
            System.out.println("Invalid amount!");
        }
    }

    public void withdraw(double amount) // Method to withdraw money with security check
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else
        {
            System.out.println("Insufficient funds or invalid amount!");
        }
    }
}
```

```java
public class Main
{
    public static void main(String[] args)
    {
        BankAccount acc = new BankAccount(1000);
        acc.deposit(500);
        acc.withdraw(200);
        System.out.println("Final Balance: " + acc.getBalance());
    }
}
```

Explanation:

- ✓ The balance variable is private, so no one can modify it directly.
- ✓ Instead, we use getter (getBalance()) to access it securely.
- ✓ The deposit() and withdraw() methods control access, ensuring valid operations.
- ✓ This ensures data integrity and security, similar to how banks restrict direct account balance modifications.

Kodewala

Let's take one more example.

The ATM lets you **deposit** and **withdraw** money, but it keeps important information like your **balance** and **PIN** private and secure. You can only access this information through specific actions (methods) provided by the ATM, ensuring that everything stays safe.

```java
public class ATM
{
    // Private data (balance and PIN)
    private double balance = 1000.0;
    private String pin = "1234";

    // Public method to withdraw money
    public void withdraw(double amountToWithdraw, String enteredPin)
    {
        System.out.println("Current Balance : " + balance);
        if (enteredPin.equals(pin) && amountToWithdraw <= balance)
        {
            balance = balance - amountToWithdraw;
            System.out.println("Balance Post Withdraw : " + balance);
        } else
        {
            System.out.println("Error: Incorrect PIN or insufficient funds.");
        }
    }

    // Public method to deposit money
    public void deposit(double amount, String enteredPin)
    {
        if (enteredPin.equals(pin))
        {
            balance = balance + amount;
            System.out.println("₹" + amount + " deposited. New balance: ₹" + balance);
        } else
        {
            System.out.println("Error: Incorrect PIN.");
        }
    }

}
```

The current balance in the ATM account is a private variable. This means that it cannot be accessed directly from outside the class. Only methods inside the class can modify or access it. This ensures that the balance is protected and only modified via the ATM's methods.

This method allows the user to withdraw money from the ATM

```java
public class User
{

    public static void main(String[] args)
    {

        ATM atm = new ATM();

        atm.balance =12000;   // not allowed to access balance directly. This is encapsulated

        atm.deposit(120, "1234");// Correct PIN, valid deposit

        atm.withdraw(200, "1234"); // Correct PIN, valid withdrawal

    }
}
```

Explanation:

Private Variables (Data)

- ✓ **balance**: The current balance in the ATM account is a private variable. This means that it cannot be accessed directly from outside the class. Only methods inside the class can modify or access it. This ensures that the balance is protected and only modified via the ATM's methods.
- ✓ **pin:** The PIN for the ATM is also private. This prevents unauthorized access. The user can't see or change the PIN directly. It is only checked inside the class during authentication.

Kodewala

Public Methods:

- ✓ ***withdraw***(double amount, String enteredPin) : This method allows the user to withdraw money from the ATM, but only if the following conditions are met:
  - The entered PIN matches the stored PIN.
  - The user has sufficient funds in their account (i.e., the amount requested is less than or equal to the available balance).
- ✓ ***deposit***(double amount, String enteredPin) : This method allows the user to deposit money into their ATM account, but it also requires the correct PIN for authentication. After the PIN is verified, the deposit is added to the account balance.

Encapsulation in Action:

- ✓ The ATM encapsulates the **balance** and **PIN** to ensure that they cannot be accessed or modified directly by the user. This prevents accidental or malicious changes to the account.
- ✓ **Controlled Access:** The user can only interact with the ATM using the public methods (**withdraw**() and **deposit**()). They can't access or modify the **balance** or **PIN** directly, ensuring data security and integrity.

Summary: Encapsulation is a core concept in Object-Oriented Programming (OOP) that **hides** sensitive data and **provides controlled access** to it through public methods. It ensures that data is protected and only modified in a controlled manner.

Industry Experience:

- ✓ Banks use encapsulation in ATM software to protect customer data. Account balance and PIN are private, and users can only access them through controlled methods like withdraw() and deposit(), ensuring security

Interview Experience:

- ✓ What is encapsulation in Java, and why is it important?

  Tip: Explain that encapsulation **hides sensitive data** and provides **controlled access** using **getters and setters**

- ✓ How do you implement encapsulation in Java?

  Tip: Write a **BankAccount class** with **private balance and PIN** and provide **public methods** for deposit and withdrawal.

- ✓ Can you give a real-world example of encapsulation?

  Tip: Use an ATM system where balance and PIN are private, and users access them only through withdraw() and deposit() methods.

Kodewala

- ✓ How does encapsulation improve security and maintainability?
  Tip: It prevents unauthorized data modification, allows controlled updates, and makes code easier to maintain and scale.

Class Work:

- ✓ Build an ATM class where balance and PIN are private. Provide only deposit() and withdraw() methods for interaction.

Home Work:

- ✓ Create a BankAccount class with private balance and PIN. Add getter/setter methods for controlled access. Implement validations for security.

Kodewala

Kodewala Academy

Java Constructors

A **constructor** in Java is a special method that is used to initialize objects. It is called automatically when an object of a class is created. The constructor's main purpose is to set initial values for the object's attributes.

What are Constructors in Java?

In Java, a Constructor is a block of codes similar to the method. It is called when an instance (Object) of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory. It is a special type of method that is used to initialize the object. Every time an object is created using the **new()** keyword, at least one constructor is called.

Let's understand the constructor with an example.

```
public class Account
{
    public Account()
    {
        super();
        System.out.println("I'm Inside the constructor!");
    }

    public static void main(String[] args)
    {
        Account account = new Account();

    }

}
```

Constructor

Explanation:

1. Define the constructor

```
public Account()
{
  super();
  System.out.println("I'm Inside the constructor!");
}
```

Kodewala

- ✓ The constructor **Account()** has the same name as the class and no **return type**.
- ✓ **super**(); implicitly calls the parent class (**Object**) constructor.
- ✓ The constructor prints "I'm Inside the constructor!" when an object is created.

2. **Constructor Invocation (**How and When Constructor Code will be Executed**)**

```
Account account = new Account();
```

- ✓ When new **Account**(); is executed, the constructor automatically runs.
- ✓ It initializes the object and executes the constructor body.

3. The constructor message is printed, confirming that it got executed.

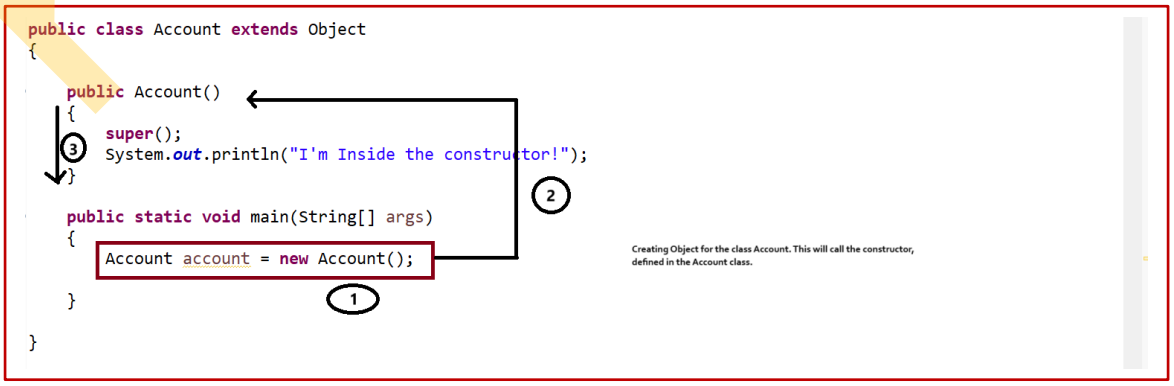
```
I'm Inside the constructor!
```

Note: Even if you do not explicitly extend any class in Java, your class is still a subclass of Object. This is because every class in Java automatically inherits from the Object class by default. So, even if you don't write **extends** Object, your class still has access to methods like **toString**(), **equals**(), and **hashCode**() from the Object class.

How are Java Constructors Different from Methods?

- ✓ A **constructor** must have the same name as the class, whereas a **method** can have any name.
- ✓ A **constructor** does not have a **return type**, while a **method** must have a return type (or void if it returns nothing).
- ✓ A constructor is called **only once** when an object is created, whereas a method can be called multiple times.

Now, let's define the syntax for invoking a constructor when creating an object or instance.



Kodewala

- ✓ We are creating the new object for Account class
- ✓ When new Account(); is executed, the constructor Account() is called automatically.
- ✓ Code inside the constructor's body will get executed.

Let's dive deeper and explore how a constructor works internally.

- ✓ If you **don't write a constructor**, the **compiler automatically provides a default constructor** with an implicit super() call.
- ✓ The first line of a constructor must call **super()** (superclass constructor) or this() (another constructor in the same class).
- ✓ If you don't explicitly write **super**(), the compiler automatically adds a no-argument super() at the beginning.
- ✓ The superclass constructor must be called to ensure the proper creation of an object.

Why Do We Need Constructors?

Let's take an example of Account class.

- ✓ In the **Account** class, an account should have details like **account number, holder name, and balance**. When we create an account object, can it exist without any values? Answer is **No.**
- ✓ **Constructors help us by automatically setting initial values** when an object is created. Without a constructor, the object would exist but values will not get initialize for the variables/attributes.
- ✓ If we do not define a constructor, Java **adds a default one** to ensure the object is properly created. This makes sure every account starts with some initial setup instead of empty values.
- ✓ A **default constructor** is a constructor that **takes no parameters** and is either explicitly defined by the developer or automatically provided by Java compiler if no constructor is written by developer.

When is a Java Constructor Called?

- ✓ When object is created using the new keyword, a constructor is automatically called.

kodewala

Rules for Writing Constructors

- ✓ A constructor must have the same name as the class.
- ✓ A constructor cannot be abstract, final, static, or synchronized.
- ✓ Access modifiers (public, private, protected, or default) can be used to control which classes can call the constructor.

Constructor Chaining

What is Constructor Chaining?

- ✓ It is the process of calling one constructor from another.
- ✓ Used to reuse code and reduce redundancy.
- ✓ Can happen within the same class or between parent and child classes

Types of Constructor Chaining

1. Within the Same Class (this can be done using **this())**

- ✓ Calls another constructor in the same class.
- ✓ Must be the first statement in the constructor.

Let's understand this with an example.

When creating a database connection, we might need default values or custom values. Instead of writing duplicate code, we use constructor chaining (this()).

```
class Database
{
    String dbName, user, password;

    // Default constructor (calls parameterized constructor)
    Database()
    {
        this("mydb", "root", "admin123"); // Calls the next constructor
    }

    // Parameterized constructor
    Database(String dbName, String user, String password)
    {
        this.dbName = dbName;
        this.user = user;
        this.password = password;
        connect();
    }

    void connect()
    {
        System.out.println("Connecting to DB: " + dbName + " as " + user);
    }

    public static void main(String[] args)
    {
        Database db1 = new Database(); // Uses default values
        Database db2 = new Database("testdb", "admin", "pass123"); // Uses custom values
    }
}
```

From the default constructor, we are calling parameterized constructor. This helps avoid repeating code and makes object creation flexible.

Kodewala

- ✓ The default constructor (**Database()**) calls another constructor using **this().**
- ✓ The parameterized constructor initializes values and calls connect().
- ✓ This helps avoid repeating code and makes object creation flexible.

2. Between Parent & Child Class (super())

- ✓ Calls the parent class constructor from the child class.
- ✓ Ensures base class initialization before subclass logic.
- ✓ Must be the first statement in the child class constructor.

Let's understand this with an example

A **Manager** is also an **Employee**, but with additional responsibilities. Instead of rewriting common code, we use super() to call the **Employee constructor** from the **Manager constructor**.

```
package com.kodewala;

class Employee
{
    String name;

    // Employee constructor
    Employee(String name)
    {
        this.name = name;
        System.out.println(name + " is an Employee");
    }
}

class Manager extends Employee
{
    String department;

    // Manager constructor calls Employee constructor
    Manager(String name, String department)
    {
        super(name); // Calls Employee constructor
        this.department = department;
        System.out.println(name + " is a Manager of " + department);
    }

    public static void main(String[] args)
    {
        Manager m = new Manager("Jay", "IT");
    }
}
```

- ✓ The Employee constructor initializes the name.
- ✓ The Manager constructor calls super(name), so the Employee constructor runs first.
- ✓ This avoids duplicate code and ensures the parent class is properly initialized.

Kodewala

Industry Experience:

- ✓ In real-world applications, constructors are used to initialize objects with necessary configurations.
- ✓ **Example**: In a banking application, an **Account** object can be initialized with default values or user-specific details.

Interview Experience:

- ✓ What is a constructor in Java?
- ✓ Can a constructor be static?
- ✓ What is constructor chaining?
- ✓ Difference between constructor and method?
- ✓ Can a constructor be private?
- ✓ What happens if you don't define a constructor in a class?
- ✓ Can a constructor call another constructor in the same class?
- ✓ Can a constructor throw an exception?
- ✓ Can we call a constructor explicitly using an object?
- ✓ What is the order of execution in constructor chaining?
- ✓ What happens if a constructor in a subclass does not explicitly call super()?
- ✓ What is the role of constructors in the Singleton design pattern?

Class Work:

- ✓ Create a Company class with a constructor that initializes the company name. Create an Employee class that inherits from Company and uses super() to initialize the company name and also stores employee details.

Home Work:

- ✓ Create a Bank class with a constructor that initializes the bank name. Create a Customer class that inherits from Bank and uses super() to pass the bank name while adding customer details.

Kodewala

Inheritance

Inheritance is a key concept in Object-Oriented Programming (OOP). Inheritance allows one class to use the properties (fields) and behaviors (methods) of another class.

In simple terms, inheritance helps us to create new classes based on existing ones, so we do not have to write the same code again. The new class can use the features of the existing class and also add its own new features.

Why Do We Need Inheritance in Java?

1. **Code Reusability** – Instead of writing the same code again, child classes can reuse the methods and fields of the parent class. We (developers) can reuse the code from one class in another class using **inheritance**. This helps avoid code duplication and makes the program more efficient and easier to maintain.
2. **Method Overriding** – It allows subclasses to provide a specific implementation of a method already defined in the parent class, enabling **runtime polymorphism**. This means, using **inheritance**, we can modify or extend the functionality of the **parent** class in the **child** class.
3. **Abstraction** – It helps hide unnecessary details and show only the essential functionality to the user.

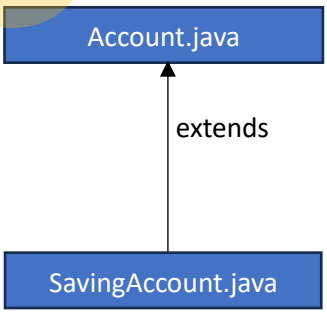
In short, inheritance makes code **more reusable, maintainable, and flexible**.

Terms used Inheritance

- ✓ **Super Class (Parent Class):** The class whose properties and methods are inherited by another class.
- ✓ **Sub Class (Child Class):** The class that inherits from a parent class and can add its own fields and methods.

How to Implement/Achieve Inheritance?

The **extends** keyword is used for inheritance.



- ✓ **Account.java** – This is the **parent (super) class** that have common fields and methods for all accounts.
- ✓ **SavingAccount.java** – This is the **child (sub) class** that **inherits** from **Account.java** using the **extends** keyword. This means SavingAccount can use the fields and methods of Account while also adding its own features(Functionalities)

Kodewala

Here, extends is used to create an **inheritance relationship** between **SavingAccount** and **Account**.

```
class Parent
{
  //methods and fields
}

class Child extends Parent

{

  //methods and fields of Child Class

// Methods and fields of Parents class will be also available as well. This is due to inheritance.

}
```

Understanding Inheritance with an Example

Let us understand **inheritance** in Java with a real-world example of a bank account system.

```
//Parent Class
class Account
{
    double balance;

    // Constructor
    public Account(double balance)
    {
        this.balance = balance;
    }

    // Method to show the balance
    public void showBalance()
    {
        System.out.println("Account Balance: " + balance);
    }
}
```

This is a Parent class

Method show the balance

```
//Child Class inheriting Account using 'extends'
class SavingAccount extends Account
{
    double interestRate;

    // Constructor
    public SavingAccount(double balance, double interestRate)
    {
        super(balance); // Calling parent class constructor
        this.interestRate = interestRate;
    }

    // Method to calculate interest
    public void calculateInterest()
    {
        double interest = balance * interestRate / 100;
        System.out.println("Interest Earned: " + interest);
    }
}

//Main Class to Test Inheritance
public class BankSystem
{
    public static void main(String[] args)
    {
        SavingAccount sa = new SavingAccount(10000, 5); // Creating SavingAccount object
        sa.showBalance(); // Inherited method from Account class
        sa.calculateInterest(); // Method from SavingAccount class
    }
}
```

showBalance() method is Inheriting from Parent Class(Account.java). That is the reason we able to call here in child class

Kodewala

Explanation:

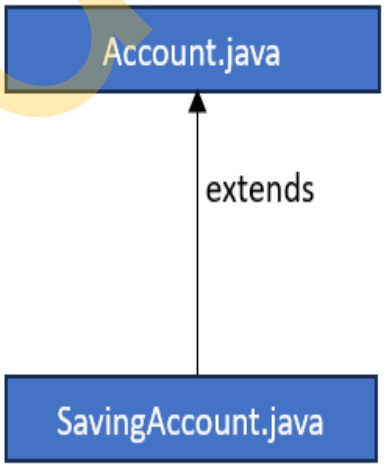
- **Account.java (Parent Class):** This represents a general bank account. It has a balance variable and a method showBalance() to display the account balance.
- **SavingAccount.java (Child Class):** This is a specialized account that **inherits** from Account using the **extends** keyword. It adds an interestRate and a method calculateInterest() to compute interest on the balance.
- **super(balance);** is used to call the parent class constructor and initialize the balance.
- In the **main method**, we create a SavingAccount object, display the balance (inherited method), and calculate interest (child class method).

This example shows how **inheritance helps reuse code (**We are reusing the showBalance() method of parent class inside our class**)** and allows the child class to extend the functionality of the parent class.

Types of Inheritance: Below are the different types of inheritance which are supported by Java.

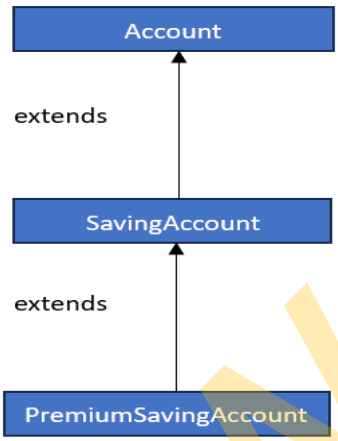
- ✓ Single Inheritance
- ✓ Multilevel Inheritance
- ✓ Hierarchical Inheritance
- ✓ Multiple Inheritance
- ✓ Hybrid Inheritance

1. **Single Inheritance:** In single inheritance, a subclass (Child) is derived from a single superclass (Parent). It inherits the properties and behaviours of its parent class, allowing code reuse. This is also known as **simple inheritance**. In the figure below, **Account** is the parent class, and **SavingAccount** is the child class, which inherits all the features of **Account**.



Kodewala

2. **Multilevel Inheritance:** In Multilevel Inheritance, a **child** class inherits from a **parent** class, and that **child** class also acts as a **parent** class for another **child** class.
In the below example, class **Account** serves as a parent class for class **SavingAccount**, which in turn serves as a parent class for class **PremiumSavingAccount**.



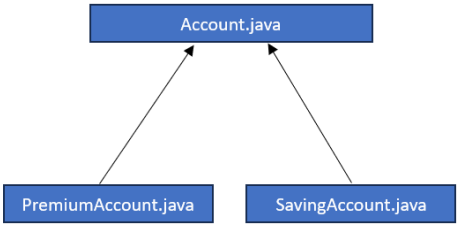
Code example of multi-level inheritance.

```
class Account
{
    public void whichOne()
    {
        System.out.println("Kodewala");
    }
}

class SavingAccount extends Account
{
    public void what()
    {
        System.out.println("Academy");
    }
}

class PremiumSavingAccount extends SavingAccount
{
    public void where()
    {
        System.out.println("Bangalore");
    }

    public static void main(String[] args)
    {
        PremiumSavingAccount premiumSavingAccount = new PremiumSavingAccount();
        premiumSavingAccount.whichOne();
        premiumSavingAccount.what();
        premiumSavingAccount.where();
    }
}
```

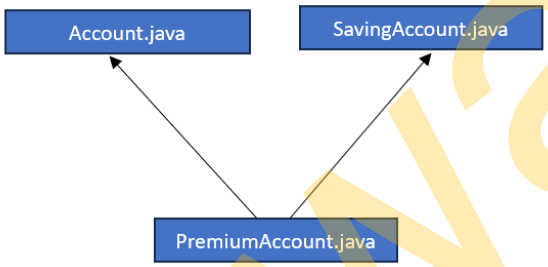
Calling the methods of parents classes

Kodewala

3. **Hierarchical Inheritance:** In Hierarchical Inheritance, a single parent class is inherited by multiple child classes.



4. **Multiple Inheritance (Through Interfaces):** In Multiple Inheritance, a class can inherit features from multiple parent classes. However, **Java does not support multiple inheritance with classes to avoid ambiguity**. Instead, Java achieves multiple inheritance through interfaces.



Why Java Does Not Support Multiple Inheritance?

From the above picture (multiple inheritance), If two **Account** and **SavingAccount** classes have the same method and a child class (**PremiumAccount**) inherits from both, Java would not know which method to call.

For Example: If **Account** and **SavingAccount** both have **showBalance()** method, and PremiumAccount inherits both, calling **showBalance()** would be ambiguous.

The **diamond problem** occurs in multiple inheritance when a class inherits from two parent classes that have the same method, creating ambiguity. Java **avoids this issue by disallowing multiple inheritance with classes** but allows multiple inheritance through **interfaces**.

```
class Account {
    void showBalance() {
        System.out.println("Account Balance");
    }
}

class SavingAccount {
    void showBalance() {
        System.out.println("Saving Account Balance");
    }
}

// ✕ Not Allowed: Java does not support multiple inheritance
class PremiumAccount extends Account, SavingAccount {
    // Which showBalance() should be called? ✕
}
```

Not Allowed

Kodewala

Diamond Problem in Java 8 (Using Interfaces with showBalance())

Java 8 introduced default methods in interfaces, which can lead to a similar issue.

```java
interface Account
{
    default void showBalance()
    {
        System.out.println("Account Balance");
    }
}

interface SavingAccount
{
    default void showBalance()
    {
        System.out.println("Saving Account Balance");
    }
}

// ⚠ Ambiguity arises when implementing both interfaces
class PremiumAccount implements Account, SavingAccount
{
    // Must override showBalance() to resolve ambiguity
    public void showBalance()
    {
        System.out.println("Premium Account Resolving Conflict");
    }
}

public class Bank
{
    public static void main(String[] args)
    {
        PremiumAccount pa = new PremiumAccount();
        pa.showBalance(); // No ambiguity because it's overridden
    }
}
```

Java **requires** the child class (PremiumAccount) to override the conflicting method and explicitly define which one to use.

- ✓ Java avoids the diamond problem by disallowing multiple inheritance with classes.
- ✓ Java allows multiple inheritance via interfaces, but it requires the child class to resolve conflicts explicitly.
- ✓ The diamond problem still exists in interfaces, but Java handles it by enforcing explicit method resolution.

Kodewala

If we are using inheritance, what can be done in a child class?

1. **Use Inherited Fields Directly** – The **child class** can access fields from the **parent** class.
2. **Add New Fields** – The **child class** can declare its own fields that are not present in the **parent** class.
3. **Use Inherited Methods Directly** – The **child** class can call methods from the **parent** class without modification.
4. **Override Methods** – The **child** class can provide a new implementation for a method from the **parent** class.
5. **Hide Static Methods** – A static method in the **child** class with the same signature as one in the **parent** class hides the parent method.
6. **Add New Methods** – The **child** class can define additional methods that are not in the **parent** class.
7. **Call Parent Class Constructor** – The **child** class constructor can invoke the superclass constructor, either implicitly or using super ().

Advantages of Inheritance in Java

1. **Code Reusability** – Inheritance helps reuse existing code, reducing duplication. The subclass inherits properties and methods from the superclass.
2. **Abstraction** – Enables the creation of abstract classes that define a common structure for related classes, improving maintainability and extensibility.
3. **Class Hierarchy** – Establishes a structured relationship between classes, making it easier to model real-world entities.
4. **Polymorphism** – Allows objects to take multiple forms. Subclasses can override superclass methods to modify behaviour as needed.

Industry Experience:

✓ Inheritance is widely used in enterprise applications for code reuse and maintainability.

Example: Banking System

- Account (Parent class)
- SavingAccount and CurrentAccount (Child classes inheriting common functionality like showBalance()).

Kodewala

Interview Experience:

- ✓ What is inheritance, and why do we need it?
- ✓ How does method override work?
- ✓ Why is multiple inheritance not supported in Java?
- ✓ Difference between method overriding and method hiding.
- ✓ Can we override a private method? Why or why not?
- ✓ Can we override a static method in Java?
- ✓ Can a subclass access private members of the parent class? If not, how can it use them?
- ✓ What happens if a parent and child class have the same variable name? How can we access both?
- ✓ Implement **Employee** class and inherit it into **Manager** and **Developer**, overriding a **calculateSalary**() method.

Class Work:

- ✓ Create a **User** parent class with **name** and **email** fields, and inherit it in **AdminUser** and **CustomerUser**.
- ✓ Explore access modifiers and how they impact inheritance.

Home Work:

- ✓ **E-commerce System:**
  - Create a Product parent class with fields like name and price.
  - Inherit ElectronicsProduct and ClothingProduct, overriding a applyDiscount() method.
- ✓ **Company Hierarchy:**
  - Employee → SoftwareEngineer → SeniorSoftwareEngineer (Multi-level Inheritance).
- ✓ **Hierarchical Inheritance:**
  - Vehicle as a parent class, with Car and Bike as child classes implementing their own fuelType().

Kodewala

Abstract

An **abstract class** is a special type of class that **cannot be directly used to create objects**. Instead, it acts as a **blueprint** for other classes. Some methods in an abstract class **do not have a body (implementation)**, meaning the child class must define them.

A class that is declared with the **abstract** keyword is known as an abstract class in Java. We cannot create an object of Abstract class.

Key Points:

- ✓ Declared using the **abstract** keyword.
- ✓ Can have both **abstract methods (without a body)** and **regular methods (with a body)**.
- ✓ Helps in **code reusability** and provides a **structure** for related classes.

Declaring an Abstract Class

Let us take an example of a **Bank Account** system where we define an **abstract** class **Account** and implement different types of accounts like **SavingAccount** and **CurrentAccount**.

Defining the Abstract Class (Account)

```
//Abstract class
abstract class Account
{
    double balance;

    Account(double balance)
    {
        this.balance = balance;
    }
    abstract void calculateInterest(); // Abstract method (to be implemented by subclasses)

    void showBalance()
    {
        // Concrete method
        System.out.println("Current Balance: " + balance);
    }
}
```

Account (Abstract Class)

- ✓ This is the parent class that defines the common behaviour of all accounts.
- ✓ It has a balance variable and a showBalance() method.
- ✓ It has an abstract method calculateInterest(), which must be implemented by subclasses.
- ✓ Cannot create an object of this class directly.

Kodewala

Creating Subclasses (SavingAccount & CurrentAccount)

```
class SavingAccount extends Account
{
    double interestRate = 5.0; // 5% Interest rate

    SavingAccount(double balance)
    {
        super(balance);
    }

    @Override
    void calculateInterest()
    {
        double interest = (balance * interestRate) / 100;
        balance += interest;
        System.out.println("Interest added: " + interest);
    }
}

class CurrentAccount extends Account
{
    CurrentAccount(double balance)
    {
        super(balance);
    }

    @Override
    void calculateInterest()
    {
        System.out.println("No interest for Current Account.");
    }
}
```

SavingAccount (Child Class - Extends Account)

- ✓ Inherits from Account and adds an interest rate.
- ✓ Implements the calculateInterest() method to add interest to the balance.

CurrentAccount (Child Class - Extends Account)

- ✓ Inherits from Account.
- ✓ Implements calculateInterest() but does not add interest, since current accounts usually don't have interest.

Using the Abstract Class in main()

```
public class BankSystem
{
    public static void main(String[] args)
    {
        Account saving = new SavingAccount(10000);
        saving.showBalance();
        saving.calculateInterest();
        saving.showBalance();

        Account current = new CurrentAccount(15000);
        current.showBalance();
        current.calculateInterest();
        current.showBalance();
    }
}
```

Kodewala

Key Features of Abstract Classes

- ✓ **Abstract Methods:** Can contain methods without a body, which must be implemented by subclasses.
- ✓ **Concrete Methods:** Can also include methods with a body, which subclasses inherit directly.
- ✓ **Cannot Be Instantiated:** Abstract classes cannot be used to create objects; they serve as blueprints for subclasses.
- ✓ **Can Have Constructors:** Abstract classes can define constructors, which are called when a subclass object is created to initialize common properties.

Points to Remember

- ✓ An abstract class must be declared using the abstract keyword.
- ✓ It can contain both abstract (without a body) and non-abstract (with a body) methods.
- ✓ Cannot be instantiated (you cannot create objects of an abstract class).
- ✓ It can have constructors and static methods.
- ✓ It can have final methods, preventing subclasses from modifying them.

Abstraction

Abstraction in Java is the concept of hiding implementation details and exposing only the required functionality to the user. This allows users to focus on what an object does rather than how it works, making the system simpler and more efficient. The unnecessary details or complexities are not displayed to the user.

Here are a few real-world examples of abstraction:

ATM Machine – When using an ATM, you can withdraw cash, check your balance, or transfer money by simply pressing buttons. You don't need to know the internal workings of the banking system or how transactions are processed in the backend.

Car Driving – When you drive a car, you use the steering wheel, accelerator, and brakes without knowing the internal mechanics of the engine, fuel injection, or braking system. The complexity is hidden from the user.

Mobile Phone – A smartphone allows you to make calls, send messages, and browse the internet without knowing the underlying operating system's working or hardware interactions.

Kodewala

Hiding Implementation (Using Abstraction)

```
abstract class Bank
{
    abstract void loanInterestRate(); // No implementation, only declaration
}

class HDFC extends Bank
{
    void loanInterestRate()
    {
        System.out.println("HDFC Loan Interest Rate: 8.2%");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Bank obj = new HDFC(); // Using Bank reference, unaware of exact implementation
        obj.loanInterestRate();
    }
}
```

Here is where abstraction works:

- ✓ The user only knows about Bank (which defines loanInterestRate()).
- ✓ The actual logic is inside HDFC, but the user doesn't need to worry about it when using Bank.
- ✓ If you use HDFC directly, you do see the logic.
- ✓ But if you work with Bank (abstract class), you only care about what it does, not how it's implemented.
- ✓ Abstraction hides unnecessary complexity at the top level and exposes only essential details.

When to Use Abstract Classes and Abstract Methods?

Abstract classes and methods are used when we need to define a common structure for a group of related classes without providing full implementations for all methods. A superclass can serve as a general blueprint, ensuring consistency while allowing subclasses to define their own specific behaviour.

Kodewala

Kodewala Academy

Industry Experience:

- ✓ In large-scale applications, abstraction is used to design modular and maintainable systems.
- ✓ Example: In a banking system, different types of accounts (Savings, Current, Loan) implement common behaviors through abstract classes.
- ✓ Abstract classes help define core functionalities while allowing flexibility in implementation.

Interview Experience:

- ✓ What is abstraction in Java? How is it implemented?
- ✓ What is an abstract class? How is it different from an interface?
- ✓ Can an abstract class have a constructor? If yes, what is its purpose?
- ✓ Can we create an object of an abstract class? Why or why not?
- ✓ What happens if an abstract class has all concrete (non-abstract) methods?
- ✓ Can an abstract class be declared as final? Why or why not?
- ✓ Can an abstract class have a main() method? If yes, can it be executed?
- ✓ Can an abstract class implement an interface? If yes, what happens if the interface has unimplemented methods?
- ✓ What happens if an abstract class does not have any abstract methods?
- ✓ Can an abstract class be private or static? Why or why not?
- ✓ If a class extends an abstract class but does not implement its abstract methods, what will happen?
- ✓ Can an abstract class have a static method? Why or why not?
- ✓ Can an abstract method be final, static, or private? Why?
- ✓ What is the difference between **100% abstraction** and **partial abstraction** in Java?
- ✓ If a method is marked as abstract, can it have a method body?
- ✓ Suppose you are designing a **payment gateway system**. How would you use abstraction to define different payment methods like CreditCardPayment, UPIPayment, and NetBankingPayment?
- ✓ If you need to enforce a method in multiple unrelated classes, would you use an abstract class or an interface? Why?

Class Work:

- ✓ Implement an abstract class Vehicle with an abstract method startEngine() and create subclasses like Car and Bike that provide their own implementations.

Home Work:

- ✓ Create an abstract class BankAccount with an abstract method calculateInterest().
- ✓ Implement SavingAccount and FixedDepositAccount classes that provide different interest calculation logic.

Kodewala

Kodewala Academy

Polymorphism

Polymorphism means "**many forms**" OR "**many behaviour**" In Java, it allows the same method to behave differently based on the object calling it. This is a key feature of Object-Oriented Programming, making code more **flexible** and **reusable** (You can reuse the super class code inside your class).

Real-life example of Polymorphism in Java:

A person can have multiple roles in different situations. For example, the same person can be a **Student** in college, a **Boyfriend** in a relationship, and an **Employee** at work. In each role, their behaviour changes based on the situation (Here situation is: College, Relationship, Work).

Similarly, in Java, polymorphism allows the same object or method to behave differently depending on the context. (Actual Object Type).

```java
package com.kodewala;
// Parent class representing a Person
class Person
{
    void showRole()
    {
        System.out.println("I am a person with multiple roles.");
    }}
// Child class representing a Student
class Student extends Person
{
    @Override
    void showRole()
    {
        System.out.println("I am a Student. I study and attend classes.");
    } }
// Child class representing a Boyfriend
class Boyfriend extends Person
{
    @Override
    void showRole()
    {
        System.out.println("I am a Boyfriend. I spend time with my partner.");
    }
}
// Child class demonstrating polymorphism
public class PolymorphismExample
{
    public static void main(String[] args)
    {
        // Reference of type Person pointing to different subclass objects
        Person person;
        person = new Student();
        person.showRole(); // Outputs: I am a Student. I study and attend classes.
        person = new Boyfriend();
        person.showRole(); // Outputs: I am a Boyfriend. I spend time with my partner.
    }}
```

At the runtime, method would be called based on the actual type of the object.(Student , Boyfriend etc.)

How Polymorphism Works Internally? → If you understand this, that means you understand the polymorphism perfectly.

kodewala

1. **Compile-time Check (Method Binding)**
    - ✓ At **compile time**, the compiler checks whether the method (showRole()) exists in the **Person class** (the reference type).
    - ✓ If the method does not exist in Person, a **compilation** error occurs.
2. **Runtime Execution (Dynamic Method Dispatch)**
    - ✓ At **runtime**, Java determines the actual object (Student, Boyfriend) and calls the overridden method of that object.
    - ✓ This is known as **dynamic method dispatch** (method overriding in runtime polymorphism).

This is **runtime polymorphism in Java**, where the method behaviour is determined dynamically based on the actual object.

Key Points about Polymorphism in Java (Need to Remember):

Method Resolution at Compile Time – The compiler ensures the method exists in the reference type (Person).
Method Execution at Runtime – The actual method called depends on the object (Student, Boyfriend).

Increases Flexibility – The same reference can point to different objects, allowing easy extensibility.

Types of Java Polymorphism

In Java, polymorphism is mainly divided into two types:

1. **Compile-time Polymorphism (Method Overloading)**
2. **Runtime Polymorphism (Method Overriding)**

Compile-time Polymorphism (Method Overloading) / Static Polymorphism

- ✓ Achieved by defining multiple methods with the same name but different parameters (Order of Param, Type of Param, Number of Param). Method overloading happens with in a class. This means having multiple versions of method with the same name.

  void **doRegistration**(String aadhar)

  void **doRegistration**(String mobile, int otp) → Number of param (2 prams, String and Int)

  void **doRegistration**(String name, String email) → Type of param (String, String)

  void **doRegistration**(int otp, String mobile) → order of param(int, String)
- ✓ Method resolution happens at **compile time**.
- ✓ Improves code readability and reusability.

kodewala

Let's understand this with an example.

```java
class Registration
{
    // Method 1: Registration using Aadhar
    void doRegistration(String aadhar)
    {
        System.out.println("Registered using Aadhar: " + aadhar);
    }

    // Method 2: Registration using Mobile and OTP (Different Number of Parameters)
    void doRegistration(String mobile, int otp)
    {
        System.out.println("Registered using Mobile: " + mobile + " with OTP: " + otp);
    }

    // Method 3: Registration using Name and Email (Different Type of Parameters)
    void doRegistration(String name, String email)
    {
        System.out.println("Registered using Name: " + name + " and Email: " + email);
    }

    // Method 4: Registration using OTP and Mobile (Different Order of Parameters)
    void doRegistration(int otp, String mobile)
    {
        System.out.println("Registered using OTP: " + otp + " for Mobile: " + mobile);
    }
}
public class MethodOverloadingExample
{
    public static void main(String[] args)
    {
        Registration reg = new Registration();
        reg.doRegistration("1234-5678-9012"); // Calls Method 1
        reg.doRegistration("9876543210", 123456); // Calls Method 2
        reg.doRegistration("John Doe", "john@example.com"); // Calls Method 3
        reg.doRegistration(123456, "9876543210"); // Calls Method 4
    }
}
```

These are overloaded methods

Explanation: (Also read the comments in the code)

- ✓ **Method Overloading** allows multiple methods with the same name but different parameters.
- ✓ Java **determines which method to call at compile time** based on the arguments passed.
- ✓ **Overloading is done by:**
  - ✓ **Changing the number of parameters** (String mobile, int otp)
  - ✓ **Changing the type of parameters** (String name, String email)
  - ✓ **Changing the order of parameters** (int otp, String mobile)

Kodewala

Runtime Polymorphism (Dynamic Method Dispatch/Binding)

Runtime Polymorphism, also known as **Dynamic Method Dispatch (Binding)**, is a process where **the call to an overridden method is resolved at runtime** based on the actual object. This is achieved through **method overriding**. Inheritance is must for the runtime polymorphism.

Let's understand the "Runtime Polymorphism" using Payment System Example

In below example, we implement **Runtime Polymorphism (Dynamic Method Dispatch)** using a **Payment System**. The Payment class (parent) has a method processPayment(), which is overridden by its child classes **UPIPayment** and **CCPay**.

```
class Payment
{
    // Method to be overridden
    void processPayment(double amount)
    {
        System.out.println("Processing generic payment of ₹" + amount);
    }
}
//Child class - UPI Payment
class UPIPayment extends Payment
{
    @Override
    void processPayment(double amount)
    {
        System.out.println("Processing UPI payment of ₹" + amount);
    }
}
//Child class - Credit Card Payment
class CCPay extends Payment
{
    @Override
    void processPayment(double amount)
    {
        System.out.println("Processing Credit Card payment of ₹" + amount);
    }
}
//Main class demonstrating Runtime Polymorphism
public class PaymentSystem
{
    public static void main(String[] args)
    {
        Payment payment; // Parent class reference pointing to different child class objects
        payment = new UPIPayment();  // Processing UPI payment
        payment.processPayment(500.75); // Calls overridden method in UPIPayment
        payment = new CCPay();              // Processing Credit Card payment
        payment.processPayment(1200.50);       // Calls overridden method in CCPay
    }
}
```

Runtime Polymorphism

How Runtime Polymorphism Works Here?

Method Overriding:

- ✓ The processPayment() method in the Payment (parent) class is **overridden** by UPIPayment and CCPay (child classes).

Dynamic Method Dispatch/Binding (Runtime Resolution):

Kodewala

- ✓ The Payment class reference (payment) is used to point to different **child class objects** (UPIPayment and CCPay).
- ✓ At **compile time**, Java checks that processPayment() exists in Payment.
- ✓ At **runtime**, the method that gets executed is based on the **actual object (UPIPayment or CCPay)**.

This **real-world example** of a **Payment System** demonstrates how **Runtime Polymorphism** makes the code more **flexible and scalable**.

Features of Runtime Polymorphism:

Method Resolution at Runtime – The actual method to be executed is determined at runtime.
Method Overriding is Used – Child classes override the method from the parent class.
Achieved Using Inheritance & Upcasting – Parent class reference holds child class objects.
Enhances Flexibility – Allows new payment methods (e.g., **NetBanking, Wallets**) to be added without modifying existing code.

Advantages of Polymorphism:

Code Reusability – Allows different classes to be treated as a common type, reducing duplicate code.
Better Readability & Maintenance – Less code to write and manage, making it easier to understand.
Dynamic Method Selection – Calls the correct method at runtime based on the actual object.
Flexibility – Enables writing generic code that works with multiple object types.

Disadvantages of Polymorphism:

Complexity – Understanding object behaviour can be tricky in large codebases.
Performance Overhead – Runtime method selection may add extra processing time.

Industry Experience:

Compile-Time Polymorphism (Method Overloading) – Payment Processing System

Example: Payment Gateway APIs
A payment service provides multiple ways to process payments:

- ✓ processPayment(String cardNumber) → Pay using a credit card.
- ✓ processPayment(String upiId, double amount) → Pay using UPI.
- ✓ processPayment(String walletId, int otp, double amount) → Pay using a wallet.

Industry Use Case: Allows a single payment service to handle different payment methods efficiently without creating separate method names.

Kodewala

Runtime Polymorphism (Method Overriding) – Payment & Logging System

Example 1: Payment Processing (UPI & Credit Card Payment)
A base class Payment has a method processPayment(), which is overridden by:

- ✓ UPIPayment.processPayment() → Handles UPI transactions.
- ✓ CCPay.processPayment() → Handles credit card transactions.

Industry Use Case: The actual method executed depends on the payment type selected at runtime, ensuring flexibility and easy future expansion.

Example 2: Logging System (Console, File, Database Logging)
A base class Logger has a method log(), overridden by:

- ✓ ConsoleLogger.log() → Logs messages to the console.
- ✓ FileLogger.log() → Writes logs to a file.
- ✓ DatabaseLogger.log() → Stores logs in a database.

Industry Use Case: Enables businesses to switch logging mechanisms dynamically based on requirements without modifying core logic.

Interview Experience:

1. What is polymorphism in Java?
2. What are the types of polymorphism in Java?
3. How is compile-time polymorphism achieved in Java?
4. How is runtime polymorphism achieved in Java?
5. What is the difference between method overloading and method overriding?
6. Can we overload a method by changing only the return type?
7. Can we overload a static method in Java?
8. Can we overload the main() method in Java?
9. What happens if two overloaded methods have the same number of parameters but different data types?
10. What is constructor overloading? Provide an example.
11. What are the rules for method overriding in Java?
12. Can a static method be overridden? Why or why not?
13. What is the role of the super keyword in method overriding?
14. What happens when a parent class reference points to a child class object?
15. What is covariant return type in method overriding?

kodewala

Class Work:

- ✓ Implement a payment system where users can pay using a credit card, UPI, or wallet by overloading the processPayment method.
- ✓ Create a Payment class with processPayment method, overridden by UPIPayment and CCPay, executing dynamically at runtime.

Home Work:

- ✓ Create a booking system where users can book a ticket using name, name + email, or name + phone number using method overloading.

- ✓ Design a Notification system where a parent class Notification has a send() method, overridden by EmailNotification and SMSNotification, executing based on the actual object at runtime.

Kodewala

Method Overloading

Method Overloading allows multiple methods to have the **same name** but differ in the **number of parameters**, the **order of parameters**, or the **type of parameters**.

It is also known as **Compile-time Polymorphism**, **Static Polymorphism**, or Early Binding. When resolving overloaded methods, Java prioritizes the most specific (child) argument over the parent argument.

Why Method Overloading Required?

Improves Code Readability & Maintainability

- ✓ Instead of using different method names for similar operations, we can use a single, meaningful method name with different parameters.
- ✓ This makes the code easier to understand, maintain, and debug.

Eliminates the Need for Multiple Method Names

- ✓ Without overloading, every method performing a similar task would need a different name.
- ✓ This leads to inconsistent naming and confusion for developers.

Enhances Code Reusability

- ✓ A single method name can be reused for different types and numbers of arguments.
- ✓ This avoids duplicate code, reducing redundancy and making updates easier.

Achieves Compile-Time Polymorphism (Static Binding)

- ✓ The correct method is determined at compile-time based on the arguments passed.
- ✓ This improves performance compared to runtime decisions.

Let's understand method overloading with an example.

```
class Payment
{

    void processPayment(int amount) // Method with one parameter (Processing payment by amount)
    {
        System.out.println("Processing payment of $" + amount);
    }

    void processPayment(String upiID, int amount) // Method with two parameters (Processing payment using UPI)
    {
        System.out.println("Processing UPI payment of $" + amount + " from UPI ID: " + upiID);
    }

    void processPayment(boolean confirmation, int amount)// Method with different parameter order (Processing cash
                                                         // payment with confirmation)
    {
        System.out.println("Processing cash payment of $" + amount + " with confirmation: " + confirmation);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Payment payment = new Payment();

        payment.processPayment(500); // Calls method with one int parameter
        payment.processPayment("kodewala@upi", 1000); // Calls method with String and int
        payment.processPayment(true, 2000); // Calls method with boolean and int
    }
}
```

Calling overloaded method based on the provided args.

Kodewala

Explanation

- ✓ Same method name (processPayment) is used multiple times.
- ✓ Different parameter types, number, and order allow Java to differentiate between methods.
- ✓ Compile-time Polymorphism: Java decides which method to call during compilation based on the arguments passed.

Different Ways of Method Overloading in Java

- ✓ Changing the Number of Parameters.
- ✓ Changing Data Types of the Arguments.
- ✓ Changing the Order of the Parameters of Methods

Let's understand it with code example.

```
class Payment {
    // 1 Changing the number of parameters
    void processPayment(int amount) {
        System.out.println("Processing payment of INR " + amount);
    }

    void processPayment(int amount, String mode) {
        System.out.println("Processing INR " + amount + " via " + mode);
    }

    // 2 Changing the type of parameters
    void processPayment(String upiID, double amount) {
        System.out.println("Processing UPI payment of INR " + amount + " from UPI ID: " + upiID);
    }

    // 3 Changing the order of parameters
    void processPayment(boolean confirmation, int amount) {
        System.out.println("Processing cash payment of INR " + amount + " with confirmation: " + confirmation);
    }
}

public class PaymentDemo {  // Better name for the class executing the methods
    public static void main(String[] args) {
        Payment payment = new Payment();

        payment.processPayment(500);                    // Calls method with 1 int parameter
        payment.processPayment(1000, "Credit Card");    // Calls method with int and String
        payment.processPayment("kodewala@upi", 1500.50);    // Calls method with String and double
        payment.processPayment(true, 2000);          // Calls method with boolean and int
    }
}
```

What Happens If No Exact Match is Found for Arguments?

If a method with the exact parameters is not available, the compiler follows these steps to find the closest match:

Convert to a Bigger Type (Same Family)

- ✓ If the exact data type is missing, Java tries to convert the argument to a larger type within the same category (e.g., int → long).

Convert to a Different Family (Wider Type)

Kodewala

- If no match is found in the same family, Java looks for a method with a broader type (e.g., int → float).

Example: If a method expects long, but only int is given, Java will convert int to long. If no long method is found, it may try float.

Method Overriding

Method Overriding happens when a child class provides a new version (new functionality) of a method that already exists in the parent class.

- ✓ The method in the child class must have the same name and parameters as in the parent class.
- ✓ It allows the child class to change the behaviour of the inherited method.

Let's understand it with an example.

```
//Parent class
class Payment
{
    void processPayment()
    {
        System.out.println("Processing payment...");
    }
}

//Child class overriding the method
class UPIPayment extends Payment
{
    @Override
    void processPayment()
    {
        System.out.println("Processing UPI payment...");
    }
}

public class PaymentDemo
{
    public static void main(String[] args)
    {
        Payment payment = new UPIPayment(); // Parent reference, Child object
        payment.processPayment(); // Calls the overridden method in UPIPayment
    }
}
```

Explanation:

- ✓ processPayment() is first defined in the Payment class.
- ✓ The UPIPayment class overrides it with its own version. That mean, UPIPayment class changed the functionality of processPayment()
- ✓ When called using a Payment reference, the child class method runs due to runtime polymorphism.

Kodewala

Rules for Method Overriding in Java

Same Method Signature

- ✓ The method in the child class must have the same name, return type, and parameters as in the parent class.

Inheritance is Required

- ✓ Overriding happens only in an inheritance (i.e., between a parent and a child class).

Access Modifier Should Not be More Restrictive

- ✓ The overridden method cannot have a more restrictive access modifier than the parent method.
  - ✅ public → public (Allowed)
  - ❌ public → private (Not Allowed)

Static Methods Cannot be Overridden

- ✓ Static methods belong to the class, not instances, so they cannot be overridden.
- ✓ If a static method is redefined in a subclass, it's called method hiding, not overriding.

Final Methods Cannot be Overridden

- ✓ If a method is marked as final in the parent class, it cannot be overridden in the child class.

Constructor Cannot Be Overridden

- ✓ Constructors cannot be inherited, so they cannot be overridden.

Child Class Method Can Have a Covariant Return Type

- ✓ The overridden method can return a subclass of the return type of the parent method.

Interview Experience:

1. What is method overloading in Java?
2. How does Java support method overloading?
3. What are the different ways to overload a method?
4. Can method overloading be done by changing the return type only? Why or why not?
5. Is it possible to overload a main() method in Java?
6. Can we overload static methods in Java? Explain.
7. Can a method be overloaded based on different return types if the parameters remain the same?
8. How does Java resolve ambiguity in method overloading?
9. What happens if no exact match is found for overloaded methods?

Kodewala

10. What is method overriding in Java?
11. What are the rules for method overriding?
12. Can a private method be overridden in Java? Why or why not?
13. Can a final method be overridden? Explain.
14. What happens when we override a method with a different return type?
15. Can static methods be overridden? Explain the concept of method hiding.
16. How does method overriding support runtime polymorphism?
17. What is the difference between method overloading and method overriding?
18. How does Java determine which overridden method to call at runtime?
19. What is the role of the @Override annotation in Java?

Kodewala

Core Java Week – 6

(Interface, Types of Interfaces, Exception Handling, JDBC)

Kodewala

Kodewala Academy

Java Interface

An **interface** in Java is like a **blueprint** that defines what a class should do but not how it should do it.

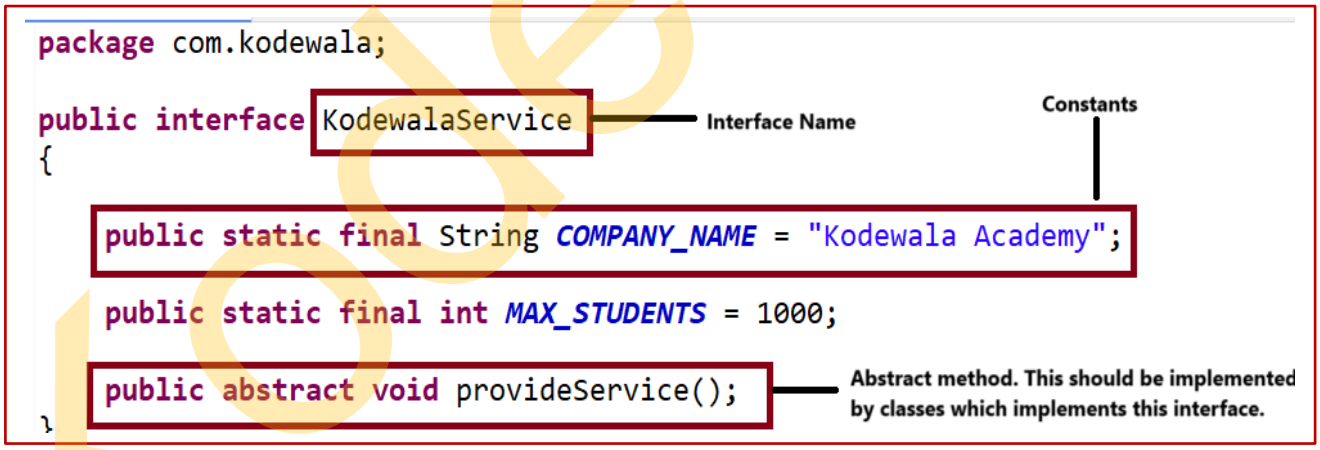
- ✓ It contains **only abstract methods** (before Java 8) and **static constants**.
- ✓ All variables in an interface are **public, static, and final** by default.
- ✓ Helps in achieving **abstraction** and **multiple inheritance** in Java.
- ✓ Promotes **loose coupling** (less dependency between classes).

Think of it as a contract → Any class that implements an interface **must** provide implementations for all its methods.

Syntax of an Interface:

```
Interface <Name Of The Interface> {

// declare constant fields
// declare methods that abstract( by default)

}
```

Let's declare the interface and understand it in detail.



Explanation:

- ✓ **public interface KodewalaService** → This is the interface. Classes will implement it.
- ✓ **public static final String COMPANY = "KodewalaPay";**
  - A constant (**final** means value cannot change).
  - **static** means it belongs to the interface itself, not individual objects.
  - **public** makes it accessible from anywhere.
- ✓ **public abstract void provideService();**
  - This is a method **without a body.**
  - Any class that implements this interface **must** define how it works.

Kodewala

Kodewala Academy

Point to Note:

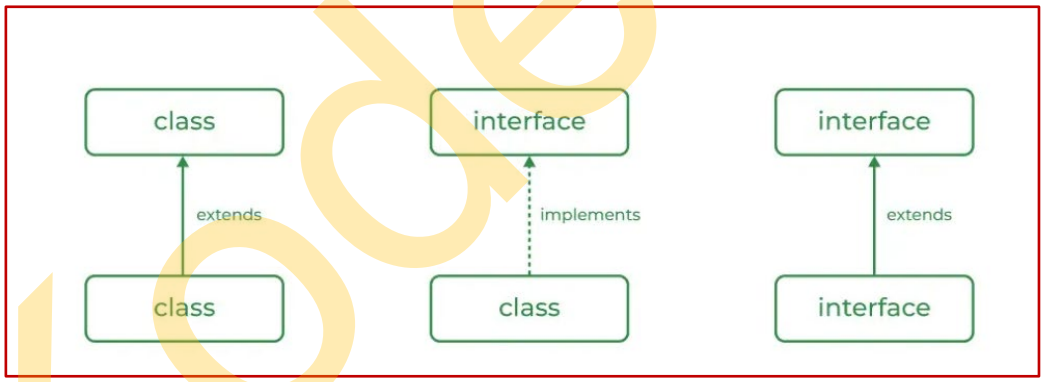
- ✓ Use the **interface** keyword to declare an interface.
- ✓ An interface provides **100% abstraction** (only method **declarations**, no **implementations**).
- ✓ All methods in an interface are **public** and **abstract** by default.
- ✓ All fields are public, static, and final (constants).
- ✓ A class uses the implements keyword to follow the interface and must provide implementations for all its methods.

Why Use Interfaces?

- ✓ **Ensures Consistency** → All classes following an interface, must have the same methods.
- ✓ **Supports Multiple Inheritance** → A class can implement multiple interfaces, avoiding Java's single inheritance rule.
- ✓ **Loose Coupling** → Classes depend on the interface, not on specific implementations, making code flexible.
- ✓ **Improves Code Organization** → Helps separate what a class should do from how it does it.
- ✓ **Reusable & Scalable** → New classes can easily implement the interface without modifying existing

Class vs. Interface Relationship. Refer below diagram.

- ✓ A **class** can **extend** another class (inherit features).
- ✓ A **class** can **implement** an **interface** (follow the rules).
- ✓ An **interface** can **extend** another interface (add more rules).
- ✓ An **interface** cannot implement a class (because interfaces only define behaviour, not implementation).



Kodewala

Difference Between Class and Interface

Feature	Class	Interface
Definition	A blueprint for creating objects with attributes and methods (can have both implementation and behavior).	A blueprint that only defines methods (without body) and constants. No implementation.
Methods	Can have **both implemented and abstract** methods.	Can only have **public abstract** methods (before Java 8). From Java 8, it can have **default** and **static methods** with a body.
Variables	Can have **instance variables** (non-static, changeable).	Only has **public static final** variables (constants).
Inheritance	A class can **extend only one** class (single inheritance).	An interface can **extend multiple interfaces**.
Implementation	A class inherits from another class using `extends`.	A class implements an interface using `implements` and **must** define all its methods.
Object Creation	Can be **instantiated** (if it's not abstract).	**Cannot be instantiated directly** (only used as a blueprint).
Access Modifiers	Can have `public`, `private`, `protected`, or default methods and fields.	All methods are **public** by default. All fields are **public static final** by default.
Multiple Inheritance	Not supported (can only extend one class).	Supported (a class can implement multiple interfaces).
Use Case	Used when objects need to have both **properties (variables)** and **behavior (methods)**.	Used when different classes need to follow a **common set of rules** (methods without implementation).

Let's explore the concept of interfaces and their implementation with a Banking System example.

In a banking system, different types of accounts (e.g., **SavingsAccount**, **CurrentAccount**) share common functionalities like **depositing money, withdrawing money, and checking balance**.

We can define these common behaviours in an **interface** and let each account type implement them in its own way.

Interface: BankAccount

```
public interface BankAccount
{
    public static final double MIN_BALANCE = 1000.00; // Constant

    // Abstract Methods
    public abstract void deposit(double amount);
    public abstract void withdraw(double amount);
    public abstract double checkBalance();
}
```

Kodewala

Implementation: Savings Account

```java
public class SavingsAccount implements BankAccount {
    private double balance;

    public SavingsAccount(double balance) {
        this.balance = balance;
    }

    @Override
    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited " + amount + " in Savings Account. New Balance: " + balance);
    }

    @Override
    public void withdraw(double amount) {
        if (balance - amount >= MIN_BALANCE) {
            balance -= amount;
            System.out.println("Withdrawn " + amount + " from Savings Account. New Balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }

    @Override
    public double checkBalance() {
        return balance;
    }
}
```

SavingsAccount is implementing BankAccount Interface

Implementation: Current Account

```java
public class CurrentAccount implements BankAccount
{
    private double balance;

    public CurrentAccount(double balance)
    {
        this.balance = balance;
    }

    @Override
    public void deposit(double amount)
    {
        balance += amount;
        System.out.println("Deposited " + amount + " in Current Account. New Balance: " + balance);
    }

    @Override
    public void withdraw(double amount)
    {
        balance -= amount;
        System.out.println("Withdrawn " + amount + " from Current Account. New Balance: " + balance);
    }

    @Override
    public double checkBalance()
    {
        return balance;
    }
}
```

Understanding Interfaces with a Banking System Example

What is the **BankAccount** Interface?

An interface in Java acts as a **contract** that defines a set of methods that all implementing classes must follow. Instead of specifying how a method works, an interface only declares what methods should exist. In our banking system example, we use an interface named **BankAccount** to define basic banking operations that all bank accounts must implement.

Kodewala

The **BankAccount** interface defines three essential methods:

- ✓ **deposit(double amount):** Adds money to the account.
- ✓ **withdraw(double amount):** Withdraws money from the account (with some rules, like minimum balance).
- ✓ **checkBalance():** Returns the current balance of the account.

The interface also defines a constant:

- ✓ **MIN_BALANCE = 1000.00 →** This means that savings accounts must maintain at least INR 1000 at all times.

How Do Different Bank Accounts Use This Interface?

Since every bank account must have common functionalities (**deposit**, **withdraw**, **check balance**), we define them once in the interface. Then, different types of bank accounts implement them in their own way.

1. Savings Account:

- ✓ Can store money for long-term.
- ✓ Allows depositing money anytime.
- ✓ Allows withdrawal only if the remaining balance is at least ₹1000.

2. Current Account (for Businesses)

- ✓ Allows both deposits and withdrawals without maintaining a minimum balance.

Why did we use an Interface for a Banking System?

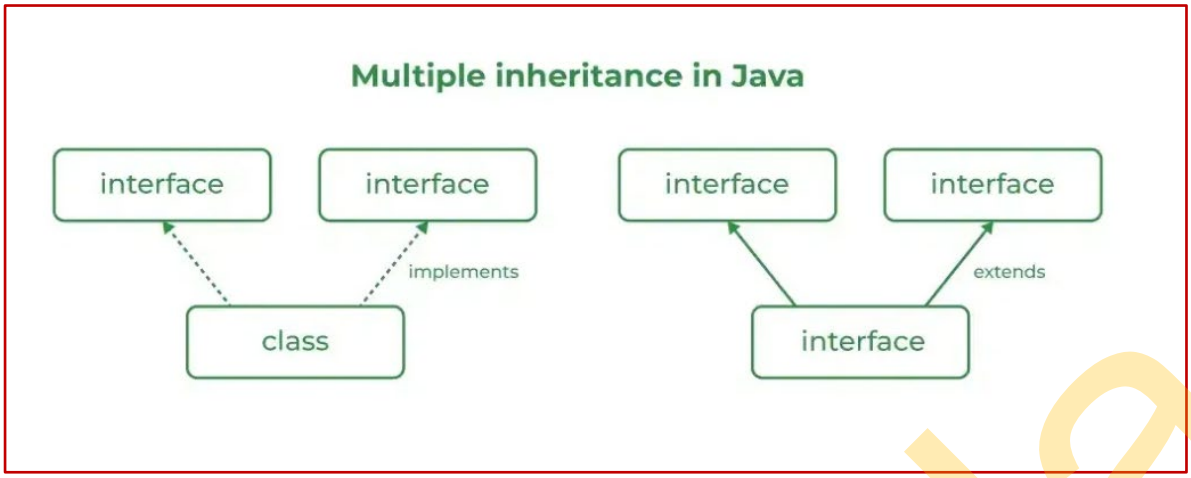
- ✓ **Code Reusability** → We don't need to write the same deposit(), withdraw(), and checkBalance() methods for every type of account. They are already defined in the BankAccount interface.
- ✓ **Standardization** → All accounts must follow the same structure, reducing errors.
- ✓ **Scalability** → We can add new types of accounts like FixedDepositAccount or LoanAccount easily by just implementing the BankAccount interface.
- ✓ **Flexibility with Multiple Inheritance** → A class can implement multiple interfaces, which means a bank account could follow rules from other financial entities as well (like a loan or credit system). This method keeps the **code clean**, **reusable**, and **easy to manage**, making it easier to **expand** and **maintain** the banking system over time.

Multiple Inheritance in Java Using Interfaces

In Java, **multiple inheritance** (inheriting from more than one class) is **not allowed** with classes to avoid conflicts and ambiguity. However, we can achieve multiple inheritance using **interfaces**.

In this approach, a class can **implement multiple interfaces**, allowing it to inherit behaviour from different sources without conflicts.

kodewala



Let's understand with an example.

```java
interface BankServices
{
    void deposit(double amount);
}

interface LoanServices
{
    void applyLoan(double amount);
}

class Customer implements BankServices, LoanServices
{
    public void deposit(double amount){
        System.out.println("Deposited: ₹" + amount);
    }

    public void applyLoan(double amount){
        System.out.println("Loan applied for: ₹" + amount);
    }
}

public class BankingSystem
{
    public static void main(String[] args){
        Customer customer = new Customer();
        customer.deposit(5000);
        customer.applyLoan(200000);
    }
}
```

Explanation:

- ✓ **BankServices** interface defines the deposit() method.
- ✓ **LoanServices** interface defines the applyLoan() method.
- ✓ Customer implements both, allowing a customer to **deposit money** and **apply for a loan**.
- ✓ The **BankingSystem** class tests this by creating a **Customer** object and calling both methods.

Kodewala

New Features in Interfaces (JDK 8)

In **Java 8**, two important features were added to interfaces:

1. Default Methods

- ✓ Interfaces can now have **methods with a default implementation** using the default keyword.
- ✓ This helps **add new methods** to interfaces without breaking existing implementations. This is done to achieve backward completability.

2. Static Methods

- ✓ Interfaces can now have static methods, which belong to the interface itself and cannot be overridden by implementing classes.
- ✓ These are useful for utility methods related to the interface and any common functionalities can be implemented. Implementing classes will be able to use the static method directly.

New Features in Interfaces (JDK 9)

In **Java 9**, **private methods** were introduced in interfaces to improve code reusability and maintainability.

Private Methods in Interfaces

- ✓ Interfaces can now have private methods to avoid code duplication inside default and static methods.
- ✓ These methods cannot be accessed by implementing classes.

Why Private Methods in Interfaces?

- ✓ **Code Reusability** – Avoids repeating the same logic in multiple default/static methods.
- ✓ **Encapsulation** – Keeps helper methods hidden from implementing classes.
- ✓ **Improves Readability** – Makes interfaces cleaner and more maintainable.

Real-time Example: Online Payment System (Using Default, Static, and Private Methods in an Interface)

In an **online payment system**, different payment methods like **Credit Card and UPI** follow common rules but have their own implementations.

- ✓ **Default Method (validateTransaction())** → Ensures transaction validation before payment.
- ✓ **Static Method (checkServiceStatus())** → Checks if the payment service is active.
- ✓ **Private Method (log())** → Handles internal logging (used inside default and static methods).
- ✓ **Abstract Method (processPayment())** → Each payment method implements its own payment logic.

This ensures **code reusability, better organization, and maintainability** in a real-world payment system.

kodewala

```java
interface PaymentGateway {
    default void validateTransaction() {
        log("Validating transaction...");
        System.out.println("Transaction validated!");
    }

    static void checkServiceStatus() {
        log("Payment service is active.");
    }

    private static void log(String message) {
        System.out.println("[LOG] " + message);
    }

    void processPayment(double amount);
}

class CreditCardPayment implements PaymentGateway {
    public void processPayment(double amount) {
        System.out.println("Paid ₹" + amount + " via Credit Card.");
    }
}

class UPIPayment implements PaymentGateway {
    public void processPayment(double amount) {
        System.out.println("Paid ₹" + amount + " via UPI.");
    }
}

public class PaymentApp {
    public static void main(String[] args) {
        PaymentGateway.checkServiceStatus();

        PaymentGateway cc = new CreditCardPayment();
        cc.validateTransaction();
        cc.processPayment(5000);

        PaymentGateway upi = new UPIPayment();
        upi.validateTransaction();
        upi.processPayment(1500);
    }
}
```

Default Method

Static Method

Private Method

Points to remember about Interface

- ✓ **Interfaces cannot be instantiated**, but we can create a reference of an interface pointing to an object of its implementing class.
- ✓ A **class can implement multiple interfaces** in Java.
- ✓ An **interface can extend another interface (or multiple interfaces)** using extends.
- ✓ A **class implementing an interface must implement all its methods** unless it's an abstract class.
- ✓ **All methods in an interface are public and abstract by default** (except default, static, and private methods in Java 8+).
- ✓ **All fields in an interface are public, static, and final by default** (constants).

Kodewala

- ✓ Interfaces **enable multiple inheritance** in Java.
- ✓ **Interfaces support loose coupling**, making code flexible and maintainable.
- ✓ **Instance variables cannot be declared inside interfaces** (only constants allowed).
- ✓ **Constructors are not allowed** inside interfaces.
- ✓ **The main() method is allowed inside an interface from Java 8**, but it must be static.
- ✓ **Static, final, and private methods** are **allowed from Java 8+**, but instance methods (without default) are not.

Interview Question:

1. What is an interface in Java?
2. Can an interface have instance variables? Why or why not?
3. Can an interface have constructors? Why?
4. Can an interface be instantiated? If not, how can we use it?
5. What is the default access modifier for methods in an interface?
6. How do you implement an interface in Java?
7. Can a class implement multiple interfaces? Provide an example.
8. What happens if a class does not implement all methods of an interface?
9. Can an interface extend another interface? How?
10. Can an interface extend multiple interfaces?
11. What are default methods in an interface? Give an example.
12. Why were default methods introduced in Java 8?
13. What are static methods in an interface? How are they different from default methods?
14. What are private methods in an interface? Why are they useful?
15. Can an interface contain a main() method? If yes, how?
16. How does an interface achieve multiple inheritance in Java?
17. What happens if two interfaces have methods with the same name? How is ambiguity resolved?
18. Can an interface contain a method with a body (implementation)? When?
19. What is the difference between an abstract class and an interface?
20. When should we use an interface instead of an abstract class?

kodewala

Type of Interface

In java there are 4 types of **interfaces**.

Normal Interface - A standard interface that can have multiple abstract methods, default methods, and static methods. Example: List, Map. This we have already discussed in previous session.

Nested Interface - An interface declared inside a class or another interface, usually for better encapsulation. Example: Map.Entry in Map. This is not used much in our day-to-day coding life.

Marker Interface - An empty interface without methods, used as a tag to provide special behaviour to implementing classes. Example: Serializable, Cloneable.

Functional Interface - An interface with exactly one abstract method, enabling lambda expressions and method references. This is also called Single Abstract Method (SAM) Example: Runnable, Callable.

Here, we will be discussing only Functional Interface and Marker Interface.

1. **Functional Interface -** A functional interface in Java has only one **abstract method** but can have **multiple default** or **static methods**. Examples include **Runnable**, **Callable**. From Java 8, we can use lambda expressions and method references to create instances of functional interfaces.

```
public class LambdaRunnableExample
{

    public static void main(String[] args)
    {
        // Using a lambda expression to implement Runnable
        Thread thread = new Thread(() -> System.out.println("New thread created"));
        thread.start();
    }
}
```

Lambda Expression: **(** () -> System.out.println(...) **)**

- ✓ Instead of creating a separate Runnable class, we use a **lambda expression** to define the run() method.
- ✓ The lambda **() -> System.out.println("New thread created")** replaces the need for implementing the Runnable interface explicitly.
- ✓ This means that when the thread starts, it will execute the System.out.println("New thread created") statement.

A **Functional Interface** is also called a **Single Abstract Method (SAM) Interface**. It has only **one abstract method** and allows a simpler way to write code. Java introduced functional interfaces to support **modern programming techniques**, like using **lambda expressions**.

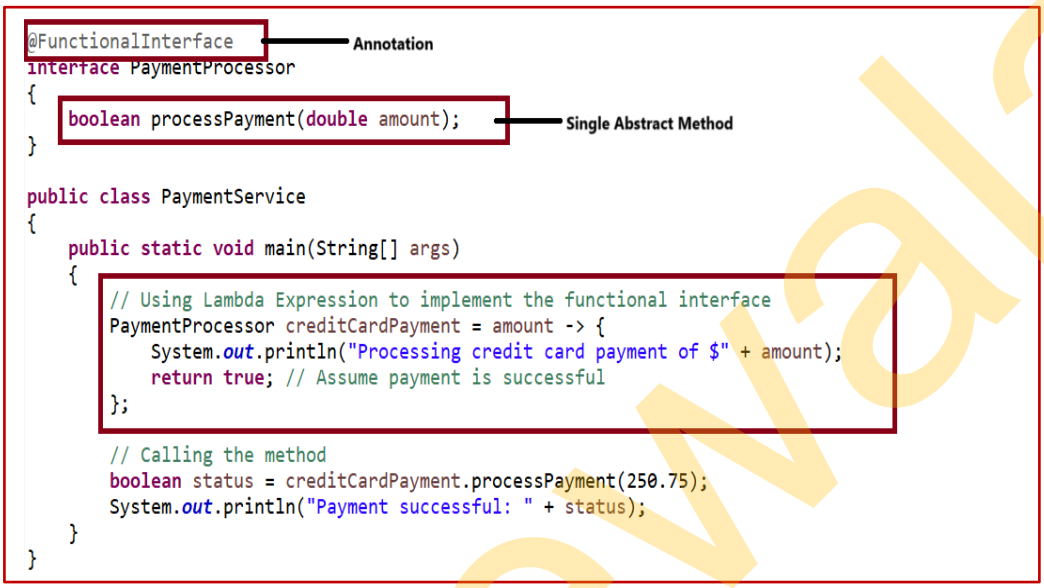
Kodewala

@FunctionalInterface Annotation

The @**FunctionalInterface** annotation ensures that an interface has **only one abstract method**. If more than one abstract method is present, the compiler gives an error:
"Unexpected @FunctionalInterface annotation."

However, using this annotation is **optional** - An interface is still a functional interface even without it. It simply helps catch mistakes at compile time.

Let's define a Functional Interface with **@FunctionalInterface Annotation**



Defined using @FunctionalInterface to ensure only one abstract method.

Method processPayment(double amount) represents a payment operation returning **true** for success and **false** for failure.

Defining the Functional Interface (PaymentProcessor)

- ✓ The **@FunctionalInterface** annotation ensures that the **interface** has only one abstract method.
- ✓ The interface **PaymentProcessor** defines a single method, **processPayment**(**double amount**), which takes a payment amount and returns a boolean value (true for success, false for failure).
- ✓ This method represents a standard payment processing action, making it suitable for lambda expressions.

Implementing Payment Processing in the Main Class (PaymentService)

- ✓ The PaymentService class contains the main method, where we implement the payment processing logic.
- ✓ Instead of creating a separate class to implement PaymentProcessor, we use a lambda expression, making the code more concise and readable.

kodewala

Implementing the Functional Interface Using a Lambda Expression

- ✓ We assign a lambda expression to PaymentProcessor, representing a credit card payment processor.
- ✓ The lambda takes a payment amount as input and prints a message indicating the payment is being processed.

Calling the processPayment() Method

- ✓ The lambda expression is executed by calling processPayment(250.75), passing the amount $250.75 as an argument.

Functional Interfaces in Java Before Java 8

Before Java 8, functional interfaces were implemented using **anonymous inner classes** or by creating separate implementing classes. Below is an example of how the Runnable interface was traditionally implemented before **lambda expressions** were introduced.

```
public class RunnableExample
{
    public static void main(String[] args)
    {
        // Implementing Runnable using an Anonymous Inner Class
        Runnable task = new Runnable()
        {
            @Override
            public void run()
            {
                System.out.println("Thread is running...");
            }
        };

        // Creating and starting the thread
        Thread thread = new Thread(task);
        thread.start();
    }
}
```

Anonymous Inner Class – Before Java 8, we had to create an anonymous inner class to implement Runnable.
Overriding run() Method - The run() method contains the logic to execute in a separate thread.
Creating a Thread - A Thread object is created and started using thread.start().
More Code, Less Readability - This approach is verbose and less readable, which was improved with lambda expressions in Java 8.

Kodewala

Built-In Functional Interfaces in Java

Since Java 8, many existing interfaces have been converted into functional interfaces and are annotated with **@FunctionalInterface**. Some common examples include:

- **Runnable -** Contains a single method run(), used for executing threads.
- **Comparable -** Defines the compareTo() method for comparing objects.
- **ActionListener -** Has the actionPerformed() method to handle UI events.
- **Callable -** Contains the call() method, similar to Runnable, but can return a result.

Only One Abstract Method - A functional interface can have only one abstract method. If more than one abstract method is present, it becomes a non-functional interface.

@FunctionalInterface Annotation (Optional) - This annotation is not mandatory, but it helps the compiler enforce the functional interface rule.

Static and Default Methods Allowed - A functional interface can have any number of static and default methods without violating its definition.

Overriding Parent Methods - If an interface inherits methods from a parent class, it does not break the functional interface rule.

Built-in Functional Interfaces - Java 8 introduced many pre-defined functional interfaces in the java.util.function package to support lambda expressions and functional programming.

2. **Marker Interface:** A **Marker Interface** in Java is an interface that does not contain any methods or fields. It is used to indicate a specific behaviour or property to the Java runtime or compiler. Examples of marker interfaces include **Serializable**, **Cloneable.** These interfaces are empty but serve important purposes in Java.

Why Do We Need Marker Interfaces in Java?

- ✓ Marker interfaces are used to indicate a specific behaviour or capability to the JVM or compiler without defining any methods. They act as a **tag** to classify classes.
- ✓ They help the JVM recognize certain classes and apply special processing (e.g., Serializable for object serialization).
- ✓ They allow frameworks and APIs to check if a class has a certain property and take appropriate action.
- ✓ They were widely used before Java introduced annotations and are still relevant in many legacy systems.

kodewala

```java
class Employee implements Serializable
{
    private static final long serialVersionUID = 1L; // Ensures version consistency
    String name;
    int id;

    public Employee(String name, int id)
    {
        this.name = name;
        this.id = id;
    }
}
```

Explanation:

- ✓ Serializable is a marker interface (it has no methods).
- ✓ When a class implements Serializable, it allows objects of that class to be converted into a byte stream and saved to a file.

Interview Question:

1. What is a functional interface in Java?
2. How many abstract methods can a functional interface have?
3. Is the @FunctionalInterface annotation mandatory?
4. Can a functional interface have static and default methods?
5. Name some built-in functional interfaces in Java 8.
6. How does lambda expression work with functional interfaces?
7. What happens if a functional interface has more than one abstract method?
8. Can a functional interface extend another interface?
9. How do method references work with functional interfaces?
10. What is the difference between Predicate, Function, Consumer, and Supplier?
11. What is a marker interface in Java?
12. Why do we need marker interfaces?
13. Can a marker interface have methods?
14. Name some examples of marker interfaces in Java.
15. How does the Serializable marker interface work?
16. Can we create a custom marker interface?
17. What is the difference between Cloneable and Serializable marker interfaces?
18. Are marker interfaces still useful after Java 8, considering annotations?

Kodewala

Kodewala Academy

Exception Handling

Exception handling helps developers manage exception that occur during program execution at **runtime**.

It uses followings:

- ✓ **try-catch block** - to catch and handle the exception.
- ✓ **finally block** - to run code after try-catch, whether an exception occurs or not. This is mandatory executable block.
- ✓ **throwing exceptions** - to create and throw exception when needed.
- ✓ **custom exceptions** - to define your own exception. This is also called as User Defined Exception.

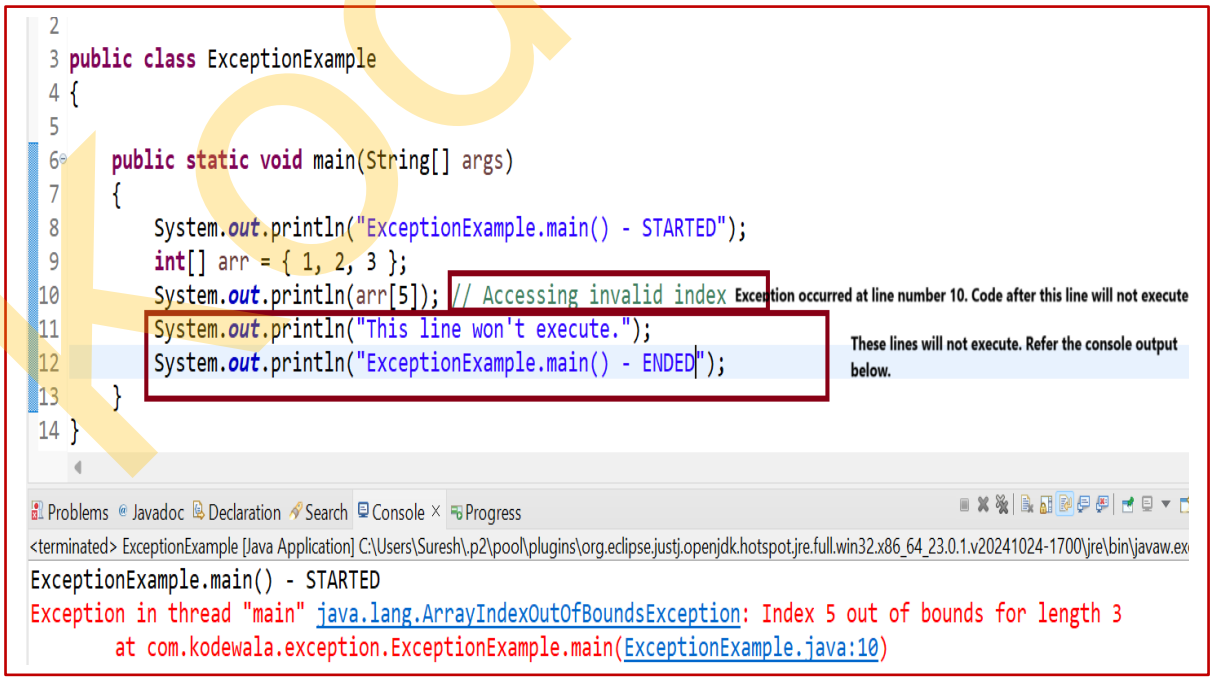
What is Exception?

An Exception is an unexpected situation that happens while a program is running. It stops the normal flow of code. This can happen due to following reason:

- ✓ Accessing an invalid index from an array. This throws **Array Index Out Of Bound Exception**.
- ✓ Dividing by zero. This will throw **Arithmetic Exception**.
- ✓ Trying to open a file that does not exist. This will throw **File Not Found Exception**

An Exception in Java is an error that happens when something goes wrong while running the program

Let us understand **exceptions** and how they affect program flow with a simple **Array Index Out of Bound Exception** example.



Kodewala

Note: When an exception occurs and is not handled, the program stops immediately, and any code after the error will not execute.

How do we handle the Exception?

Exception handling in Java helps manage exceptions during program execution and keeps the application running smoothly. Common exceptions include:

- ✓ **ClassNotFoundException**
- ✓ **IOException**
- ✓ **SQLException**
- ✓ **RemoteException**

Handling these exception helps developers build strong and reliable applications.

Let's handle the **ArrayIndexBoundException**

```
2
3  public class ExceptionExample
4  {
5
6      public static void main(String[] args)
7      {
8          int[] arr = { 5, 12, 33, 87 };
9
10         try
11         {
12             System.out.println(arr[5]); // Accessing invalid index
13         }
14
15         catch (ArrayIndexOutOfBoundsException e)
16         {
17             System.out.println("Invalid index! Array size is only " + arr.length);
18         }
19
20         System.out.println("Program continues after handling the exception.");
21     }
22 }
23
```

Exception thrown here

Exception Caught here and handled

This line will get executed after the exception caught. This means program will not terminate(exception handled) and continue running

Explanation:

- ✓ **try block -** Contains the code that might cause an error. In above example, it will case ArrayIndexOutOfBoundsException.
- ✓ **catch block -** Catches the exception if an invalid array index is accessed and prints an error message.
- ✓ **After handling the exception**, the program continues executing smoothly without crashing.

Note: Exception handling helps detect and manage errors smoothly, allowing the program to continue running normally.

Kodewala

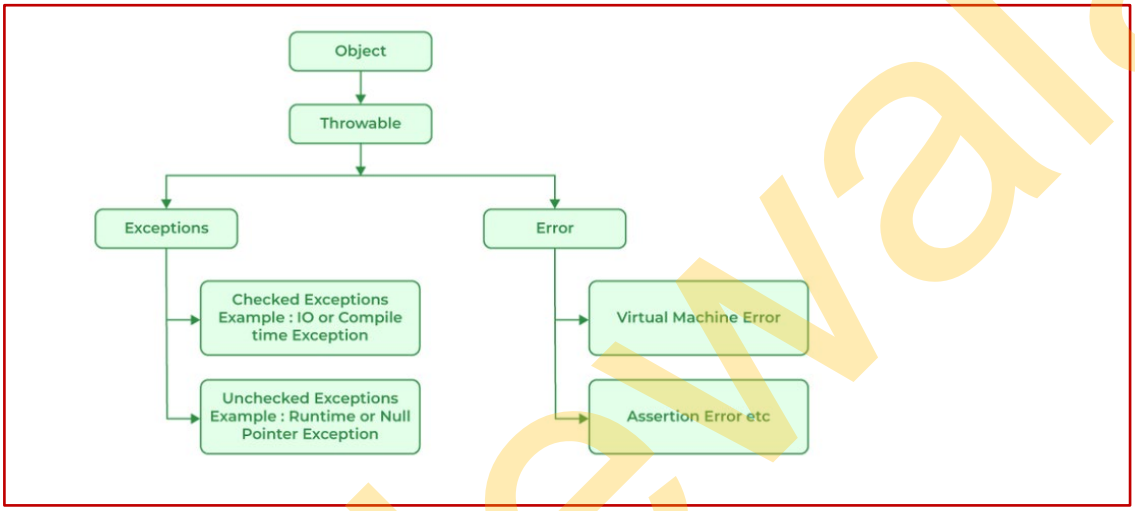
Note: The **finally** block always runs, whether an exception occurs or not, making it useful for cleanup tasks.

Exception Class Hierarchy

All exception and error types in Java inherit from the **Throwable** class. It has two branches:

- ✓ **Exception** - For errors that programs should handle, e.g., **NullPointerException**.
- ✓ **Error** - For critical errors related to the Java runtime, e.g., **StackOverflowError**.

Here is a simple representation of the **exception hierarchy** in Java:



Main Causes of Exceptions in Java:

Exceptions in Java can occur due to various reasons, such as:

- ✓ Invalid user input
- ✓ Network connection loss
- ✓ Physical limitations (like out-of-disk space or memory)
- ✓ Code errors
- ✓ Array index out of bounds
- ✓ Null reference access
- ✓ Type mismatch
- ✓ Attempting to open an unavailable file
- ✓ Database errors
- ✓ Arithmetic errors (like division by zero)

Kodewala

What is Error?

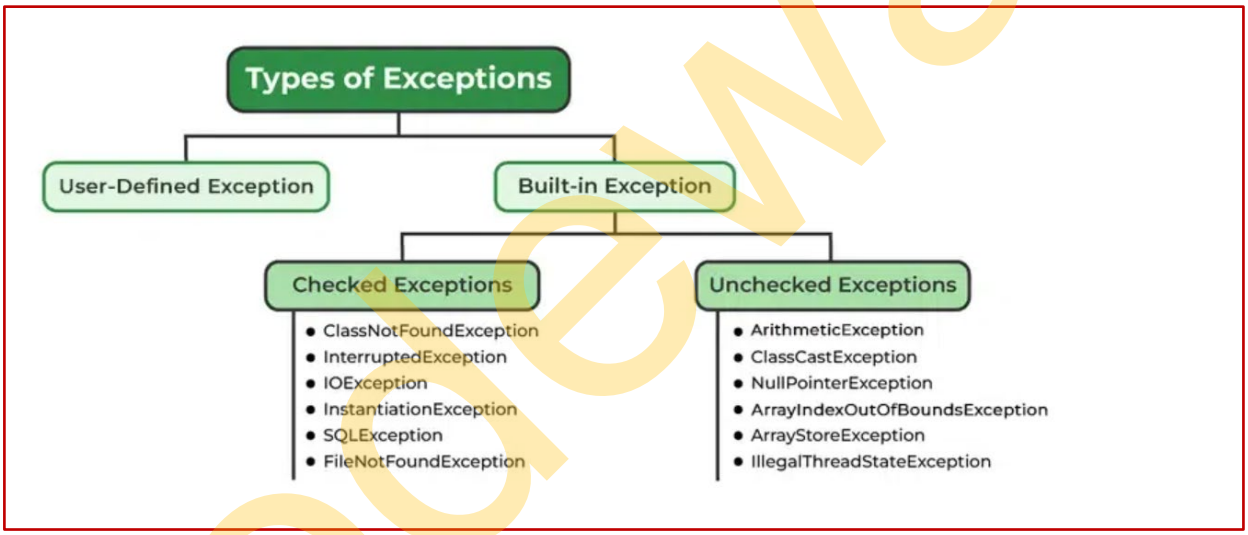
Errors in Java are irrecoverable issues, such as:

- ✓ JVM running out of memory
- ✓ Memory leaks
- ✓ Stack overflow
- ✓ Library incompatibility
- ✓ Infinite recursion

These errors are usually beyond the developer's control and should not be handled.

Types of Java Exceptions

Java provides several types of exceptions for its class libraries and also allows users to create custom exceptions.



Exceptions in Java are divided into two parts:

- ✓ **Built-in Exceptions**
    1. Checked Exceptions
    2. Unchecked Exceptions
- ✓ **User-Defined Exceptions / Custom Exception**

Built-in Exception - Build-in Exception are pre-defined exception classes provided by Java to handle common errors during program execution.

1. **Checked Exception -** These exceptions are known by the compiler. Checked exceptions are known as compile-time exceptions because they are checked by the compiler during compilation. Examples include:

Kodewala

- ✓ **ClassNotFoundException** - Thrown when a class is not found at runtime due to an incorrect location or missing file.
- ✓ **IOException** - Thrown when an input/output operation fails.
- ✓ **SQLException** - Thrown when a database error occurs.
- ✓ **FileNotFoundException** - Thrown when trying to open a file that doesn't exist.

2. **Unchecked Exceptions:** Unchecked exceptions are the opposite of checked exceptions. The compiler does not check these exceptions at compile time. Even if a program throws an unchecked exception and we don't handle it, there will be no compilation error. Examples include:

- ✓ **ArithmeticException -** Thrown for illegal math operations (e.g., division by zero).
- ✓ **ClassCastException -** Thrown when trying to cast an object to an incompatible class.
- ✓ **NullPointerException -** Thrown when accessing methods or fields of a null object.
- ✓ **ArrayIndexOutOfBoundsException -** Thrown when accessing an array with an invalid index.

User-Defined Exception / Custom Exception

Java allows us to create our own exceptions by extending the Exception class. This is known as a **custom exception or user-defined exception**. Custom exceptions help handle errors based on user needs. In simple terms, a custom exception is created by defining your own exception class and throwing it using the throw keyword.

```
class UserAlreadyExists extends Exception
{
    public UserAlreadyExists(String message)
    {
        super(message);
    }
}

public class ExceptionExample
{
    public static void main(String[] args)
    {
        try
        {
            String user = "Kodewala";
            if (user.equals("Kodewala"))
            {
                throw new UserAlreadyExists("User already exists in the system.");
            }
        } catch (UserAlreadyExists e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

User Defined Exception

Kodewala

Rules for Creating Custom Exceptions in Java:

- ✓ **Extend Exception Class -** Create a class that extends Exception or RuntimeException.
- ✓ **Constructor with Message -** Define a constructor to pass a custom error message to the parent class.
- ✓ **Throw Custom Exception -** Use the throw keyword to throw the custom exception.
- ✓ **Handle Exception -** Use try-catch to handle the custom exception.

Why Use Custom Exceptions in Java?

- ✓ To handle application-specific errors.
- ✓ To provide clear and descriptive error messages for easier debugging.
- ✓ To represent business logic errors in a meaningful way

Try-Catch Block

In Java, a try-catch block is used to handle exceptions. The try block contains code that may throw an exception, and the catch block handles the exception if it occurs.

```
try {
// Code that may throw an exception
}
 catch (ExceptionType e)
 {
// Code to handle the exception
}
```

finally Block

The finally block is used to run important code, whether an exception occurs or not.

Note: The finally block always executes after the try-catch block and is often used for tasks like closing database connections and closing files.

```
try {
// Code that may throw an exception
} catch (ExceptionType e) {
// Code to handle the exception
} finally
{
// cleanup code
}
```

Kodewala

Handling Multiple Exception

We can handle multiple type of exceptions in Java by using multiple catch blocks, each catching a different type of exception.

```
try {
// Code that may throw an exception
} catch (ArithmeticException e)
 {
// Code to handle the exception
} catch(ArrayIndexOutOfBoundsException e)
{
//Code to handle the anothert exception
}catch(NumberFormatException e)
{
//Code to handle the anothert exception
}
```

How Programmer Handle an Exception?

Java uses **five keywords** for exception handling: **try, catch, throw, throws, and finally.**

- ✓ **try -** Contains code that may cause an exception.
- ✓ **catch -** Handles the exception if it occurs.
- ✓ **throw -** Manually throws an exception.
- ✓ **throws** - Declares exceptions that a method can throw.
- ✓ **finally -** Runs code after try, whether an exception occurs or not.

Advantages of Exception Handling

- ✓ Ensures complete program execution.
- ✓ Separates program logic from error-handling code.
- ✓ Propagates errors effectively.
- ✓ Provides meaningful error messages.
- ✓ Identifies different types of errors.

kodewala

Kodewala Academy

Interview Question:

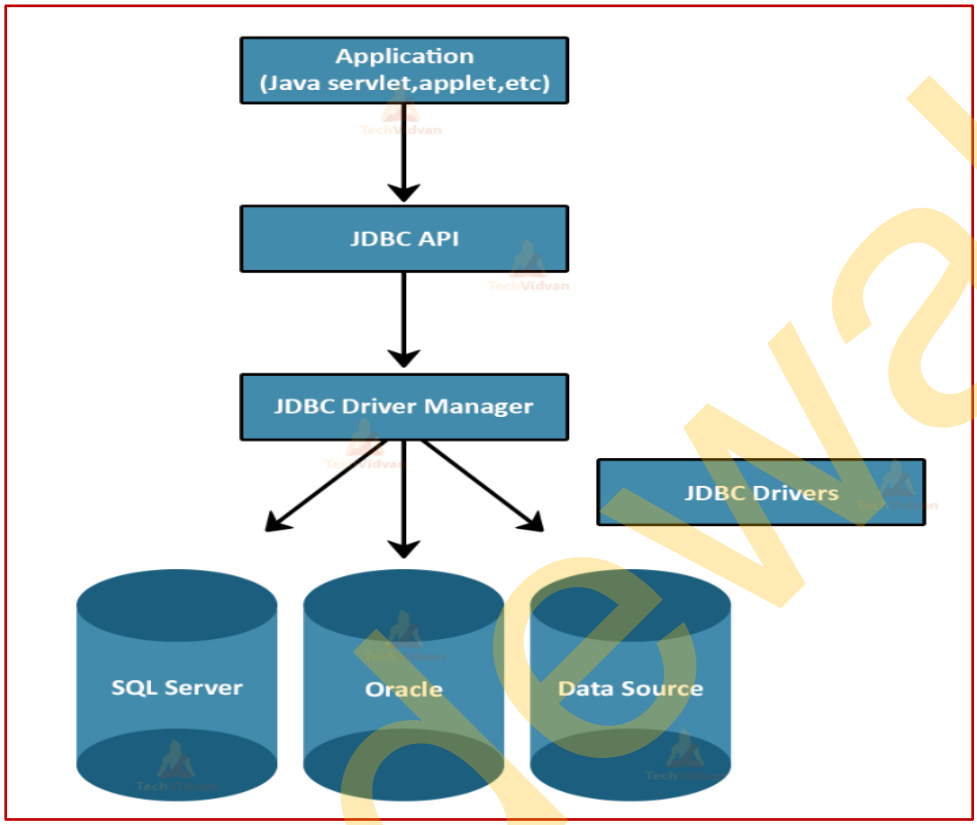
1. What is an exception in Java?
2. How is an exception different from an error?
3. What is the difference between checked and unchecked exceptions?
4. Can we handle errors in Java?
5. What is the difference between throw and throws?
6. What is the purpose of the finally block?
7. Can a finally block be skipped?
8. When does the finally block not execute?
9. Is it mandatory to have a catch block after try?
10. Can we have multiple catch blocks for a single try?
11. What happens if an exception is not caught?
12. What is the base class for all exceptions?
13. Can we throw multiple exceptions in Java?
14. What is a custom exception? How do you create one?
15. What is the difference between final, finally, and finalize?
16. Can we have a try block without a catch block?
17. What happens if an exception is thrown in the finally block?
18. What is the use of the throws keyword in Java?
19. Can we rethrow an exception in Java?
20. How do you handle multiple exceptions in a single catch block?

Kodewala

JDBC (Java Database Connectivity)

JDBC (Java Database Connectivity) is a Java API that allows applications to communicate with databases. It enables Java programs to connect to a database, run queries, and manage data. With a standard interface, JDBC ensures that Java applications can work seamlessly with various relational databases such as MySQL, Oracle, PostgreSQL, and others.

Let's understand the **JDBC Architecture**



Explanation:

1. **Application**: A Java application(Web, Applet etc) that connects and interacts with a database.
2. **JDBC API**: Allows Java programs to run SQL queries and get results. Key interfaces include Driver, ResultSet, RowSet, PreparedStatement, and Connection. Important classes include DriverManager, Types, Blob, and Clob.
3. **DriverManager**: Manages database connections using specific drivers to link applications with databases.
4. **JDBC Drivers**: These drivers facilitate communication between the application and the database.

Kodewala

The **JDBC architecture** uses two models to access a database:

1. **Two-Tier Architecture:**
   The application directly communicates with the database using JDBC.

   Client Application (Java) → JDBC Driver → Database (MySql, Oracle etc)

2. **Three-Tier Architecture:**
   The application interacts with a middle layer (like a server), which then communicates with the database.

   Client Application (Java) → Application Server → JDBC Driver → Database (MySql, Oracle etc)

JDBC Components

JDBC uses 4 main components to interact with a database:

1. **JDBC API:** Provides methods and interfaces for database communication. It includes:
   - ✓ **java.sql**: Part of Java SE, it contains core interfaces and classes for connecting to databases, running queries, and handling results.
   - ✓ **javax.sql**: Part of Java EE, it extends java.sql with features like connection pooling and data source management.
2. **JDBC Driver Manager:** Loads the appropriate database driver and manages database connections, ensuring the correct driver processes user requests.
3. **JDBC Test Suite:** Tests operations like insertion, deletion, and updating performed by JDBC drivers.
4. **JDBC Drivers:** These are client-side adapters that convert Java requests into a protocol the database understands. There are 4 types:
   - ✓ Type-1: JDBC-ODBC Bridge Driver
   - ✓ Type-2: Native-API Driver (partially Java)
   - ✓ Type-3: Network Protocol Driver (fully Java)
   - ✓ Type-4: Thin Driver (fully Java, deprecated since Java 8) – Modern applications commonly use Type-4 drivers.

Kodewala

JDBC Classes and Interfaces

1. Driver Manager: Manages JDBC drivers and establishes connections.

2. Connection: Represents a session with a database.

3. Statement: Executes static SQL queries.

4. PreparedStatement: Precompiled SQL statement for dynamic queries with parameters.

5. CallableStatement: Executes stored procedures in the database.

6. ResultSet: Represents query results and allows navigation through rows.

7. SQLException: Handles SQL-related exceptions.

Steps to Connect to a MySQL Database Using JDBC

Step 1: Load the JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver"
```

Step 2: Establish a Connection

```
Connection connection = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/database_name",
"DB_username",
"DB_password");
```

Step 3: Create a Statement

```
Statement statement = connection.createStatement();
```

Step 4: Execute a Query

```
String query = "INSERT INTO students (id, name) VALUES (101,
'Kodewala')";
int rowsAffected = statement.executeUpdate(query);
System.out.println("Rows affected: " + rowsAffected);
```

Step 5: Close the Connection

```
statement.close();
connection.close();
```

Kodewala

Let's create sample JDBC application and understand the flow end to end.

Create a Simple JDBC Application

Prerequisite:

1. **Before connecting to MySQL**, ensure that **MySQL Server** is installed on your system.
   You can download the installer from: https://dev.mysql.com/downloads/installer/

2. **Connect to mysql server from cmd.**

```
C:\Users\Suresh>mysql -u root -p
Enter password: **********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

3. **Show the databases**

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| kodewala           |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)
```

Kodewala

4. **Create a new database**

```
mysql> create database demo;
Query OK, 1 row affected (0.02 sec)

mysql>
```

5. **Use the database**

```
mysql> use demo;
Database changed
mysql>
```

6. **Create new table**

```
mysql> CREATE TABLE students (
    ->     id INT NOT NULL AUTO_INCREMENT,
    ->     name VARCHAR(100) NOT NULL,
    ->     address VARCHAR(255),
    ->     PRIMARY KEY (id)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
```

7. **Describe the newly created table**

```
mysql> describe students;
+---------+--------------+------+-----+---------+----------------+
| Field   | Type         | Null | Key | Default | Extra          |
+---------+--------------+------+-----+---------+----------------+
| id      | int          | NO   | PRI | NULL    | auto_increment |
| name    | varchar(100) | NO   |     | NULL    |                |
| address | varchar(255) | YES  |     | NULL    |                |
+---------+--------------+------+-----+---------+----------------+
3 rows in set (0.00 sec)
```

Now we will be using “demo” database and “students” table to insert the records using Java JDBC application.

Kodewala

```java
public class MyFirstJDBCApplication
{
    public static void main(String[] args)
    {
        String url = "jdbc:mysql://localhost:3306/demo"; // Database URL, username, and password
        String username = "root";
        String password = "Test@12345";
        // Query to insert data into the students table
        String query = "INSERT INTO students (id, name, address) VALUES (102, 'Academy', 'India')";
        try
        {
            Class.forName("com.mysql.cj.jdbc.Driver"); // Step #1 : Load MySQL Type-4 Driver
            Connection c = DriverManager.getConnection(url, username, password); // Step #2 : Establish connection
            Statement st = c.createStatement(); // Step #3 : Create a statement
            int count = st.executeUpdate(query); // Step #3 : Execute the query
            System.out.println("Number of rows affected by this query: " + count);
            st.close(); // Close the connection
            c.close();
            System.out.println("Connection closed.");
        } catch (ClassNotFoundException e)
        {
            System.err.println("JDBC Driver not found: " + e.getMessage());
        } catch (SQLException e)
        {
            System.err.println("SQL Error: " + e.getMessage());
        }
    }
}
```

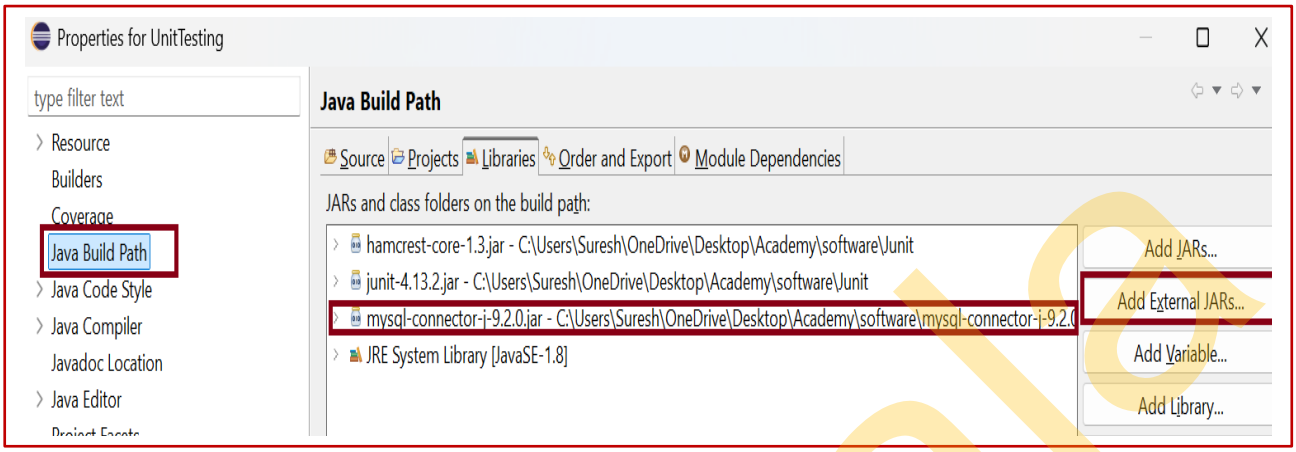
Explanation

1. **Import Required Packages:** Imports java.sql.* to use JDBC classes.
2. **Class and Main Method:** Defines the class with the main() method as the starting point.
3. **Database Connection Details:** Defines url, username, and password to connect to the MySQL database.
4. **SQL Query:** Inserts a record into the students table with id, name, and address.
5. **Load JDBC Driver:** Loads the MySQL Type-4 Driver using Class.forName().
6. **Establish Connection:** Establishes a connection using DriverManager.getConnection().
7. **Create Statement:** Creates a Statement object to run the SQL query.
8. **Execute Query:** executeUpdate() inserts data and returns the number of rows affected.
9. **Close Connection:** Closes the Statement and Connection to release resources.
10. **Exception Handling:**
    - ✓ Catches ClassNotFoundException if the driver is not found.
    - ✓ Catches SQLException for database connection or query errors.

Kodewala

Kodewala Academy

Note: Download mysql-connector driver from https://dev.mysql.com/downloads/connector/j/ and add an external jar.



Now run the program and check the data in mysql table(students)

```
mysql> select * from students;
+-----+----------+---------+
| id  | name     | address |
+-----+----------+---------+
| 101 | Kodewala | India   |
| 102 | Academy  | India   |
+-----+----------+---------+
2 rows in set (0.00 sec)
```

Interview Experience:

1. Explain the different types of JDBC drivers.
2. What is the difference between Statement and PreparedStatement?
3. What is a Connection in JDBC?
4. What is the DriverManager class used for in JDBC?
5. What is a ResultSet in JDBC?
6. What is the difference between executeQuery() and executeUpdate()?
7. What is the purpose of the execute() method in JDBC?
8. What is autocommit in JDBC?

Kodewala

9. How do you handle SQL exceptions in JDBC?
10. What is the difference between Connection.close() and Statement.close() in JDBC?
11. Explain how to update records in a database using JDBC.
12. What is the PreparedStatement used for in JDBC?
13. What are batch processing and how does JDBC support it?
14. What is the purpose of the CallableStatement interface in JDBC?
15. Explain how to establish a database connection using JDBC.
16. How do you retrieve the generated keys after executing an insert query in JDBC?
17. What is the difference between executeUpdate() and execute()?
18. What is ResultSetMetaData in JDBC?
19. What is the Connection interface used for in JDBC
20. What is a BatchUpdateException in JDBC, and how do you handle it?
21. How can you handle transactions in JDBC? Explain with an example.
22. What is connection pooling in JDBC, and why is it important?
23. What are the differences between Statement, PreparedStatement, and CallableStatement in terms of performance and usage?
24. How do you prevent SQL injection attacks when using JDBC?
25. Explain the concept of "two-phase commit" in JDBC and its use cases.
26. How does JDBC handle large data types such as BLOB and CLOB? Explain the methods to read and write them.
27. What are the performance considerations when using JDBC for database operations, and how do you optimize query performance?

Kodewala

Kodewala Academy

Transforming code into career

Core Java Week – 7

(Collection Framework)

Kodewala

Collection Framework

Any group of individual objects that are represented as a single unit is known as **a Java Collection** of **Objects**. In Java, a separate framework named the "Collection Framework" has been introduced in JDK 1.2 which holds all the Java Collection Classes and Interface in it.

The two main root interfaces in collection framework are:

- ✓ **Collection interface** (java.util.Collection)
- ✓ **Map interface** (java.util.Map)

What is a Framework? A framework is a collection of classes and interfaces that provides a ready-made architecture. When implementing a new feature or class, defining a framework is not required. However, a good object-oriented design usually includes a framework with related classes, where all the classes perform similar tasks.

Why Java Needed a Collection Framework?

Before the **Collection Framework** (before JDK 1.2), Java used **Arrays, Vectors,** and **Hashtables** to group objects. However, these collections did not share a common interface, so each one had a different way of working. This made it hard for users to remember all the methods, syntax, and constructors for each collection.

Let's understand this with an example of adding an element in a **hashtable** and a **vector**.

```
import java.util.*;

public class CollectionExp
{

    public static void main(String[] args)
    {
        // Creating instances of the array, vector, and hashtable
        int arr[] = new int[] { 1, 2, 3, 4 };
        Vector<Integer> v = new Vector<>();
        Hashtable<Integer, String> h = new Hashtable<>();

        // Adding elements into the vector
        v.add(1);
        v.add(2);

        // Adding elements into the hashtable
        h.put(1, "Kodewala");
        h.put(2, "Academy");

        // Accessing the first element of the array, vector, and hashtable
        System.out.println(arr[0]); // Array access
        System.out.println(v.get(0)); // Vector access
        System.out.println(h.get(1)); // Hashtable access
    }
}
```

Kodewala

As we can see, none of these collections (Array, Vector, or Hashtable) shared a standard way to access elements, making it difficult for programmers to write code that work with all types of collections.

Another issue was that many methods in the Vector class were final, preventing developers from extending it to create similar collections. To solve these problems, Java introduced the **Collection Framework** in JDK 1.2, and legacy collections like Vector and Hashtable were updated to follow this framework.

Advantages of the Collection Framework?

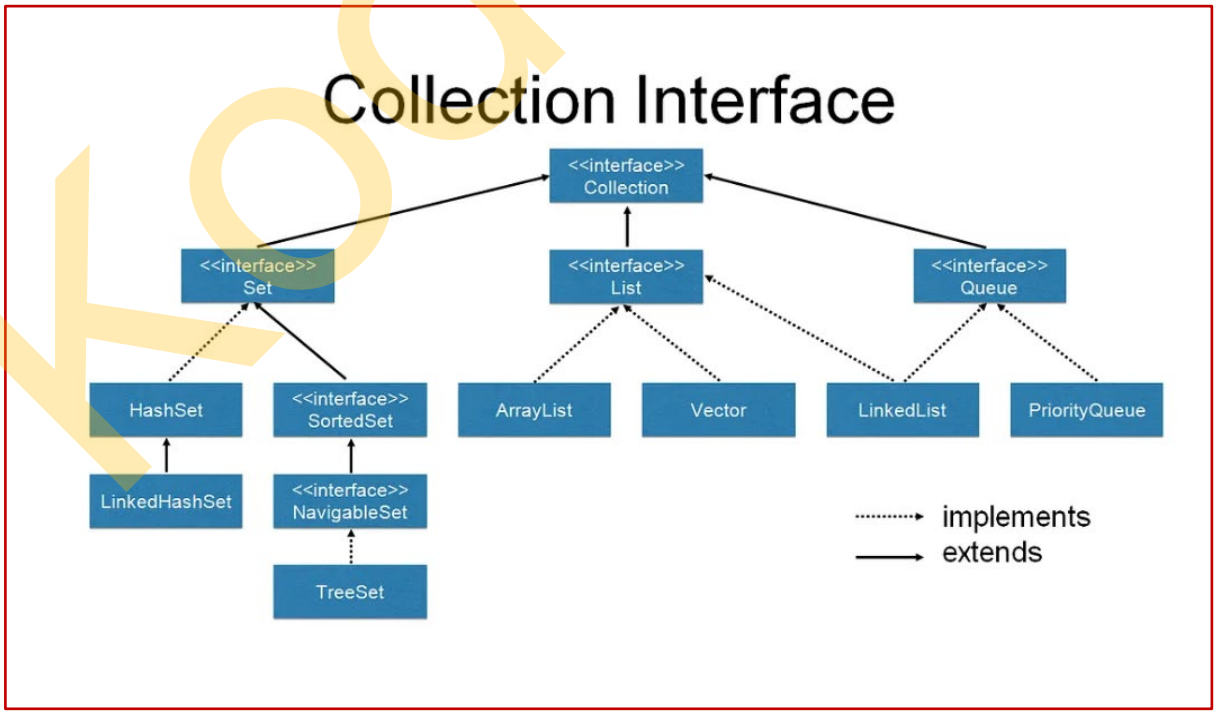
Consistent API: The API provides a set of basic interfaces like **Collection**, **Set**, **List**, and **Map**. All classes that implement these interfaces (like ArrayList, LinkedList, Vector, etc.) share a common set of methods.

Reduces Programming Effort: Programmers don't need to worry about designing the Collection.

Increases Program Speed and Quality: The Collection Framework offers high-performance implementations of data structures and algorithms, allowing programmers to use the best implementation for boosting the performance of their code.

Hierarchy of the Collection Framework

The "java.util" package includes all the classes and interfaces for the **Collection Framework**. It has an **Iterable interface** that provides an iterator to go through collections. The main **Collection** interface extends **Iterable** and acts as the root of the collection framework. All collections extend this interface, inheriting its iterator and methods. The hierarchy of the collection framework is shown in the figure below.



Kodewala

The Collection Framework has several interfaces, each designed to store a specific type of data. Here are the main interfaces in the framework.

1. Iterable Interface

This is the root interface of the entire **Collection Framework**. The **Collection** interface extends **Iterable**, so all other interfaces and classes in the framework also implement it. It's main purpose is to provide an iterator(looping) for collections. It has only one abstract method, iterator(), which returns an iterator.

```
Iterator iterator();
```

2. Collection Interface

This interface extends the **Iterable** interface and is implemented by all classes in the **Collection Framework**. It provides the basic methods which are common to all collections, such as adding, removing, and clearing elements. These methods are defined here to ensure consistency across all collection classes, regardless of how they work internally. In short, the "**Collection**" interface acts as the foundation for all collection classes.

Note: "**Collection**" is an interface (as discussed above), whereas "**Collections**" is a utility class, which we will cover later. This is a common point of confusion and is sometimes asked in interviews.

3. List Interface

The List interface is a child (sub class) of the Collection interface. It is used to store **ordered** collections of objects and **allows duplicate elements**. This interface is implemented by classes like **ArrayList**, **LinkedList**, **Vector**, and **Stack**. Since all these classes implement the List interface, we can create a **List** object using any of them.

Use Case: In a classroom, you might store student names in the order they registered, and some students might have the same name (Duplicate).

```
List <T> arrayList  = new ArrayList<> ();
List <T> linkedList = new LinkedList<> ();


Where T is the type of the object. Like String, Person, Account
```

The following classes implement the **List** interface:

ArrayList: ArrayList provides dynamic arrays in Java. Although it may be slower than traditional arrays, it is useful when frequent manipulation of array elements is needed. The size of an **ArrayList** automatically adjusts as elements are added or removed. You can randomly access elements in the list. However, **ArrayList** does not support primitive types (e.g., int, char). You must use their wrapper classes (e.g., Integer, Character) for such cases.

kodewala

Let's understand the **ArrayList** with the below code example:

```
public class ArrayListExample
{
    public static void main(String[] args)
    {
        ArrayList<String> products = new ArrayList<>(); // Creating an ArrayList of electronic products

        // Adding elements
        products.add("Laptop");
        products.add("Smartphone");
        products.add("Headphones");

        System.out.println("Electronic Products: " + products); // Printing the list

        System.out.println("First product: " + products.get(0)); // Accessing elements by index

        products.remove("Smartphone");      // Removing an element
        System.out.println("After removing Smartphone: " + products);

        System.out.println("Total products: " + products.size()); // Checking size
    }
}
```

How does ArrayList works internally?

The **ArrayList** in Java is a part of the **java.util** package and is backed by a dynamic array (i.e., an array that can grow as needed)

Backed by Array:

- Internally, ArrayList uses an array of Object[] to store elements.

Default Capacity:

- When you create an **ArrayList** with new ArrayList(), it initializes with default capacity of 10.
- You can also set initial capacity using: new ArrayList<>**(20);**

Dynamic Resizing:

- When the internal array is full and you add more elements, ArrayList grows automatically.
- When you create an **ArrayList** with new ArrayList(), it initializes with default capacity of 10.
- It creates a new array with **increased capacity**, usually 1.5 times the old size (newCapacity = oldCapacity + oldCapacity / 2), and copies the old elements to the new array.

Index-Based Access:

- Since it's an array under the hood, it allows fast access by index (constant time O(1)).

Shifting on Insert/Remove:

- When you insert or remove elements (not at the end), all elements after that index are shifted towards right, which can be costly (linear time O(n)).

Kodewala

Here is summary:

Feature	Behavior
Storage	`Object[]` array
Default capacity	10
Resizing strategy	Grows 1.5x when full
Access time	`O(1)`
Insertion/Removal time	`O(n)` (in worst case due to shifting)

ArrayList Time Complexity (with Explanation) → Very Important (Interview point of view)

Operation	Time Complexity	Reason
`get(index)` / `set(index, value)`	`O(1)`	Direct index access using array
`add(E e)` (end)	`O(1)`	Fast unless internal resizing occurs
`add(index, E e)`	`O(n)`	Shifts elements right after index
`remove(index)`	`O(n)`	Shifts elements left after index
`contains()` / `indexOf()`	`O(n)`	Linear search

When to Use ArrayList?

Use when:

- You need fast random access.
- You mostly add/remove at the end.
- Insertion order is important.
- Duplicate allowed.

Avoid When: You frequently add/remove in the middle or beginning → prefer **LinkedList**.

Kodewala

LinkedList: A **LinkedList** is a data structure where elements are stored as a **chain of nodes**. Each node has two parts - one to store the **data**, and another to store the **address of the next node**. So, the elements are **not stored in continuous memory** like arrays.

LinkedList is useful when we need to **frequently add or remove elements**.

Let's understand the **LinkedList** with the below code example:

```
public class LinkedListExample
{
    public static void main(String[] args)
    {
        LinkedList<String> electronics = new LinkedList<>(); // Creating a LinkedList of electronic items

        electronics.add("Laptop");          // Adding items to the LinkedList
        electronics.add("Smartphone");
        electronics.add("Tablet");
        electronics.add("Smartwatch");

        System.out.println("Electronic Products: " + electronics);      // Displaying the LinkedList

        electronics.addFirst("TV");         // Adding item at the beginning and end
        electronics.addLast("Camera");

        electronics.remove("Tablet");       // Removing an item

        System.out.println("Updated Products: " + electronics);         // Display updated list
    }
}
```

How does LinkedList work internally?

The LinkedList in Java is a part of the java.util package and is implemented as a doubly linked list.

Node-Based Structure

- Internally, it uses nodes instead of arrays.
- Each node contains:
    1) `data` - the value (TV, Camera, Tablet).
    2) `next` - address to the next node.
    3) `prev` - address to the previous node (since it's a doubly linked list)

Dynamic Size

- No fixed capacity.
- Grows or shrinks dynamically as elements are added or removed.

No Index-Based Storage

- Unlike ArrayList, it does not store elements in continuous memory.
- No direct access by index - access requires traversal from head/tail.
- Hence, getting an element at a specific index is O(n).

Kodewala

Efficient Insertions/Deletions

- Adding/removing from the start or end is fast - only address are updated (O(1)).
- Inserting/removing in the middle requires traversal but still avoids shifting.

Access (Search) Time Slower Than ArrayList

- Since there is no indexing, random access is slower (O(n)).
- Best used when frequent insertions/deletions are needed.

Here is summary:

Operation	Time Complexity	Reason
Add/Remove at Start/End	`O(1)`	Direct pointer update (no traversal or shifting)
Add/Remove in Middle	`O(n)`	Requires traversal to the index
Get element by index	`O(n)`	No direct index access; needs traversal
Search element	`O(n)`	Linear search through nodes

If both ArrayList and LinkedList take O(n) for insertion in the middle, why is LinkedList sometimes preferred?

Feature	`ArrayList`	`LinkedList`
Insert in middle (time)	`O(n)` (due to **shifting**)	`O(n)` (due to **traversal**)
What causes the cost?	Shift elements after the insert index	Traverse to index, then just re-link
Insert once reached index	Shift + insert (`O(n)`)	Only pointer change (`O(1)` once there)
Insert at head	`O(n)` (needs shifting)	`O(1)`
Insert at tail	`O(1)` *(amortized)*	`O(1)`

So, why prefer LinkedList?

Frequent insertions/deletions at start/middle → LinkedList is better, because:

- It **avoids shifting** elements.
- It **updates links**, which is faster once position is found.

If you're doing **lots of random access** → ArrayList is better, because:

- It provides **O(1) index access**, while LinkedList needs traversal (O(n)).

Kodewala

In short:

LinkedList is better when:

- Frequent insert/delete operations (especially at head/middle).
- You don't need random access often.

ArrayList is better when:

- You access elements often by index.
- You insert/remove mostly at the end.

Common Misunderstanding: *LinkedList has O(1) time for random insertion.*

Reality:

- LinkedList insertion is O(1) only after reaching the node.
- But to reach the node at a specific index, it must traverse the list, which takes O(n) time.

Let's breakdown and understand it.

- **Traverse to the index** → O(n)
- **Insert by changing pointers** → O(1)

So overall:

- **Random insertion in LinkedList = O(n)** due to traversal
- Only **insertion at head/tail** is truly O(1)

Conclusion: The **LinkedList** is a type of list where elements are connected using links (nodes), not stored in continuous memory like arrays.

It is good for adding or removing elements from the beginning or end because it doesn't need to shift data. But it is not the best choice if you need to access elements by index often, since it has to go through the list one by one. Overall, it is useful when you work with lots of insertions and deletions.

4. Set Interface

A **Set** is a collection that **does not allow duplicate** elements and is typically **unordered**. It is useful when you want to store only unique objects and avoid duplication.

The Set interface is implemented by several classes such as **HashSet**, **LinkedHashSet**, and **TreeSet**. Since all these classes implement the Set interface, you can create a Set instance using any of them depending on your requirements.

```
Set<T> hashSet = new HashSet<> ();
Set<T> linkedHashSet = new LinkedHashSet<> ();
Set<T> treeSet = new TreeSet<> ();


Where T is the type of the object. Like Person, String, Employee, Account etc
```

Kodewala

The following are the classes that implement the Set interface:

HashSet: The **HashSet** class provides an implementation of a hash table. It does not maintain the order of insertion, as elements are stored based on their hash codes (formula used to find the bucket location). HashSet also allows one null element.

Let's understand HashSet with an example:

```
public class HashSetDemo
{

    public static void main(String args[])
    {
        HashSet<String> products = new HashSet<String>(); // Creating a HashSet to store product names

        products.add("Laptop"); // Adding electronic products to the HashSet
        products.add("Smartphone");
        products.add("Tablet");
        products.add("Smartphone"); // Duplicate, will not be added
        products.add("Headphones");
        products.add(null); // HashSet allows one null value

        Iterator<String> itr = products.iterator(); // Iterating and displaying the unique product name
        while (itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Run the program and observe the output and notice how **duplicate entries** like **"Smartphone"** are ignored, and the elements are printed in an **unordered** manner due to the nature of HashSet.

Why Elements are unordered?

HashSet uses an array of buckets (also **called buckets array**) to store its elements. Each element is placed in a specific bucket based on its **hash code**, and the position of each element in the array is determined by the formula:

index = hashCode(key) & (capacity - 1)

Since elements are stored in these buckets based on their hash code, the order of insertion is not preserved, leading to an **unordered** collection.

Let's understand the **hashCode()** and **equals()** methods**.** These are part of Object class.

hashCode(): It generates a unique integer for each object, which is used to determine the **bucket** in which the object is stored in a collection like **HashSet**. Objects with the same **hash code** are placed in the same bucket, **but that doesn't mean they are equal. (In some cases JVM return same hashcode for two different objects.)**

equals(): Once two objects have the same **hash code** and are in the same bucket, **equals()** is used to compare the objects to check if they are **truly equal**.

Kodewala

Important Rules:

- ✓ If two objects are equal according to equals(), they must have the same hash code.
- ✓ If two objects have different hash codes, they are considered unequal.

Why is this important for HashSet?

- ✓ HashSet uses hashCode() to quickly narrow down the possible location of an object.
- ✓ If two objects have the same hash code, equals() is used to verify if they are duplicates (and thus should not be added).

Internal Working of HashSet?

1. **HashSet is backed by a HashMap**

   - ✓ Internally, HashSet uses a HashMap to store its elements.
   - ✓ When you add an element to a HashSet, it internally adds that element as a key in the HashMap with a constant dummy value (like PRESENT).

```
private transient HashMap<E,Object> map;

private static final Object PRESENT = new Object();
```

   When you call: set.add("apple");

   Internally, it does:

   map.put("A", PRESENT); Note: go through the internal implementation of **set.add()** method.

2. **Hashing Mechanism**
   When an element is added:
   - ✓ Its hashCode() is computed.
   - ✓ That hash code is processed (with some bitwise operations) to determine the bucket index.
   - ✓ The element is stored in that bucket (in a Node object).

```
int hash = hash(key.hashCode());

int bucketIndex = (n - 1) & hash;
```

This formula (n - 1) & hash is a bitwise AND operation used to calculate the index in the array (n is the capacity).

3. **Handling Collisions**

   - ✓ If two different objects have the same hash code, they go into the same bucket.
   - ✓ Then Java uses the equals() method to check if they are equal.
   - ✓ If equals() returns true, it won't be added (duplicate).
   - ✓ If not equal, it gets added in a linked list or a balanced tree (since Java 8) inside that bucket.

kodewala

4. **Load Factor and Rehashing**
   - ✓ Default capacity: 16
   - ✓ Default load factor: 0.75
   - ✓ When the number of **elements** > **capacity** * **load factor**, the capacity is doubled, and all elements are rehashed (i.e., redistributed into new buckets).

LinkedHashSet: A **LinkedHashSet** is quite similar to a HashSet, but the key difference is that it uses a doubly linked list to store the elements, which allows it to maintain the insertion order of the elements. The LinkedHashSet class is a **Set** implementation that **maintains insertion order**.
It is built using a combination of **HashSet** (for uniqueness and hashing) and a **doubly-linked list** (to maintain the order).

Let's understand it with an example.

```
public class LinkedHashSetExample{
    public static void main(String[] args){
        LinkedHashSet<String> electronicItems = new LinkedHashSet<>();
        electronicItems.add("Laptop");      // Add electronic items to the set
        electronicItems.add("Smartphone");
        electronicItems.add("Smartwatch");
        electronicItems.add("Smartphone"); // Duplicate, won't be added
        System.out.println("Electronic Items are :");// Display the items(insertion order will be preserved)
        for (String item : electronicItems)
        {
            System.out.println(item);
        }
        // Checking the contents of the LinkedHashSet
        System.out.println("\nDoes the set contain 'Laptop'? " + electronicItems.contains("Laptop"));
        // Removing an item
        electronicItems.remove("Smartwatch");
        System.out.println("\nAfter removing 'Smartwatch':");
        for (String item : electronicItems)
        {
            System.out.println(item);
        }
    }
}
```

Observation:

- ✓ Duplicate "Smartphone" is ignored.
- ✓ Order is preserved (unlike HashSet).

Why Elements are Ordered?

- ✓ LinkedHashSet uses a LinkedHashMap internally to store its elements.
- ✓ The key in this map is the element added to the set.
- ✓ The linked list in the LinkedHashMap ensures that the insertion order is preserved.

Internal Working of LinkedHashSet

1. **Backed by LinkedHashMap**

   When you add an element to a LinkedHashSet, internally it does:

   map.put(element, PRESENT); // PRESENT is a dummy constant

kodewala

2. **Maintaining Order**

- ✓ LinkedHashMap maintains a doubly-linked list across all entries.
- ✓ So the order in which elements are added is maintained during iteration.

3. **Hashing Mechanism**

- ✓ Like HashSet, it calculates the hashCode() of the object.
- ✓ Applies the formula: (n - 1) & hash to find the bucket index.
- ✓ If multiple elements hash to the same bucket, collision is handled using chaining (linked list or tree after Java 8).

4. **equals() and hashCode()**

- ✓ hashCode() determines the bucket.
- ✓ equals() is used to check for actual object equality (to avoid duplicates).

Note:

- ✓ If two objects are equal as per equals(), they must have the same hash code.
- ✓ LinkedHashSet uses both hashCode() and equals() to detect duplicates.

TreeSet: A TreeSet is a Set implementation that stores elements in sorted (ascending) order. It uses a TreeMap internally to store its elements and guarantees no duplicates and sorted order.

Unlike HashSet and LinkedHashSet, which rely on hashing, TreeSet is backed by a Red-Black Tree (a self-balancing binary search tree). This allows log(n) time complexity for insertion, removal, and lookup.

Let's understand it with an example:

```
import java.util.TreeSet;

public class TreeSetExample
{
    public static void main(String[] args)
    {
        TreeSet<String> set = new TreeSet<>();
        set.add("Laptop");
        set.add("Smartphone");
        set.add("Tablet");
        set.add("Smartphone"); // duplicate will be removed.

        System.out.println(set); // Sorted Order
    }
}
```

Kodewala

- ✓ Duplicate “Smartphone” is ignored.
- ✓ Elements are **automatically sorted** in ascending order (unlike HashSet or LinkedHashSet).
- ✓ No null elements (throws NullPointerException if added).

Why Elements Are Ordered?

- ✓ TreeSet uses a TreeMap internally.
- ✓ The element is both the key and value in the internal TreeMap.
- ✓ TreeMap uses natural ordering (Comparable) or custom ordering (Comparator) to keep keys sorted. Comparable and Comparator will be covered in next session.
- ✓ So, TreeSet elements follow the same sorting behavior.

Internal Working of TreeSet:

1. **Backed by TreeMap:**

   When we add an element to a TreeSet, internally it does:

   ```
   map.put(“IPhone”, DUMMY_OBJECT);
   ```

   So, the element becomes the key in a TreeMap, and a dummy value is stored.

2. **Maintaining Sorted Order:**

- ✓ TreeMap maintains keys in ascending order by default.
- ✓ Sorting is based on the natural order (compareTo) or a provided Comparator.

3. **No Hashing Used:**

- ✓ Unlike HashSet/LinkedHashSet, it does not calculate hashCode().
- ✓ Instead, it uses compareTo() or compare() for positioning the elements in the tree.

4. **compareTo() vs equals():**

- ✓ TreeSet relies on compareTo() (or compare()) to check for duplicates.
- ✓ So two elements that return 0 from compareTo() are considered duplicates—even if equals() says otherwise.

Note:

- ✓ No duplicate elements are allowed.
- ✓ Null elements are not allowed (throws NullPointerException).
- ✓ Not thread-safe. Use Collections.synchronizedSortedSet() if thread safety is needed.
- ✓ All operations like add(), remove(), contains() take O(log n) time.
- ✓ Use TreeSet when sorted, unique elements are needed.

kodewala

How Can we iterate the collections (List and Set)?

1. **For-each Loop**

- ✓ Introduced in Java 5.
- ✓ Simplest and most readable way to iterate over all elements of a collection.
- ✓ Internally uses an Iterator behind the scenes.
- ✓ Works well when you just want to read each element (not modify the collection).

```
List<String> items = new ArrayList<String>();

items.add("Apple");
items.add("LG");
items.add("Moto");

for (String item : items)
{
    System.out.println(item);
}
```

2. **Using Iterator**

- ✓ Iterator is an interface in Java used to traverse collections.
- ✓ Offers more control than a for-each loop.

```
List<String> items = new ArrayList<String>();

items.add("Apple");
items.add("LG");
items.add("Moto");

Iterator<String> itr = items.iterator();
while (itr.hasNext())
{
    String fruit = itr.next();
    System.out.println(fruit);
}
```

5. **Map Interface:** A Map is a data structure used to store **key-value (K, V)** pairs.
It means you store a value, and you access it using a unique key.

- ✓ **Duplicate keys are not allowed**, because each key must point to only one value.
- ✓ **Duplicate values are allowed**, as different keys can point to the same value.

We use a map when we want to look up data based on a key, like finding a student's name by their ID.

Kodewala

The Map interface is implemented by classes like:

- ✓ HashMap (no order)
- ✓ LinkedHashMap (insertion order)
- ✓ TreeMap (sorted order)

We can create a Map like this:

```
Map<Integer, String> map = new HashMap<>();
```

HashMap: HashMap is a commonly used class that provides the basic implementation of the **Map** interface in Java. It stores data as **key-value pairs**, to get a value, you must know its **key**(Must Remember this)

- ✓ It uses a technique called **hashing** to make searching and indexing faster.
- ✓ **Hashing** means converting a large piece of data (like a String) into a smaller, fixed-size value called a **hash code**. This helps in quickly finding the right bucket to store or search data.
- ✓ Internally, even **HashSet** uses a **HashMap** to store its elements, the elements act as keys, and the values are dummy object.

Let's understand the HashMap with State(K) and Capitals(V) example.

```
import java.util.HashMap;
import java.util.Map;

public class StateCapitalMap
{
    public static void main(String[] args)
    {
        Map<String, String> stateCapital = new HashMap<>(); // Create a Map to store state and capital

        stateCapital.put("Maharashtra", "Mumbai"); // Add entries (state as key, capital as value)
        stateCapital.put("Karnataka", "Bengaluru");
        stateCapital.put("Tamil Nadu", "Chennai");
        stateCapital.put("Uttar Pradesh", "Lucknow");
        stateCapital.put("West Bengal", "Kolkata");

        for (Map.Entry<String, String> entry : stateCapital.entrySet()) // Print the map entries
        {
            System.out.println("State: " + entry.getKey() + ", Capital: " + entry.getValue());
        }
    }
}
```

Kodewala

Kodewala Academy

How does HashMap Works Internally / Can you Explain how put() method works?

When you put Key and Values into Map:

```
map.put("Apple", "iPhone11");
```

Here is what happens inside:

1. **hashCode() is Calculated**

   - ✓ Hashmap calls hashCode() on the key "Apple".
   - ✓ This gives a number (called hash code).

```
int hash = key.hashCode();

"Apple".hashCode(); → Some int number assume 24543534
```

2. **Index (Bucket) is Found**
   - ✓ The hash is converted into a bucket index using:

```
index = hash & (n - 1)

where n is the capacity of the array (bucket array).
```

3. **Node Entry is Placed in the Bucket**
   - ✓ If that bucket(Index) is empty → just add the new entry.
   - ✓ If not → check if any key in that bucket matches the new key using equals().
4. **Collision Handling**
   - ✓ If multiple keys lands to the same bucket (same index), it's called a collision.
   - ✓ If keys are same then it will overwrite the values.
   - ✓ If Keys are not same but returned same hash code, then new node will be added to existing LinkedList.
   - ✓ Internally Data Stored into Linked list (Java 7 and earlier).
   - ✓ Internally Data Stored into Tree structure (Red-Black Tree) if the number of entries in a bucket is more than 8 (Java 8+). This is called "treeify"

5. **Retrieving Values (map.get(key))**

   HashMap again uses:

   - ✓ hashCode() to find the bucket.
   - ✓ equals() to find the exact key if there are multiple Nodes/Entries due to collision.

Kodewala

LinkedHashMap: It maintains insertion order of key-value pairs using a doubly-linked list.

TreeMap: Stores key-value pairs in sorted order of keys using a Red-Black tree.

Here is the Quick Summary : Interview point of view.

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order	Maintains insertion order	Sorted by keys
Performance (Get/Put)	O(1)	O(1) (slightly slower than HashMap)	O(log n)
Underlying Structure	Hash table	Hash table + Doubly linked list	Red-Black tree
Null Keys/Values	1 null key, multiple null values	1 null key, multiple null values	No null keys (null values allowed)
Use Case	Fast access, no order needed	Fast access with predictable iteration order	Sorted map needed by keys

ConcurrentHashMap: It is **thread-safe (**only one thread can access at a time**)** version of HashMap from "java.util.concurrent". It allows **concurrent access** by multiple threads without locking the entire map.

In Java 8+, it uses **bucket-level locking** (via a synchronized block on segments), not the entire map.

Allows **high concurrency** with minimal blocking, multiple threads can read and write safely.

Null Not allowed - throws NullPointerException.

It is ideal for multi-threaded environments where high performance and data consistency are required, like caching, counters, etc.

Collections: The **Collections** class is a utility class in Java that consists of **static methods** for operating on or returning collections such as List, Set, Map.

Please Note: It is **different** from the **Collection** interface. **Collection** is the root interface of Java collections hierarchy, while **Collections** is a helper/utility class.

Kodewala

Commonly Used Methods from Collections class:

Method	Description
`Collections.sort(list)`	Sorts the list in natural (ascending) order.
`Collections.reverse(list)`	Reverses the order of elements in the list.
`Collections.shuffle(list)`	Randomly shuffles the list elements.
`Collections.max(collection)`	Returns the maximum element.
`Collections.min(collection)`	Returns the minimum element.
`Collections.synchronizedList(list)`	Returns a thread-safe (synchronized) list.
`Collections.unmodifiableList(list)`	Returns an unmodifiable view of the list.

Thread Safety Utilities: The Collections class provides methods to make collections thread-safe(only one thread can access the object/collection at a time):

- ✓ synchronizedList(List<T> list)
- ✓ synchronizedSet(Set<T> set)
- ✓ synchronizedMap(Map<K, V> map)

These methods return synchronized (thread-safe) versions of the specified collections.

Refer the code example below.

```
public class SynchronizedListExample
{

    public static void main(String[] args)
    {

        List<String> electronicsList = new ArrayList<>(); // Create a normal list of electronic items

        List<String> synchronizedElectronicsList = Collections.synchronizedList(electronicsList); // Wrap it with a
                                                                                                   // synchronized list
        synchronizedElectronicsList.add("Laptop"); // Adding items to the synchronized list
        synchronizedElectronicsList.add("Smartphone");
        synchronizedElectronicsList.add("Tablet");

        System.out.println("Synchronized List: " + synchronizedElectronicsList); // Display the synchronized list

        synchronized (synchronizedElectronicsList) // Iterating over the synchronized list (must use synchronized block)
        {
            for (String item : synchronizedElectronicsList)
            {
                System.out.println("List Item: " + item);
            }
        }
    }
}
```

kodewala

Immutable Collections: To create read-only collections:

- ✓ unmodifiableList(List<? extends T> list)
- ✓ unmodifiableSet(Set<? extends T> set)
- ✓ unmodifiableMap(Map<? extends K, ? extends V> map)

These methods return unmodifiable views of the specified collections, preventing any modifications.

```
public class UnmodifiableSetExample {

    public static void main(String[] args) {

        // Create a normal set of electronic items
        Set<String> electronicsSet = new HashSet<>();
        electronicsSet.add("Laptop");
        electronicsSet.add("Smartphone");
        electronicsSet.add("Tablet");

        // Make the set unmodifiable
        Set<String> unmodifiableElectronicsSet = Collections.unmodifiableSet(electronicsSet);

        // Print the unmodifiable set
        System.out.println("Unmodifiable Set: " + unmodifiableElectronicsSet);

        // Try to modify the unmodifiable set (will throw exception)
        // unmodifiableElectronicsSet.add("Smartwatch"); // Uncommenting this line will throw java.lang.UnsupportedOperationException
    }
}
```

Interview questions

1. What are the main interfaces in the Java Collection Framework?
2. What is the difference between Collection and Collections?
3. How does the Map interface differ from Collection?
4. What are the key differences between List, Set, and Map?
5. What is the hierarchy of the Collection Framework?
6. What is the difference between ArrayList and LinkedList?
7. When would you use Vector instead of ArrayList?
8. How does ListIterator differ from Iterator?
9. What is the difference between HashSet, LinkedHashSet, and TreeSet?
10. How does HashSet ensure uniqueness?
11. Can a Set contain null elements?
12. How is the ordering of elements maintained in LinkedHashSet and TreeSet?
13. What happens if we mutate an object used in a HashSet?
14. How does a HashMap work internally?
15. What is the difference between HashMap and Hashtable?
16. What are the differences between HashMap, LinkedHashMap, and TreeMap?
17. Can you store null keys and values in a HashMap?
18. What happens when the hashCode() method returns the same value for multiple keys?

Kodewala

19. What is the load factor in HashMap?
20. What is rehashing in a HashMap?
21. What is the time complexity for adding and retrieving elements in ArrayList, HashSet, and HashMap?
22. How is the capacity and size of a HashMap different?
23. How does Java handle collisions in HashMap?
24. What is the difference between Iterator and Enumeration?
25. What is a fail-fast iterator?
26. What is the difference between fail-fast and fail-safe?
27. How can we avoid ConcurrentModificationException?
28. Can you modify a collection while iterating?
29. What is ConcurrentHashMap and how is it different from Collections.synchronizedMap()?
30. What is CopyOnWriteArrayList?
31. How does Java handle thread safety in collections?
32. When should we use concurrent collections?
33. What are some examples of fail-safe iterators?
34. How do you sort a List of custom objects?
35. What is the difference between Comparable and Comparator?
36. How does TreeMap or TreeSet maintain order?
37. Can you store heterogeneous types in a TreeSet?
38. What happens if compareTo() and equals() are not consistent?
39. How do you convert a List to a Set and vice versa?
40. How do you make a collection read-only?
41. How do you synchronize a collection manually?
42. Can we sort a Map by its values?
43. Why is it important to override hashCode() and equals() together?
44. What happens if we don't override equals() in a key class used in HashMap?
45. Can we use a mutable object as a key in a HashMap?
46. Why is it recommended to program to interfaces rather than implementations in Java collections?

Kodewala

Core Java Week – 8 & 9

Kodewala

Stream API

The **Java Stream API** is used to process collections (like **List**, **Set**) in a **declarative, functional style**. It allows you to **filter**, **transform**, and **collect** data without writing repetitive code like loops and conditions.

Introduced in **Java 8**, the Stream API helps you work with collections in a **clean and concise** way.

Note: The Stream API is **not the same** as Java's **InputStream** and **OutputStream**, which handle byte streams. The Stream API deals with **objects**, not bytes.

Java Stream Definition

A **Java Stream** is a feature that can **automatically loop through elements** in a collection.

Unlike using a **for-each loop** or an **Iterator**, where **you write the loop**, a Stream **handles the looping internally**.

Stream Processing

In stream processing, you can attach **operations** to a stream. These operations are performed on each element in the stream, one by one.

The operations are connected in a **chain**, where:

- ✓ The **first operation** processes the element and passes it to the next.
- ✓ Each **next operation** can either modify the element or leave it unchanged.
- ✓ This continues until all operations have run.

This is how each element flows through a series of processing **steps.**

```
List<Integer> numbers = Arrays.asList(2, 5, 1, 2, 4, 6, 7, 8, 9);

numbers.stream().filter(n -> n % 2 == 0) // Operation 1: Keep even numbers
                .map(n -> n * 10) // Operation 2: Multiply by 10
 .forEach(n -> System.out.println(n)); // Operation 3: Print each result
```

Kodewala

How to Create a Stream?

There are several ways to create a **Stream** in Java. One of the most common is by using a **Java Collection**, such as a **List**. For example, you can call the **stream()** method on a List to get a Stream.

```java
// Create a List of Strings using Arrays.asList
List<String> items = Arrays.asList("Kodewala", "Academy", "Bangalore");

// Create a Stream from the list
Stream<String> stream = items.stream();

// Use a lambda expression to print each item
stream.forEach(item -> System.out.println(item));
```

- ✓ The list contains 3 elements: "**Kodewala**", "**Academy**", and "**Bangalore**".
- ✓ The stream allows you to process each item one by one, in a functional way.
- ✓ The forEach method applies the lambda (item -> System.out.println(item)) to each item, printing them to the console.

Features of stream

- ✓ **Not a data structure** - Stream does not store data. It just processes it.
- ✓ **Functional style** - Uses lambdas to write logic.
- ✓ **Lazily evaluated** - Operations are not executed until a terminal operation is called.
- ✓ **Can be parallel** - supports parallel processing using .parallelStream().
- ✓ **Chained operations** - Allows multiple operations to be chained (like filter → map → collect).

```java
// Create a List of Strings
List<String> items = Arrays.asList("Kodewala", "Academy", "Java", "Spring", "Stream");

// Stream pipeline demonstrating multiple stream features
List<String> result = items.stream()        // Not a data structure: Stream is just a pipeline
        .filter(item -> item.length() > 5) // Functional style: Uses lambda to filter
        .map(item -> item.toUpperCase())  // Chained operation: filter → map
        .sorted()                        // Another chained operation
        .collect(Collectors.toList()); // Terminal operation triggers lazy evaluation

// Print the result
result.forEach(r -> System.out.println(r));

// Parallel processing example (can improve performance on large data)

System.out.println("Parallel Stream Output:");
items.parallelStream().forEach(item -> System.out.println(Thread.currentThread().getName() + " - " + item));
```

Kodewala

Types of Streams:

- ✓ **Sequential Stream** - Processes data in a single thread (default).
- ✓ **Parallel Stream** - Splits tasks across multiple threads for faster performance on large data sets.

How Many Parallel Stream will use?

- ✓ **Number of Threads = Number of CPU Cores (Processors) – 1**
- ✓ If your system has 8 cores, then parallel stream will typically use 7 threads.

Code to Check It:

```
System.out.println(ForkJoinPool.commonPool().getParallelism());
```

This will show how many threads the parallel stream is using.

Stream Operations: Streams support two main types of operations:

1. Intermediate Operations (Lazy - return another stream)

- ✓ Intermediate operations are **stream methods that return another Stream**. They are used to **build a processing pipeline**.
- ✓ **But they don't process data immediately** - they just set up the steps. They are **lazy**, and **only execute when a terminal operation** is finally called.
- ✓ Used to transform or filter data.
- ✓ **Intermediate operations can be called multiple times(chaining)** - but they don't actually do anything **until a terminal operation is called**.
- ✓ You can **chain multiple** intermediate operations together.
- ✓ Laziness allows streams to work with **infinite or large data sources** efficiently.

kodewala

Common Intermediate Operations:

Operation	Description
`filter()`	Select elements based on a condition
`map()`	Transform each element
`flatMap()`	Flatten nested structures
`distinct()`	Remove duplicates
`sorted()`	Sort elements
`limit(n)`	Keep first n elements
`skip(n)`	Skip first n elements

2. Terminal Operations (Eager - Trigger the pipeline)

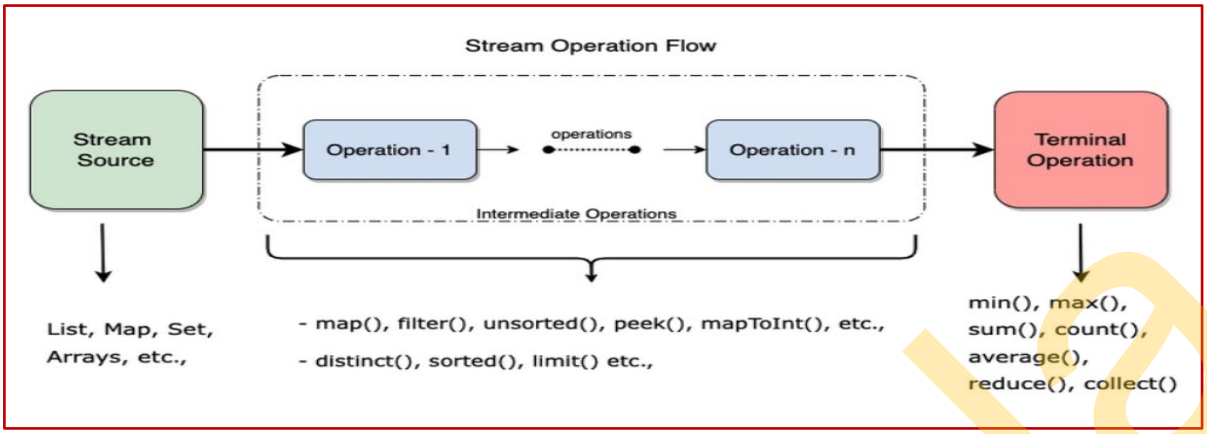
- ✓ These end the stream pipeline.
- ✓ They trigger the processing of the stream.
- ✓ Return a final result (non-stream).
- ✓ Terminal operation can be called only once per stream.
- ✓ **Streams are consumed** when a terminal operation is executed. After that, the stream is considered **"closed"**, and you cannot reuse it.

Code Example

```
List<String> allNames = Arrays.asList("Bangalore", "Chennai", "Delhi");

List<String> outPut = allNames.stream().filter(name -> name.startsWith("B")).map(word -> word.toUpperCase())
        .collect(Collectors.toList());
System.out.println(outPut);
```

Kodewala

Internal Working Flow of Stream



Lazy Evaluation - How It Saves Performance

Only when a terminal operation is triggered does each element flow one-by-one through the entire pipeline. This avoids creating intermediate collections and improves performance.

Intermediate Operations

filter()

- ✓ The **filter()** method is used to keep only the elements that match a condition.
- ✓ It takes a Predicate (a function that returns true or false) and checks each element in the stream.
- ✓ If the condition returns true → the element stays in the stream.
- ✓ If it returns false → the element is removed.

```
// Input: List of strings
List<String> words = Arrays.asList("Hi", "Java", "Go", "Kodewala", "AI");

// Create a stream and filter using a lambda expression
Stream<String> output = words.stream().filter(word -> word.length() >= 3); // Lambda: keep words with
                                                                           // 3 or more characters

// Terminal operation using lambda to print each element
output.forEach(word -> System.out.println(word));
```

Kodewala

map()

The map() method is used to transform each element in a stream.

- ✓ It takes a lambda to define how each item should be changed.
- ✓ The result is a new stream with the transformed values.

```
List<String> list = Arrays.asList("kodewala", "academy", "java");

// Convert each string to uppercase using map()
Stream<String> streamMapped = list.stream().map(value -> value.toUpperCase());

// Print the mapped stream
streamMapped.forEach(value -> System.out.println(value));
```

flatMap()

flatMap() takes each element from a stream and turns it into zero or more elements - then "flattens" all those elements into a single stream.

- ✓ Use it when you have nested structures (like lists inside lists, or strings with words).
- ✓ It helps convert a stream of complex elements into a flat stream of simple elements.
- ✓ flatMap() flattens nested streams into a single stream of words

```
// List of sentences
List<String> stringList = Arrays.asList("This is from Kodewala", "We are in Bangalore", "Gone with the wind");

// Stream of words from all sentences, flattened
stringlist.stream().flatMap(sentence -> Arrays.stream(sentence.split(" "))) // Split each sentence into words
                                                                            // and flatten
        .forEach(word -> System.out.println(word)); // Print each word
```

distinct()

The **distinct()** method removes duplicate elements from a stream, keeping only the unique ones.

- ✓ It returns a new stream with no duplicates.
- ✓ Useful when you want to filter out repeated values.

Kodewala

```java
// List with duplicates
List<String> stringList = Arrays.asList("one", "two", "three", "one");

// Stream, apply distinct(), collect result to a list
List<String> output = stringList.stream().distinct() // Remove duplicates
        .collect(Collectors.toList()); // Collect to list

// Print the distinct list
System.out.println(output);
```

limit()

The limit() method is used to restrict the number of elements in a stream.

- ✓ It returns a new stream with only the first **N** elements.
- ✓ Useful when you want to take only a few items from a large collection.

```java
// Input list
List<String> stringList = Arrays.asList("one", "two", "three", "one");

// Stream limited to first 2 elements
stringList.stream().limit(2) // Take only first 2 elements
        .forEach(element -> System.out.println(element)); // Print each
```

peak()

The peek() method lets you look at each element in the stream without changing it.

- ✓ It is useful for debugging or logging.
- ✓ It is an intermediate operation, so it would not run unless followed by a terminal operation (like forEach(), collect(), etc.).

```java
// Input list
List<String> list = Arrays.asList("abc", "def");

// Use peek() to observe each value (for debugging/logging)
Stream<String> peeked = list.stream().peek(
        value -> System.out.println("Peeking: " + value)); // This won't print anything yet

// Terminal operation to trigger the stream
peeked.forEach(value -> System.out.println("Final: " + value));
```

Kodewala

sorted()

The sorted() method is used to sort elements in a stream.

- ✓ By default, it sorts in natural order (e.g., A → Z, 1 → 9).
- ✓ You can also provide a custom comparator using a lambda for custom sorting.
- ✓ It is a non-terminal operation, so it must be followed by a terminal operation like forEach() or collect().

Default (natural) sorting

```
List<String> list = Arrays.asList("banana", "apple", "cherry");

list.stream().sorted() // Sorts alphabetically
        .forEach(value -> System.out.println("Sorted: " + value));
```

Custom sorting (by string length)

```
List<String> list = Arrays.asList("Kodewala", "Academy", "Bangalore");

list.stream().sorted((s1, s2) -> s1.length() - s2.length()) // Sort by string length (shortest first
       .forEach(v -> System.out.println("Sorted by length: " + v));
```

- ✓ sorted() doesn't change the original list — it returns a **new sorted stream**.
- ✓ It's **lazy** - won't run until a terminal operation is called.
- ✓ Use lambda for custom sorting rules.

mapToInt(), mapToDouble(), mapToLong()

Used to convert stream elements to primitive streams.

```
List<String> list = Arrays.asList("10", "20", "30");

list.stream().mapToInt(s -> Integer.parseInt(s)) // Lambda to convert String to int
       .forEach(n -> System.out.println(n)); // Lambda to print each int
```

Kodewala

skip(n)

The skip(long n) method is an **intermediate operation** that returns a new stream **with the first n elements discarded**.

- ✓ It **skips** the first n elements of the stream.
- ✓ If the stream contains fewer than n elements, the result is an **empty stream**.
- ✓ It is a **lazy operation** - won't execute until a **terminal operation** (like forEach, collect, etc.) is called.
- ✓ Often used with **limit()** for pagination or slicing data.

```
List<String> list = Arrays.asList("Kodewala", "Academy", "Test", "Bangalore");

list.stream().skip(2) // Skip the first 2 elements
        .forEach(item -> System.out.println(item)); // Lambda to print each remaining item
```

Terminal Operations

A terminal operation in a Stream starts the actual processing of data and produces a single result (like a value, collection etc.).

- ✓ Once you call a terminal operation, the stream is consumed and cannot be used again.
- ✓ Terminal operations don't return another stream, so you can't chain more operations afterward.

One-line definition:

A terminal operation in Java Streams triggers processing, returns a final result, and closes the stream so it can't be reused.

Example:

```
List<String> names = Arrays.asList("Kodewala", "BTM", "Bangalore");
long count = names.stream()
        .filter(name -> name.startsWith("B")).count(); // Terminal operation
System.out.println(count); // Output: 2
```

Kodewala

1. **forEach :** Perform an action for each element. It iterates over each element in the stream and performs the given action. Does not return anything.

```
List<String> names = Arrays.asList("Bangalore", "Kodewala", "BTM");
names.stream().forEach(name -> System.out.println(name));
```

2. **toArray()** - Converts the stream elements into an array using the provided array generator function.

```
List<String> names = Arrays.asList("kodewala", "Bangalore", "btm");
String[] nameArray = names.stream().toArray(size -> new String[size]);
System.out.println(Arrays.toString(nameArray));
```

3. **reduce()** - Combine elements into a single value. Performs a reduction on the elements, using an identity value and an accumulator function.

   List = [1, 2, 3, 4]
   Identity = **0**

1. Start → accumulator = **0**
2. Add 1 → accumulator = 0 + 1 = **1**
3. Add 2 → accumulator = 1 + 2 = **3**
4. Add 3 → accumulator = 3 + 3 = **6**
5. Add 4 → accumulator = 6 + 4 = **10**

   Final Result: 10

```
int sum = Arrays.asList(1, 2, 3, 4).stream().reduce(0, (a, b) -> a + b);
System.out.println(sum); // 10
```

4. **collect()** - Collects stream elements into a collection, map, or other container using a Collector.

```
List<String> names = Arrays.asList("Bangalore", "BTM", "Kodewala");
List<String> upperCaseNames = names.stream().
        map(n -> n.toUpperCase()).collect(Collectors.toList());
System.out.println(upperCaseNames);
```

Kodewala

5. **min() -** Returns the smallest element in the stream according to the provided comparator.

```
int minValue = Arrays.asList(10, 5, 20).stream().min((a, b) -> a - b).get();
System.out.println(minValue); // 5
```

6. **max() -** Returns the largest element in the stream according to the provided comparator.

```
int maxValue = Arrays.asList(10, 5, 20).stream().max((a, b) -> a - b).get();
System.out.println(maxValue); // 20
```

7. **count()** - Count number of elements. Returns the number of elements in the stream as a long.

```
List<String> names = Arrays.asList("Bangalore", "BTM", "Kodewala");
long count = names.stream().count();
System.out.println(count); // 3
```

Lazy Operations in Java Streams

In Streams, intermediate operations (like map(), filter(), sorted()) are lazy, meaning they don't process data immediately.
They just define a pipeline of transformations to be applied later, when a terminal operation is invoked.

a. **No processing happens** until a terminal operation is called.
b. Multiple intermediate operations are **fused into a single pass** over the data for efficiency.
c. This enables **short-circuiting** - stopping processing early if the result is determined (e.g findFirst(), anyMatch())

```
List<String> names = Arrays.asList("Bangalore", "BTM", "Bombay");

// Define stream pipeline (lazy)
names.stream().filter(n -> {
    System.out.println("Filtering: " + n);
    return n.startsWith("B");
}).map(n -> {
    System.out.println("Mapping: " + n);
    return n.toUpperCase();
}).forEach(word -> System.out.println(word)); // Terminal operation triggers processing
```

kodewala

Explanation

- ✓ When .filter() and .map() are declared, **nothing is executed** yet.
- ✓ The moment .forEach() (a terminal operation) is called:
    1. The stream starts reading data.
    2. For each element:
        - filter() is applied
        - If the filter passes, map() is applied
        - The result is passed to forEach()

Interview Tip

Intermediate operations in streams are lazy - they don't execute until a terminal operation triggers the pipeline. This improves efficiency by processing elements only as needed.

Collectors Methods

toList() - Collect elements into a List

```
List<String> list = Stream.of("A", "B", "C").collect(Collectors.toList());
```

toSet() - Collect elements into a Set

```
Set<String> set = Stream.of("A", "B", "A").collect(Collectors.toSet());
```

toCollection() - Collect elements into a specific collection

```
LinkedList<String> linkedList = Stream.of("A", "B", "C")
        .collect(Collectors.toCollection(() -> new LinkedList<>()));
```

joining() - Concatenate strings

```
String joined = Stream.of("Kodewala", "Bangalore").collect(Collectors.joining(", "));
```

Kodewala

counting() - Count elements

```
long count = Stream.of("A", "B", "C").collect(Collectors.counting());
```

groupingBy() - Group elements by classifier lambda

```
Map<Integer, List<String>> grouped = Stream.of("one", "two", "three", "four")
        .collect(Collectors.groupingBy(s -> s.length()));
```

partitioningBy() - Partition by predicate lambda

```
Map<Boolean, List<Integer>> partitioned = Stream.of(1, 2, 3, 4, 5)
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Interview Questions:

1. Explain the difference between map() and flatMap() with examples.
2. What is lazy evaluation in Streams? Give an example.
3. How do parallel streams improve performance? What are the pitfalls?
4. What is a Spliterator? How is it related to streams?
5. What is the difference between findFirst() and findAny() in parallel streams?
6. Explain Stream.generate() and Stream.iterate() with examples.
7. How to handle checked exceptions inside stream operations?
8. Explain the difference between peek() and forEach().
9. How do you measure the performance of streams?
10. What is the behavior of sorted() in parallel streams?
11. How do you flatten a nested collection of lists into a single list using streams?
12. How do you chain multiple filters and maps in a stream efficiently?
13. What is short-circuiting in streams? Give examples.

Kodewala

Interview Questions (Programs)

1. Find the second-highest number in a list using streams

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40, 50);
Optional<Integer> secondMax = nums.stream().sorted(Comparator.reverseOrder()).skip(1).findFirst();
System.out.println("Second Highest: " + secondMax.get());
```

2. Count the frequency of each character in a string

```
String input = "banana";
Map<Character, Long> freq = input.chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
System.out.println(freq);
```

3. Find all duplicate elements in a list

```
List<Integer> nums = Arrays.asList(1, 2, 3, 2, 4, 5, 1);
Set<Integer> duplicates = nums.stream().filter(n -> Collections.frequency(nums, n) > 1
        .collect(Collectors.toSet());
System.out.println(duplicates);
```

4. Reverse each word in a sentence

```
String sentence = "Kodewala is Java Training Academy";
String result = Arrays.stream(sentence.split(" ")).map(word -> new StringBuilder(word).reverse().toString()
        .collect(Collectors.joining(" "));
System.out.println(result);
```

5. Find the longest string in a list

```
List<String> words = Arrays.asList("cat", "elephant", "tiger", "hippopotamus");
String longest = words.stream().max(Comparator.comparingInt(String::length)).orElse("");
System.out.println("Longest word: " + longest);
```

6. Sort a list of strings by length

```
List<String> words = Arrays.asList("banana", "apple", "kiwi", "grape");
words.stream().sorted(Comparator.comparingInt(w -> w.length())).forEach(word -> System.out.println(word));
```

Kodewala

7. Merge two lists and remove duplicates

```
List<Integer> list1 = Arrays.asList(1, 2, 3);
List<Integer> list2 = Arrays.asList(3, 4, 5);

List<Integer> merged = Stream.concat(list1.stream(), list2.stream()).distinct().collect(Collectors.toList());
System.out.println(merged);
```

8. Find first non-repeated character in a string

```
String input = "HelloH";
Character firstNonRepeat = input.chars().mapToObj(c -> (char) c)
        .filter(ch -> input.indexOf(ch) == input.lastIndexOf(ch)).findFirst().orElse(null);
System.out.println("First non-repeated: " + firstNonRepeat);
```

9. Group strings by their length

```
List<String> words = Arrays.asList("one", "three", "five", "four", "six");
Map<Integer, List<String>> grouped = words.stream().collect(Collectors.groupingBy(s -> s.length()));
System.out.println(grouped);
```

10. Find the sum of squares of even numbers

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
int sum = nums.stream().filter(n -> n % 2 == 0).mapToInt(n -> n * n).sum();
System.out.println("Sum of squares of even numbers: " + sum);
```

ss

Operation/Function Used

stream() – Creates a stream from a collection

sorted() – Sorts stream elements

sorted(Comparator.reverseOrder()) – Sorts in descending order

skip(n) – Skips first n elements

findFirst() – Returns first element of the stream

orElse() – Returns value or default if empty

orElseThrow() – Throws exception if stream is empty

Kodewala

chars() – Converts String to IntStream of Unicode values

mapToObj() – Converts primitive stream to object stream

collect() – Performs a mutable reduction into a collection or map

Collectors.groupingBy() – Groups elements by classifier function

Collectors.counting() – Counts elements in each group

filter() – Selects elements matching a predicate

Collections.frequency() – Counts occurrences in a collection

Collectors.toSet() – Collects elements into a Set

Collectors.joining() – Joins elements into a single string

map() – Transforms elements

StringBuilder.reverse() – Reverses characters in a StringBuilder

max() – Finds maximum element by comparator

Comparator.comparingInt() – Comparator based on integer property

forEach() – Performs action on each element

Stream.concat() – Concatenates two streams

distinct() – Removes duplicate elements

indexOf() – Finds first index of a character/string

lastIndexOf() – Finds last index of a character/string

Collectors.toList() – Collects elements into a List

mapToInt() – Converts to IntStream for numeric operations

sum() – Sums IntStream values

---------- END OF THE STREAMS ----------

kodewala

Maven

Maven is a **build automation and project management tool** for Java (and other JVM-based languages) that helps you:

1. **Manage libraries** - Automatically download and use external jars.
2. **Build projects** - Compile code, run tests, and create JAR/WAR files.
3. **Organize code** - Uses a standard folder structure.

It works using a **pom.xml** file where you list your dependencies and build settings. Maven then downloads what's needed and builds your project.

pom.xml (Project Object Model)

- ✓ It's the **main configuration file** for a Maven project.
- ✓ Written in **XML** format.
- ✓ Defines:

    1. **Project Information** → name, version, packaging type (JAR/WAR).
    2. **Dependencies** → libraries your project needs.
    3. **Build Settings** → plugins, compiler settings.
    4. **Repositories** → where Maven looks for dependencies.

```xml
<project>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.kodewala</groupId>
    <artifactId>my-app</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <version>3.1.0</version>
        </dependency>
    </dependencies>
</project>
```

- ✓ <groupId> → Unique ID for your project's organization.
- ✓ <artifactId> → Name of your project.
- ✓ <version> → Project version.
- ✓ <packaging> → jar / war.
- ✓ <dependencies> → Libraries you need.
- ✓ <plugins> → Tools that add extra build features.

Kodewala

Maven Repositories

- Repositories are **places where Maven stores and retrieves JAR files**.

Types of Repositories:

1. **Local Repository**
   - Location: ~/.m2/repository on your machine.
   - Stores downloaded dependencies for reuse.
2. **Central Repository**
   - Maven's default global library store.
   - URL: https://repo.maven.apache.org/maven2
   - Contains thousands of open-source Java libraries.
3. **Remote Repository**
   - Any custom server where dependencies are hosted.
   - Used by companies to store internal libraries.

Flow:

1. You run mvn install.
2. Maven checks your **local repo**.
3. If not found → downloads from **central repo** (or remote repo) → saves to local repo.

Interview Tip:

Where does Maven download dependencies from?

- ✓ First checks **local repo**,
- ✓ Then **remote/company repo**,
- ✓ Then **Maven Central**.

Maven Commands

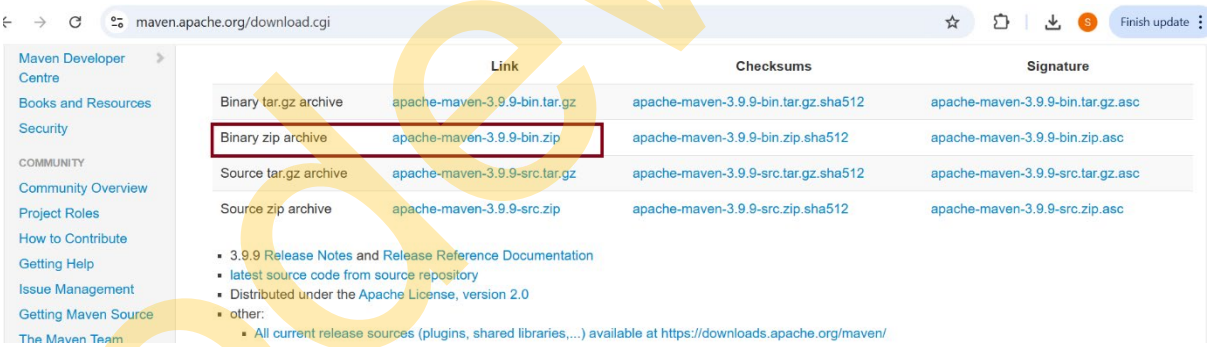
- ✓ **mvn clean** - Clean up previous builds (delete target/ folder)
- ✓ **mvn compile** - Compile the source code
- ✓ **mvn test** - Run the tests
- ✓ **mvn package** - Build the JAR/WAR file
- ✓ **mvn install** - Install the package into local Maven repository
- ✓ **mvn deploy** - Deploy the package to a remote repository

Install and use Maven in your local machine

kodewala

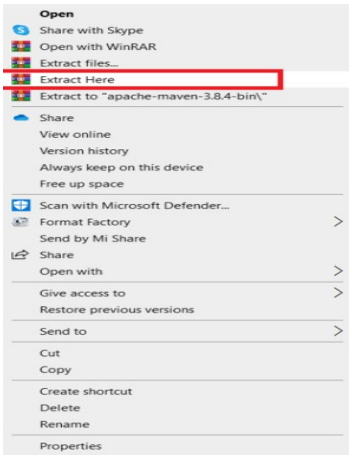
1. Open your web browser and navigate to [Maven's official website](#).
2. Click on the **"Download"** link on the Maven homepage.

3. For Windows, click on the **Binary zip archive** link. If you are using a different operating system, select the appropriate binary package for your system.

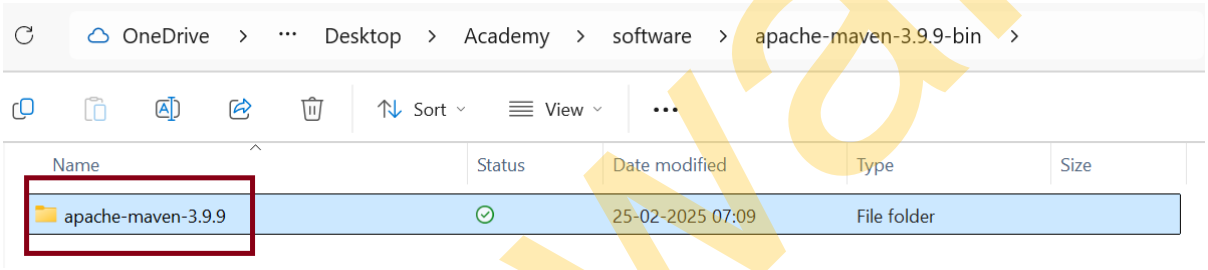


4. Click on the **apache-maven-3.9.9-bin.zip** button. (for Window)
5. Now check for the executable file in downloads in your system.
6. Now right-click on the file and click on extract here to extract the file.

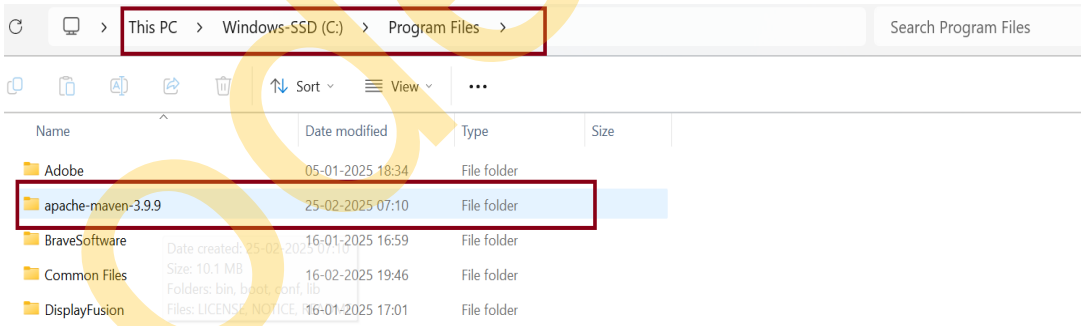
Koolewala



7. After extracting, you get the extracted folder.
8. Now copy the extracted folder.

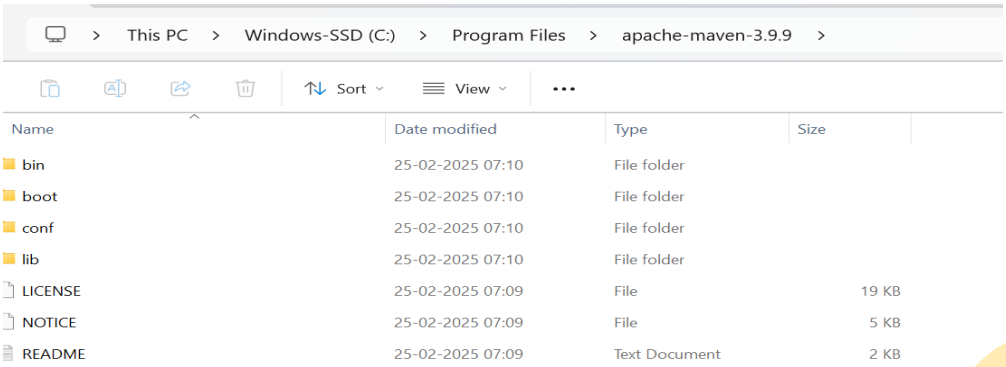


9. Paste the copy folder in your windows drive in the Program files folder.



10. Permission Windows appears to paste the folder in program files then click on "Continue".
11. Open the folder apache maven.

kodewala



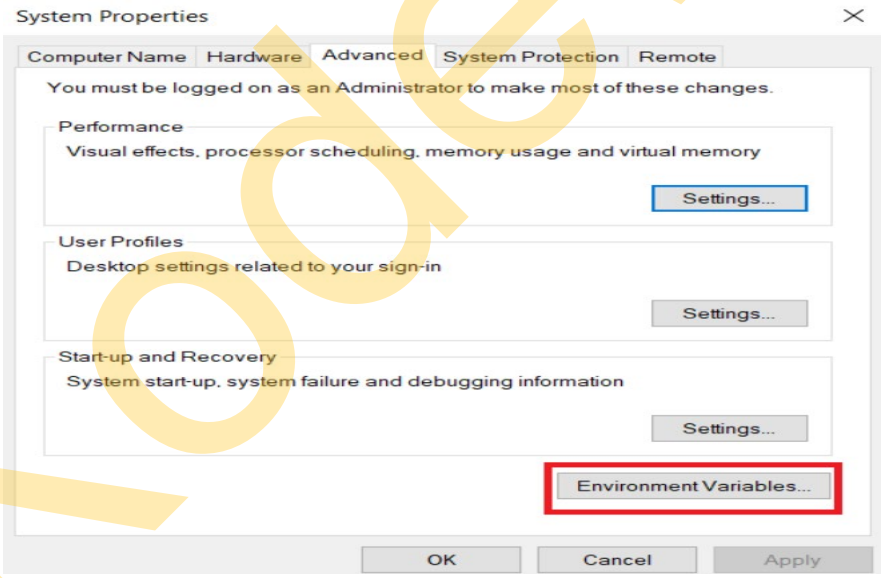
12. After opening the folder then copy the address of the folder in program files.

    Example: **C:\Program Files\apache-maven-3.9.9**

13. Add **MAVEN_HOME** System Variable

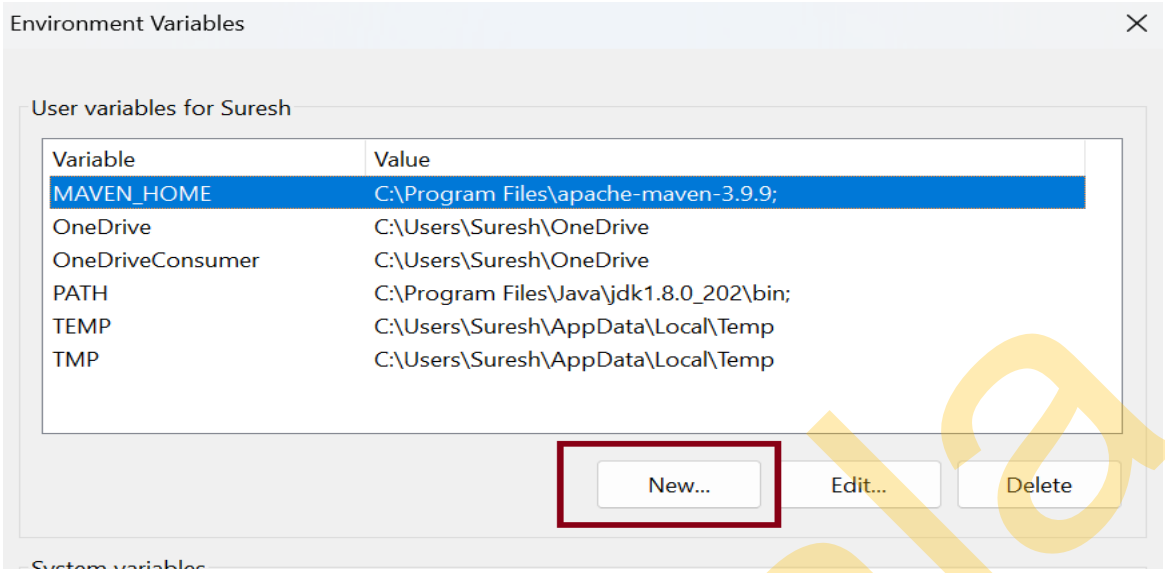
14. Click on Start Menu and search "**Edit the system environment variables**" and open it.

15. After opening System, Variable New window appears, and click on "**Environment Variables**"

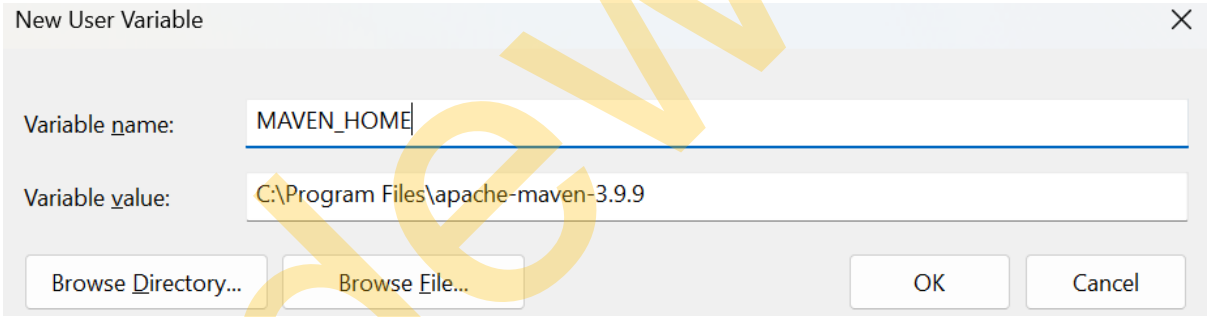


16. Now click on New under user variable.

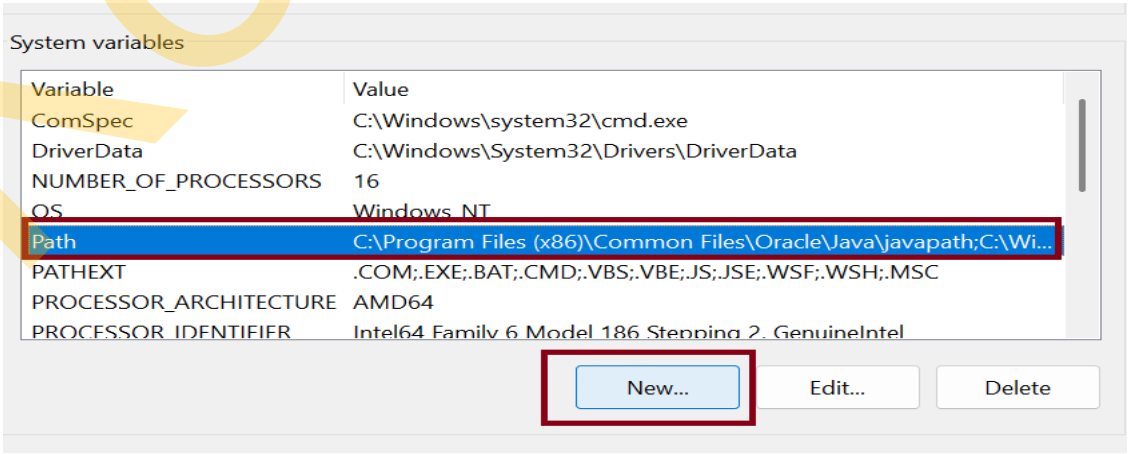
kodewala



17. Now you can write “MAVEN_HOME” variable name and then paste the address we copy from program files in the variable value and then click on OK.



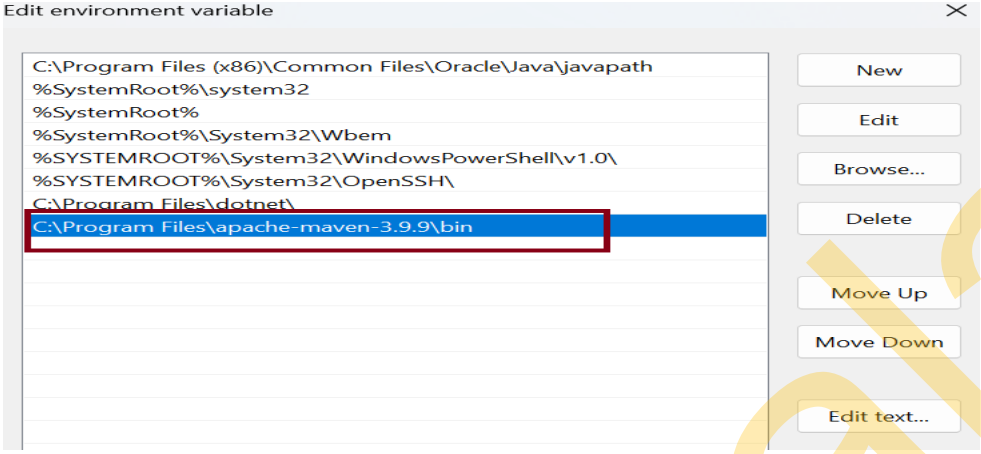
18. Now go to the “System variables” Path option and double click on Path or Click on the Edit button.



19. Now click on “New” Button.

koolewala

20. Copy the path of bin dir from maven installation. Example : “C:\Program Files\apache-maven-3.9.9**bin**”
21. Paste as shown below and click Ok OK OK etc…..



22. Maven installs done

23. Verify Maven Installation → Open new “cmd” / command prompt and type “mvn -version”

24. If you see above message means, maven installation is done.

Develop Sample Java Application using Mavan

1. Create Sample Standalone Project:(Single line command only)

```
mvn archetype:generate -DgroupId=com.example -DartifactId=helloworld -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Note: You should have internet connected to your compute.

2. Go to project Directory “helloworld”
3. Compile the code using below command.

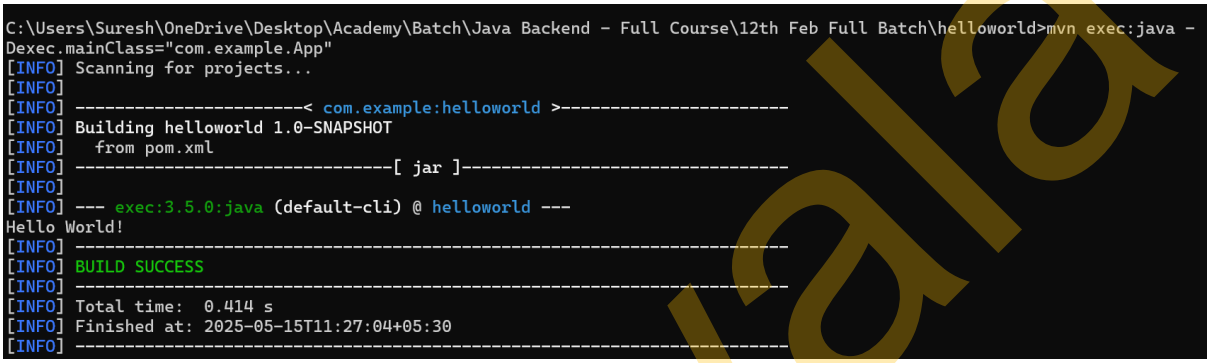
```
mvn compile
```

Koderwala

4. Run the code

```
mvn exec:java -Dexec.mainClass="com.example.App"
```

If you see below message, then your program ran/executed successfully.



Kodewala

Junit Framework

Before we start Junit, let's understand about SDLC.

What is SDLC?

- ✓ The **Software Development Life Cycle** is the step-by-step process used to plan, develop, test, and deploy software.
- ✓ It ensures software is **high quality, cost-effective, and delivered on time**.

Phases of SDLC

1. **Requirement Analysis**
   - ✓ Gather and document what the client needs.
   - ✓ Example: "The calculator must support addition, subtraction, multiplication, division."

2. **Design**
   - ✓ Create system architecture, database design, and user interface design.
   - ✓ Example: Decide classes, methods, and data flow.

3. **Implementation (Coding)**
   - ✓ Developers write code as per the design.
   - ✓ Example: Write Calculator.java with methods like add(), divide().

4. **Testing (Where JUnit Fits)**
   - ✓ Verify if the code works as expected.
   - ✓ JUnit is used here for unit testing - testing small pieces of code (like a method or class) in isolation.
   - ✓ Example: Test if add(2,3) returns 5.

5. **Deployment**
   - ✓ Release the software to production.
   - ✓ Example: Upload the app to a server or publish it to a store.

6. **Maintenance**
   - ✓ Fix bugs, make updates, and add new features after deployment.
   - ✓ Old JUnit tests can be rerun to ensure changes don't break existing functionality (regression testing).

Kodewala

Where JUnit Fits

- ✓ **Stage: Testing phase** of SDLC
- ✓ **Type of Testing: Unit Testing** (lowest level of testing)
- ✓ **Purpose:**
    1. Verify correctness of individual methods/classes.
    2. Catch bugs early in development.

JUnit

- ✓ JUnit is a Java unit testing framework to test small parts of code (unit tests).
- ✓ **JUnit 4** is annotation-based, easy to integrate with Maven/Gradle, and widely used.

Junit-4 Annotations

Annotation	Description
`@Test`	Marks a method as a test case.
`@Before`	Runs before each test method.
`@After`	Runs after each test method.
`@BeforeClass`	Runs once before all tests (static method).
`@AfterClass`	Runs once after all tests (static method).
`@Ignore`	Skips a test method.
`@Test(expected=Exception.class)`	Expects an exception in the test.
`@Test(timeout=milliseconds)`	Fails if test takes longer than given time.

Benefits of JUnit

- ✓ **Automated Testing**
    - No need to manually check outputs - tests run automatically and verify results.
- ✓ **Early Bug Detection**
    - Catch issues during development before they reach production.
- ✓ **Faster Development**
    - Run hundreds of tests in seconds, saving developer time.

Kodewala

- ✓ **Improved Code Quality**
  - Ensures code works as expected for both new and existing features.
- ✓ **Supports Regression Testing**
  - Old tests can be run after code changes to ensure nothing breaks.
- ✓ **Readable & Maintainable Tests**
  - Annotations (@Test, @Before, @After) make tests easy to understand.
- ✓ **Integration with Tools**
  - Works with Maven, Gradle, Jenkins, GitHub Actions for CI/CD pipelines.

Let's write Junit test cases for **Calculator.java** class

1. **Add JUnit 4 Dependency to pom.xml**

```xml
<dependencies>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Save the file - Maven will download JUnit automatically

2. **Create the Class to Test**

In src/test/java:

1. Right-click → **New → Package** → Name it com.kodewala.
2. Inside the package, create CalculatorTest.java:

```java
public class CalculatorTest {

    private static Calculator calc;

    @BeforeClass
    public static void setupBeforeClass() {
        System.out.println("Before all tests");
        calc = new Calculator();
    }

    @Test
    public void testAdd() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    public void testSubtract() {
        assertEquals(1, calc.subtract(3, 2));
    }

    @Test(expected = ArithmeticException.class)
    public void testDivideByZero() {
        calc.divide(10, 0);
    }
}
```

Kodewala

3. **Run the Test in Eclipse**

   ✓ Right-click CalculatorTest.java → **Run As → JUnit Test**.

   You will see:

   ✓ **Green bar** → All tests passed.

   ✓ **Red bar** → Some tests failed, with error details.

Code Coverage

- ✓ Code coverage is a **metric** that shows how much of your source code is executed when your automated tests run. It helps identify which parts of the code are tested and which are not.
- ✓ Ensures that tests cover critical parts of the application.
- ✓ Helps find untested or dead code.
- ✓ Improves software reliability and reduces bugs.

Ideal Code Coverage Percentage?

- ✓ 80% or higher is generally considered a good target for most projects.
- ✓ 100% coverage is often unrealistic and not always necessary; some code (e.g., error handling or logging) might be difficult or unnecessary to cover.
- ✓ Focus on meaningful tests rather than just coverage numbers.

Interview Questions

1. How would you handle testing a method that interacts with a database using JUnit?
2. Explain how you can test for exceptions in JUnit 4 with a real example.
3. What steps would you take if your code coverage report shows low coverage in critical modules?
4. Describe a situation where 100% code coverage was not achievable or practical, and how you managed testing in that case.
5. How do you organize your test classes and test methods in a large Java project using Maven and JUnit?
6. How can you run a specific JUnit test or group of tests using Maven commands?
7. What are some common challenges you have faced while writing unit tests and how did you overcome them?
8. How do you use assertions effectively in JUnit to make tests more reliable?
9. Can you explain the difference between @Before and @BeforeClass with real usage scenarios?

Kodewala

Thread

What is a Thread?

- ✓ A **thread** is a lightweight subprocess. It is a unit of execution within a process.
- ✓ In Java, every program has at least one thread: the **main thread**.
- ✓ Java supports **multithreading** which means executing multiple threads in parallel.

Real-world analogy:

> Think of a restaurant (process) with multiple waiters (threads), each taking orders from different tables.

Who Create main thread?

1. JVM starts your application by creating a thread named "main"

   - ✓ This is the first thread of your program.
   - ✓ It executes all code in the main() method.

2. Any **additional threads** must be explicitly created by you(developers) in the main() method or other methods it calls.

3. To verify that the main method is executed by the main thread, you can print the name of the currently executing thread inside the main method:

```
public static void main(String[] args)
{
    // Print the name of the current thread
    System.out.println("Inside main method");
    System.out.println("Thread name: " + Thread.currentThread().getName());
}
```

Ways to Create a Thread

There are 2 major ways:

1. Extending **Thread** class

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Running thread by extending Thread");
    }
}
```

kodewala

2. Implementing **Runnable** interface

```
class MyRunnable implements Runnable
{
    public void run()
    {
        System.out.println("Running thread by implementing Runnable");
    }
}
```

Note: There is another way - Using **ExecutorService** (from Java 5). We will discuss it later.

Which one is better?

- ✓ **Runnable** is better than Thread

  - Because Java supports multiple inheritance via interfaces, not classes.
  - Makes your class more flexible so that it can extend any other class as well.

- ✓ Prefer **ExecutorService** in real-world apps for thread pooling and lifecycle management. (Will discuss in detail)

Thread Life Cycle

- ✓ **New** → A thread is created but not yet started. i.e **Thread t = new Thread();**
- ✓ **Runnable** → The thread is ready to run and waiting for CPU time. i.e **t.start();**
- ✓ **Running** → picked by CPU and thread is actively executing its run() method.
- ✓ **Blocked** → Waiting to acquire a monitor lock (e.g., trying to enter a synchronized block but another thread holds the lock).
- ✓ **Waiting/Time_Waiting** → Temporarily paused, waiting to be notified (via wait(), join(), or sleep()).
- ✓ **Terminated** → run() completes.

Interview One Liner:

Can a thread move directly from the WAITING state to the RUNNING state?

No, a thread cannot go directly from **WAITING** to **RUNNING**. It must first move to the **RUNNABLE** state before it can be scheduled and actually run.

Summary of transitions:

WAITING -> RUNNABLE -> RUNNING

Kodewala

WAITING Vs TIMED_WAITING

- ✓ **WAITING**: waits indefinitely for notification.
- ✓ **TIMED_WAITING**: waits for a specified time (sleep(), wait(timeout), join(timeout)).

Both follow the same transition path through RUNNABLE before RUNNING.

Interview One Liner:

Can a thread that has completed execution be started again? If not, what exception is thrown?

- ✓ A completed thread cannot be started again.
- ✓ If you try to start a thread that has already completed (i.e., its run() method has finished), the JVM throws a java.lang.IllegalThreadStateException.
- ✓ Once a thread completes execution, it is considered **dead**.
- ✓ Calling start() on a dead thread will throw java.lang.IllegalThreadStateException.

Interview One Liner:

What happens if you call the run() method of a thread directly instead of calling start()?

- ✓ Calling the **run()** method directly does not start a new thread. Instead, it executes the run() method in the current thread just like a normal method call.
- ✓ The new thread is not created, and no concurrent execution happens.

Synchronization

Synchronization is a mechanism to **control access** to **shared resources(code)** by multiple threads to avoid data **inconsistency**, **race conditions**, and **unexpected behaviour**.

Why Synchronization is Needed?

- ✓ When multiple threads access and modify the same shared resource (e.g., variable, object etc) concurrently without proper control, it leads to race conditions.

- ✓ If two threads call **increment()** at the same time, both might read the same value and increment it, resulting in a **wrong final count**.

Kodewala

```
class Counter
{
    int count = 0;

    public void increment()
    {
        count++;
    }
}
```

Synchronized Keyword

- ✓ Java provides the **synchronized** keyword to ensure mutual exclusion, i.e - Only one thread can execute the critical piece of code at a time.

Ways to Synchronize Code

- ✓ Synchronized Method

```
int count = 0;

public synchronized void increment()
{
    count++;
}
```

- ✓ Synchronized Block

```
public void increment()
{
    synchronized (this)
    {
        count++;
    }
}
```

Synchronization Internal Working (Monitor & Lock)

- ✓ Every object and class in Java has an intrinsic lock (monitor).
- ✓ When a thread enters a synchronized method/block, it must acquire the object's/class's lock.
- ✓ If another thread holds the lock, the current thread goes into BLOCKED state until the lock is released.
- ✓ Once the lock is acquired, the thread enters the RUNNING state and executes the code.

Kodewala

Best Practices:

Use synchronized block over synchronized method when possible (better performance)

Advanced Concurrency (Synchronization)

- ✓ advanced concurrency utilities in "**java.util.concurrent**" used for thread synchronization and coordination in Java

1. **ReentrantLock:** A lock with the same behaviour as the synchronized keyword, but with additional features like:

- ✓ **Try locking without blocking** - thread can **attempt to acquire a lock**, but **would not wait** if the lock is already held.

boolean tryLock();

Use case: When you do not want your thread to get stuck waiting for a lock.

- ✓ **Timeout while waiting for a lock** - This allows a thread to **wait for a specified time** to acquire the lock and give up if it can't.

boolean tryLock(long timeout, TimeUnit unit)

Use case: Use case: In scenarios where a thread can **wait some time** but not forever for a lock

```
class TryLockExample
{
    private final ReentrantLock lock = new ReentrantLock();

    public void attemptTask()
    {
        try
        {
            if (lock.tryLock(2, TimeUnit.SECONDS))
            {
                try
                {
                    System.out.println("Lock acquired, performing task...");
                    Thread.sleep(1000);
                } finally
                {
                    lock.unlock();
                }
            } else
            {
                System.out.println("Couldn't acquire lock within 2 seconds, exiting...");
            }
        } catch (InterruptedException e)
        {
            e.printStackTrace();
        }
    }
}
```

kodewala

2. ReadWriteLock (and ReentrantReadWriteLock)

Used when:

- ✓ **Multiple threads** can read the resource concurrently.
- ✓ **Only one thread can write**, and no readers are allowed during write.

```
class SharedData
{
    private int data = 0;
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public void write(int value)
    {
        rwLock.writeLock().lock();
        try
        {
            data = value;
            System.out.println("Written: " + value);
        } finally
        {
            rwLock.writeLock().unlock();
        }
    }

    public void read()
    {
        rwLock.readLock().lock();
        try
        {
            System.out.println("Read: " + data);
        } finally
        {
            rwLock.readLock().unlock();
        }
    }
}
```

Use Case: Shared Bank Account Details (Read-Heavy Scenario)

- ✓ You have a system where **multiple users are frequently checking** their account balance (read operations).
- ✓ Occasionally, a transaction (like a deposit or withdrawal) occurs (write operation).

Why **ReadWriteLock** is Ideal Here?

- ✓ Multiple threads can read the balance at the same time → **High performance.**
- ✓ Only one thread should update the balance at a time → **Data consistency.**

Kodewala

3. **Semaphore:** Controls access to a resource with limited permits, e.g. only N threads can access at a time.

```
class ParkingLot
{
    private final Semaphore slots = new Semaphore(3); // Only 3 cars(threads) allowed

    public void parkCar(String carName)
    {
        try
        {
            slots.acquire();
            System.out.println(carName + " parked.");
            Thread.sleep(1000);
        } catch (InterruptedException e)
        {
            Thread.currentThread().interrupt();
        } finally
        {
            System.out.println(carName + " left.");
            slots.release();
        }
    }
}
```

When to Use Dynamic Permits?

Semaphore slots = new Semaphore(initialCount, true); // true = fair (FIFO)

Semaphore in one line:

A Semaphore is a concurrency control tool that restricts access to a limited number of permits, allowing only a fixed number of threads to access a shared resource simultaneously.

When to Use What:

ReentrantLock ➔ Full control over locking, fairness, etc.

ReadWriteLock ➔ Many readers, few writers (read-heavy)

Semaphore ➔ concurrent access to a resource (DB connections, threads in pool)

Koolewala

Inter-Thread Communication

It is the mechanism by which multiple threads communicate with each other by sharing resources and coordinating execution (often using wait(), notify(), and notifyAll())

Interview One Liner:

wait(), notify(), and notifyAll() are defined in the **Object** class, not in the **Thread** class.

Method used:

wait() - Current thread to **release the lock** and go to waiting state.

notify() - Wakes up one waiting thread on the same object.

notifyAll() - Wakes up all waiting threads on the same object.

Producer/Consumer with wait/notify

Problem Statement:

- ✓ One thread produces data.
- ✓ Another thread consumes it.
- ✓ Consumer should wait if there's nothing to consume.

```
class Shared
{
    int num;
    boolean ready = false;

    synchronized void produce(int n) throws InterruptedException
    {
        while (ready)
        {
            wait();
        }
        num = n;
        System.out.println("Produced: " + n);
        ready = true;
        notify();
    }

    synchronized void consume() throws InterruptedException
    {
        while (!ready)
        {
            wait();
        }
        System.out.println("Consumed: " + num);
        ready = false;
        notify();
    }
}
```

Kodewala

```java
class Producer extends Thread
{
    Shared shared;

    Producer(Shared shared)
    {
        this.shared = shared;
    }

    @Override
    public void run()
    {
        for (int i = 1; i <= 3; i++)
        {
            try
            {
                shared.produce(i);
            } catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
}
```

```java
class Consumer extends Thread
{
    Shared shared;

    Consumer(Shared shared)
    {
        this.shared = shared;
    }

    @Override
    public void run()
    {
        for (int i = 1; i <= 3; i++)
        {
            try
            {
                shared.consume();
            } catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
}
```

Kodewala

```java
public class Main
{
    public static void main(String[] args)
    {
        Shared shared = new Shared();
        Producer producer = new Producer(shared);
        Consumer consumer = new Consumer(shared);

        producer.start();
        consumer.start();
    }
}
```

- ✓ Shared is the **common object** shared by two threads: Producer and Consumer.
- ✓ Producer **waits** if data hasn't been consumed yet.
- ✓ Consumer **waits** if no new data has been produced yet.

wait() vs sleep() Methods

wait() releases the lock and pauses the thread until notified or timed out.

sleep() holds the lock and pauses for a fixed time.

Why should wait()/notify() be called inside a synchronized block?

Because they require a lock on the object to avoid IllegalMonitorStateException.

join() Method

join() tells the **main thread** (or any calling thread) to **wait** until the specified thread **completes its execution**.

```java
MyThread t1 = new MyThread();
MyThread t2 = new MyThread();

t1.setName("Thread-1");
t2.setName("Thread-2");

t1.start();
t1.join(); // main waits for t1 to finish

t2.start(); // t2 starts only after t1 completes

System.out.println("Main ends.");
```

Kodewala

Executor Framework

The **Executor Framework** is a high-level API introduced in Java 5 (JDK 1.5) as part of the "**java.util.concurrent**" package to manage thread execution efficiently without manually handling Thread objects.

Why Was It Introduced?

Before Java 5, developers had to manage thread creation, execution, and lifecycle manually, which was error-prone and hard to scale.

The Executor Framework solved this by:

- ✓ Decoupling task submission from thread management
- ✓ Offering built-in thread pools.

Core Interfaces & Classes

Executor → Basic interface with **execute**(Runnable).

ExecutorService → Extended interface with more control like **submit**(), **shutdown**().

Executors → Utility class with static factory methods to create **thread pools.**

Callable<V> → Like **Runnable**, but **returns a result** and can throw exceptions.

Future<V> → Represents the result of an **async** computation.

How to Use?

```
public class ExecutorExample
{
    public static void main(String[] args)
    {
        // Create a thread pool with 2 threads
        ExecutorService executor = Executors.newFixedThreadPool(2);

        for (int i = 0; i < 6; i++)
        {
            // Submit tasks
            executor.execute(new Task1());
            executor.execute(new Task2());
        }
        // Shutdown the executor
        executor.shutdown();
    }
```

Kodewala

Executor Service using Callable and Future

```
class MyCallableTask implements Callable<String>
{
    public String call() throws Exception
    {
        Thread.sleep(1000); // Simulate long task
        return "Callable Task Completed by " + Thread.currentThread().getName();
    }
}
public class CallableExample
{
    public static void main(String[] args) throws Exception
    {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // Submit Callable task and get Future
        Future<String> future = executor.submit(new MyCallableTask());

        // Get the result (blocks until result is ready)
        String result = future.get();
        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

Types of Thread Pools

newFixedThreadPool(int n) → Creates a pool with **fixed number** of threads. Used for stable number of concurrent tasks.

newCachedThreadPool() → Creates pool with **unbounded threads**, reuses idle ones. Used for many short-lived tasks.

newSingleThreadExecutor() → Creates a pool with **only one thread.** Used for sequential task execution.

newScheduledThreadPool(int n) → Executes tasks periodically or with delay. Used for timers, background jobs.

Callable vs Runnable

Runnable

Interface used to represent a task that executes without returning a result. It has a **run()** method and cannot throw checked exceptions.

Callable

Interface designed for tasks that return a result and can throw checked exceptions. It uses the **call()** method and works with **Future**

Kodewala

What is a Future?

A **Future** is an object that represents the result of an **asynchronous task** which may not be available yet but will be in the future.

- ✓ Used with ExecutorService.submit(Callable)
- ✓ It allows developers to:
  - Check if the task is done
  - Cancel the task
  - Get the result when it's ready

```
class MyTask implements Callable<String>
{
    public String call() throws Exception
    {
        Thread.sleep(1000); // Simulate delay
        return "Task completed!";
    }
}

public class FutureExample
{
    public static void main(String[] args) throws Exception
    {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Future<String> future = executor.submit(new MyTask());

        System.out.println("Waiting for result...");
        String result = future.get(); // blocks until result is ready
        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

One-liner Interview:

A Future in Java holds the **future result** of an async task and provides methods to **retrieve, cancel, or check** its status.

Kodewala

Interview question & Answers

1. **You have a shared counter variable accessed by multiple threads. Sometimes the final count is less than expected. What could be causing this? How would you fix it?**

   Answer: This is caused by a race condition where multiple threads update the counter simultaneously without synchronization. To fix it, use synchronization (synchronized block), atomic classes like AtomicInteger, or locks to ensure only one thread updates the counter at a time.

2. **Two threads each hold a lock the other thread needs. How would you detect this deadlock? How can you design your code to avoid such deadlocks?**

   Answer: Deadlocks can be detected using thread dumps or monitoring tools showing threads waiting forever. To prevent deadlocks, acquire locks in a consistent global order, avoid nested locks when possible, or use tryLock() with timeout to fail gracefully instead of waiting indefinitely.

3. **Explain how you would implement a producer-consumer system using threads in Java. Which classes or methods would you use to ensure correct synchronization**?

   Answer: Producers add items using put(), which waits if the queue is full. Consumers remove items using take(), which waits if the queue is empty. This ensures proper synchronization without explicit wait()/notify() calls.

4. **You want one thread to wait for another thread to complete a task before proceeding. How would you implement this? Discuss the use of wait(), notify(), and join().**

   Answer: The simplest way is to use thread.join(), which blocks the calling thread until the other thread finishes. For more complex coordination, use wait() and notify() on a shared monitor object inside synchronized blocks to pause and resume threads based on conditions.

kodewala

5. **You have thousands of short-lived tasks that need to run concurrently. How would you manage the threads efficiently? Why is creating a new thread for each task a bad idea?**

   Answer: Creating a new thread for every task is expensive due to overhead in creating and destroying threads. Instead, use ExecutorService with a thread pool (Executors.newFixedThreadPool()), which reuses a fixed number of threads to handle many tasks efficiently, reducing resource consumption.

6. You are debugging a multithreaded application. Sometimes the output is inconsistent or wrong. How do you approach identifying race conditions and fixing them?

   Answer: Add synchronized blocks or use thread-safe classes to protect these shared variables. Use logging and thread analyzers to track how threads interact and interleave.

7. **What are the differences between using synchronized and ReentrantLock? When would you prefer one over the other?**

   Answer: synchronized is simpler and automatically releases locks, while ReentrantLock provides advanced features like timed waits, interruptible locking, and explicit lock release. Use ReentrantLock when you need more control, otherwise synchronized suffices.

Intermediate-level Questions

1. How do you create a thread in Java?
2. What is the difference between start() and run() methods?
3. What does the synchronized keyword do?
4. What is a race condition?
5. What is deadlock?
6. How can deadlocks be avoided?
7. What is the purpose of wait() and notify() methods?
8. What does Thread.sleep() do?
9. What is the difference between Runnable and Callable interfaces?
10. What is a thread pool?
11. What is the volatile keyword used for?
12. What does the join() method do?
13. What is ExecutorService?
14. What happens if you call start() twice on the same thread?

Kodewala

Advance Java

Spring Framework

Introduction to Spring

Spring Framework is a lightweight, open-source framework for building Java applications, especially enterprise-level applications.

Lightweight Means?

- ✓ You can use only the **modules you need**, instead of loading everything.
- ✓ It does **not require heavy or bulky application servers** (like JEE containers) to run.
- ✓ Spring runs even in simple environments like Tomcat or Jetty server instead of full-blown servers like WebLogic or JBoss.

Why Spring is Famous?

Spring is famous because it made **Java development simpler**, **faster**, and **more flexible** by eliminating the **complexity of traditional** Java EE**.**

Spring Modules

1. **Spring Core Framework**

Spring Core - Provides the base for **dependency injection (DI)** and **inversion of control (IOC).**

Spring Beans - Handles **bean creation**, **wiring**, and **lifecycle**.

Spring Context - Offers **ApplicationContext** and resource loading.

2. **Spring Web & REST Development**

Spring Web - Supports REST APIs and HTTP handling.

Spring Web MVC - Framework for building full-featured web app using the MVC pattern.

3. **Spring Data**

Spring JDBC - Simplifies JDBC database operations and reduces boilerplate code.

Spring ORM - Integrates Hibernate, JPA, and other ORM tools for object-relational mapping.

Spring Data JPA - Provides ready-to-use CRUD repositories and custom query creation.

Kodewala

4. **Spring Security**

Spring Security - Adds authentication, authorization, password hashing and OAuth2.

5. **Spring Test**

Spring Test - Enables integration and unit testing using mock environments.

6. **Modern Add-ons (Frequently Used in Microservices & Cloud Apps)**

Spring Boot - Simplifies Spring application setup with auto-configuration and embedded servers.

Spring Actuator - Provides production-ready monitoring and health check endpoints.

Spring Cloud - Supports distributed systems patterns (config server, service registry, load balancing).

Spring Batch - Helps build robust and scalable batch processing jobs.

7. **Other Modules**

Spring Scheduling - Enables scheduled tasks using @Scheduled.

Spring Core - Inversion of Control & Dependency Injection (DI)

Spring – Inversion of Control (IoC)

- ✓ **IoC (Inversion of Control)** is a **design principle** where the control of object (**Bean**) creation and dependency management is **transferred from the program (developer)** to the **framework or container** (like Spring).

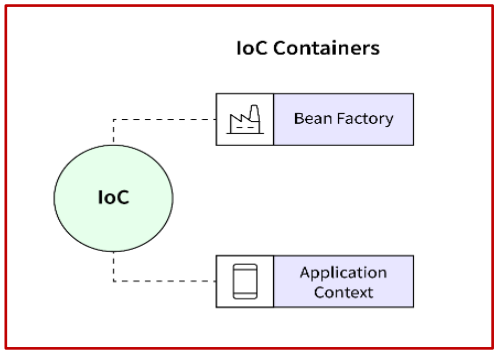
- ✓ It is responsible for **creating**, **initializing**, and **assembling** objects (inject required objects into each other), so we do not need to create them manually.

- ✓ **IoC Container** is responsible for overall object (**Bean**) lifecycle management.

kodewala

Spring IoC Container Types

There are two types of IoC containers.



BeanFactory - The **BeanFactory** is the simplest form of the Spring IoC container. It is the foundation interface for the Spring IoC container and provides the fundamental features of the container, including the ability to manage beans.

Key Features:

- ✓ **Lazy loading of beans**: Beans are only created when requested.
- ✓ **Lightweight**: Suitable for resource-constrained environments(Small Apps)

ApplicationContext - The **ApplicationContext** is a more advanced and feature-rich container than BeanFactory. It extends the BeanFactory interface and adds additional functionalities and integration with other Spring modules.

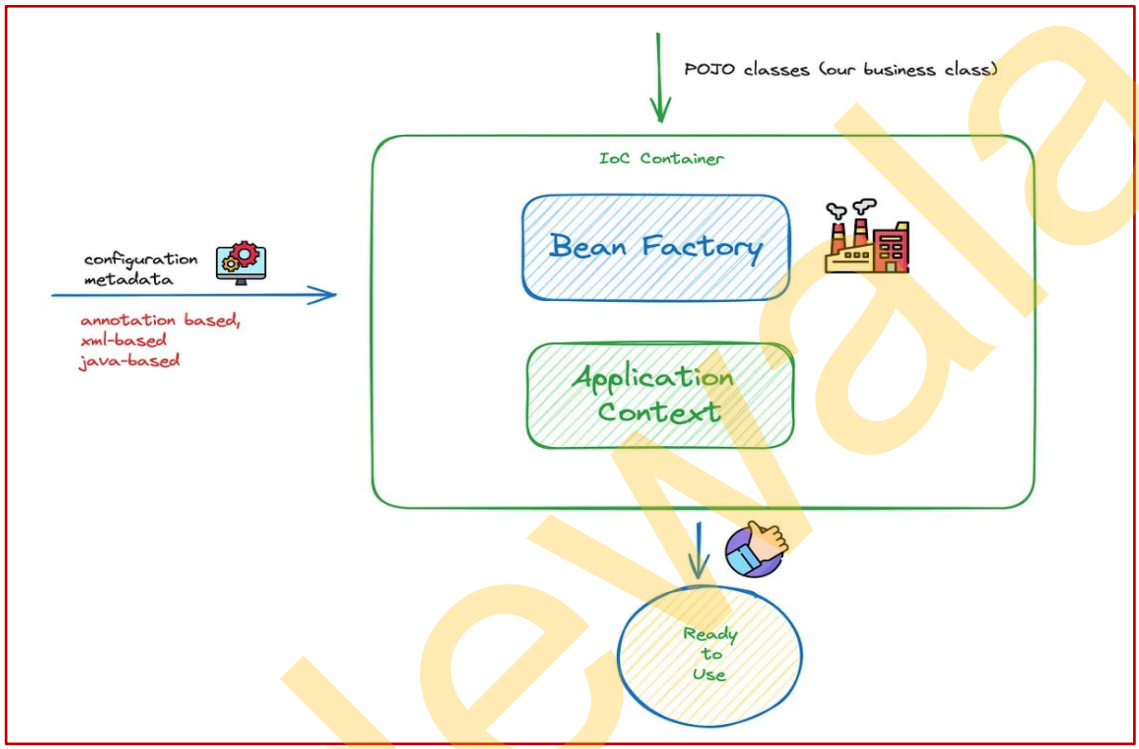
Key Features:

- ✓ **Eager loading of beans:** Beans are preloaded on container startup.
- ✓ **Suitable for enterprise applications:** Offers a wide range of functionalities for complex applications.
- ✓ **Supports multiple configurations:** XML-based, annotation-based, and Java-based configurations.

Kodewala

Spring IOC Architecture

- ✓ The IoC container requires certain info from the application to create and manage beans.
- ✓ We need to provide **POJO** class and **configuration** (XML, JAVA and Annotation)
- ✓ IoC container will create and manage the bean. We will be able to access/use the bean.



1. **Creating Bean (Using XML Type)**

XML Config(applicationContext.xml)

```
<bean id="myEmpObj" class="com.kodewala.main.Employee">

    <property name="firstName" value="Kodewala" />
    <property name="lastName" value="Academy" />
    <property name="email" value="kw@kodewala.com" />

</bean>
```

POJO

```
package com.kodewala.main;

public class Employee
{

    private String firstName;
    private String lastName;
    private String email;

    public String getFirstName()
    {
        return firstName;
    }
```

IoC Container

```
String configFile = "/com/kodewala/resources/applicationContext.xml"; // Rela

ApplicationContext context = new ClassPathXmlApplicationContext(configFile);

Employee emp = (Employee) context.getBean("myEmpObj");
```

Kodewala

2. Creating Bean (Using Java Type)

Java (SpringConfig.java)

```java
@Configuration
public class SpringConfig
{

    @Bean("user")
    public User createUser()
    {
        User user = new User();
        user.setFirstName("Kodewala");
        user.setLastName("Academy");
        user.setEmail("kw@kodewala.com");

        return user;
    }
```

POJO

```java
public class User
{
    private String firstName;
    private String lastName;
    private String email;

    public String getFirstName()
    {
        return firstName;
    }
```

IoC Container

```java
ApplicationContext context = new AnnotationConfigApplicationContext(SpringConfig.class)

User user = (User) context.getBean(User.class);

user.showInfo();
```

3. Creating Bean (Using Annotation)

Java (AppConfig.java)

```java
@Configuration
@ComponentScan("com.kodewala.main.bean")
public class AppConfig
{

}
```

POJO

```java
@Component
public class Employee
{
    @Value("kodewala")
    private String firstName;

    public String getFirstName()
    {
        return firstName;
    }
```

IoC Container

```java
ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);

Employee emp = (Employee) context.getBean(Employee.class);
```

Kodewala

When to Use Which Type of Bean Configuration

1. **XML Configuration**
   Use when:
   - ✓ You are working on a legacy project that **already uses XML**.
   - ✓ You want a clear separation between **configuration** and **code**.
   - ✓ The team is more comfortable with **XML-based** config.
2. **Java-Based Configuration**
   Use when:
   - ✓ You want full control over bean creation.
   - ✓ You prefer type-safe config.
   - ✓ You need to configure third-party or complex beans programmatically.
3. **Annotation-Based Configuration**
   Use when:
   - ✓ You are building a modern Spring application.
   - ✓ You prefer less boilerplate and quicker setup.
   - ✓ You want to keep configuration close to the classes.
   - ✓ Your project structure is modular, and you use component scanning.

What is a Spring Bean?

- ✓ A **Spring Bean** is an **object managed** by the **Spring IoC container.**
- ✓ In simple terms, it is a regular **Java object** whose **lifecycle** and **dependencies** are controlled by **Spring**.
- ✓ The class whose life cycle is managing by IOC then that class is called as **Spring Bean**

How Does Spring Identify a Bean?

A class becomes a spring bean when:

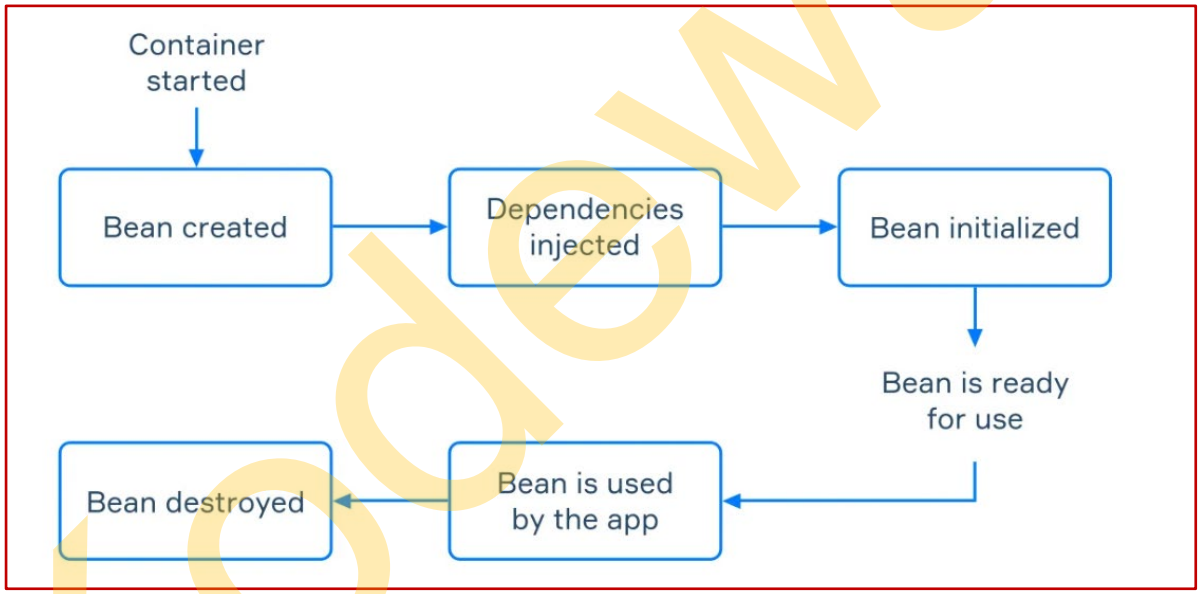
- ✓ It is scanned and registered using annotations like **@Component**, **@Service**, **@Repository**, etc.

It is returned from a **@Bean** method inside a **@Configuration** class.

kodewala

Spring Bean Life Cycle

- ✓ **Bean Instantiation**: Spring creates the bean instance using the bean definition.
- ✓ **Dependency Injection**: Spring injects dependencies into the bean through constructor or setter injection.
- ✓ **Bean Initialization:** Initialization methods like **@PostConstruct** or **InitializingBean** are called to perform custom setup.
- ✓ **Bean Usage**: The bean is used within the application, interacting with other beans.
- ✓ **Bean Destruction:** Cleanup methods like **@PreDestroy** or **DisposableBean** are invoked before the bean is destroyed.



Interview line:

In real-time, we prefer **@PostConstruct** for bean setup as it is clean, requires no interface, and works well with Spring's DI.

We use **afterPropertiesSet()** when we need Spring lifecycle control, often in framework-level or reusable components.

Kodewala

Code Example:

```
public class Employee implements InitializingBean, DisposableBean
{
    @Value("kodewala")
    private String firstName;

    @PostConstruct
    public void init()  1
    {
        System.out.println("Custom initialization logic executed!");
    }

    @Override
    public void afterPropertiesSet() throws Exception   2
    {
        System.out.println("Initializing logic after properties set");
    }

    @PreDestroy
    public void cleanup()   3
    {
        System.out.println("Cleanup logic executed before destruction");
    }

    @Override
    public void destroy() throws Exception  4
    {
        System.out.println("Destroying bean and releasing resources");
    }
```

Spring Bean scopes

Bean scope defines the lifecycle and visibility of a bean within the container. It tells Spring how many instances of a particular bean should be created and how long they should be available. Choosing the right scope allows for efficient resource management, better performance, and optimized memory usage.

singleton (default): A single shared instance per Spring container.

prototype: A new bean instance is created every time it's requested.

request: One bean instance per HTTP request (Web apps only).

session: One bean instance per user session (Web apps only).

application: Shared across all requests and sessions within a web app context.

Kodewala

Dependency Injection (DI)

The process of injecting **dependent object** into **target object** is called **Dependency Injection** (DI).

We can achieve dependency injection in 3 ways.

Setter Injection (SI)

- ✓ **Injects the dependency** by calling the setter method of the target class.

```
@Bean
public Employee createEmployee()
{
    Employee employee = new Employee();
    employee.setFirstName("Kodewala");
    employee.setLastName("Academy");
    employee.setEmail("kw@kodewala.com");
    employee.setBand("B3");
    // DI
    employee.setAddress(createAddress());
```

- ✓ **Flexible**, allows partial injection.
- ✓ **Used** when dependencies are optional.

Constructor Injection (CI)

- ✓ Injects the dependency via the constructor of the target class.

```
@Bean
public Employee createEmp()
{
    // Constructor based DI
    Employee employee = new Employee("Kodewala", createSalary());
    return employee;
}
```

- ✓ **Immutable**, ensures dependencies are not null.
- ✓ **Used** when dependencies are mandatory.

Field Injection

- ✓ Dependencies are injected directly into fields using annotations like @Autowired.
- ✓ No need for constructor or setter methods.

kodewala

When to Use Constructor vs Setter Injection

Constructor Injection

- ✓ **Use when:** Dependencies are mandatory
- ✓ **Advantages:** Ensures immutability, simplifies unit testing, enforces required fields.
- ✓ **Best for:** Services, configuration classes

Setter Injection

- ✓ **Use when:** Dependencies are optional or configurable
- ✓ **Advantages:** More flexible, allows reconfiguration.
- ✓ **Best for:** Beans with optional properties.

Field Injection

- ✓ **Use when:** Quick development or legacy code
- ✓ **Advantages:** Least boilerplate.
- ✓ **Disadvantages:** Hard to test, violates encapsulation.
- ✓ **Not recommended for:** Clean architecture or unit-testable designs.

Spring @Qualifier Annotation?

- ✓ **It resolves the ambiguity** when multiple beans of the same type are available for autowiring.
- ✓ To avoid this, we use @Qualifier("beanName") to **specify the exact bean** to inject.

What are Spring Profiles?

Spring Profiles allow you to **segregate parts of your application configuration** and **activate different beans** for **different environments** (like dev, test, prod).

Kodewala

Why Use Spring Profiles?

In real-world projects, the environment determines:

- ✓ Which DB to connect to (H2 for dev, Oracle for prod)
- ✓ Whether to enable caching or not

Profiles let you **activate only the relevant configuration or beans for that environment**.

Interview questions and answers

Scenario: You are building a payment module. You need to perform some setup before a bean is ready and cleanup after it is destroyed.

Question: How will you manage initialization and destruction of a Spring bean? What are the options?
Answer: Think of @PostConstruct, @PreDestroy, InitializingBean, DisposableBean and custom init-method/destroy-method.

Scenario: In your Spring Boot application, the **UserService** should have only one instance, but a **ReportGenerator** should be newly created for each request.
Question: How would you configure these two beans with different scopes in Spring?

Answer: Use @Scope("singleton"), @Scope("prototype")

Scenario: A new developer used @**Autowired** on a **private** field, but the application is failing with a **NullPointerException** at runtime.

Question: What could be the cause? How would you debug and resolve it? Should field injection be used at all?
Answer: Check if component scanning is enabled, and if the class is managed by Spring. Also discuss constructor injection as a better practice.

Scenario: You are refactoring a legacy Spring app where some beans are defined using **@Component** and others using **@Bean**.

Kodewala

Question: When should you use @Component over @Bean? Can you give a scenario where one is preferred over the other?
Answer: Use @Component for classes you own. Use @Bean when integrating third-party classes or need custom creation logic.

Scenario: You have two implementations of an interface **INotificationService**: **EmailService** and **SMSService**.

Question: How would you ensure Spring injects the correct implementation into different components? What would happen if you didn't use @Qualifier?

Answer: Use @Component("emailService"), @Component("smsService"), and @Qualifier("emailService"). Without qualifier, Spring throws NoUniqueBeanDefinitionException.

Scenario: Your application is low on resources, and the initialization time is critical.
Question: Would you use BeanFactory or ApplicationContext? Why? Can you explain their differences with respect to lazy loading and features?

Answer: BeanFactory is the core container, supports lazy loading. ApplicationContext is more feature-rich.

Scenario: Your service class uses a property for an API key which was hardcoded. You want to externalize it for better maintainability.

Question: How can you inject a value from **application.properties** into a Spring bean? What precautions should you take for production secrets?
Answer: Use @Value("${api.key}"). Use application.properties or application.yml. Use external config server or encrypted vaults in prod.

Kodewala

Scenario: You added a new field to a service and annotated it with @Autowired, but Spring throws a NoSuchBeanDefinitionException.
Question: What are the possible reasons for this error? How would you go about fixing it?
Answer: Check if the bean is registered, if component scan path includes it, or if you missed annotating with @Component, @Service, etc.

Scenario: Your app should use different data sources in dev and prod environments.
Question: How will you implement this using Spring Profiles? How do you activate the correct profile in different environments?
Answer: Use @Profile("dev") and @Profile("prod"). Activate using application.properties, command line, or environment variables.

Kodewala

Spring Data

- ✓ Spring Data is a part of the Spring ecosystem that simplifies data access and persistence.
- ✓ Spring data providing built-in methods to perform curd operation in Database.

Advantage of Spring Data

- ✓ Simplify DAO(Data Access) layer implementation.
- ✓ Eliminate writing repetitive JDBC/JPA code.
- ✓ Unify access patterns across different types of databases.
- ✓ Provide extensible repository support with paging, sorting, query derivation (from method name)
- ✓ Enable integration with relational (JPA) and NoSQL (MongoDB, Redis, etc.) stores.

Core Components

Features of Spring Data:

1. **Entity**

   The domain object mapped to a database table.

2. **Repository**

   Interface to perform CRUD operations.

3. **JpaRepository**

   Predefined interface with ready-made methods.

4. **Query Derivation**

   Allows you to write method names like **findByNameAndAge(...)** and Spring will generate the query behind the scenes.

5. **Paging and Sorting**

   Enable pagination across all datastore implementations.

   Classes: Page, Pageable, Sort

Kodewala

6. **Auditing Support**

Common annotations and logic for auto-filling fields like: @CreatedDate, @LastModifiedDate, etc.

7. **@Query Annotation**

For custom queries native SQL.

Basic Flow

- ✓ Define a JPA Entity
- ✓ Create a Repository interface
- ✓ Use the repository in your service/controller

1. Entity

```
@Entity
@Table
public class Payment
{

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int paymentId;

    @Column
    private int amount;
    @Column
    private String paymentStatus;
```

AUTO : GenerationType - GenerationType
IDENTITY : GenerationType - GenerationType
SEQUENCE : GenerationType - GenerationType
TABLE : GenerationType - GenerationType
class : Class<javax.persistence.GenerationType>

@Entity

- ✓ This annotation tells JPA that this class is an entity, i.e. it should be mapped to a database table.
- ✓ By default, the table name will be the same as the class name (Payment), unless specified with @Table(name = "table_name").

@Id

- ✓ Marks the id field as the primary key of the entity

Kodewala

@GeneratedValue(strategy = GenerationType.IDENTITY)

- ✓ IDENTITY - value of the primary key id will be **auto-generated by the database.**
- ✓ AUTO - JPA to automatically choose the best generation strategy based on the database
- ✓ SEQUENCE - Uses a database sequence.
- ✓ TABLE - Uses a separate table (e.g., id_generator) to track ID values.

Interview One Liner:

If you are building **production-grade applications**, always prefer:

- ✓ SEQUENCE (if your DB supports it) → it is **faster** and **more tunable** using allocationSize.
- ✓ Avoid TABLE unless absolutely necessary.
- ✓ Avoid AUTO in large-scale systems because its behavior varies by DB.

2. Repository

- ✓ StudentRepository: This is a **DAO layer** interface (Data Access Object)
- ✓ JpaRepository<Users, Long>: We are extending **JpaRepository**, a Spring Data interface that provides **CRUD operations**, **pagination**, and **sorting**.

```
public interface StudentRepository extends JpaRepository<Users, Long>
{
    List<Users> findByName(String name);

    List<Users> findByCourseContaining(String course);

}
```

In repository, you might use **@Query** as well.

```
public interface StudentRepository extends JpaRepository<Users, Long>
{
    List<Users> findByName(String name);

    List<Users> findByCourseContaining(String course);

    @Query("SELECT s FROM Student s WHERE s.course = :course")
    List<Users> getByCourse(@Param("course") String course);
```

Kodewala

Interview discussion:

When should you use **@Query** if Spring Data JPA already provides **derived query methods** (like **findByCourse**)?

Use Spring Data JPA **Method Names** When:

- ✓ Your query is **simple** and follows **naming conventions**.

```
List<Student> findByCourse(String course);
```

auto-generates

```
SELECT * FROM student WHERE course = ?
```

Use **@Query** When:

- ✓ You want to join multiple tables/entities.
- ✓ You want to fetch only selected fields
- ✓ Filters don't fit Spring's method naming rules.
- ✓ You want full control using real SQL

What is @Transactional?

- ✓ Defines a transactional boundary.
  - Begins a transaction at method start, commits on success, rolls back on exception.
- ✓ Applies to public methods only.
- ✓ Used at method or class level
  - Method-level overrides class-level settings.

Best Practice: Use @Transactional in the Service Layer

Why Service Layer?

- ✓ Business operations (like register, placeOrder, etc.) often involve **multiple DB calls.**
- ✓ Managing transactions here ensures **atomicity** (either all succeed or all fail).
- ✓ Keeps Repository methods reusable and simple.

Kodewala

```java
@Service
public class StudentService
{

    @Autowired
    private UserRepository repository;

    @Transactional
    public void registerStudent(Users users)
    {
        repository.save(users);
        repository.sendWelcomeMail(users); // another DB call
    }
}
```

Confused about JPA, Hibernate, and Spring Data JPA?

1. **JPA (Java Persistence API)**
   - ✓ Think of JPA like a **rulebook**.
   - ✓ It tells us how to map your Java objects to database tables and how to perform operations like **saving**, **updating**, or **deleting** records.

     Example:

   - ✓ We decide that your "Users" Java class will be stored in a table called "users" with columns for id and name.
   - ✓ JPA defines ***how*** we describe this mapping and what operations you can do.

2. **Hibernate**
   - ✓ Hibernate is like the **worker** who actually builds the house using the rule book. (Provided by JPA)
   - ✓ It implements the rules from **JPA** and talks directly to the database to save, fetch, or update your data.

     Example:

   - ✓ You tell Hibernate, "Save this User object." Hibernate translates that into a database command and runs it.

3. **Spring Data JPA**
   - ✓ Spring Data JPA is a part of the Spring Framework that provides a higher-level, easier-to-use API for working with JPA.

Kodewala

Popular JPA annotations

- ✓ **@Entity:** Marks a Java class as a JPA entity, which can be persisted to a database.
- ✓ **@Table:** Specifies the name of the database table to map the entity to.
- ✓ **@Id:** Specifies the primary key field of the entity.
- ✓ **@GeneratedValue:** Specifies how the primary key value should be generated(Auto, Sequence, Identity etc.)
- ✓ **@Column:** Specifies the mapping of the entity field to a database column.

```java
@Entity
@Table
public class Users
{
    @Id
    @Column
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column
    private String name;
    @Column
    private String email;
```

- ✓ **@JoinColumn:** Specifies the mapping of a foreign key column between two entities.
- ✓ **@OneToOne:** Defines a one-to-one relationship between two entities.

```java
@Entity
public class Student
{

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id") // This creates a foreign key column in the Student table
    private Address address;
```

cascade = CascadeType.ALL : If you perform any operation (like save, delete, update) on the parent entity, the same operation will automatically apply to the child entity.

Kodewala

- ✓ **@OneToMany:** Defines a one-to-many relationship between two entities.
- ✓ **@ManyToOne:** Defines a many-to-one relationship between two entities.
- ✓ **@ManyToMany:** Defines a many-to-many relationship between two entities.
- ✓ **@NamedQuery:** Defines a named query for an entity.
- ✓ **@NamedQueries:** Defines a set of named queries for an entity.

```
@NamedQueries({
    @NamedQuery(name = "User.findAll", query = "SELECT u FROM User u"),
    @NamedQuery(name = "User.findByEmail", query = "SELECT u FROM User u WHERE u.email = :email")
    })
@Entity
@Table
public class Users
{
    @Id
    @Column
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column
```

- ✓ **@Transient:** Specifies that an entity field should not be persisted to the database.

```
@Entity
@Table
public class Users
{
    @Id
    @Column
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column
    private String name;

    @Transient
    private String email;
```

- ✓ **@Version:** Used to mark a field as the **version number** for an entity. It is primarily used for **optimistic locking** to avoid **lost updates** in concurrent transactions.

```
@Entity
@Table
public class Users
{
    @Id
    @Column
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column
    private String name;
    @Column
    private String email;

    @Version
    private int version; // or long, Integer, Long – supported types
```

kodewala

How It Works

- ✓ Read an entity (version = 1).
- ✓ Modify it and try to update.
- ✓ JPA checks:
  - If version in DB == version in memory → update and increment version.
  - If not → throws OptimisticLockException (someone else modified it).

Optimistic Locking ?

- ✓ checks for conflicts only at the time of commit/update, rather than locking data while reading.

```
UPDATE account SET balance = 1000, version = 2 WHERE id = 201 AND version = 1;
```

- ✓ If no rows are affected → it means someone else updated it, and OptimisticLockException is thrown.

Interview Questions

1. What is JdbcTemplate used for?
   Answer: To execute SQL queries, updates, and extract data from ResultSets in a simplified manner.

2. What is Spring Data JPA?
   Answer: An abstraction over JPA to reduce boilerplate code for database operations using repositories.

3. What is the purpose of CrudRepository?
   Answer: Provides basic CRUD operations like save, findById, deleteById, etc.

4. What is the difference between JpaRepository and CrudRepository?
   Answer: JpaRepository extends CrudRepository and adds JPA-specific methods like batch operations etc.

5. How does Spring Data handle exceptions?
   Answer: Converts SQLException and JPA exceptions to Spring's DataAccessException hierarchy.

Koolewala

6. What is @Query in Spring Data JPA?
   Answer: Used to define custom JPQL or native SQL queries in repository interfaces.

7. What is the role of RowMapper in Spring JDBC?
   Answer: Converts each row of a ResultSet into a Java object.

8. What is a derived query method?
   Answer: A method name like findByUsername() where the query is derived from the method name.

9. What is @EnableJpaRepositories used for?
   Answer: Enables scanning of repository interfaces in Spring Data JPA.

10. What is the difference between findAll() and findById()?
    Answer: findAll() returns all records; findById() returns a specific record wrapped in Optional.

11. What is NamedParameterJdbcTemplate?
    Answer: A JdbcTemplate variant that allows using named parameters in queries.

12. How do you handle pagination in Spring Data?
    Answer: Use Pageable and Page<T> return types in repository methods.

13. What is optimistic locking in Spring Data JPA?
    Answer: A concurrency control mechanism using @Version field to prevent lost updates.

14. What is the purpose of @Repository annotation?
    Answer: Marks a class as a Spring-managed DAO component and enables exception translation.

15. How do you perform a batch insert using Spring JDBC?
    Answer: Use JdbcTemplate.batchUpdate() or named parameter template with batch operations.

16. How do you define a custom repository method in Spring Data JPA?
    Answer: Create a separate interface and implementation, then extend it in your main repository.

17. What is queryForObject() vs query() in JdbcTemplate?
    Answer: queryForObject() returns a single row; query() returns multiple rows.

Kodewala

18. What is @Transactional(readOnly = true)?
    Answer: Marks a transaction as read-only, which can help optimize performance.

19. What is @GeneratedValue in JPA?
    Answer: Specifies strategy for generating primary key values (AUTO, IDENTITY, SEQUENCE, etc.)

20. Can we use native SQL in Spring Data JPA?
    Answer: Yes, using @Query(value = "...", nativeQuery = true).

21. What is Pageable in Spring Data?
    Answer: Interface for pagination information like page number, size, sort, etc.

22. What is the purpose of @Table annotation in JPA?
    Answer: Specifies the table name to map the entity.

Kodewala

Spring Web MVC

- ✓ Spring MVC (Model-View-Controller) is a web framework built on top of the Spring Framework for building web applications.

- ✓ It separates the application logic (**Controller**), business logic (**Model**), and user interface (**View**) to promote a **clean** and **maintainable** structure.

Spring MVC Architecture

Model:

- ✓ Holds the **application data** and **business logic.**
- ✓ A **User** class or **Service** that fetches user details from the database.

View:

- ✓ Defines how the data is presented to the user.
- ✓ **JSP** and HTML page that shows user details.

Controller:

- ✓ **Handles user requests**, interacts with the model, and decides which view to return.
- ✓ A method mapped to a URL like **/getUser** that calls a **service** and returns a **page**.

DispatcherServlet(Front controller)

The central dispatcher that routes requests to the appropriate **controllers** and **views**. It is the **front controller** in Spring MVC architecture.

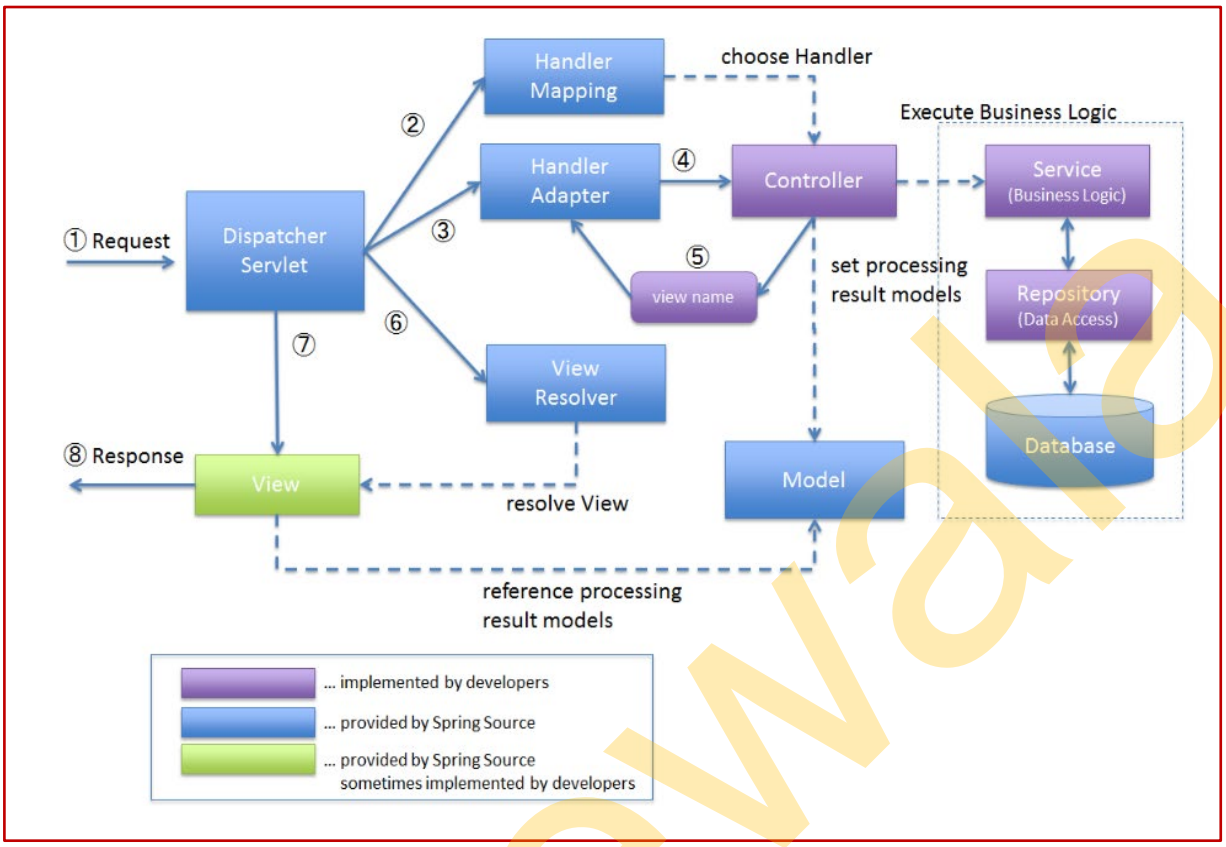
Spring MVC Request Flow

1. User sends a request to the DispatcherServlet.
2. DispatcherServlet consults HandlerMapping to find the correct controller.
3. The controller processes the request and returns a ModelAndView object.
4. DispatcherServlet consults ViewResolver to resolve the view.
5. The view is rendered and sent back to the user.

Client --> DispatcherServlet --> HandlerMapping --> Controller --> ViewResolver --> View --> Client

Kodewala

Spring MVC Processing Sequence (Internals)



1. **DispatcherServlet** receives the request.
2. **DispatcherServlet** dispatches the task of selecting an appropriate controller to **HandlerMapping**. **HandlerMapping** selects the controller which is mapped to the incoming **request URL** and returns the (**selected Handler**) and **Controller** to **DispatcherServlet**.
3. **DispatcherServlet** dispatches the task of executing of business logic of **Controller** to **HandlerAdapter**.
4. **HandlerAdapter** calls the business logic process of **Controller**.
5. **Controller** executes the business logic, sets the processing result in **Model** and returns the logical name of view to **HandlerAdapter**.
6. **DispatcherServlet** dispatches the task of resolving the View corresponding to the View name to **ViewResolver**. **ViewResolver** returns the View mapped to View name.
7. **DispatcherServlet** dispatches the rendering process to returned View.
8. **View** renders Model data and returns the response

Kodewala

DispatcherServlet & Configuration

How can you configure **DispatcherServlet?**

1. **XML-based**: Define DispatcherServlet in web.xml

```xml
<web-app>
  <display-name>Archetype Created Web Application</display-name>

    <servlet>
        <servlet-name>myds</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>myds</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>
```

2. **Java-based**: Use @Configuration, @EnableWebMvc, and extend WebMvcConfigurer

```java
@Configuration
@EnableWebMvc // Enables Spring MVC features (like @Controller, @RequestMapping)
@ComponentScan(basePackages = "com.kodewala.booking") // Scans for controllers and other beans
public class AppConfig implements WebMvcConfigurer
{


    @Bean
    public InternalResourceViewResolver viewResolver()
    {
        InternalResourceViewResolver resolver = new InternalResourceViewResolver();
        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");
        return resolver;
    }
```

Please Note: In case of Spring boot, @EnableAutoConfiguration takes care of it.

DispatcherServlet Lifecycle?

- ✓ Loaded by **web.xml** or **SpringBootServletInitializer**
- ✓ Initialized during application startup. i.e – load on startup.
- ✓ Delegates requests to controllers.

kodewala

Controller Layer – Basic annotation

- ✓ **@Controller**: Marks a class as a Spring MVC Controller.

```
@Controller
public class HelloController
{
    @GetMapping("/hello")
    @ResponseBody
    public String sayHello(Model model)
    {
        model.addAttribute("message", "Hello, Spring MVC!");
        return "welcome";
    }
}
```

- ✓ **@RestController**: Combines **@Controller** and **@ResponseBody**.
  - Automatically applies @ResponseBody to **all methods**, so you don't have to write it again and again.
  - **@RestController** = @Controller + @ResponseBody
  - Used to build **RESTful web services**
  - Best choice when building **REST APIs** that return JSON/XML/text.

```
@RestController
public class MyRestController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello from @RestController!";
    }
}
```

Interview One-Liner:

@RestController is suitable annotation that combines **@Controller** and **@ResponseBody**, making it ideal for REST APIs, while **@Controller** is typically used for web pages or when you need fine control over response handling.

- ✓ **@RequestMapping:**
  - Used to **map web requests to a controller method**.
  - It can handle **any HTTP method** (GET, POST, PUT, DELETE, etc.) using the method attribute.

Kodewala

```java
@Controller
public class MyController
{

    @RequestMapping(value = "/hello", method = RequestMethod.GET)
    @ResponseBody
    public String getHello()
    {
        return "Hello via RequestMapping (GET)";
    }

    @RequestMapping(value = "/hello", method = RequestMethod.POST)
    @ResponseBody
    public String postHello()
    {
        return "Hello via RequestMapping (POST)";
    }
```

- ✓ **@GetMapping -** Shortcut for @RequestMapping(method = RequestMethod.GET)
- ✓ **@PostMapping -** Shortcut for @RequestMapping(method = RequestMethod.POST)

```java
@GetMapping("/greet")
public String greet()
{
    return "Hello via GetMapping";
}

@PostMapping("/submit")
public String submitData(@RequestBody String data)
{
    return "Data received: " + data;
}
```

Request Handling

@RequestParam - Binds a query parameter (from the URL) to a method parameter.

URL: GET /greet?name=Kodewala **Response:** Hello, Kodewala

```java
@RestController
public class GreetingController
{

    @GetMapping("/greet")
    public String greet(@RequestParam String name)
    {
        return "Hello, " + name;
    }
}
```

✓Common for GET requests and form submissions and filtering etc.

Kodewala

✓ **@PathVariable -** Binds a **placeholder variable** in the URL path to a method parameter.

URL: GET /user/101 **Response: User ID: 101**

```
@RestController
public class UserController
{

    @GetMapping("/user/{id}")
    public String getUserById(@PathVariable int id)
    {
        return "User ID: " + id;
    }
}
```

One-liner for Interview:

Use **@PathVariable** when the value is part of the URL path identifying the resource (like an ID), and use **@RequestParam** for optional or extra information sent as query parameters

✓ **@RequestBody -** Bind Request Body Content (JSON, XML, etc.)

✓ Binds the HTTP request body to a Java object. Used mainly for POST and PUT.

Request Json:

```
{
  "name": "Kodewala",
  "age": 40
}
```

```
@RestController
public class UserController
{

    @PostMapping("/user")
    public String createUser(@RequestBody User user)
    {
        return "User: " + user.name + ", Age: " + user.age;
    }
```

Response Handling

1. String - Return View Name (page name)

- **Use when:** You are returning a **view (like JSP)** - **not** JSON or plain data.

```
@Controller
public class PageController
{

    @GetMapping("/home")
    public String showHomePage()
    {
        return "home"; // Resolves to /WEB-INF/views/home.jsp (or .html)
    }
}
```

Kodewala

✓ Spring uses **ViewResolver** to map "home" to a JSP

2. ModelAndView - Return View(page) + Data(what you want to display)

- **Use when**: You want to return **both the view name and the model data** in a single object.

```
@Controller
public class UserController
{

    @GetMapping("/user")
    public ModelAndView getUser()
    {
        ModelAndView mv = new ModelAndView("userView");
        mv.addObject("name", "Suresh");
        return mv;
    }
}
```

- Combines view name and data: returns **userView.jsp** with **name** = "Suresh" in the model.

3. ResponseEntity - For REST APIs

- **Use when**: You are building **REST APIs** and want **full control** over:
  - ✓ Response body
  - ✓ HTTP status
  - ✓ Headers

```
@RestController
public class ApiController
{

    @GetMapping("/api/data")
    public ResponseEntity<String> getData()
    {
        return ResponseEntity.status(200).header("Custom-Header", "value").body("Data sent successfully");
    }
}
```

kodewala

Exception Handling in Spring MVC

1. Local Exception Handling in Spring MVC (Per Controller)

- ✓ Local Exception Handling means handling exceptions within a specific controller, instead of globally.
- ✓ How? → Using **@ExceptionHandler** annotation

```
@Controller
public class CalculationController
{

    @GetMapping("/divide")
    @ResponseBody
    public String divide()
    {
        int result = 10 / 0; // Throws ArithmeticException
        return "Result: " + result;
    }

    // Local exception handler
    @ExceptionHandler(ArithmeticException.class)
    @ResponseBody
    public String handleArithmeticException(ArithmeticException ex)
    {
        return "Error: Division by zero is not allowed!";
    }
}
```

- ✓ This handler method will be triggered only for exceptions thrown inside this controller.

When to use local exception handling?

- ✓ When only one controller needs custom logic for an exception.
- ✓ When different controllers need different error messages or handling.
- ✓ For fine-grained control over controller-level errors

2. Global Exception Handling in Spring MVC

- ✓ Global Exception Handling means handling exceptions from all controllers in one centralized place, so you don't have to repeat error handling logic in each controller.
- ✓ How? → Using **@ControllerAdvice** + **@ExceptionHandler**

Let's handle **NullPointerException** globally.

kodewala

```java
@ControllerAdvice
public class GlobalExceptionHandler
{

    @ExceptionHandler(NullPointerException.class)
    @ResponseBody
    public String handleNullPointer(NullPointerException ex)
    {
        return "Global Error: Null value encountered!";
    }

    @ExceptionHandler(Exception.class)
    @ResponseBody
    public String handleGeneric(Exception ex)
    {
        return "Global Error: Something went wrong!";
    }
}
```

```java
@RestController
public class DemoController
{

    @GetMapping("/null")
    public String throwNull()
    {
        String str = null;
        return str.toUpperCase(); // Will cause NullPointerException
    }
}
```

HTTP Methods

GET: is used to retrieve data from the server.
Example use cases: fetching user details, listing all products etc.

POST is used to create new data on the server.
Example use cases: registering a new user, submitting a contact form etc.

PUT is used to fully update an existing resource.
Example use cases: updating a user profile completely, replacing product details etc.

PATCH is used to partially update a resource.
Example use cases: changing only the user's email, updating just the order status etc.

DELETE is used to remove data from the server.
Example use cases: deleting a user account, canceling an order, removing a product from the catalog.

HEAD is used to fetch only the headers, not the response body.
Example use cases: checking if a file exists, verifying file availability without downloading.

OPTIONS is used to find out which HTTP methods are supported by an endpoint.
Example use cases: CORS preflight requests etc.

kodewala

HTTP Status Codes

2xx - Success

- ✓ **200 OK** - Standard successful response.
- ✓ **201 Created** - When a new resource is created.
- ✓ **204 No Content** - Successful response but no content (like after DELETE).

3xx - Redirection

- ✓ **301 Moved Permanently** - For permanent URL redirects.
- ✓ **302 Found** - Temporary redirect (common in web apps).
- ✓ **304 Not Modified** - Used in caching to save bandwidth**.**

4xx – Client Errors (Very common in APIs)

- ✓ **400 Bad Request** - Invalid request data or syntax.
- ✓ **401 Unauthorized** - Missing or invalid authentication.
- ✓ **403 Forbidden** - Authenticated but access denied.
- ✓ **404 Not Found** - Resource doesn't exist.

5xx – Server Errors (Common server-side failures)

- ✓ **500 Internal Server Error** - Generic server error.
- ✓ **502 Bad Gateway** - Upstream server error.
- ✓ **503 Service Unavailable** - Server overloaded or down.
- ✓ **504 Gateway Timeout** - Timeout waiting for upstream.

Kodewala

Interview question and Answer

1. **What is the role of DispatcherServlet in Spring MVC?**
   Answer: It acts as the front controller to intercept requests and delegate them to appropriate handlers.

2. **How do you map a controller method to a URL using annotations?**
   Answer: Use @RequestMapping or its shortcuts like @GetMapping with the URL path.

3. **What is the difference between @Controller and @RestController?**
   Answer: @RestController combines @Controller and @ResponseBody, returning data directly instead of views.

4. **How do you handle HTTP GET and POST requests using Spring MVC annotations?**
   Answer: Use @GetMapping for GET and @PostMapping for POST methods on controller methods.

5. **Explain the use of @RequestMapping and its common attributes.**
   Answer: Maps URLs to methods; attributes include method, params, headers etc.

6. **What is the difference between @RequestParam and @PathVariable?**
   Answer: @RequestParam extracts query parameters; @PathVariable extracts values from the URL path.

7. **How do you bind form data to a Java object in Spring MVC?**
   Answer: Use @ModelAttribute on a method parameter representing the form object.

8. **What is the purpose of the @ModelAttribute annotation?**
   Answer: Binds request parameters to a model object and makes it available to the view.

9. **How do you return JSON response from a Spring MVC controller?**
   Answer: Use @ResponseBody or @RestController with Java objects, which are auto-converted by Jackson.

10. **Explain how Spring MVC supports content negotiation.**
    Answer: It uses the Accept header and configured converters to return the response in requested formats like JSON or XML.

11. **What is the use of @ResponseBody?**
    Answer: Indicates that the method return value should be bound to the web response body.

12. **How do you handle exceptions locally within a controller?**
    Answer: Use @ExceptionHandler annotated methods inside the controller.

Kodevala

13. **What is the purpose of @ExceptionHandler annotation?**
    Answer: Specifies a method to handle exceptions thrown by controller methods.

14. **How can you define global exception handling in Spring MVC?**
    Answer: Use @ControllerAdvice to handle exceptions across all controllers.

15. **How do you configure ViewResolvers in Spring MVC?**
    Answer: Define bean like InternalResourceViewResolver to map logical view names to actual views.

16. **How do you pass data from a controller to a view?**
    Answer: Use Model, ModelMap, or ModelAndView to add attributes accessible in the view.

17. **How do you enable CORS for a Spring MVC controller?**
    Answer: Use the @CrossOrigin annotation at class or method level.

18. **What is the use of @InitBinder annotation?**
    Answer: Initializes WebDataBinder for customizing request parameter binding.

19. **How do you upload files in Spring MVC using annotations?**
    Answer: Accept MultipartFile parameters in controller methods with multipart resolver configured.

20. **How can you validate form input in Spring MVC?**
    Answer: Use @Valid on form objects and check BindingResult for validation errors.

21. **How do you handle multipart file uploads configuration in Spring MVC?**
    Answer: Configure MultipartResolver bean and set max upload size limits.

22. **What are the differences between @GetMapping, @PostMapping, and @RequestMapping?**
    Answer: @GetMapping and @PostMapping are shortcuts for @RequestMapping with method GET or POST.

kodewala

23. **How do you use @RequestHeader to access HTTP headers?**
    Answer: Annotate method parameters with @RequestHeader and specify header name.

24. **How can you access raw HttpServletRequest or HttpServletResponse objects in Spring MVC?**
    Answer: Declare them as method parameters in controller methods.

Kodewala

Spring Supporting Modules

Security: Protects applications from unauthorized access and threats. Implemented for User login and role-based access control.

AOP (Aspect-Oriented Programming): Separates cross-cutting concerns like logging or transaction management from business logic.

Validation: Ensures that input data meets required rules before processing. i.e. Checking that a user's email is properly formatted during registration.

Cache: Stores frequently accessed data temporarily in memory to improve performance. i.e. Caching product details to reduce database load.

Spring Security

- ✓ Security is very important part of every enterprise web application.
- ✓ Security will protect our application and application's data.
- ✓ We can use Spring Security concept to secure our application.
- ✓ Before we implement spring security, we need to know about following concepts.

Authentication vs. Authorization:

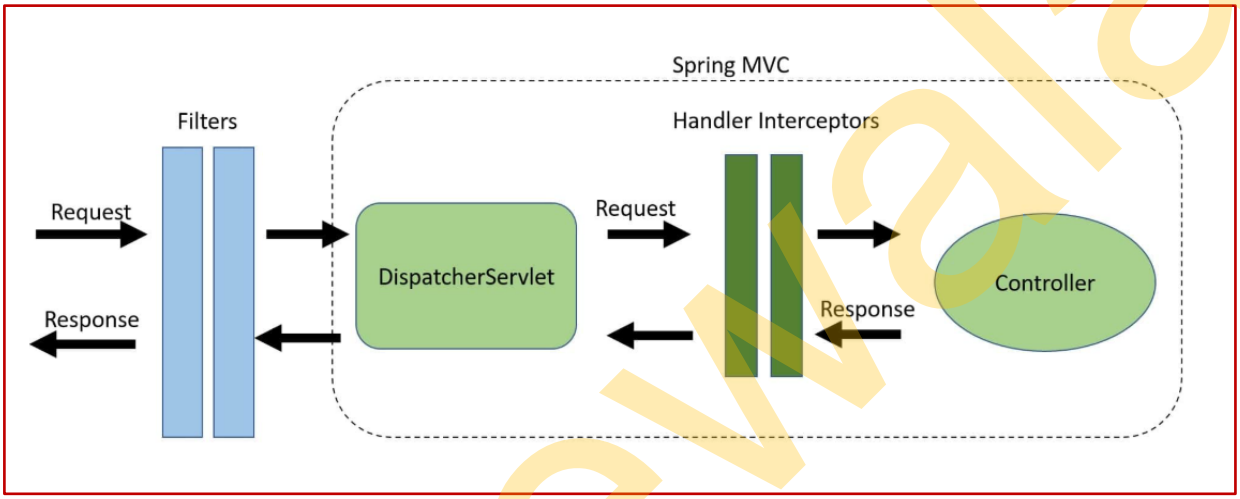
- ✓ **Authentication** verifies a user's identity. It validates **user name** and **password**.
- ✓ **Authorization** determines the access levels and permissions of an authenticated user. Once logged in, what can he access?

Koderwala

Before we move forward, lets understand the **filters**. Spring Security uses **filters** heavily to control security aspects of requests.

What is a Filter?

- ✓ A **Filter** is a component in **Java web applications** that can **intercept requests** and **responses**.
- ✓ It allows you to **process**, **modify**, or **block** HTTP requests and responses before they reach the servlet (**DispatcherServlet**).
- ✓ **Filters** are part of the Servlet API (javax.servlet.Filter).



Why Filters are important in Spring Security?

- ✓ Spring Security is built around the concept of a **filter chain** - a series of filters that process every HTTP request to secure your application.
- ✓ Each filter has a specific responsibility related to security.

Client -> Filter1 -> Filter2 -> ... -> Servlet -> ... -> Filter2 -> Filter1 -> Client

Spring Security request Flow in Spring MVC

Request → Security Filters → DispatcherServlet → Controller

Kodewala

Types of authentications in Spring Security?

Session-Based Authentication (Default)

JWT-Based Authentication

OAuth 2.0 Authentication (Third-party Login)

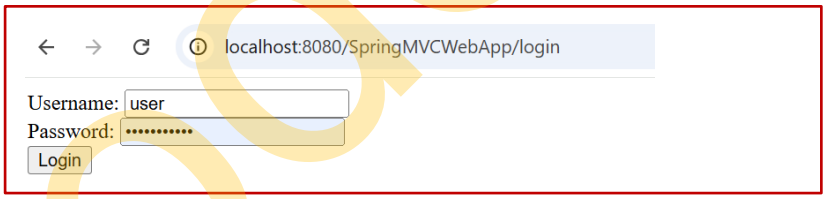
Session-Based Authentication (Default)

- ✓ In Spring Security, session-based authentication is the default mechanism.
- ✓ Once a user logs in successfully via the login form, Spring creates an HTTP session and stores the authentication object in the SecurityContext.
- ✓ This session is then used to authorize subsequent requests without requiring re-login, until logout or session expiry occurs.

1. **User enters credentials on login form (/login)**
   - login.jsp returned by controller method:

```
@GetMapping("/login")
public String login()
{
    return "login"; // Returns login.jsp
}
```

   - User fills username/password and submits form (/login).



2. **Servlet Container (Tomcat) receives request**
   - ✓ The container (e.g., Tomcat) receives the HTTP request.
   - ✓ It initiates the Servlet Filter Chain configured in web.xml (or via Java config).

```
<!-- Spring Security Filter -->
<filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>

<filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

Kodewala

UsernamePasswordAuthenticationFilter intercepts login request

- ✓ Extract username and password from HTTP POST /login.
- ✓ **Behind the scenes**: Spring Security auto-registers this filter in the filter chain.
- ✓ The filter creates an **UsernamePasswordAuthenticationToken** with submitted credentials.

```
public class SecurityConfig
{

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
    {
        http.authorizeHttpRequests(auth -> auth.requestMatchers(new AntPathRequestMatcher("/login")).permitAll()
                .requestMatchers(new AntPathRequestMatcher("/logout")).permitAll()
                .requestMatchers(new AntPathRequestMatcher("/admin/**")).hasRole("ADMIN")
                .requestMatchers(new AntPathRequestMatcher("/user/**")).hasAnyRole("USER", "ADMIN").anyRequest()
                .authenticated())
                .formLogin(form -> form.loginPage("/login").loginProcessingUrl("/login")
                        .defaultSuccessUrl("/home", true).failureUrl("/login?error=true").permitAll())
                .logout(logout -> logout.logoutSuccessUrl("/login?logout=true").permitAll())
                .csrf(csrf -> csrf.disable());

        return http.build();
    }
```

3. AuthenticationManager receives the authentication token

- ✓ The filter calls:

Authentication authentication = authenticationManager.authenticate(token);

- ✓ **AuthenticationManager:** Central interface for authentication process.

```
@Override
public Authentication attemptAuthentication(HttpServletRequest request, HttpServletResponse response)
        throws AuthenticationException {
    if (this.postOnly && !request.getMethod().equals("POST")) {
        throw new AuthenticationServiceException("Authentication method not supported: " + request.getMethod());
    }
    String username = obtainUsername(request);
    username = (username != null) ? username.trim() : "";
    String password = obtainPassword(request);
    password = (password != null) ? password : "";
    UsernamePasswordAuthenticationToken authRequest = UsernamePasswordAuthenticationToken.unauthenticated(username
            password);
    // Allow subclasses to set the "details" property
    setDetails(request, authRequest);
    return this.getAuthenticationManager().authenticate(authRequest);
}
```

4. AuthenticationProvider handles authentication logic

- ✓ Typically, **DaoAuthenticationProvider** is called.
- ✓ It uses your **UserDetailsService** and **PasswordEncoder** to authenticate.
- ✓ This is wired automatically by Spring Security due to our InMemoryUserDetailsManager,s

Kodewala

```java
@Bean
public InMemoryUserDetailsManager userDetailsService()
{
    UserDetails user = User.withUsername("user").password("{noop}user@12345").roles("USER").build();

    UserDetails admin = User.withUsername("admin").password("{noop}admin@12345").roles("ADMIN").build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

5. UserDetailsService loads user information

- ✓ User service loads user info by username.
- ✓ **This returns a UserDetails object** containing username, password, and roles.

6. PasswordEncoder verifies submitted password against stored one

- ✓ {noop} means no encoding (plain text comparison).
- ✓ Real apps should use encoders like BCryptPasswordEncoder.

7. AuthenticationProvider returns Authentication token if valid

- ✓ On successful authentication, it returns the token.
- ✓ On failure, throws exception caught by filter.

8. AuthenticationManager returns authentication result to filter

- ✓ The filter receives the authenticated token or an exception.
- ✓ If success, filter continues processing.
- ✓ If failure, redirects to login page with error.

9. SecurityContextPersistenceFilter stores Authentication in SecurityContext

✓ Authentication stored in SecurityContextHolder:

```
SecurityContextHolder.getContext().setAuthentication(authenticatedToken);
```

✓ This context is kept in HTTP session for subsequent requests.

Kodewala

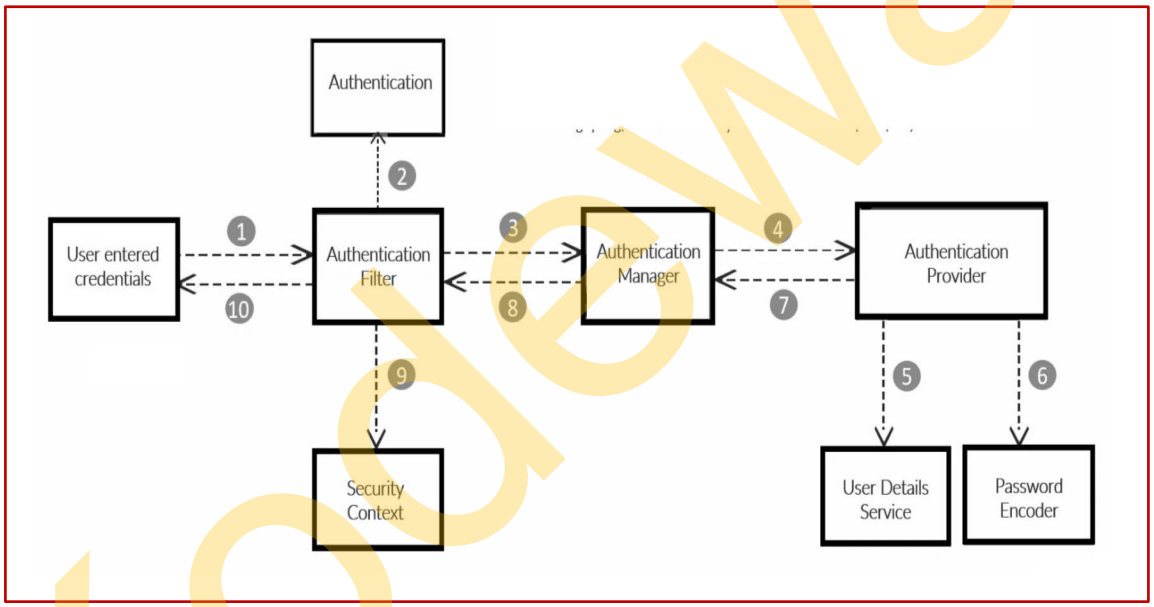
Interview Question - What happens on subsequent requests?

- ✓ The session keeps SecurityContext (usually in HttpSession).
- ✓ SecurityContextPersistenceFilter loads it for each request.
- ✓ So user stays authenticated without re-logging in.
- ✓ Authorization is checked on every request.

10. Request proceeds to DispatcherServlet

- ✓ With user authenticated, request reaches your controller.

Spring Security Process Flow(Session-Based Authentication - Default)



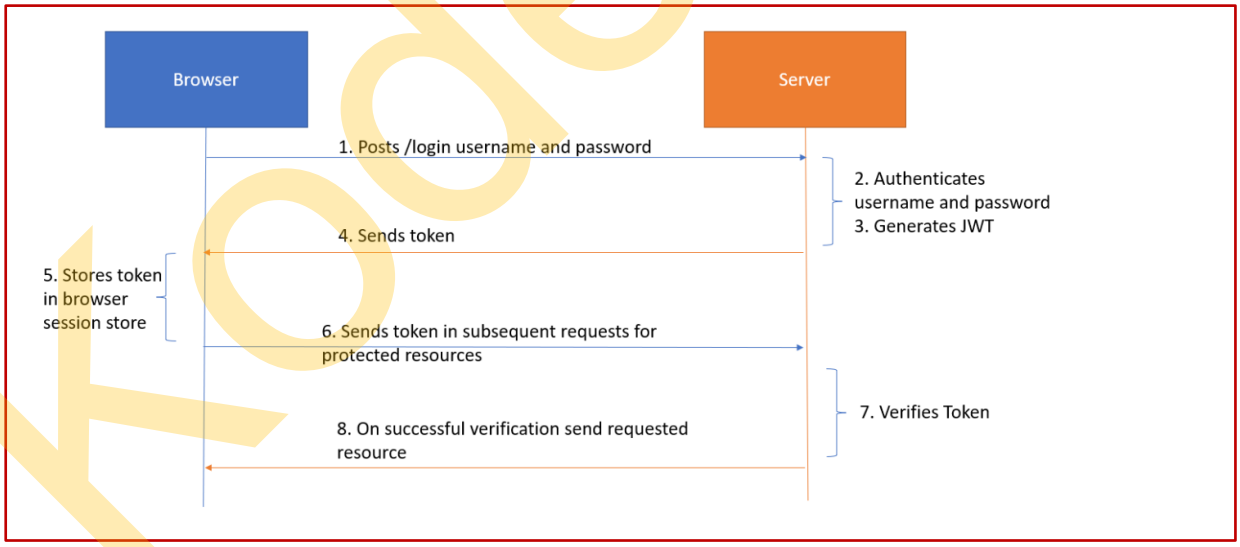
Koderwala

JWT (JSON Web Token) Authentication

- ✓ JWT is a stateless authentication mechanism where, upon successful login, a token is generated and sent to the client.
- ✓ The client stores this token (usually in localStorage or sessionStorage) and includes it in the Authorization header (Bearer <token>) of every request.
- ✓ The server validates the token without using any session, enabling scalability and decoupled services like in REST APIs or microservices.

Spring Security Process Flow(JWT Auth)

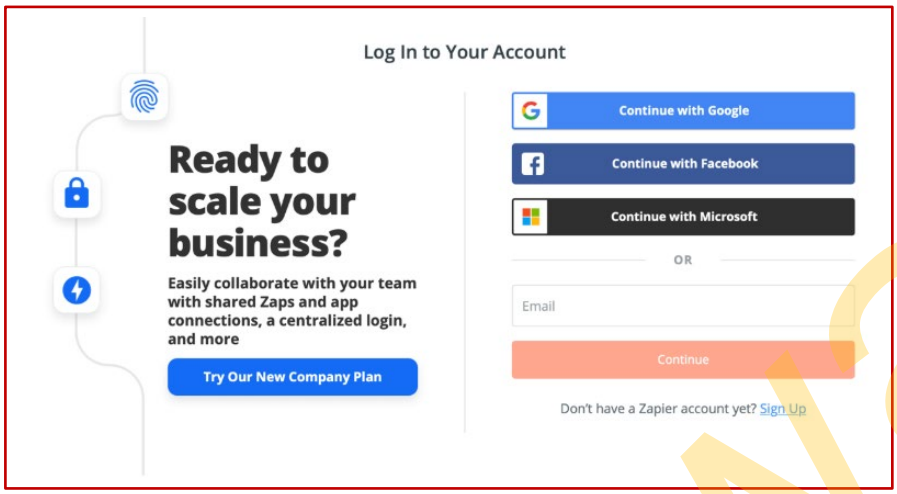
1. User sends /login request with username & password.
2. **JwtAuthenticationFilter** intercepts, extracts credentials, calls **AuthenticationManager**.
3. **AuthenticationManager** → **DaoAuthenticationProvider** calls **UserDetailsService** to load user and check password.
4. If valid, a JWT token is generated and returned to client.
5. Client stores JWT (e.g. in localStorage) and sends it in Authorization: Bearer <token> header in future requests.
6. JwtAuthorizationFilter intercepts every request, validates JWT, sets user in SecurityContext.
7. Spring uses this context to allow/deny access based on roles.



Kodewala

OAuth 2.0 Authentication (Third-party Login)

- ✓ OAuth (Open Authorization) is a token-based authorization framework.
- ✓ It allows a third-party application to access a user's resources (like profile, photos, or files) without exposing the user's credentials (like password).
- ✓ It's commonly used for **Login with Google, GitHub, Facebook**, etc.



OAuth2 Token Exchange Flow (Spring Security)

1. **User Clicks Login**
   Redirected to Identity Provider (e.g. Google).
2. **User Authenticates & Consents**
   Identity Provider redirects back with an **authorization code**.
3. **Spring Exchanges Code for Token**
   Sends code to token endpoint, receives **access token** (and optionally ID token).
4. **User Info Retrieved**
   Spring uses access token to fetch user details from the user info endpoint.
5. **Authentication Object Created**
   User info is wrapped into a DefaultOAuth2User, stored in **SecurityContext**.
6. **Access Granted**
   User is redirected to the protected resource (e.g., /home) as an authenticated user.

kodewala

When to use Session-based, JWT, and OAuth2 authentication mechanisms:

Session-Based Authentication

- ✓ Traditional **web apps** (Spring MVC, JSP etc.)
- ✓ **Small to medium-scale apps**
- ✓ App want to **store session on the server**

JWT (JSON Web Token)

- ✓ **REST** APIs
- ✓ Mobile apps where client and server are on **different domains**
- ✓ You want a **stateless** backend (microservices)
- ✓ Tokens shared between **multiple services**

OAuth 2.0

- ✓ **Third-party** login (Google, Facebook login)
- ✓ Delegated access (allow app to access user's Dropbox files)
- ✓ **Authorization** for APIs across different domains or organizations

Interview Question: In an e-commerce application like Amazon, which authentication mechanisms would you use for the web app, mobile app, and internal microservices and why?

- ✓ Use **session-based authentication** for secure, server-managed login.
- ✓ For mobile apps and microservices, use **JWT** for stateless, scalable, token-based authentication.

Spring Security Annotations

@EnableWebSecurity - Enable web security configuration.

@EnableMethodSecurity - Enable method-level security.

@PreAuthorize - Secures methods by allowing access based on roles. Fine-grained access control before method execution.

@PostAuthorize - Secures methods by allowing access based on roles. Fine-grained access control after method execution.

@Secured - Spring-specific, simple role-based access. (little flexible)

@RolesAllowed - JSR-250(Java Specification Request) standard, simple role-based access.

Kodewala

@Secured - Simple role check before method execution

```
@Secured("ROLE_ADMIN")
public void deleteDocument(Long id)
{
    // delete document logic
}
```

@RolesAllowed - Simple role check before method execution

```
@RolesAllowed("ROLE_USER")
public Document getDocument(Long id)
{
    // fetch and return document
    return null;
}
```

@PreAuthorize - Rich pre-execution check with method parameter access

```
@PreAuthorize("hasRole('ADMIN') or #owner == authentication.name")
public Document updateDocument(Long id, String owner)
{
    // update logic here
    return null;
}
```

@PostAuthorize - Authorization based on method return value

```
@PostAuthorize("returnObject.owner == authentication.name or hasRole('ADMIN')")
public Document getDocument(Long id)
{
    // fetch document logic
    return null;
}
```

Kodewala

Interview Question and answer

1. How does Spring Security work under the hood?

   Answer: You need to clearly explain that how Spring Security uses a filter chain (SecurityFilterChain) to intercept requests and how AuthenticationManager, AuthenticationProvider, and SecurityContextHolder collaborate internally.

2. Explain the flow of JWT authentication in a Spring application.

   Answer: Explain token generation on login, token validation on each request, and setting the authenticated user in SecurityContext.

3. Difference between session-based and JWT-based authentication? Which one is better for microservices?

   Answer: JWT is stateless and ideal for distributed microservices, while session-based requires central session storage.

4. What is the difference between @Secured, @PreAuthorize, and @RolesAllowed? When to use each?

   Answer: Expected to mention that @PreAuthorize supports SpEL and is preferred for fine-grained access control, while @Secured and @RolesAllowed are role-based.

5. How is OAuth2 login integrated with Spring Security?

   Answer: Explain knowledge of configuring Google/GitHub login and handling OAuth2 user details after login.

6. What is CSRF and how does Spring Security protect against it? When should you disable it?

   Answer: Explaining CSRF token mechanism and when to disable it (e.g., for stateless APIs) is a must-know.

7. What is the role of SecurityContext and SecurityContextHolder?

   Answer: Explain how Spring stores the authenticated user's details per request.

8. How would you implement custom authentication logic in Spring Security?

Kodewala

Answer: Good answers include implementing a custom AuthenticationProvider or UserDetailsService for DB or external service validation.

Kodewala

Spring Supporting Modules

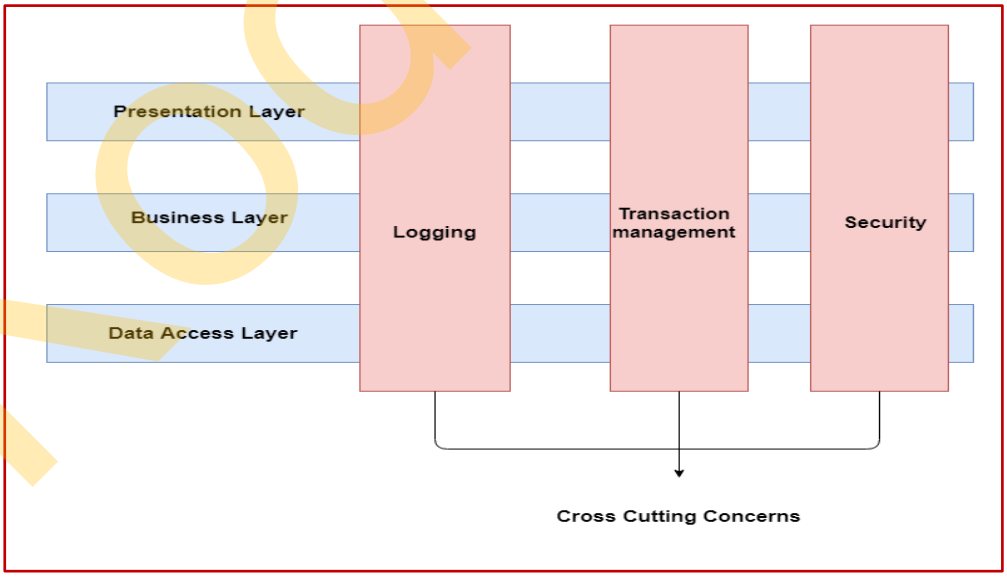
AOP (Aspect-Oriented Programming): Separates cross-cutting concerns like logging or transaction management from business logic.

Validation: Ensures that input data meets required rules before processing. i.e. Checking that a user's email is properly formatted during registration.

AOP (Aspect-Oriented Programming)

- ✓ **AOP (Aspect-Oriented Programming)** is a programming paradigm that allows you to modularize **cross-cutting concerns** like logging, security, transactions etc which are scattered across multiple classes.

- ✓ In **Spring**, AOP allows you to define **aspects** (common functionalities) separately and then apply them declaratively (like annotation) to different parts of your application (like method execution).



kodewala

Advantages of AOP in Spring

Separation of Concerns

- ✓ AOP cleanly separates cross-cutting concerns (logging, security, transactions, audit, caching etc) from business logic.
- ✓ Your service code focuses only on core functionality, making it cleaner and more maintainable.

Less Code Duplication

- ✓ No need to write repetitive code (logging, security checks) in every class/method.
- ✓ Write once in an aspect, and apply it wherever needed.

Better Modularity

- ✓ Each aspect is a modular unit that encapsulates a specific concern, improving code structure and modularity.

Let's understand with an example.

Without AOP:

```
if (isAdmin(user))
{
    log("Deleting user...");
    deleteUser(userId);
    log("User deleted");
}
```

With AOP:

```
@AdminOnly
@LogExecution
public void deleteUser(String userId)
{
   System.out.println("User with ID " + userId + " has been deleted.");
}
```

- ✓ Cleaner, modular, testable, and maintainable codebase.
- ✓ Our business method (deleteUser) is **clean and focused only on business rules**, not polluted with logging, auth, or exception handling.
- ✓ Logging, validation, security, or auditing can be **uniformly applied across methods** or even across modules without developers manually adding code everywhere.

Kodewala

Developing and Using Your Own Aspect

1. Add Maven Dependencies

```xml
<!-- Spring AOP support -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>6.2.2</version>
</dependency>

<!-- AspectJ Weaver (for @Aspect and runtime weaving) -->
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.19</version>
</dependency>
```

2. Create the Aspect Class

```java
@Aspect
@Component
public class LoggingAspect
{

    @Pointcut("execution(* com.kodewala.controller.*.*(..))")
    public void controllerMethods()
    {
    }

    @Before("controllerMethods()")
    public void logBefore(JoinPoint jp)
    {
        System.out.println(
                "[Before] " + jp.getSignature().toShortString() + " Args: " + java.util.Arrays.toString(jp.getArgs()));
    }

    @AfterReturning(pointcut = "controllerMethods()", returning = "result")
    public void logAfterReturning(JoinPoint jp, Object result)
    {
        System.out.println("[AfterReturning] " + jp.getSignature().toShortString() + " Returned: " + result);
    }
```

3. Enable AOP inside you SpringConfig or xml

```xml
<aop:aspectj-autoproxy />
```

OR **@EnableAspectJAutoProxy**

AOP Common Annotation

@Aspect - Marks a class as an Aspect (cross-cutting logic).
@Before - Executes before the matched method is invoked.
@After - Executes after the matched method finishes.
@AfterReturning - Executes only after the matched method returns successfully.
@Around - Executes before and after the matched method; can control whether the method executes at all.
@Pointcut - Used to define reusable expressions that match method executions (called join points).
Instead of repeating the same execution(...) logic in multiple places, you define it once using @Pointcut and reuse it.

Kodewala

Spring Validation

- ✓ Used to validate Java objects (like form beans) to ensure data integrity and correctness before processing.
- ✓ Commonly used in Spring MVC for validating user input.

How Do we Implement it?

1. Add required Dependencies.
2. Annotate bean class with proper required validation**.**

```
public class User {

    private int id;

    @Size(min = 5, message = "Enter atleast 4 characters")
    private String name;

    @NotEmpty(message = "City is required")
    private String city;
```

3. Add **@Valid** annotation to validate user input.

```
@PostMapping("user-detail")
public String postUsers(ModelMap map, @Valid User user, BindingResult result) {

    // go back to error page in case of any validation error
    if(result.hasErrors()) {
        return "user";
    }

    map.addAttribute("user", new User());
    System.out.println(user.getName() + " = " +user.getCity());
    return "welcome";
}
```

Common annotations

@Valid

- ✓ Tells Spring to validate the input object based on annotations like @NotNull, @Size etc.
- ✓ Placed before the @ModelAttribute or request body.

@Validator

- ✓ A Spring interface to write custom validation logic (if annotations are not enough).

Koolewala

What is BindingResult?

- ✓ Holds the result of validation (errors, if any).
- ✓ Must come immediately after the @Valid parameter

What is @Validated ?

```
@Controller
@Validated // Needed for method-level validation
public class MyController
{

    @GetMapping("/search")
    public String search(@RequestParam @NotBlank @Size(min = 3) String query)
    {
        // Validated automatically
        return "result";
    }
}
```

Caching

- ✓ It is a high-speed (Fast Access) data storage layer that stores a subset of data.
- ✓ In case requests for the data are served faster than accessing the data's primary storage location (such as a database, API, Filesystem etc.)

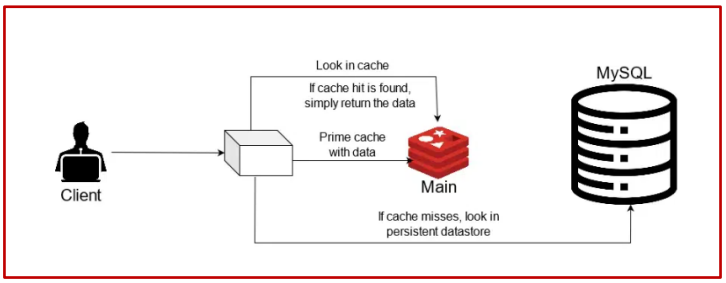
Use Case:

- ✓ Product details in an e-commerce site. Product info doesn't change often, but users request it frequently.
- ✓ Weather API or currency exchange rates. Data changes slowly but is requested often.
- ✓ User Session or Authentication Data. Fast lookup, avoids hitting the database for each request.
- ✓ Country codes, language mappings. Rarely changes but is accessed frequently.

One Liner: When do you use cache?

Data changes slowly but is requested often.

kodewala



Why is Caching Needed

1. **Performance Optimization**
   - ✓ **Reduced Latency**: Frequently accessed data is served from memory (RAM), significantly reducing retrieval time.
   - ✓ **Faster Response** Times: Improves user experience in web/mobile apps and APIs.
   - ✓ **Load Reduction**: Relieves backend systems (like databases or APIs) from repeated processing.
2. **Scalability**
   - ✓ Helps systems handle increased load without proportionally increasing backend resources.
3. **Cost Efficiency**
   - ✓ Reduces database compute and read costs, especially in cloud environments.

Types of Caches

1. **Client-side** - Stored in browser or local app memory (Web browser cache)
2. **Server-side** - App-level caching on the backend (In-memory (Redis etc))
3. **CDN Cache** - Stores static content at edge locations closer to users (Cloudflare)
4. **Database Cache** - Caches frequent DB queries or results (Redis)

Common Caching Strategies

1. **Cache-aside (Lazy Loading)**
   - ✓ Data is loaded into cache only on demand.
   - ✓ If data is not found in cache (cache miss), load from DB and populate cache.
   - ✓ Good for read-heavy applications.

Kodewala

2. **Write-through**
    - ✓ Data is written to cache and backing store simultaneously.
    - ✓ Ensures consistency, but slower writes.

3. **Write-behind (Write-back)**
    - ✓ Data is written to cache first, and backing store is updated asynchronously.
    - ✓ Fast writes, but risk of data loss if cache fails before write-back.

4. **Read-through**
    - ✓ Application interacts only with the cache.
    - ✓ Cache automatically fetches and stores data on miss.

General Terms

1. **Cache Hit & Miss:** Data found in cache → fast response.
2. **Miss:** Data not found → fetched from source → slower, but then cached.
3. **TTL (Time to Live):** Data expires after a fixed time.
4. **LRU (Least Recently Used)**
5. **LFU (Least Frequently Used)**

When Not to Cache

- ✓ Highly dynamic data that changes frequently.
- ✓ Sensitive data where consistency is critical.
- ✓ When memory is constrained and performance gain is minimal.

Best Practices

- ✓ Choose the right caching layer (app, DB, CDN) for your use case.
- ✓ Use TTLs to manage stale data.
- ✓ Monitor hit/miss ratios and adjust strategy.

Kodewala

Spring Cache

- ✓ Spring Cache is a caching abstraction provided by Spring to improve performance by reducing the number of calls to expensive methods (e.g., database access).

Annotations

- ✓ **@EnableCaching**: Enables Spring's annotation-driven cache management capability.
- ✓ **@Cacheable**: Marks a method to cache its result.

```
@Cacheable("product")
public String getProductByIdFromCatalog(String id)
{
    simulateDelay();
```

- ✓ **@CachePut**: Updates the cache without interfering with method execution.

```
@CachePut(value = "product", key = "#id")
public String updateProductCatalog(String id, String updatedProductJson)
{
    System.out.println("Updating catalog in DB and cache for ID: " + id);
    // In real case, this would update the DB. Here we just simulate.
    return updatedProductJson;
}
```

- ✓ **@CacheEvict**: Removes one or more entries from the cache.

```
@CacheEvict(value = "product", key = "#id")
public void removeProductFromCatalog(String id)
{
    System.out.println("Removing product with ID " + id + " from cache");
    // Simulate DB deletion
}
```

- ✓ **@Caching**: Allows grouping of multiple cache annotations.

```
@Caching(put = { @CachePut(value = "oldproduct", key = "#id") },
        evict= { @CacheEvict(value = "products", key = "#id") })
public String updateAndSyncCaches(String id, String newData)
{
    System.out.println("Updating both 'abcd' and evicting 'products'");
    return newData;
}
```

- ✓ **@CacheConfig**: Shares common cache-related settings at the class level.

```
@CacheConfig(cacheNames = "test")
@Service
public class ProductService
{
```

kodewala

How Caching Works

When a method annotated with @Cacheable is called:

- ✓ Spring checks if the result is already in the cache.
- ✓ If yes, it returns the cached value.
- ✓ If no, the method is executed and the result is stored in the cache.

Kodewala

Spring Boot

Spring Boot is an extension of the **Spring framework** that simplifies the **setup**, **configuration**, and **deployment** of Spring applications.

Advantage of Using Boot?

- ✓ **Auto-configuration**: It automatically configures your application based on the libraries in your class path. This reduces boilerplate(repetitive) configuration code.
- ✓ **Standalone Applications:** Spring Boot applications can run as standalone Java applications with embedded servers like Tomcat etc. and no need to deploy to external servers.
- ✓ **Rapid Development**: With minimal setup and configuration, developers can quickly start building applications.
- ✓ **Production Ready**: Comes with built-in tools for monitoring, metrics, and health checks (via **Spring Boot Actuator**).
- ✓ **Opinionated Defaults**: Provides sensible defaults for project setup, freeing developers from making low-level decisions initially.
- ✓ **Dependency Management**: Uses **spring-boot-starter** dependencies to bundle common libraries, making dependency management much easier.

In short:

- ✓ **Spring** gives you a powerful but flexible platform for Java enterprise apps.
- ✓ **Spring Boot** builds on that to make it faster and easier to get started and to go to production.

- ✓ In **Spring Boot**, the word **"Boot"** means **"to start up"** or **"to get going quickly."**
- ✓ So, **Spring Boot = a fast, easy way to get a Spring-based app up and running.**

Kodewala

Spring Framework vs Spring Boot

- ✓ **What it is:** A powerful, flexible Java framework for building web applications.
- ✓ **Requires manual setup:** You need to configure things like web servers, dependencies, XML files, etc.
- ✓ **More control:** Great for complex apps where you want full customization.
- ✓ **Steeper learning curve:** Beginners may find it hard to set up.

Example: You have to configure your own database connection, Tomcat server, and write lots of boilerplate code.

- ✓ **What it is:** Built **on top of Spring Framework** that makes it easier and faster to create Spring apps.
- ✓ **Auto-configuration:** Comes with **opinionated defaults** and auto setups (e.g. for databases, security, etc.).
- ✓ **Embedded server:** Runs apps directly with **no external server setup**.
- ✓ **Production-ready:** Includes health checks, metrics etc
- ✓ **Beginner-friendly:** Great for getting started quickly.

Example: Just add the right starter dependency (spring-boot-starter-web), and you are ready to create a REST API without any XML config or server setup.

Interview One Liner:

What is the default embedded server in Spring Boot, and how can you change it?

- ✓ Spring Boot uses Tomcat as the default embedded server, but we can easily change it to something else like Jetty etc.
- ✓ We just need to exclude the tomcat dependency and add jetty.

```
<!-- Spring Boot Web (Tomcat excluded) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>

<!-- Jetty instead of Tomcat -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

Kodewala

Convention over configuration

- ✓ Spring Boot Framework comes with sensible defaults and expects developers to follow certain naming and structural conventions.
- ✓ Allowing them to skip a lot of configurations unless they want to override the defaults.

Examples:

- ✓ Just add the right starter dependency and Spring Boot does the rest.
- ✓ Component Scanning - If you put your main class in the root package, Spring Boot will automatically scan the sub packages.
- ✓ Data Source Configuration: If you include a DB dependency (e.g. H2, MySQL), Spring Boot will, Auto-create a Data Source.

Starters in Spring Boot?

Starters = Pre-configured dependency

- ✓ **Spring Boot starters** are special **Maven** or **Gradle** dependencies(jars) that include a group of libraries commonly used for specific purposes
- ✓ We do not have to add each one manually.

Advantage

- ✓ Avoids version conflicts
- ✓ Speeds up development
- ✓ Promotes best practices

Types of Starters in Spring Boot

1. Web and API Development

- ✓ spring-boot-starter-web → Traditional Spring MVC

2. Database Access

- ✓ spring-boot-starter-data-jpa → For JPA/Hibernate

kodewala

- ✓ spring-boot-starter-jdbc → For basic JDBC
- ✓ spring-boot-starter-data-mongodb → MongoDB support

3. Security

- ✓ spring-boot-starter-security → Adds login, authentication etc

4. Templating / View Engines

- ✓ spring-boot-starter-thymeleaf → Thymeleaf

5. Testing

- ✓ spring-boot-starter-test → JUnit, Mockito etc.

What is @SpringBootApplication?

- ✓ **@SpringBootApplication** is a **meta-annotation** in Spring Boot that **marks** the main class of a Spring Boot application and enables **auto-configuration**, **component scanning**, and **Spring Boot settings**.

- ✓ It combines three key annotations

  - ***@SpringBootConfiguration*** : Marks the class as a source of bean definitions (a variant of @Configuration).

  - ***@EnableAutoConfiguration*** : Enables Spring Boot's auto-configuration based on classpath and environment.

  - ***@ComponentScan***: Scans the package of the main class and sub-packages for Spring components.

```
@SpringBootApplication
public class Application
{

    public static void main(String[] args)
    {
        SpringApplication.run(Application.class, args);
    }

}
```

Koderwala

How Can we exclude the scanning?

```
@SpringBootApplication(
        scanBasePackages = "com.kodewala",
        exclude = { SecurityAutoConfiguration.class })
public class Application
{

    public static void main(String[] args)
    {
        SpringApplication.run(Application.class, args);
    }

}
```

Why Would You Exclude a Package?

- ✓ Avoid loading test/mock classes in production.
- ✓ Prevent auto-registration of legacy or unwanted components.
- ✓ Improve startup time and memory usage.

Best Practices

- ✓ Place the main class in the root package.
- ✓ Let Spring Boot handle as much as possible via convention over configuration.
- ✓ Use application.properties(key=value) or application.yml(nested) for further configuration

Spring Initializr

- ✓ Spring Initializr : It is spring boot project generator
- ✓ A web-based project scaffolding tool for Spring Boot.
- ✓ Generates ready-to-import Maven or Gradle projects.
- ✓ Hosted at: https://start.spring.io

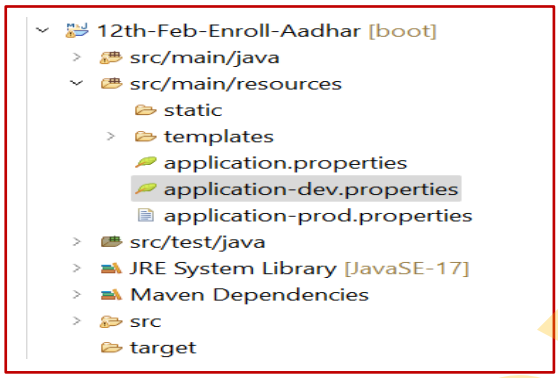
Benefits of Using Spring Initializr?

- ✓ Fast project setup
- ✓ No manual dependency management
- ✓ Ensures proper project structure
- ✓ Reduces configuration overhead

Kodewala

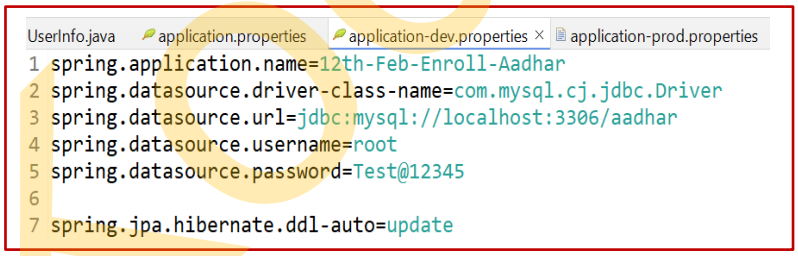
Properties and Profiles in Spring

- ✓ Spring Boot uses "**application.properties**" or "**application.yml**" files to externalize configuration.
- ✓ **application.properties** or **application.yml** is the default configuration file.
- ✓ Located in src/main/resources/



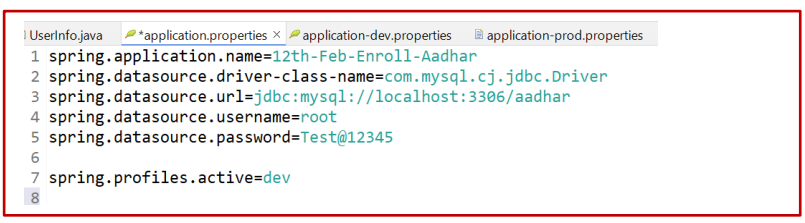
Using Profiles

- ✓ Spring Boot supports profiles to define environment-specific settings (e.g., dev, test, prod).
- ✓ Use naming like: "application-dev.properties" and "application-prod.properties"



Activating a Profile

Option 1: In application.properties:



Kodewala

Option 2: As command-line argument:

```
java -jar app.jar --spring.profiles.active=prod
```

Option 3: As environment variable:

```
SPRING_PROFILES_ACTIVE=dev
```

We can see in the log activated profile.

```
started by Suresh in C:\Users\Suresh\OneDrive\Desktop\Academy\eclipseworkspace\12th-Feb-Enroll-Aadhar)
2025-06-23T17:59:34.876+05:30  INFO 21340 --- [12th-Feb-Enroll-Aadhar] [           main] com.kodewala.main.Application               : The
following 1 profile is active: "dev"
```

Best Practice

- ✓ Keep shared/default settings in application.properties.
- ✓ Use profile-specific files to override only what's different per environment (e.g., DB URLs, logging levels, server ports, etc.).

Spring Boot Actuator

- ✓ Spring Boot Actuator provides production-ready endpoints to monitor and manage your application i.e. health checks, metrics and environment etc.

Features:

- ✓ Health checks (/actuator/health)
- ✓ Metrics (/actuator/metrics)
- ✓ Application info (/actuator/info)
- ✓ Environment details (/actuator/env)
- ✓ Integration with Prometheus, Grafana, Spring Boot Admin

Dependency: spring-boot-starter-actuator

Configuration:

application.properties | application.properties × | HelloController.java | Spring-Boot-Actuator/pom.xml

```
1 spring.application.name=Spring-Boot-Actuator
2
3 # Expose selected endpoints
4 management.endpoints.web.exposure.include=health,info,metrics,prometheus
5
6 # Show full health details
7 management.endpoint.health.show-details=always
8
9 # Change base path (optional)
10 management.endpoints.web.base-path=/actuator
```

Kodewala

Testing

✓ Specialized test annotations for Spring Boot testing:

@SpringBootTest → Full integration test

@WebMvcTest → Test Spring MVC (Controller layer) only

@DataJpaTest → Test JPA repositories with H2, in isolation

@MockBean → Spring Boot test mock injection

Spring Boot Interview Questions and Answers

1. What is Spring Boot?

Answer: Spring Boot is a framework built on top of Spring that simplifies setup and development by providing auto-configuration, starter dependencies, and embedded servers.

2. What are the key features of Spring Boot?

Answer: Auto-configuration, starter dependencies, embedded servers, opinionated defaults, actuator endpoints, and easy deployment.

3. What is a starter dependency?

Answer: A convenient dependency that aggregates commonly used libraries for a feature, e.g. spring-boot-starter-web includes Spring MVC, Tomcat, Jackson, etc.

4. What is auto-configuration in Spring Boot?

Answer: It automatically configures Spring application based on the dependencies on the classpath.

5. What is @SpringBootApplication?

Answer: A meta-annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.

6. How does Spring Boot differ from the Spring Framework?

Answer: Spring Boot reduces boilerplate and configuration by auto-configuring based on dependencies, while Spring Framework requires manual config.

Kodewala

7. How to run a Spring Boot application?

Answer: Using SpringApplication.run(Application.class, args) or by running the generated jar file.

8. What is the default embedded server in Spring Boot?

Answer: Tomcat (can be switched to Jetty etc).

9. How to externalize configuration in Spring Boot?

Answer: Using application.properties or application.yml files, environment variables or command-line arguments.

10. What are Spring profiles?

Answer: Mechanism to segregate parts of your application configuration and beans based on the environment (dev, test, prod).

11. What is Spring Boot Actuator?

Answer: A module to provide monitoring and management endpoints such as health, metrics, environment info.

12. How do you secure actuator endpoints?

Answer: By integrating Spring Security and configuring access rules or by setting management security properties.

13. What is the difference between @Component, @Service, @Repository, and @Controller?

Answer: They are stereotypes indicating the role of the bean but function similarly. @Repository handles persistence exceptions, @Controller handles web requests.

14. How to create a RESTful service in Spring Boot?

Answer: Use @RestController and map HTTP requests using @GetMapping, @PostMapping, etc.

15. What is the purpose of application.properties?

Answer: To configure application settings such as server port, datasource, logging, and custom properties.

16. How can you run code on Spring Boot application startup?

Answer: Implement CommandLineRunner or ApplicationRunner interfaces.

17. What is the use of @EnableAutoConfiguration?

Answer: Enables auto-configuration; usually included in @SpringBootApplication.

kodewala

18. How to connect Spring Boot with a database?

Answer: Configure datasource properties in application.properties and use Spring Data JPA or JDBC.

19. What is Spring Data JPA?

Answer: A module to simplify database access by implementing repositories based on JPA.

20. How to handle exceptions globally in Spring Boot?

Answer: Use @ControllerAdvice and @ExceptionHandler.

21. What is a Spring Boot starter parent?

Answer: A parent POM that manages dependency versions and plugin configuration.

22. How does Spring Boot implement dependency injection?

Answer: Through constructor, setter, or field injection using annotations like @Autowired.

23. How to test a Spring Boot application?

Answer: Use annotations like @SpringBootTest, @WebMvcTest, @DataJpaTest and tools like JUnit and Mockito.

24. What is the difference between @RestController and @Controller?

Answer: @RestController is a convenience annotation that combines @Controller and @ResponseBody, automatically serializing return values to JSON/XML.

Kodewala

Microservices

Software applications can be developed using two primary architectural approaches:

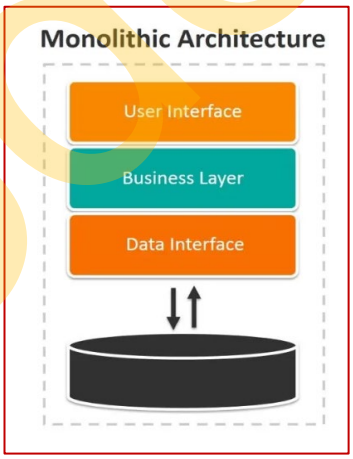
- ✓ **Monolithic Architecture**
- ✓ **Microservices Architecture**

What is Monolithic Architecture?

Monolithic Architecture is a traditional software development approach where all the components and functionalities of an application are built, packaged, and deployed as a single unit.

In **Monolithic** architecture, the entire application typically consists of:

1. **Presentation Layer** - Manages the user interface and user interaction.
2. **Business Logic Layer** - Contains the core processing logic, rules, and computations.
3. **Data Access Layer** - Handles data persistence and interactions with the database.



kodewala

Drawbacks of Monolithic Architecture?

- ✓ **Single Point of Failure**
  A failure in one module can bring down the entire application.

- ✓ **Technology Lock-In**
  The entire application is usually tied to a single technology stack, making it harder to adopt new tools or frameworks.

- ✓ **Tight Coupling**
  Components are closely interlinked, so changes in one part can affect others and increase regression risks.

- ✓ **Complex Deployment**
  Even small changes require the entire application to be rebuilt and redeployed.

- ✓ **Difficult Maintenance**
  As the codebase grows, understanding, testing, and maintaining the system becomes increasingly hard.

- ✓ **Server Load Concentration**
  All components run together, consuming more resources and potentially overloading the server.

- ✓ **Limited Scalability**
  The application must be scaled as a whole, even if only one module needs more resources.

Note:

To overcome the limitations of Monolithic, we adopt a Microservices based Architecture approach.

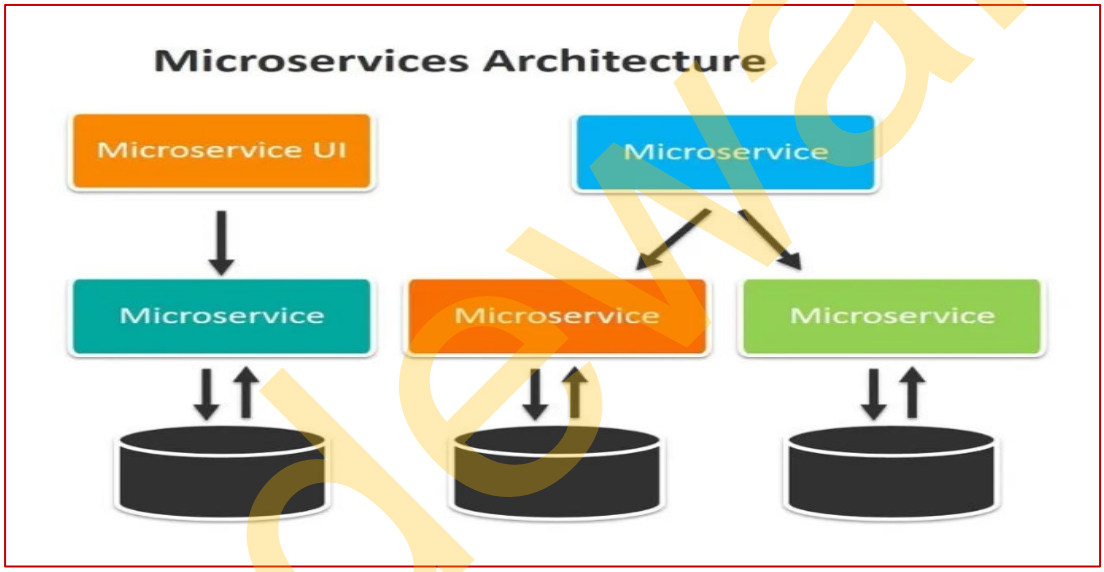
Kodewala

Introduction to Microservices

- ✓ Microservices is **not a technology**
- ✓ It is **not a programming language**
- ✓ It is **not a framework**
- ✓ It is **not an API**
- ✓ **Microservices is an architectural design pattern**

    - **Microservice** is an architecture style that **structures** an **application** as a collection of **small**, **independent**, and **loosely coupled** services.

    - **Microservice** design pattern can be implemented using **any technology stack**, allowing teams to build, deploy, and scale each service independently.



Advantages of Microservices Architecture

- ✓ **Loosely Coupled**
  Each service is independent, making it easier to develop, test, and deploy without affecting others.

- ✓ **Reduced Server Load**
  Services can be distributed across multiple servers, balancing the load and improving performance.

- ✓ **Easier Maintenance**
  Small, focused codebases make it simpler to identify and fix bugs or add new features.

Kodewala

- ✓ **No Single Point of Failure**
  If one service fails, it does not crash the entire system, other services can continue to run.

- ✓ **Technology Independent**
  Different services can use different programming languages or databases best suited to their needs.

- ✓ **Faster Delivery and Deployment**
  Teams can work on individual services in parallel, enabling quicker releases and updates.

Challenges with Microservices Architecture

- ✓ **Bounded Context**
  Bounded context means identifying how many micro services we need to develop for one application and deciding which functionality we need to add in which micro service.

- ✓ **Repeated Configurations**
  Common configurations (e.g. data sources, Kafka, Redis etc.) often need to be duplicated across multiple services, leading to redundancy.

- ✓ **Limited Visibility**
  Since services are distributed, developers may not get exposure to or control over the entire application's functionality, reducing holistic understanding.

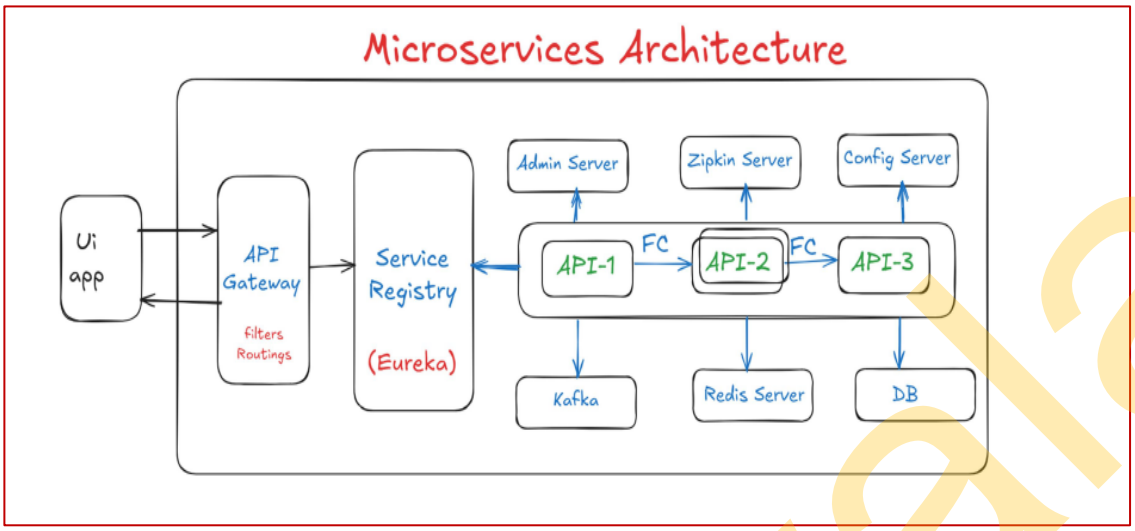
Microservices Architecture

- ✓ There is no **fixed architecture** for micro services development.
- ✓ The architecture can be **customized based on the specific needs and requirements** of the project.
- ✓ In Microservices Architecture, we commonly use several supporting components to manage service communication, configuration, discovery, security, and monitoring.

1) Service Registry (Eureka)	5) Kafka Server
2) Admin Server	6) Redis Server
3) Zipkin Server	7) FeignClient
4) Config Server	8) API Gateway

Kodewala

Note: All components are **optional and can be used based on project demand** and they are not mandatory in every microservices setup.



1. API Gateway

- ✓ Acts as the **entry point** for all backend services.
- ✓ Handles **routing, filtering, security** and **rate-limiting**.
- ✓ **Example:** Spring Cloud Gateway, Netflix Zuul

2. Service Registry (Service Discovery)

- ✓ Keeps track of all service instances with their **names, URLs, and status**.
- ✓ Helps services **discover each other** without hardcoding URLs.
- ✓ **Example:** Eureka Server, Consul, Zookeeper

3. Admin Server

- ✓ Used to monitor and manage all services in one place.
- ✓ Displays Actuator endpoints like health, metrics, environment etc.
- ✓ **Example**: Spring Boot Admin Server

4. Zipkin Server (Distributed Tracing)

- ✓ Tracks request flow across multiple services.
- ✓ Helps identify slow services or bottlenecks.
- ✓ **Example**: Zipkin, Jaeger

kodewala

5. Config Server

- ✓ Externalizes all application config (YML/Properties).
- ✓ Supports **centralized and environment-based config**.
- ✓ **Example:** Spring Cloud Config Server

6. Feign Client

- ✓ Simplifies **inter-service REST communication**.
- ✓ Reduces boilerplate code for calling other services.
- ✓ **Example:** Spring Cloud OpenFeign

7. Kafka Server (Message Broker)

- ✓ Used for **asynchronous communication** between services.
- ✓ Based on **Publish-Subscribe** model.
- ✓ **Example:** Apache Kafka, RabbitMQ

8. Redis Server (Cache)

- ✓ Stores frequently used data in key-value format.
- ✓ Helps reduce database calls and improve speed.
- ✓ **Example**: Redis

9. Circuit Breaker

- ✓ Stops calling a service that's repeatedly failing.
- ✓ Helps build fault-tolerant microservices.
- ✓ **Example**: Resilience4j, Hystrix

kodewala

10. Load Balancer

- ✓ Distributes traffic across multiple service instances.
- ✓ Improves availability and scalability.
- ✓ **Example**: Spring Cloud LoadBalancer, Netflix Ribbon, Nginx

11. API Documentation

- ✓ Auto-generates and visualizes API documentation.
- ✓ Allows testing APIs from the browser.
- ✓ **Example**: Swagger UI, Springdoc OpenAPI

12. Security Server

- ✓ Provides authentication and authorization for APIs.
- ✓ Supports login, roles, OAuth2, tokens, etc.
- ✓ **Example**: Spring Security, Keycloak, Okta

13. Prometheus (Monitoring)

- ✓ Collects metrics from services (CPU, memory, request counts).
- ✓ Helps in health monitoring and alerting.
- ✓ **Example**: Prometheus

14. Grafana (Dashboard)

- ✓ Displays visual dashboards and charts from Prometheus metrics.
- ✓ Used for real-time performance tracking.
- ✓ **Example**: Grafana

15. ELK Stack (Logging)

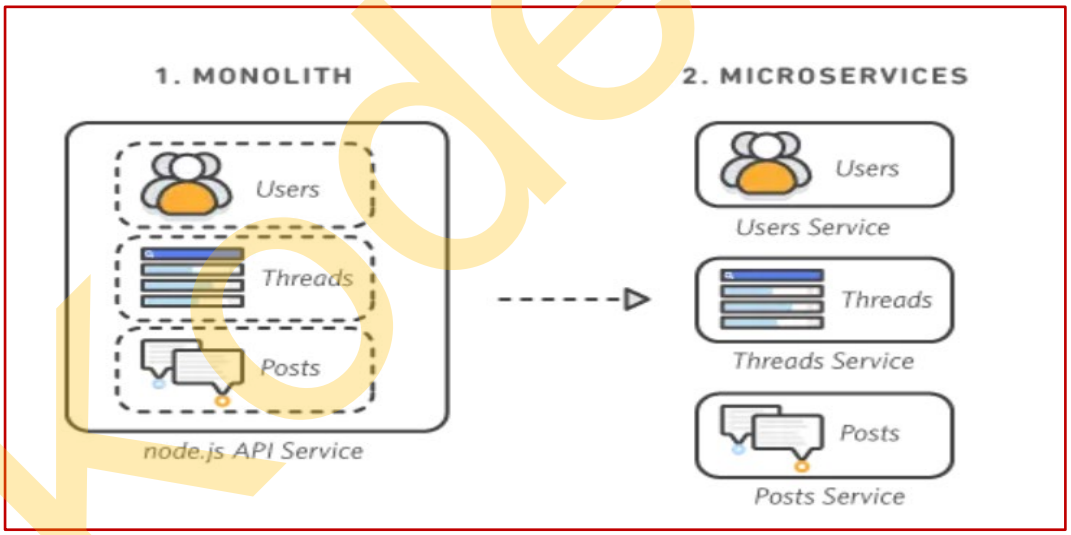
- ✓ Centralizes and visualizes logs from all services.
- ✓ Useful for debugging and tracking issues.
- ✓ Example: Elasticsearch + Logstash + Kibana

kodewala

When to Use Monolithic vs. Microservices Architecture

1. **Application Size**
    - ✓ **Monolithic** is ideal for small, simple apps or prototypes due to a single codebase and minimal setup.
    - ✓ **Microservices** suit large, complex systems with scalable and modular design.
2. **Team Competency**
    - ✓ **Monolithic** requires general development skills.
    - ✓ **Microservices** need expertise in cloud architecture, APIs, containers, and distributed systems, challenging for teams lacking this experience.
3. **Infrastructure**
    - ✓ **Monolithic** apps work on a single server.
    - ✓ **Microservices** thrive in cloud environments, offering better scalability, fault tolerance, and high availability but require a robust setup and tooling.

How to transition (Migrate) from a monolithic to a microservices architecture



1. Assess the Monolith

- ✓ **Understand** the current system structure, modules, and dependencies.
- ✓ **Identify pain points**, performance bottlenecks, and tightly coupled components.

Kodewala

2. Define Business Capabilities

- ✓ **Decompose** the application based on business domains or bounded contexts (Domain-Driven Design).
- ✓ **Each service** should map to a single business capability (e.g. user management, billing, inventory, order, payment).

3. Establish Microservices Infrastructure

- ✓ Set up infrastructure for:
  - **API Gateway**
  - **Service Discovery** (e.g. Eureka)
  - **Configuration Management** (e.g. Spring Cloud Config)
  - **Monitoring & Logging** (e.g. ELK stack, Prometheus, Grafana)
  - **CI/CD pipelines**
  - **Containerization** (Docker, Kubernetes)

4. Strangle the Monolith (Strangler Fig Pattern)

- ✓ Incrementally extract functionality **one module at a time** into microservices.
- ✓ Replace monolith functionality gradually while keeping the rest intact.
- ✓ Route specific requests to new microservices using **API Gateway**.

5. Build Communication Between Services

- ✓ Use **REST APIs** or **Message Queues** (e.g. RabbitMQ, Kafka) for communication.
- ✓ Design **robust APIs**, handle failure gracefully with **circuit breakers** (e.g., Resilience4j, Hystrix).

6. Migrate Data Carefully

- ✓ Prefer **database per service**.
- ✓ Use patterns like **Event Sourcing** or **Change Data Capture** to sync data.
- ✓ Avoid direct sharing of databases between services.

7. Refactor Authentication and Authorization

- ✓ Move to centralized auth (e.g., OAuth2, Keycloak).
- ✓ Implement **token-based security** for APIs (e.g., JWT).

Kodewala

8. Test Extensively

- ✓ Do **integration tests**, and **end-to-end testing**.
- ✓ Validate service communication, failover, and data consistency.

9. Monitor and Optimize

- ✓ Continuously monitor service performance.
- ✓ Track logs, metrics, and failures to fine-tune deployments and architecture.

10. Decommission the Monolith

- ✓ Once all services are migrated and stable, retire the monolithic codebase.

Kodewala

Apache Kafka

The problem:

- ✓ You need a way to send data to a central place quickly.
- ✓ Since machines often break down, you also need a system that makes copies of your data, so these common failures don't lead to downtime or lost information.

That is where **Apache Kafka** comes in as a powerful solution. It is a **reliable** messaging system, originally built by **LinkedIn**, that works on a **publish-subscribe model** - letting systems send and receive data **efficiently** and **durably**.

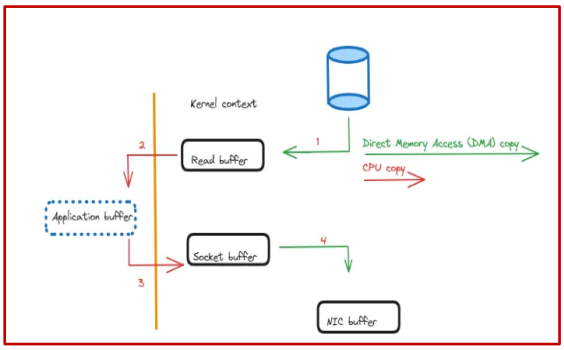
Why use Kafka?

- ✓ **Apache Kafka allows multiple producers and consumers** to work at the same time. This is different from many traditional queue systems, where once a message is read by one client, it is no longer available to others.
- ✓ **Disk-Based retention:**
  - Consumers do not always have to process messages right away.
  - Messages are saved to disk and stay there for a while, so they can be accessed later.
  - There is no risk of losing data.
- ✓ **Fast:** Kafka is a good solution for applications that require a **high throughput** (large number of operation), **low latency** (less processing time) messaging solution. Kafka can write up to 2 million requests per second.
- ✓ **Scalable:** We can add more capacity while the system is still running, without causing any downtime.
- ✓ **High Performance**: Excellent performance under high load.

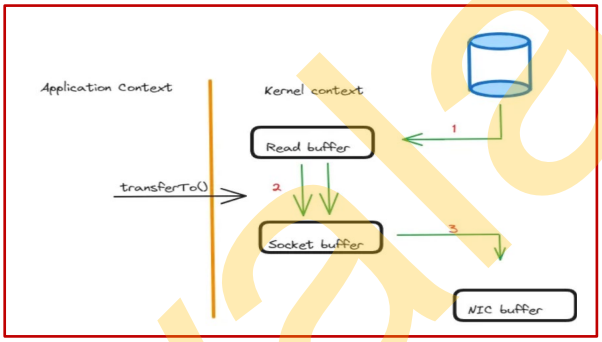
Kodewala

Why is Kafka fast? - Very Important Question from Interview point of view

- ✓ **Zero Copy**: Basically, Kafka calls the OS kernal directly rather than at the application layer to move data fast.



Normal Copy



Zero Copy

- ✓ **Batch data in chunks**: Kafka is all about batching the data into chunks. This minimises cross machine latency with all the buffering/copying that accompanies this.
- ✓ **Avoids Random Disk Access:** Kafka is designed to access the disk in sequential manner. This enables it to get similar speeds from a physical disk compared with memory.
- ✓ **Can scale Horizontally:** The ability to have thousands of partitions for a single topic spread among thousands of machines means Kafka can handle huge loads.

Compared to other message queue systems

RabbitMQ/AMQP Based System	KAFKA
AMQP based (Advanced Messaging Queueing Protocol)	Kafka is distributed event streaming platform. Became popular choice: Event driven Microservice architecture
Performance: 30-40K/Sec	Performance: 2 Million/Sec
Payload Size: No constraint	Payload size: Default limit 1 MB (configurable) *Question to ask if changing config?* *Why payload size not small and continuous stream or chucked*
Message Retention: Acknowledgement based	Message Retention: Policy based (E.g. retain 2 days etc)
Exchange Type: Direct, Fan out, Topic, header based	Publish Subscribe based

Kodewala

Kafka Use Cases

- ✓ **Activity tracking:** The original use case for Kafka, designed at LinkedIn, is that of user activity tracking.
- ✓ **Messaging:** where applications need to send notifications to users. Those can produce messages without needing to be concerned about formatting. Then another application can read all the messages and handle them consistently.
- ✓ **Metrics and logging**
- ✓ **Commit log:** Database changes can be published to Kafka and applications can easily monitor this stream to receive live updates as they happen.
- ✓ **Stream processing:** Kafka is extremely good for streaming and processing huge datasets.

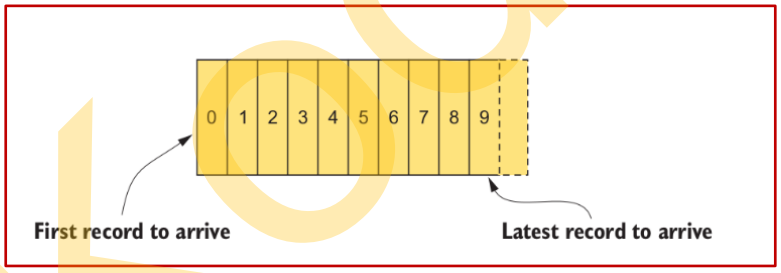
Kafka architecture

Broker

Kafka is a message broker. A **broker** is an intermediary that brings together two parties that don't necessarily know each other for a mutually beneficial exchange.

Log

Is a file that Kafka appends incoming records to. A log is an append-only, totally ordered sequence of records ordered by time



Configuration setting **log.dir**, specifies where **Kafka** stores log data on disk.

The logs directory is configured in the root at /logs.

/logs

/logs/topicA_0	topicA has one partition.
/logs/topicB_0	topicB has three partitions.
/logs/topicB_1	
/logs/topicB_2	

Kodewala

Topics

Topics are logs (group of messages) that are separated by topic name. Think, topics as labelled logs. The closest analogies for a topic are a **database table** or **a folder** in a filesystem.

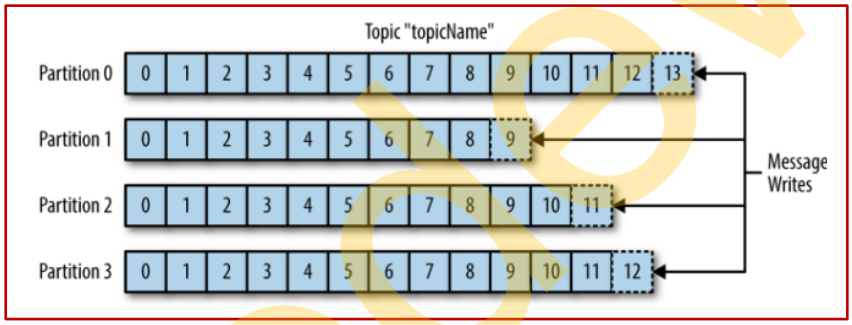
Topic examples are:

- ✓ orders
- ✓ payments
- ✓ delivery
- ✓ payments

To help manage the load of messages coming into a topic. Kafka use **partitions**

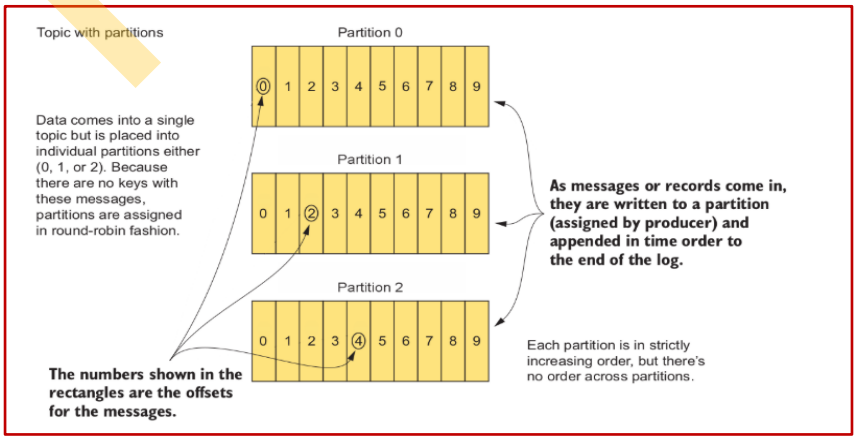
Topics are broken down into a number of **partitions**. (Default is 10)

Partitions are the way that Kafka provides redundancy and scalability. Each partition can be hosted on a different server, which means that a single topic can be scaled horizontally across multiple servers.



Partitions

- ✓ Help increasing throughput
- ✓ Allows topic messages to be spread across several machines so that the capacity of a given topic isn't limited to the available disk space one server.



koolwala

Partitions group data by key

- ✓ When a message is sent to kafka, we can specify a key option for that message.
- ✓ If the key (key will be discussed in the next section) is not null. Kafka uses the following formula to calculate which partition the message will be sent to.

```
Producer -> message(key, value) -> Kafka
// Kafka choose a partition by using the formula
target_partition = hashCode(key) % number of partitions
```

- ✓ Records with the same key will always be sent to the same partition and in order. This is famous Interview Question.

Important characteristics of Kafka

- ✓ Kafka stores and retrieves message from topic. Doesn't keep any state of producers or consumers.
- ✓ Messages are written into Kafka in batches. A batch is just a collection of messages, all of which are being produced to the same topic and partition.

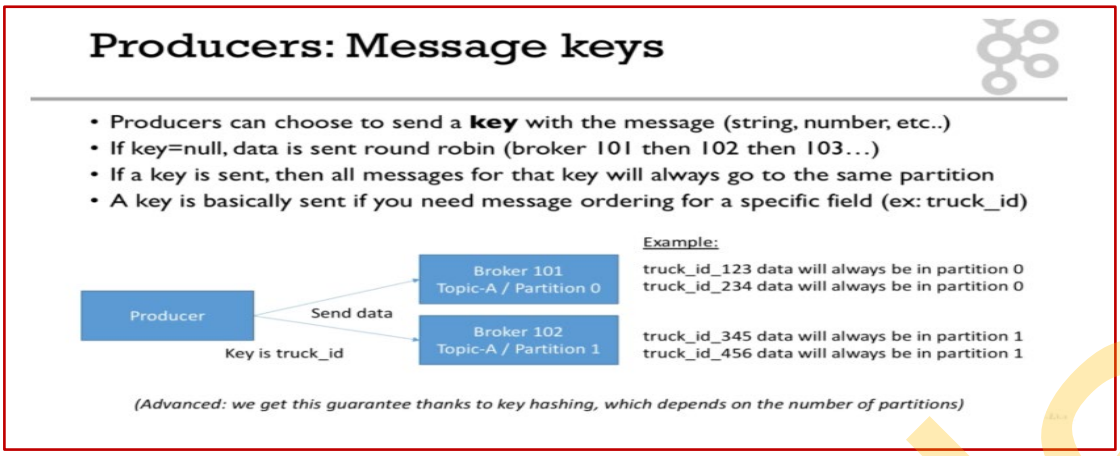
Producers and Consumers

Producers

Producers create new messages. In other publish/subscribe systems, these may be called publishers or writers. A message will be produced to a specific topic.

- ✓ The producer does not care what partition a specific message is written to and will balance messages over all partitions of a topic evenly. → Interview Question
- ✓ In some case, the producer will direct messages to specific partitions using message key. Messages with a specified message key will be ensured to come in the right order in a partition.

Koderwala

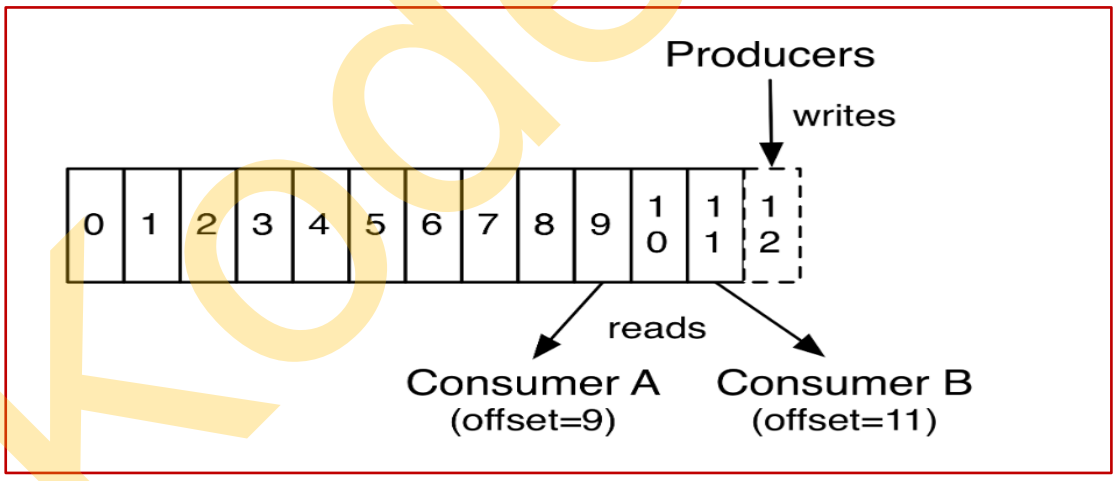


Consumer

Consumers read messages. In other publish/subscribe systems, these may be called subscribers or readers.

- ✓ The consumer subscribes to one or more topics and reads the messages in the order in which they were produced.
- ✓ The consumer keeps track of which message it has already consumed by keeping track of the offset of messages.

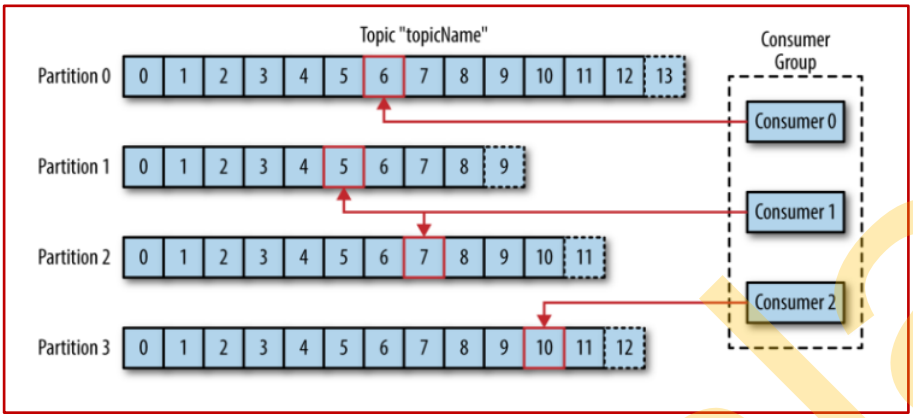
The offset is a simple integer number that is used by Kafka to maintain the current position of a consumer.



kodewala

Consumer group

- ✓ Consumers groups used to read and process data in parallel.

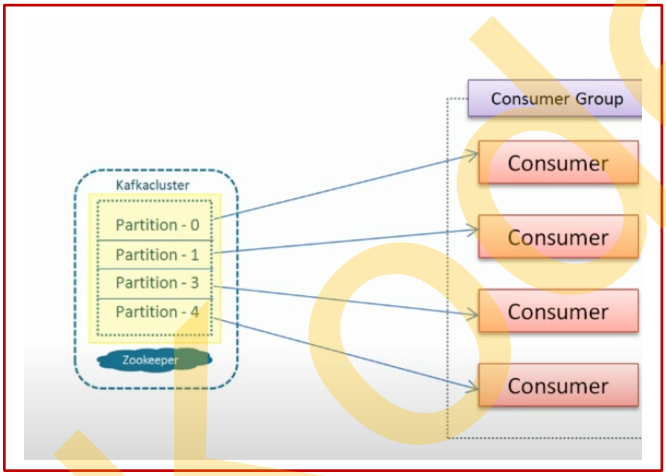


Question - How consumers can read data in parallel without duplicate reads?

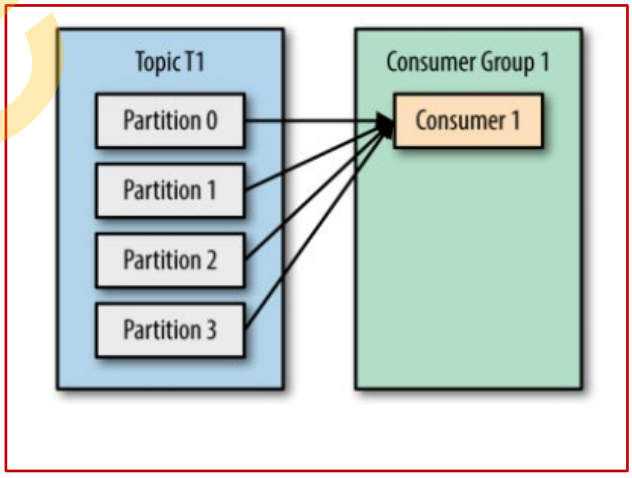
Kafka provide a simple solution for this problem.

- ✓ A partition can only be consumed by one consumer at a time.
- ✓ But a consumer can consumer multiple partitions parallelly.

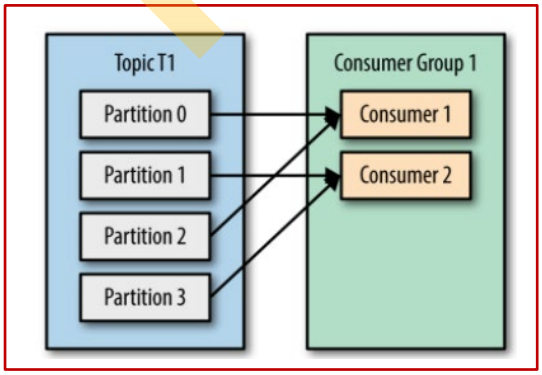
4 Consumers and 4 Partitions



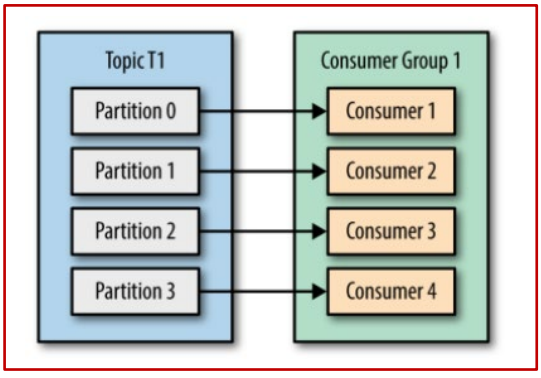
1 Consumer and 4 Partitions



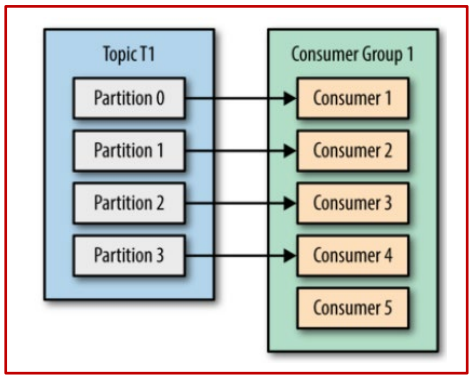
2 Consumer and 4 Partitions



4 Consumer and 4 Partitions

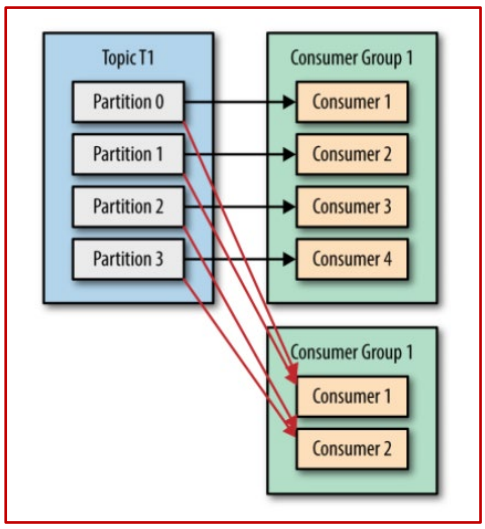


Kodewala



- ✓ If the number of consumers in a group exceeds the number of partitions in a topic.
- ✓ Then there will be some idle consumer and get no messages at all

Question: How can we read messages in parallel from the same partition?



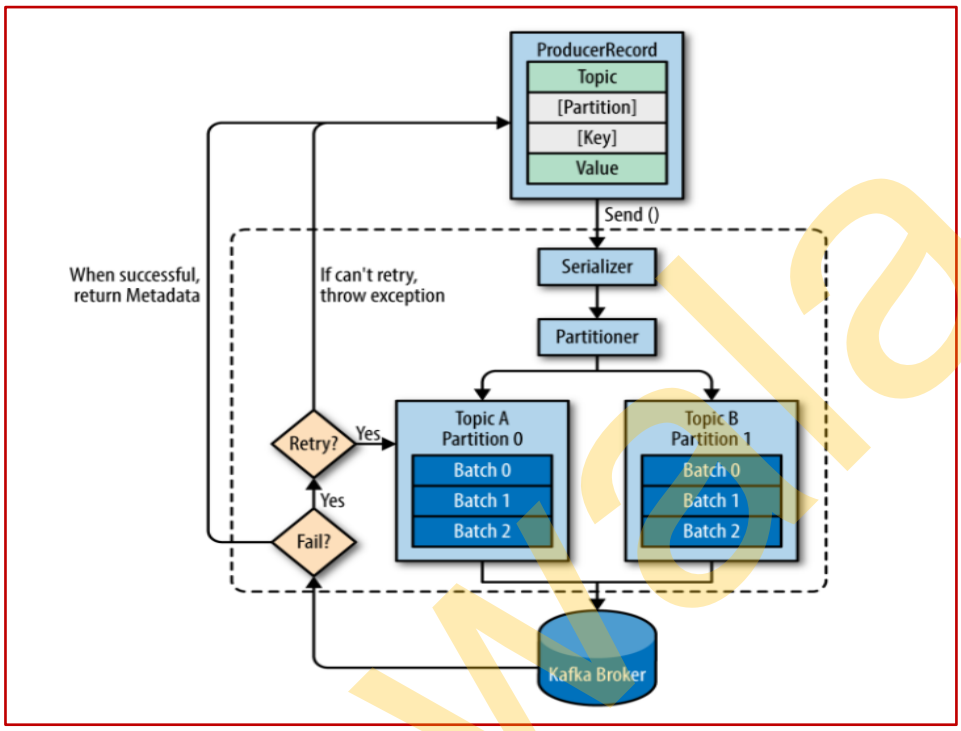
- ✓ You can create a new consumer group for each application that needs all the messages from one or more topics

Flow of sending a message

- ✓ Create a ProducerRecord, which must include the topic we want to send the record to and a value. Optionally, we can also specify a key and/or a partition.
- ✓ Then Serialized the key and value objects to ByteArrays so they can be sent over the network
- ✓ Data is sent to a partitioner. The partition check if ProducerRecord has a specifed partition option. If yes, it doesn't do anything an reply the partition we specify. If not, the partitioner will choose a partition for us.
- ✓ Once a partition is selected, the producer then add the record to a batch of records that will also be sent to the same topic and partition.
- ✓ When broker receives the messages, it sends back a response.
  - If the messages were successfully writtent to Kafka, return a RecordMetatData object contains <topic, partition, offset>

kodewala

- If failed, the broker will return an error. The producer may retry sending the message a few more times before giving up and returning an error.



Interview Question & Answers

1. Explain the ISR (In-Sync Replica) mechanism in Kafka. Why is it important for durability?

Hint: ISR tracks replicas that have fully caught up with the leader. Data is only acknowledged when written to all ISR members.

2. How does Kafka handle message ordering and what configurations impact this?

Hint: Ordering is guaranteed per partition. Use a consistent partitioning key. Partition count and key strategy affect order.

3. Describe how Kafka achieves high throughput and low latency.

Hint: Kafka uses sequential disk writes, batching, zero-copy.

Koderola

4. **What are Kafka exactly-once semantics (EOS)? How do you implement them?**

Hint: EOS ensures no duplicates or message loss. Use transactions with idempotent producers and configured brokers.

5. Describe the role of the __consumer_offsets topic.

Hint: Stores committed offsets for consumer groups. Enables fault-tolerant and recoverable consumption.

6. How do Kafka partitions affect consumer scalability and performance?

Hint: More partitions = more parallelism. But too many can impact broker memory and leadership overhead.

7. What happens when a Kafka broker goes down? How is data availability maintained?

Hint: Partitions on that broker are re-elected on ISR replicas. Availability depends on replication factor and ISR.

8. Explain the difference between Kafka producers with acks=0, acks=1, and acks=all.

Hint: Controls when a write is acknowledged. 0 = no wait, 1 = leader only, all = all ISR replicas.

9. What is the role of Zookeeper in Kafka?

Hint: Zookeeper manages cluster metadata and controller election.

10. How do Kafka Connect and Kafka Streams differ? When would you use each?

Hint: Connect = ETL integration tool, Streams = library for stream processing in your app.

11. Describe how you would design a Kafka topic schema strategy to support backward/forward compatibility.

Kodewala

Hint: Use Avro/Protobuf with Schema Registry. Maintain schema evolution rules (add optional fields, don't remove required ones).

12. How would you monitor and tune Kafka for performance (producers, brokers, consumers)?

Hint: Use JMX metrics, Prometheus/Grafana, and monitor lag, throughput, GC, disk I/O. Tune batch size, linger.ms, fetch size, etc.

13. What are the implications of using a high number of partitions in a Kafka topic?

Hint: Helps parallelism but increases metadata overhead, leader election time, and Zookeeper load.

14. Explain the rebalancing process in Kafka consumer groups. How can it be optimized?

Hint: Rebalancing happens when group membership or topic partitions change. Use cooperative rebalancing to reduce impact.

15. What strategies do you use to avoid message duplication or data loss in Kafka pipelines?

Hint: Use idempotent producers, transactions, acknowledgments (acks=all), and proper offset management.

Kodewala

Microservices Design Patterns and Principles

Introduction

The purpose of this guide is to help you with architectural patterns followed while designing Microservices architecture.

Abstract

This document aims to provide basic know-how of the 'Microservices Design Patterns.' We aim to provide a reference document for design patterns to be used when any application is designed using Microservices architecture.

We aim to publish this whitepaper as a ready reference to ten highly used design patterns related to Microservices in a detailed fashion.

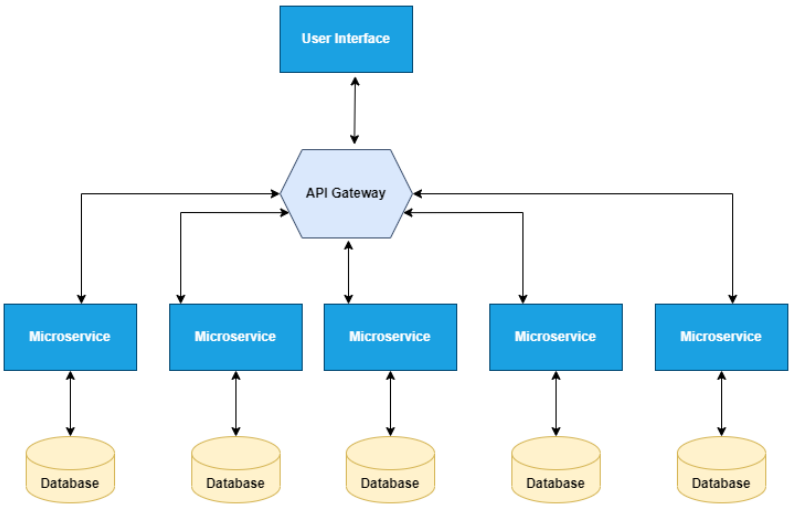
Introduction to Microservices

Microservices are self-contained and independent deployment modules. In Microservices, the application is divided into independent modules based on business domains. Microservices are designed based on **Domain Driven Design (DDD)**, which says the application should be modeled around independent modules with bounded context for the specific business domain. As we now have independent modules, we have faster rollout and quicker release cycles. All the modules can be worked on in parallel, and any changes will only affect that specific module. Therefore, Continuous Integration (CI) & Continuous Delivery (CD) are key advantages of using Microservices architecture.

> In short, the Microservices architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and are independently deployable by fully automated deployment machinery.
>
> **– Martin Fowler**

As Monolithic applications pose their own set of challenges, here are four main concepts that describe the importance of Microservices architecture over Monolithic architecture. Some of the key points about Microservices are:

1. Loosely coupled components designed around business domains
2. Application can be distributed across different clouds and data centers
3. Change management is easy in Microservices

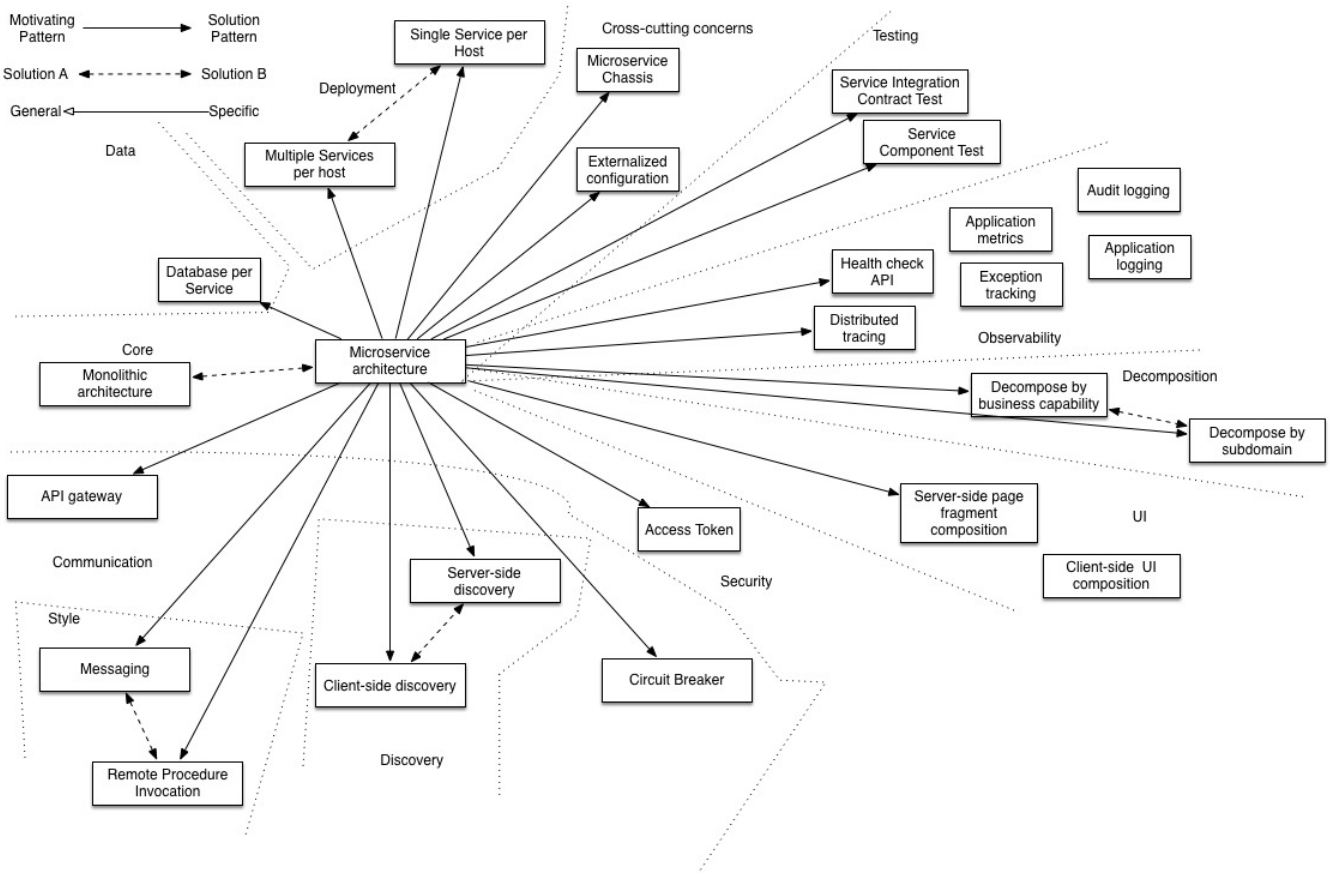


Microservices Design Patterns and Principles

As we now know, Microservices are independent components that encapsulate business domains. If applications are architected using Microservices design principles and patterns, then the application is highly scalable. So, let's list down the principles in which the Microservices architecture has been built.

1. Scalability
2. Flexibility
3. Independent and autonomous
4. Decentralized governance
5. Resiliency
6. Failure isolation
7. Continuous delivery through the DevOps

While adhering to the principles mentioned above, there are some standard sets of problems & complex scenarios that architects and developers need to address. In addressing such common problems, statements bring proven solutions to the table, which become patterns. Here are a few high-level patterns that are used around Microservices.

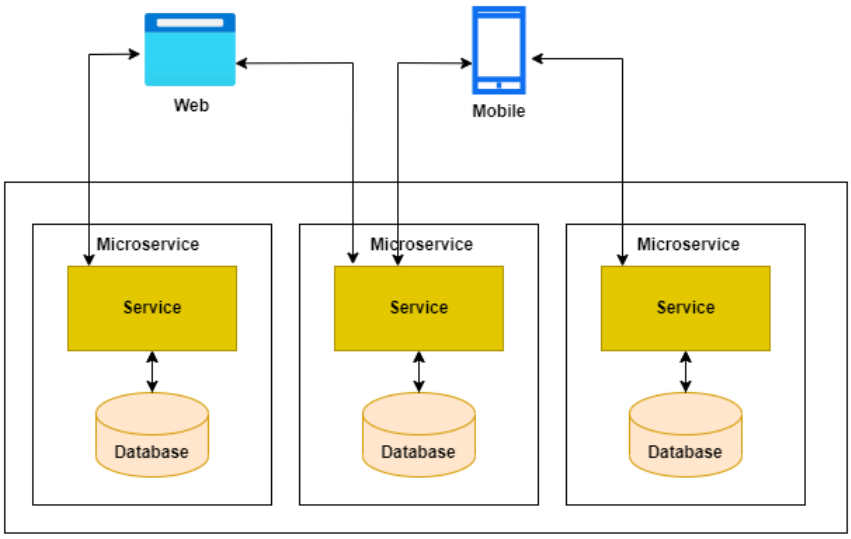


Picture Reference: https://microservices.io/patterns/microservices.html

We will cover the most used and discussed design patterns in detail, along with use cases. Here is the list of design patterns that will be covered:

1. Database per Microservices
2. CQRS
3. Event Sourcing
4. Saga
5. BFF
6. API Gateway
7. Strangler
8. Circuit Breaker
9. Externalized Configuration
10. Consumer-Driven Contract Tracing

Database per Microservices



Microservices are generally independent and loosely bound components. So, to achieve this independent nature, every Microservice must have its own database so that it can be developed and deployed independently.

Let's take the example of an e-commerce application. We will have product, ordering and SKU Microservices that each service interacts (store and retrieve) with data from their own databases. Any changes to one database don't impact other Microservices.

Other Microservices can't directly access each independent database. Each component's persistent data can only be accessed via APIs. So, Database per Microservices provides many benefits, especially to evolve rapidly and support massive scaling as they make each Microservice independent.

Database changes can be made independently without impacting other Microservices:

- Each database can scale independently
- Microservices domain data is encapsulated within the service
- If one of the database servers is down, this will not affect other services

Also ***Polyglot data persistence** gives the ability to select the best-optimized storage needs per Microservices. So, we can use this as an example - an e-commerce application with Microservices, as mentioned earlier. Here are the optimized choices:

- The product Microservices use a NoSQL document database for storing catalog-related data, which is storing JSON objects to accommodate high volumes of read operations
- The shopping cart Microservices use a distributed cache that supports its simple, key-value datastore
- The ordering Microservices use a relational database to accommodate the rich relational structure of its underlying data

Because of the ability to scale and high availability, NoSQL databases are getting highly popular and are widely used in enterprise applications nowadays. Also, their support for unstructured data gives flexibility to developments on Microservices components.

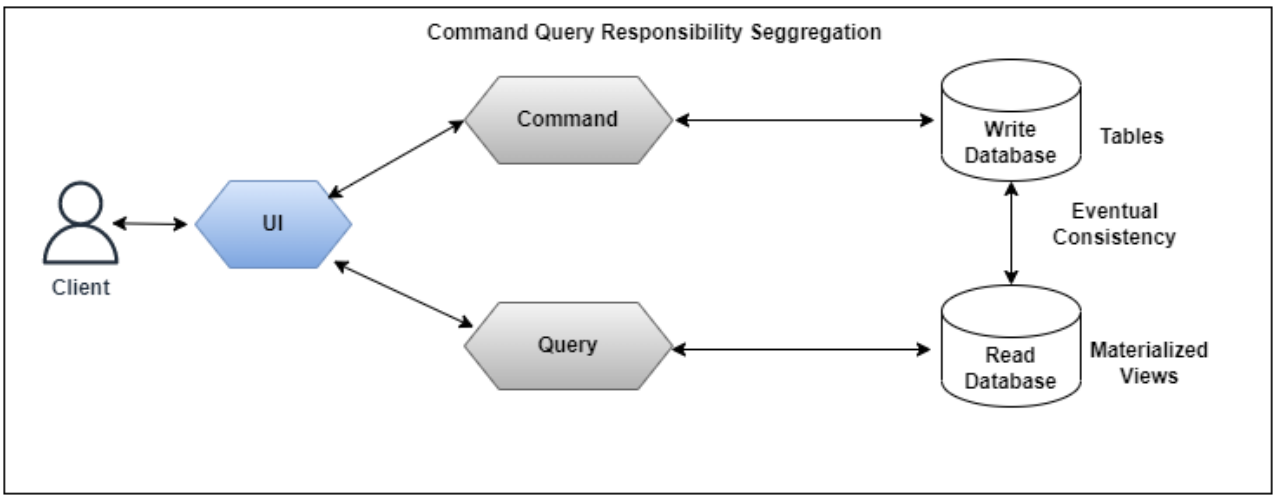
CQRS Design Pattern

CQRS stands for Command and Query Responsibility Segregation. It is one of the most widely used patterns for querying the database in Microservices architecture. CQRS is very handy when we need to eliminate complex queries involving inefficient joins. This pattern promotes the complete separation of read and write concerns in the database.

Traditionally, in Monolithic or SOA architecture, we have a single database for the entire application, and these databases will respond to both read and write requests. As the application becomes more complex over time, reading and writing to the database become non-performant. Sometimes, it has been observed that some applications follow an active-passive database model. However, even then, only one database copy is active at a time, so performance issues still persist.

For instance, in the case of database reads, if an application requires a query that needs to join more than ten tables, it can lead to locking the database due to the latency of query computation. Similarly, when writing to the database, we may need to perform complex validations and process lengthy business logic for some CRUD operations, which can also cause the locking of database operations.

So, reads and writes to the database are different operations for which separate strategies can be defined. To achieve this, CQRS offers to use “separation of concerns” principles and separate reads and writes into two databases. By this principle, we can use different databases for reading and writing database types, like using NoSQL for reading and using a relational database for CRUD operations.



So, another factor that is considered here is the nature of the application. If the use cases of your application mostly need to read the data in comparison to writing, then your application is a read-intensive application. So accordingly, you should focus

and choose your read and write databases. So here, CQRS separates reads and writes into different databases, where commands perform the creation or updating of data, and queries perform read data.

Commands are actions with some defined operations like “add the item to bucket” or “check my balance.” So, commands can be published via message brokers, which help applications process them in an async manner. Queries never modify the database. Queries always return the JSON data with DTO objects. In this way, we can isolate the Commands and Queries.

How to Sync Databases with CQRS?

When we segregate read and write concerns in two different databases, the primary consideration is to sync these two databases properly. So, both databases should always be synced.

This can be achieved using **Event Driven Architecture**. As per Event Driven Architecture, when an update command is issued to the write database, it will publish an update event using message broker systems, and this will be consumed by the read database, which will pull the latest changes from the write database to keep the read database in sync.

But this option creates a consistency issue because the data would not be reflected immediately since async communication is implemented with message brokers. This will operate the principle of “**eventual consistency**.” Eventual consistency is a property of distributed computing systems such that the value for a specific data item will, given enough time without updates, be consistent across all nodes. The read database eventually synchronizes with the write database, which will take some time to sync.

So, if we return to our read and write databases in the **CQRS pattern** when you start on the design, you can initially create a read database from the replicas of the write database. Also, as we have separate read and write databases, it means that both are scalable independently.

Event Sourcing

With CQRS, as reads and writes are separated, we need to keep both the database in sync as well, which is achieved using Event-Driven Architecture. But any failure in this sync or any loss of event can make data inconsistent in one of the databases, so to overcome this scenario, Event Sourcing is used. In Event Sourcing, apart from publishing the events on message broker systems, we store the events in the write database, and this event is stored in a single source of truth. In case of failure, events can be replayed, which will help keep data consistent.

Let's understand Event Sourcing with an example. Suppose there is a user’s table and a user updates their details. Usually, the updated details will override the existing values. This is generally how most applications work, i.e., they always store the entity’s current state.

But with the Event Sourcing pattern, it is changing. Instead of storing just the latest state, we store all operations in the database. The Event Sourcing pattern suggests saving all events into the database with a sequential order of data events. This events database is called an event store.

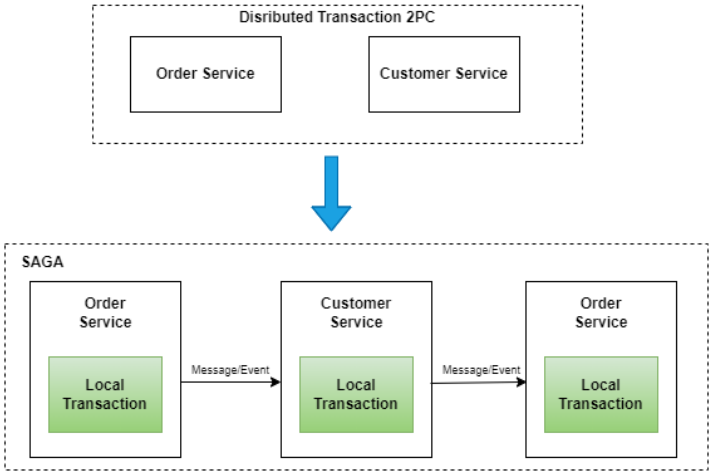
So, the event store is considered as a single source of truth for the data. After that, these event stores are converted to a read database using the materialized views. This conversion operation can also be handled by publish/subscribe patterns with events published through message broker systems. Moreover, this event list gives us the ability to replay events at a given timestamp. This allows us to build the latest status of data in case of any failure.

Let’s take the use case of a shopping cart for our e-commerce application, where we applied CQRS and Event Sourcing. As you can see in the image below, for the shopping cart, every user’s actions are recorded in the event store with appended events. All these events are combined and summarized on the read database with denormalized tables into materialized view database.

By applying these patterns, we can query the latest status of the shopping cart using materialized views, which are used to create a read database. Instead of storing actual data, it stores sequential events that denote user actions, allowing us to know the history of actions the user has taken with timestamps. This enables us to retrieve the status of shopping cart data at any point in time. We can use event store write databases with Azure Cosmos DB, Cassandra, Event Store databases, etc.

SAGA

Usually, Microservices are designed around business domains, so the application is divided into multiple Microservices. However, there is a challenge, as now, any transaction will also be distributed across the services. To handle this distributed transaction scenario Saga pattern is used.



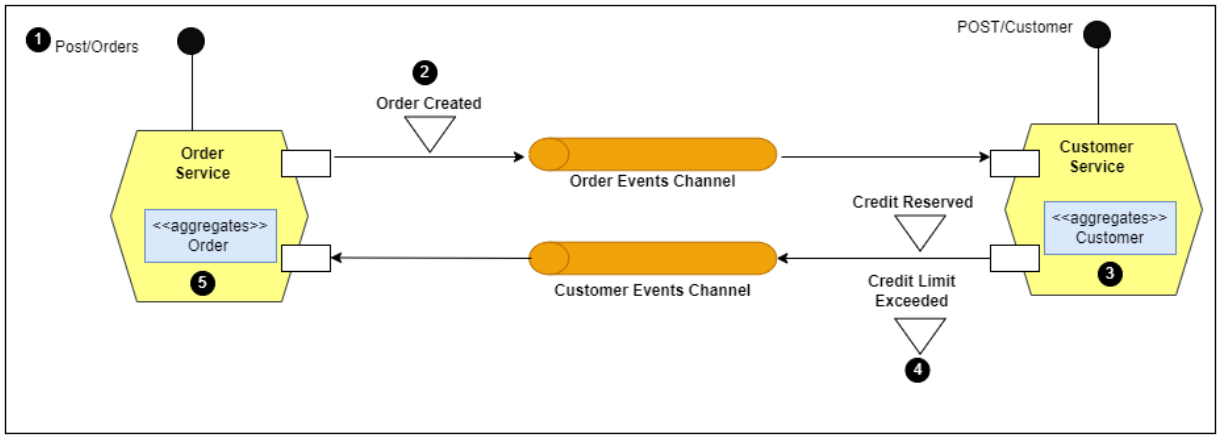
The Saga design pattern is used for managing data consistency in the case of distributed transactions. The Saga pattern tends to create a set of transactions that update each Microservice sequentially and publish an event to trigger the following transaction for the next Microservices. If the transaction fails at any step or service, then the Saga patterns trigger to rollback event, which basically does reverse operations in each of the Microservices.

A saga is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga. If a local transaction fails because it violates a business rule, then the saga executes a series of compensating transactions that undo the changes that were made by the preceding local transactions.

There are two options for how SAGA is implemented:

- **Choreography** - Each local transaction publishes domain events that trigger local transactions in other services
- **Orchestration** - An orchestrator (object) tells the participants what local transactions to execute

Choreography-based saga



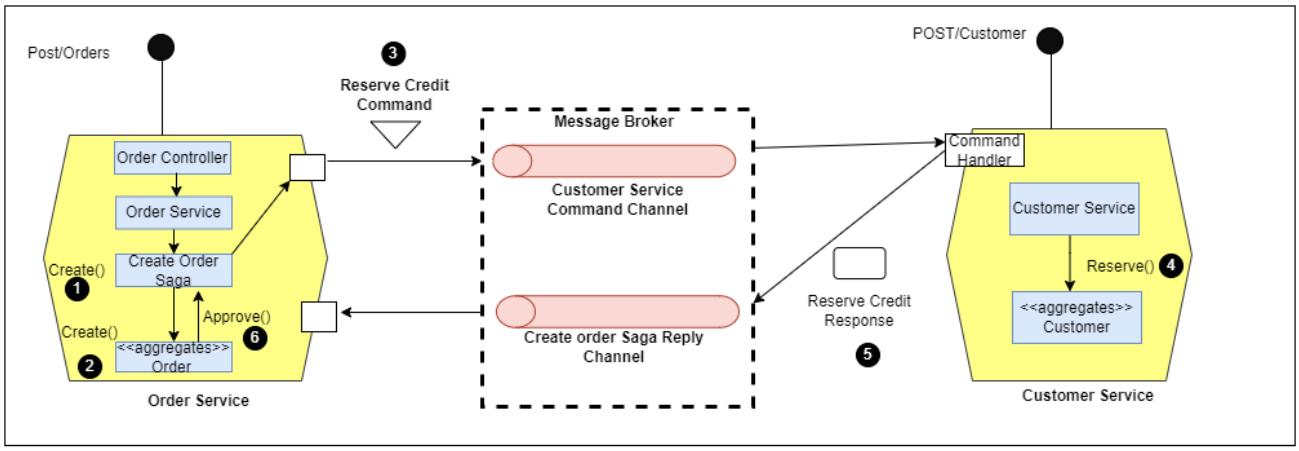
Here is the e-commerce service using events to implement choreography-based SAGA.

Choreography enables the implementation of the saga pattern by applying publish-subscribe principles. With choreography, each Microservice executes its own local transactions and publishes events to a **message broker system** that triggers **local transactions** in other Microservices.

In the above diagram, the order service receives a post request for an order at step 1 and updates the local database. Then, it publishes the 'order created' event on the order event channel. At the same time, the customer service is a subscriber to the order event channel and will receive an event notification. On receiving notification, customer service triggers its own local tasks, like checking if the customer has appropriate credits or not and then reserving credits for the order for which the notification was received. Once done, it will either emit a 'credit reserved' or 'credit limit exceeded' event in the customer event channel, which is subscribed by the order service. This is how the entire distributed transaction works.

For fewer services, this flow works well. Still, suppose the number of services are more. In that case, it becomes complex, and apart from its own local tasks, each service has the additional responsibility of emitting & consuming events as well, which makes things furthermore complex, and that's where the Orchestration-based saga will solve the issue.

Orchestration Based Saga



The Orchestrator-based saga also operates on events, but the responsibility for emitting events and ensuring the atomicity of transactions shifts from the services themselves to the orchestrator. Orchestrator commands services to execute local transactions. The orchestrator acts as an agent who tells services to execute local transactions and maintains the status of the complete transaction.

In the above diagram, the order service receives a post request for placing an order at step 1. Order service will initiate a transaction here and create a saga orchestrator. This orchestrator will ask the order service to create an order in the pending state. Now the orchestrator will send a “Reserve credit” command to customer service. Customer service will reserve credit and send the status of its local transaction channel, which the orchestrator consumes. Now, if the orchestrator receives a positive status from customer service, it will ask the order service to update the order status from “pending” to “In-Progress.” But, if the credit is not reserved, then the entire transaction will be rolled-back from the order and customer service.
As the orchestrator now takes care of emitting events, this type of implementation is suitable for complex workflows involving many services. New services might also be added in the future. Since the orchestrator controls every transaction, there is less chance of getting into cyclical dependency.

Use the Saga pattern when you need to:

- Ensure data consistency in a distributed system without tight coupling
- Roll back or compensate if one of the operations in the sequence fails

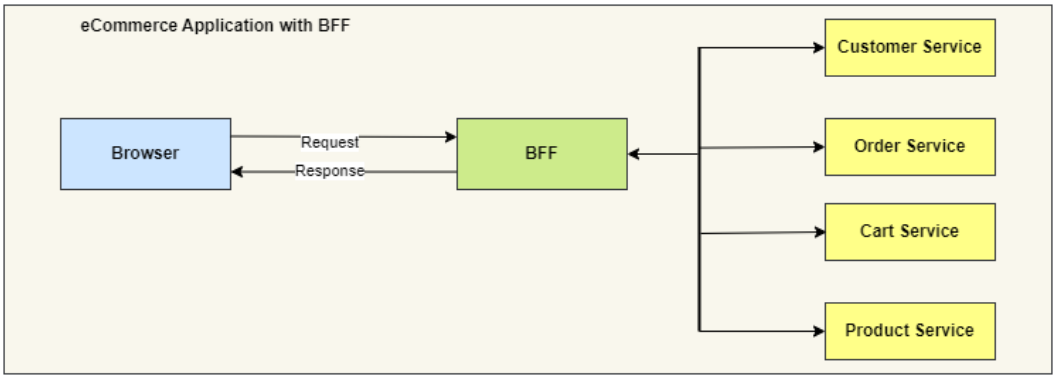
BFF (Backend for Frontend)

As and when an application is designed in Microservices architecture, there may be cases where data to be displayed in the frontend, be it web or mobile, might be coming from multiple services. In such cases, it becomes the responsibility of the frontend team to transform and aggregate responses from multiple Microservices. However, there are a few challenges with this approach:

- The frontend team needs to take care of a lot of transformations and aggregations
- Also, the browser or mobile app interface will use more resources for rendering the page

BFF patterns solve these challenges by introducing an intermediate layer between frontend and backend services. The BFF layer acts as a proxy that will call all backend services, aggregate and transform the response as per the needs of the frontend, and expose ready-to-consume responses that best fit the frontend. Therefore, there will be very minimal logic on the frontend. Hence, a BFF helps to streamline the data formatting process and takes up the responsibility of providing a well-focused response for the frontend to consume.

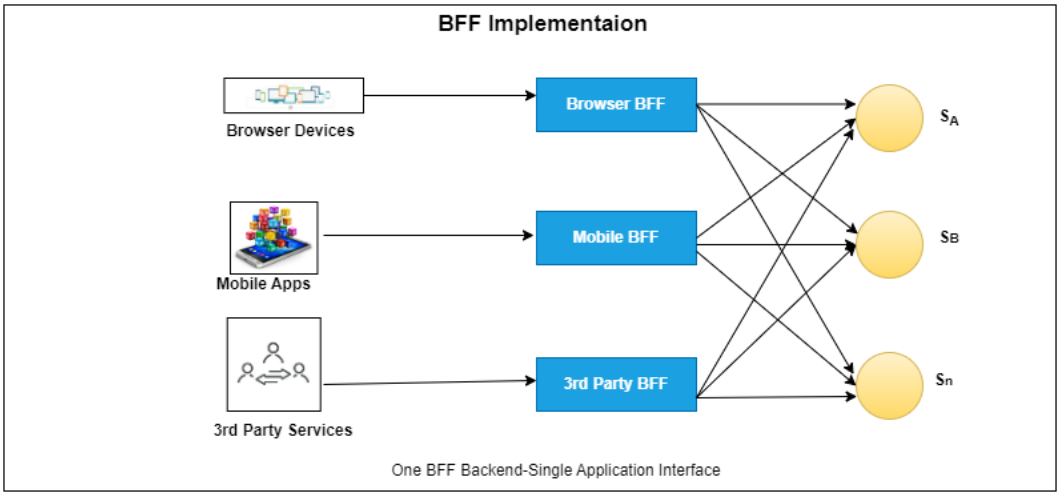
Here is an example showing BFF:



Ideally, the front-end team will be responsible for managing the BFF in most cases. A single BFF is focused on a single UI and that UI only. As a result, it will help us keep our frontends simple and see a unified view of data through its backend.

Usage of BFF again depends on the needs and architectural requirements. If the application you are designing or migrating to Microservices is simple, where each service has its own UI, then BFF is not needed. However, if the application has a complex functionality that involves multiple third-party integrations and front-end screens need to show processed data from multiple services, BFF is the best fit. Also, if the client's requirement is around optimal rendering of the frontend and backend, and you have complex Microservices, a BFF is suitable to use.

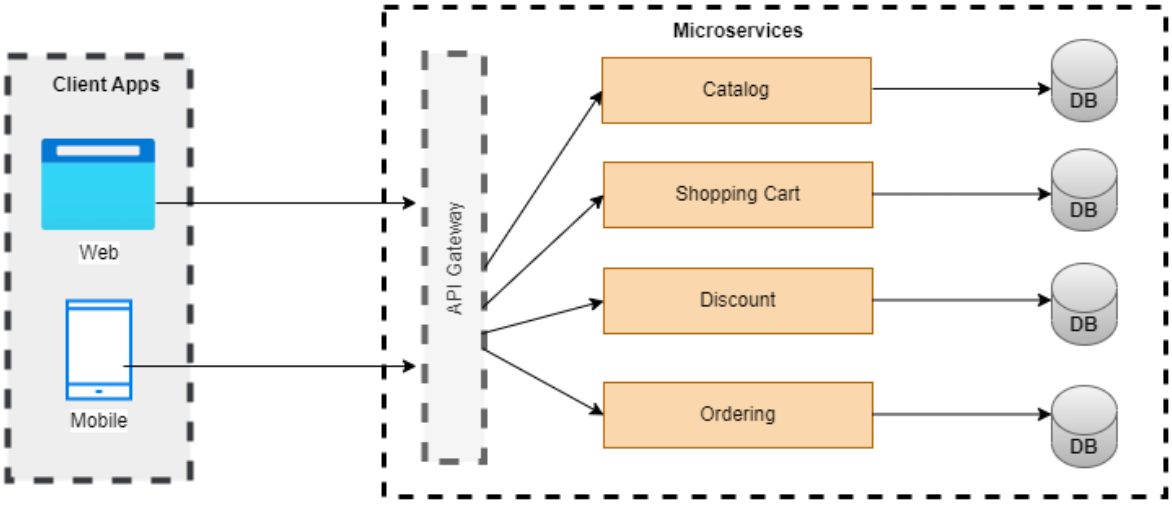
Furthermore, there can be multiple BFFs in the application depending on the needs and requirements of the application. In a scenario where the same backend code serves both web and mobile, where each shows data differently, there can be one BFF layer for the web and one BFF layer for mobile.



To conclude, implementing these patterns should be done judiciously, and code duplication should be avoided.

API Gateway

If we need to build a large, complex Microservices-based application with multiple clients, we can use the API gateway pattern. This pattern applies to distributed systems and acts as a reverse proxy or gateway routing for requests coming in from clients. API gateway acts as communication between the client and underlying services and serves as a single-entry point to the system while abstracting the underlying architecture. It also provides cross-cutting concerns like authentication, SSL termination, and caching.



So, as you can see, the API gateway provides a single entry point to multiple services. There are some things that we need to make sure of while using this pattern.

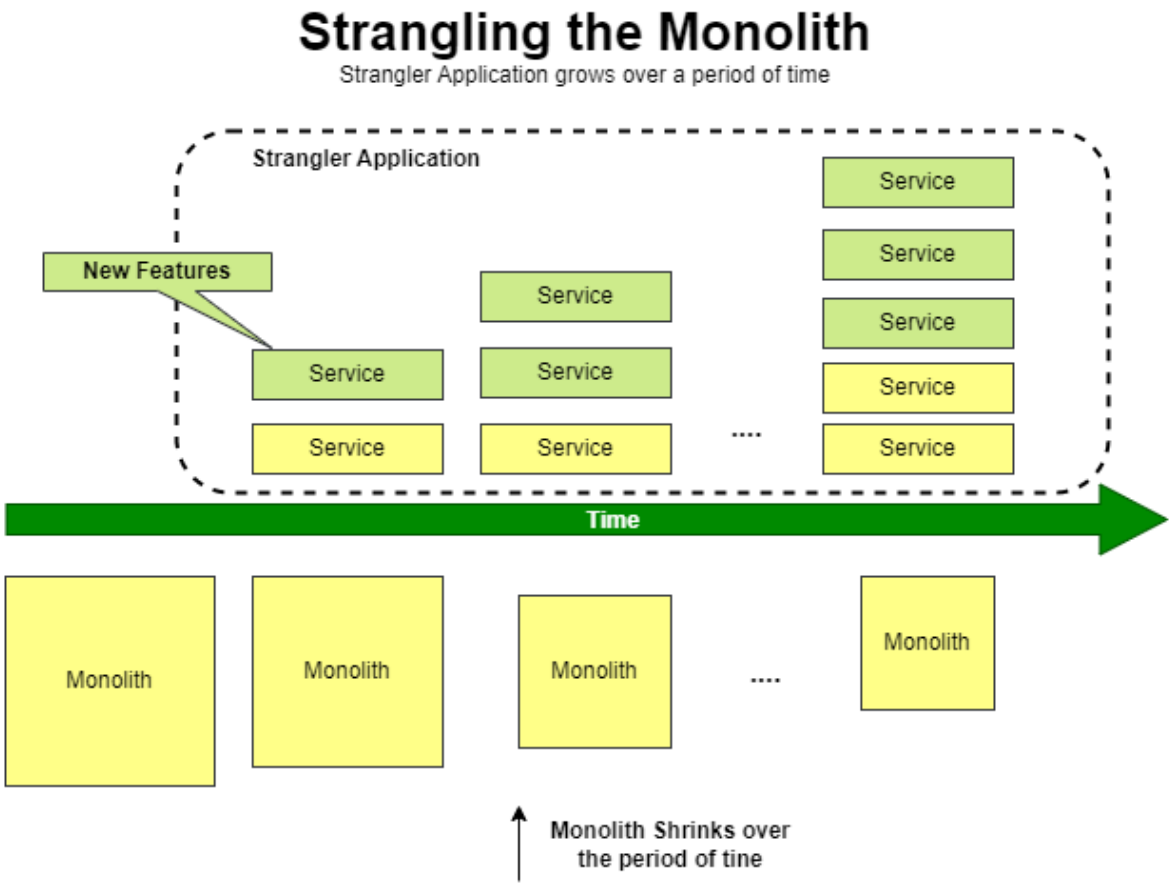
If there is only one node acting as an API gateway, there is a chance it will become a single point of failure. This situation needs to be handled, as if more complex logic is added to the API Gateway, it will become an anti-pattern. As the usage of this pattern grows, it is advised to deploy multiple API Gateways for multiple services based on the use case and the number of services.

In summary, we need to be cautious about using a single API Gateway, as it should be decided based on the business context of the client applications. Ideally, it is not advised to have one API gateway for all the internal Microservices.

Strangler Pattern

The Strangler Pattern is a methodology used in Microservices architecture to gradually migrate a Monolithic application to a Microservices-based architecture. This pattern is useful when the application is too large and complex to migrate simultaneously.

The Strangler Pattern involves designing a new set of Microservices that gradually take over the functionality of the Monolithic application. Initially, all requests for specific functionality are divided between the traditional Monolith and the new Microservice to cater to the same functionality. Over time, as the Microservices mature, they start to handle more of the traffic, and the Monolithic application handles less. Slowly, the entire complex Monolith will be replaced by more agile and scalable Microservices.

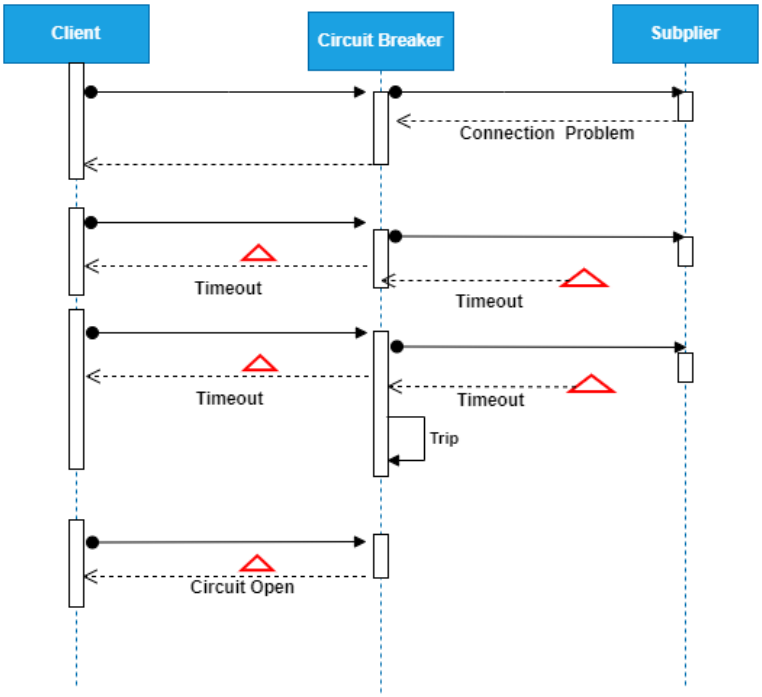


The Strangler Pattern follows these basic steps:

- Identify the functionality of the Monolithic application around the business domain that can be converted to a Microservice
- Create a new set of Microservices from scratch to handle the identified functionality
- Then, we must configure partial traffic to the new Microservices using an API gateway or any proxy server
- Over a period, gradually increase the traffic to the Microservices while decreasing the traffic to the Monolithic application
- Keep monitoring the performance of the new Microservices and adjust the traffic accordingly
- Eventually, route entire traffic to Microservices and bring down the entire Monolithic service

Circuit Breaker

The Circuit Breaker Pattern is a technique used in Microservices architecture to improve the resilience and reliability of distributed systems. The basic idea behind the circuit breaker is very simple. You wrap a function call in a circuit breaker object, which monitors it for failures. Once the failures reach a certain threshold, the circuit breaker trips and all further calls to the circuit breaker return an error, with any further calls skipped. The Circuit Breaker can also be configured to handle failures in different ways, such as returning a default value or a cached response.



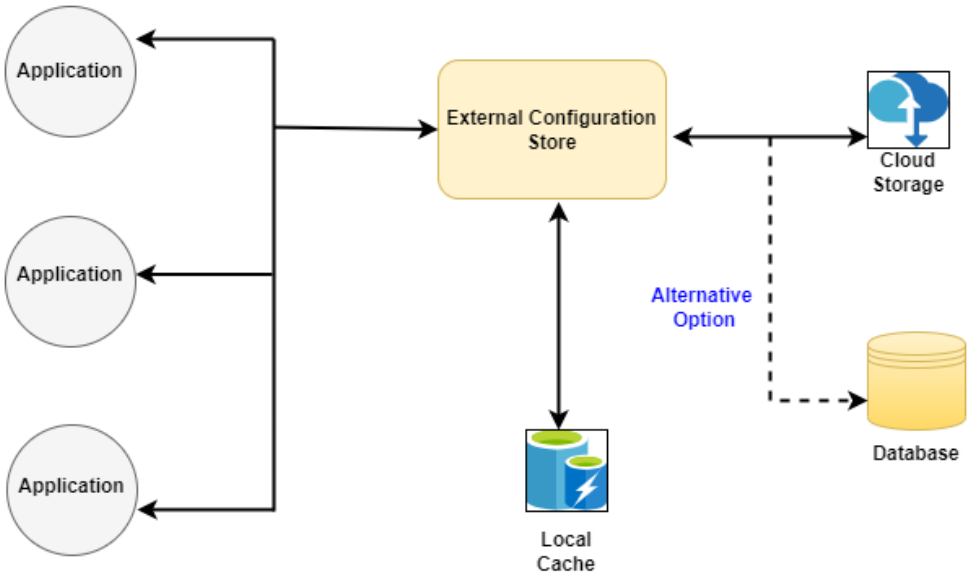
Here are the basic steps involved in the Circuit Breaker Pattern:

- Identify the Microservices that need to communicate with each other
- Add a Circuit Breaker between the Microservices
- Add configurations to monitor the status of the Microservices and detect failures
- If a Microservice fails to respond, the Circuit Breaker can prevent further requests to the Microservice and handle the failure in an appropriate way
- In some cases, the default response is also configured, which will help to handle failures gracefully.

Externalized Configuration

In Microservices architectures, systems are divided into multiple Microservices, each one running in its own container. Each process can be deployed and independently scaled, which means there may be many instances of the same microservice running at a specific time.

Let's say we want to modify the configurations for a microservice that has been replicated almost a hundred times.
If the configuration for this microservice is packaged with the service itself, we need to do deployment again for each instance running. This can lead to an inconsistent state as there are high chances that a few instances still need to upgrade to a new build and are still running on old configurations. Therefore, it is advised that services share the external configuration.



It works by keeping the configurations in an external store, such as a database, file system, or environment variables. When the Microservice is deployed and started, it loads the configuration from the external source. At runtime, if configuration changes occur, those are generally reloaded by Microservices without any new deployments.

Consumer-Driven Contract Tracing

This pattern can be referred to as the test-driven development pattern for the development of Microservices. The pattern suggests taking a design-first approach where negotiation on the expected response between provider and consumer decides the best possible outcome. The developers of a consumer service write “contracts” specifying the responses they expect from the requests made to a service provider. It is “consumer-driven” because the consumer’s developers drive the writing of the contract and lead the negotiations with the provider’s developers.

The contract is typically a JSON or XML file that the developers and their services (consumer and provider) can access. One of the main benefits of consumer-driven contract testing is that it allows collaboration from the start between service providers and consumers. It also helps service developers to understand the requirements and expectations of consumers in a proactive manner.

Another benefit of CDCT is that it helps to prevent integration issues between services. By defining the contract up front, both the consumer and provider can ensure they are on the same page, saving many testing cycles as well.

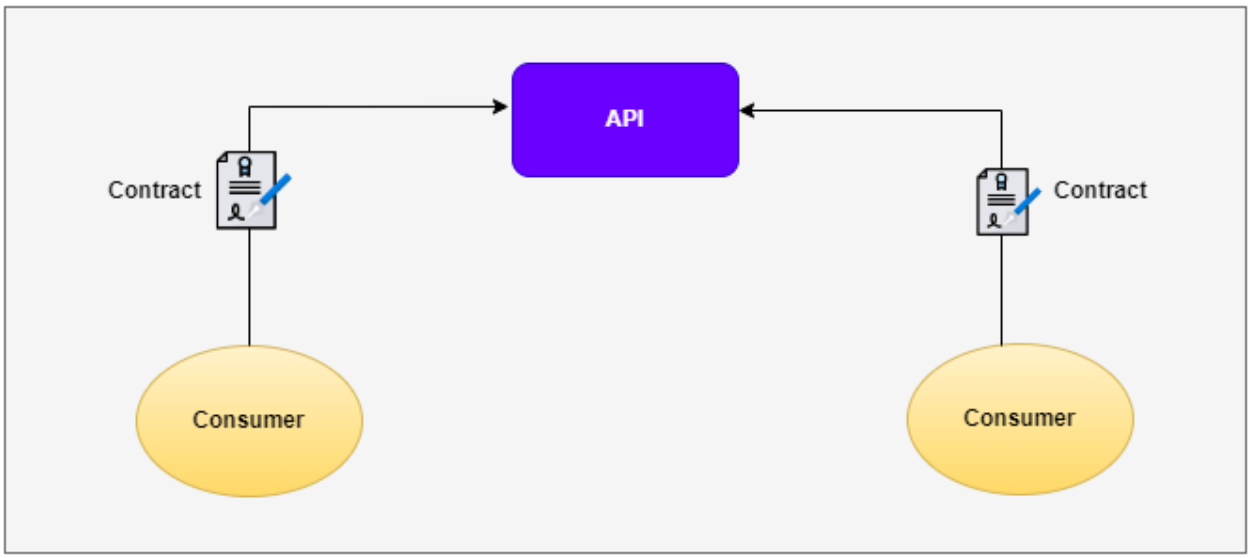
Here are the steps to implement CDCT:

1. **Define the contract**: The consumer of a service should define the contract that the service provider must follow. This should include the expected input and output of the service, as well as any other requirements that the consumer has.
2. **Implement the contract**: The service provider should implement the contract and ensure that their service meets the requirements.

3. **Run tests**: The service provider should then run test cases to ensure the service meets the requirements. This can include both unit tests and integration tests.

4. **Publish the contract:** Once the service provider has implemented the contract and passed their tests, they should publish the contract to the consumer.

5. **Verify compatibility**: Consumers can then verify that they got what was agreed upon.



Conclusion

With the increase in the usage of Microservices-based architecture, complexities arise in managing scalability or handling distributed services' transactions. However, a set of defined patterns, which have been tested repeatedly, give solutions to problems that are very common for Microservice-based architectures. Knowing each pattern provides good insight into how Microservices architecture handles performance, scalability, agility, and maintainability. We hope that the few patterns described above provide good insight for you.